Chương 6

Transformer

Cho đến nay, chúng ta đã bàn về một số mô hình cơ bản để giải các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*). Bây giờ ta sẽ tìm hiểu một lớp mô hình mới dựa trên một kiến trúc rất mạnh, gọi là **Transformer**. Transformer khác với các mô hình đã trình bày ở Chương 4 và 5 ở nhiều điểm. Thứ nhất, chúng không dựa vào mạng nơ-ron hồi quy (*recurrent neural network*) hay mạng nơ-ron tích chập (*convolutional neural network*) để mô hình hóa chuỗi từ, mà chỉ dùng cơ chế chú ý (*attention mechanism*) và mạng nơ-ron truyền thẳng (*feed-forward neural network*). Thứ hai, việc dùng tự chú ý (*self-attention*) trong Transformer giúp xử lý dễ dàng hơn các ngữ cảnh toàn cục và các quan hệ phụ thuộc giữa các từ. Thứ ba, Transformer là những kiến trúc rất linh hoạt, có thể dễ dàng sửa đổi để phù hợp với nhiều tác vụ khác nhau. Thập kỷ vừa qua đã chứng kiến sự trỗi dậy của Transformer không chỉ trong NLP mà còn trong nhiều lĩnh vực khác. Khi Transformer và các biến thể của nó ngày càng hoàn thiện, những mô hình này đang đóng vai trò ngày càng quan trọng trong nghiên cứu và ứng dụng trí tuệ nhân tạo (*artificial intelligence*).

Trong chương này, chúng ta sẽ bàn về những ý tưởng cốt lõi của Transformer. Ta bắt đầu bằng việc xem xét kiến trúc Transformer chuẩn. Sau đó ta sẽ xem một số bước phát triển đáng chú ý, chẳng hạn các cải tiến cho kiến trúc cơ bản và các phương pháp hiệu quả (về tính toán). Chúng tôi cũng sẽ giới thiệu một số ứng dụng mà ở đó mô hình Transformer đã được dùng rộng rãi. Tuy vậy, Transformer là một chủ đề rất rộng và đã có rất nhiều bài báo liên quan. Chương này không nhằm cung cấp một bản

tổng quan đầy đủ về các công trình đã có, mà là trình bày một tập hợp các chủ đề được chọn lọc, có thể được cộng đồng NLP quan tâm.

6.1 Mô hình cơ bản

Ở đây ta xem xét mô hình được trình bày trong công trình của Vaswani và cộng sự [2017]. Ta bắt đầu với kiến trúc Transformer rồi bàn chi tiết về từng thành phần của nó.

6.1.1 Kiến trúc Transformer

Hình 6.1 cho thấy mô hình Transformer chuẩn, đi theo khung bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) chuẩn. Một bộ mã hóa Transformer gồm nhiều **tầng mã hóa** (*encoding layers*) (hay **khối mã hóa** (*encoding blocks*)) xếp chồng lên nhau. Mỗi tầng mã hóa có hai tầng con (*sub-layers*) (hay khối con) khác nhau, gọi là tầng con tự chú ý và tầng con mạng truyền thẳng (*feed-forward neural network, FFN*) (FFN). Giả sử ta có một chuỗi phía nguồn $\mathbf{x} = x_1 \cdots x_m$ và một chuỗi phía đích $\mathbf{y} = y_1 \cdots y_n$. Đầu vào của một tầng mã hóa là một chuỗi gồm m vectơ $\mathbf{h}_1 \cdots \mathbf{h}_m$, mỗi vectơ có d_{model} chiều (hay d chiều cho gọn). Theo ký hiệu đã dùng ở các chương trước, ta dùng $\mathbf{H} \in \mathbb{R}^{m \times d}$ để ký hiệu các vectơ đầu vào này[1]. Tầng con tự chú ý trước hết thực hiện một phép tự chú ý $\text{Att}_{\text{self}}(\cdot)$ trên $\mathbf{H}$ để sinh ra đầu ra $\mathbf{C}$:

$$\mathbf{C} \;=\; \text{Att}_{\text{self}}(\mathbf{H}). \tag{6.1}$$

Ở đây $\mathbf{C}$ có cùng kích thước với $\mathbf{H}$, và do đó có thể được xem là một biểu diễn mới của các đầu vào. Sau đó, một kết nối tắt (residual) (*residual connection*) và một khối chuẩn hóa tầng (*layer normalization*) được thêm vào đầu ra để mô hình thu được dễ tối ưu hơn.

[1] Với $\mathbf{h}_j \in \mathbb{R}^d$ là một vectơ hàng, ta có $\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_m \end{bmatrix}$.

Chú giải của người dịch: Ma trận H: mỗi hàng là một từ

Hãy hình dung câu nguồn có $m = 3$ từ, và mỗi từ được biểu diễn bằng một vectơ $d = 4$ chiều. Xếp ba vectơ này thành ba hàng, ta được một bảng số 3×4, chính là ma trận $\mathbf{H}$:

$$\mathbf{H} = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 0 \\ 3 & 1 & 0 & 1 \end{bmatrix} \quad \text{(hàng 1 là từ thứ nhất, hàng 2 là từ thứ hai, \ldots).}$$

Mỗi tầng của Transformer nhận vào một bảng như vậy và trả ra một bảng *cùng kích thước* $m \times d$: số hàng vẫn là số từ, nhưng nội dung từng hàng đã được "làm giàu" thêm thông tin từ các từ khác trong câu. Vì kích thước không đổi nên ta có thể xếp chồng nhiều tầng giống hệt nhau lên nhau.

Mô hình Transformer gốc dùng cấu trúc **post-norm** (*chuẩn hóa sau*), trong đó kết nối tắt được tạo ra trước khi thực hiện chuẩn hóa tầng, như sau:

$$\mathbf{H}_{\text{self}} = \text{LNorm}(\mathbf{C} + \mathbf{H}), \tag{6.2}$$

trong đó phép cộng thêm $\mathbf{H}$ biểu thị kết nối tắt, còn $\text{LNorm}(\cdot)$ là hàm chuẩn hóa tầng. Thay Phương trình (6.1) vào Phương trình (6.2), ta thu được dạng của tầng con tự chú ý

$$\begin{aligned} \text{Layer}_{\text{self}}(\mathbf{H}) &= \mathbf{H}_{\text{self}} \\ &= \text{LNorm}(\text{Att}_{\text{self}}(\mathbf{H}) + \mathbf{H}). \end{aligned} \tag{6.3}$$

Định nghĩa của $\text{LNorm}(\cdot)$ và $\text{Att}_{\text{self}}(\cdot)$ đã được trình bày ở Chương 2 và 5, và chúng ta cũng sẽ bàn lại chúng ở phần sau của mục này.

Tầng con FFN nhận $\mathbf{H}_{\text{self}}$ và cho ra một biểu diễn mới $\mathbf{H}_{\text{ffn}} \in \mathbb{R}^{m \times d}$. Nó có cùng dạng với tầng con tự chú ý, chỉ khác là hàm chú ý được thay bằng hàm FFN:

$$\begin{aligned} \text{Layer}_{\text{ffn}}(\mathbf{H}_{\text{self}}) &= \mathbf{H}_{\text{ffn}} \\ &= \text{LNorm}(\text{FFN}(\mathbf{H}_{\text{self}}) + \mathbf{H}_{\text{self}}). \end{aligned} \tag{6.4}$$

Ở đây $\text{FFN}(\cdot)$ có thể là bất kỳ mạng nơ-ron truyền thẳng nào có hàm kích hoạt (*activation function*) phi tuyến. Cấu trúc phổ biến nhất của $\text{FFN}(\cdot)$ là một mạng hai tầng gồm hai phép biến đổi tuyến tính và một hàm kích hoạt ReLU ở giữa.

Chú giải của người dịch: Kết nối tắt: "cộng lại cái cũ"

Phép cộng $\mathbf{C} + \mathbf{H}$ nghĩa là cộng hai ma trận cùng kích thước theo từng phần tử (phần tử ở hàng i, cột j của ma trận này cộng với phần tử ở đúng vị trí đó của ma trận kia). Ý tưởng: tầng con chỉ cần học "phần cần sửa thêm" $\mathbf{C}$ chứ không phải tạo lại toàn bộ biểu diễn; thông tin gốc $\mathbf{H}$ luôn được chuyển thẳng lên trên. Giống như khi sửa bài văn: thay vì viết lại từ đầu, ta giữ bản cũ và chỉ ghi thêm phần chỉnh sửa. Nhờ "đường tắt" này, khi huấn luyện, tín hiệu gradient cũng đi xuyên qua nhiều tầng dễ dàng hơn, nên mô hình sâu dễ tối ưu hơn.

Với các mô hình sâu, ta có thể xếp chồng các mạng nơ-ron trên. Gọi $\mathbf{H}^l$ là đầu ra của tầng l. Khi đó ta có thể biểu diễn $\mathbf{H}^l$ như một hàm của $\mathbf{H}^{l-1}$. Ta viết nó dưới dạng hợp của hai tầng con

$$\mathbf{H}^l = \text{Layer}_{\text{ffn}}(\mathbf{H}^l_{\text{self}}), \tag{6.5}$$

$$\mathbf{H}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{H}^{l-1}). \tag{6.6}$$

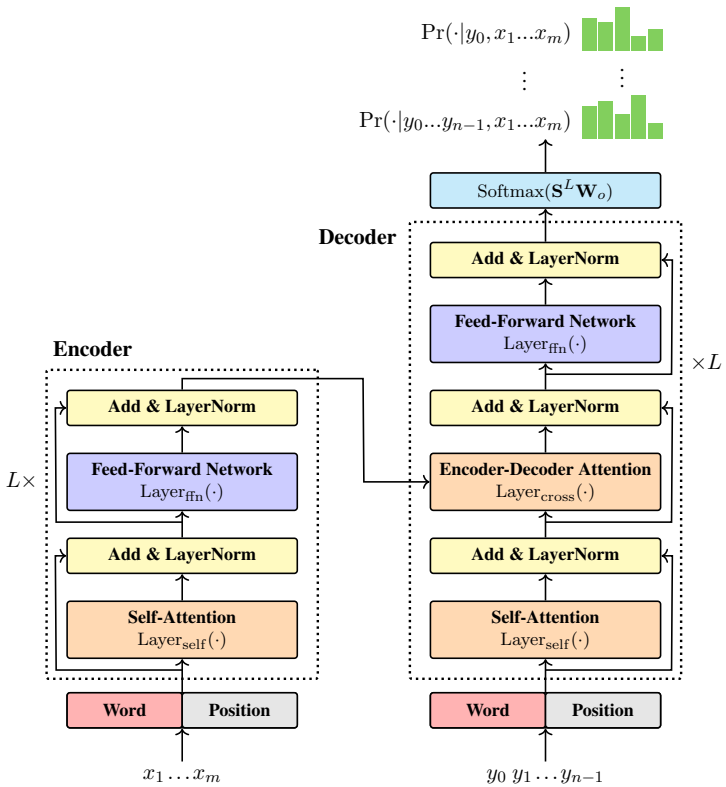
Các nhãn trong Hình 6.1: *Encoder*/*Decoder* = bộ mã hóa/bộ giải mã; *Self-Attention* = tự chú ý; *Feed-Forward Network* = mạng truyền thẳng; *Encoder-Decoder Attention* = chú ý bộ mã hóa–bộ giải mã; *Add & LayerNorm* = cộng (kết nối tắt) rồi chuẩn hóa tầng; *Word*/*Position* = (nhúng) từ/(nhúng) vị trí.

Nếu có L tầng mã hóa thì $\mathbf{H}^L$ là đầu ra cuối cùng của bộ mã hóa. Khi đó, $\mathbf{H}^L$ có thể được xem là một biểu diễn (*representation*) theo ngữ cảnh đã được học của chuỗi đầu vào. $\mathbf{H}^0$ ký hiệu đầu vào ban đầu của bộ mã hóa. Trong các mô hình hồi quy và tích chập, $\mathbf{H}^0$ thường chỉ gồm các vectơ nhúng từ (*word embeddings*) của chuỗi đầu vào. Transformer dùng một cách tiếp cận khác để biểu diễn các từ đầu vào và mã hóa thông tin vị trí một cách tường minh. Ở Mục 6.1.2, ta sẽ bàn về mô hình nhúng được dùng trong Transformer.

Bộ giải mã Transformer có cấu trúc tương tự bộ mã hóa Transformer, gồm L **tầng giải mã** (*decoding layers*) (hay **khối giải mã**, *decoding blocks*) xếp chồng. Gọi $\mathbf{S}^l$ là đầu ra của tầng giải mã thứ l. Ta có thể viết một tầng giải mã bằng các phương trình sau:

$$\mathbf{S}^l = \text{Layer}_{\text{ffn}}(\mathbf{S}^l_{\text{cross}}), \tag{6.7}$$

$$\mathbf{S}^l_{\text{cross}} = \text{Layer}_{\text{cross}}(\mathbf{H}^L, \mathbf{S}^{l-1}_{\text{self}}), \tag{6.8}$$



Hình 6.1: Kiến trúc Transformer [Vaswani và cộng sự, 2017]. Mỗi phía bộ mã hóa và bộ giải mã có L tầng xếp chồng. Một tầng mã hóa gồm một tầng con tự chú ý và một tầng con FFN. Cả hai tầng con này có chung một cấu trúc, gồm một hàm lõi ($\text{Layer}_{\text{self}}(\cdot)$ hoặc $\text{Layer}_{\text{ffn}}(\cdot)$), theo sau là một kết nối tắt và một khối chuẩn hóa tầng. Mỗi tầng giải mã có kiến trúc tương tự các tầng mã hóa, nhưng có thêm một tầng con chú ý bộ mã hóa–bộ giải mã kẹp giữa tầng con tự chú ý và tầng con FFN. Giống như hầu hết các mô hình chuỗi-sang-chuỗi, Transformer nhận $x_1 \cdots x_m$ và $y_0 \cdots y_{i-1}$ để dự đoán y_i. Biểu diễn của một từ đầu vào là tổng của một vectơ nhúng từ và một vectơ nhúng vị trí. Các phân phối $\{\Pr(\cdot|y_0 \cdots y_{i-1}, x_1 \cdots x_m)\}$ được sinh lần lượt bởi một tầng Softmax, tầng này tác động lên một phép biến đổi tuyến tính của đầu ra từ tầng giải mã cuối cùng.

$$\mathbf{S}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{S}^{l-1}). \tag{6.9}$$

Ở đây có ba tầng con của bộ giải mã. Tầng con tự chú ý và tầng con FFN giống với các tầng con dùng trong bộ mã hóa. $\text{Layer}_{\text{cross}}(\cdot)$ ký hiệu một tầng con chú ý chéo (*cross attention*) (hay tầng con bộ mã hóa–bộ giải mã), mô hình hóa phép biến đổi từ phía nguồn sang phía đích. Ở Mục 6.1.6, ta sẽ thấy rằng $\text{Layer}_{\text{cross}}(\cdot)$ có thể được cài đặt bằng chính hàm dùng cho $\text{Layer}_{\text{self}}(\cdot)$.

Bộ giải mã Transformer cho ra một phân phối trên bộ từ vựng (*vocabulary*) V_{y} tại mỗi vị trí phía đích. Điều này được thực hiện bằng một tầng Softmax, chuẩn hóa một phép biến đổi tuyến tính của $\mathbf{S}^L$ thành các phân phối trên các từ phía đích. Để làm vậy, ta ánh xạ $\mathbf{S}^L$ thành một ma trận $\mathbf{O}$ kích thước $n \times |V_{\text{y}}|$ bằng

$$\mathbf{O} \;=\; \mathbf{S}^L \cdot \mathbf{W}_{\text{o}}, \tag{6.10}$$

trong đó $\mathbf{W}_{\text{o}} \in \mathbb{R}^{d \times |V_{\text{y}}|}$ là ma trận tham số của phép biến đổi tuyến tính.

Khi đó, đầu ra của bộ giải mã Transformer có dạng

$$\begin{aligned}\begin{bmatrix} \Pr(\cdot|y_0, \mathbf{x}) \\ \vdots \\ \Pr(\cdot|y_0 \cdots y_{n-1}, \mathbf{x}) \end{bmatrix} &= \text{Softmax}(\mathbf{O}) \\ &= \begin{bmatrix} \text{Softmax}(\mathbf{o}_1) \\ \vdots \\ \text{Softmax}(\mathbf{o}_n) \end{bmatrix}, \end{aligned} \tag{6.11}$$

trong đó $\mathbf{o}_i$ là vectơ hàng thứ i của $\mathbf{O}$, và y_0 là ký hiệu bắt đầu $\langle\text{SOS}\rangle$. Theo mô hình này, xác suất của $\mathbf{y}$ khi biết $\mathbf{x}$ được định nghĩa như thường lệ,

$$\log \Pr(\mathbf{y}|\mathbf{x}) \;=\; \sum_{i=1}^{n} \log \Pr(y_i|y_0 \cdots y_{i-1}, \mathbf{x}). \tag{6.12}$$

Chú giải của người dịch: Từ ma trận $\mathbf{O}$ đến xác suất từng từ

Giả sử bộ từ vựng đích chỉ có 3 từ {*tôi*, *đi*, *học*} và $d = 2$. Nếu hàng thứ nhất

của $\mathbf{S}^L$ là $[1 \;\; 2]$ và $\mathbf{W}_o = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & -1 \end{bmatrix}$ (2×3) thì hàng thứ nhất của $\mathbf{O}$ là

$$\mathbf{o}_1 = [1 \cdot 1 + 2 \cdot 0 \quad 1 \cdot 0 + 2 \cdot 1 \quad 1 \cdot 1 + 2 \cdot (-1)] = [1 \;\; 2 \;\; -1],$$

tức một "điểm số" cho mỗi từ trong bộ từ vựng. Softmax biến các điểm thành xác suất: $\text{Softmax}(\mathbf{o}_1) = \left[\frac{e^1}{S} \;\; \frac{e^2}{S} \;\; \frac{e^{-1}}{S}\right] \approx [0.259 \;\; 0.705 \;\; 0.035]$ với $S = e^1 + e^2 + e^{-1} \approx 10.47$. Các số này không âm và có tổng bằng 1. Softmax được áp dụng *riêng cho từng hàng* (mỗi hàng ứng với một vị trí đích), như Phương trình (6.11) viết. Phương trình (6.12) thì nói: xác suất cả câu đích bằng tích xác suất của từng từ (lấy log thì tích thành tổng).

Phương trình này giống với dạng tổng quát của mô hình hóa ngôn ngữ (*language modeling*): ta dự đoán từ ở vị trí i khi biết mọi từ cho đến vị trí $i-1$. Vì vậy, chuỗi đầu ra đích được dịch đi một bước về phía trước theo thời gian so với chuỗi đầu vào, nghĩa là bộ giải mã nhận $y_0 \cdots y_{n-1}$ làm đầu vào để dự đoán chuỗi đầu ra $y_1 \cdots y_n$.

Kiến trúc Transformer vừa trình bày có một số biến thể đã được dùng thành công trong nhiều lĩnh vực khác nhau của NLP. Ví dụ, ta có thể dùng một bộ mã hóa Transformer để biểu diễn văn bản (gọi là **kiến trúc chỉ có bộ mã hóa** (*encoder-only architecture*)), một bộ giải mã Transformer để sinh văn bản (gọi là **kiến trúc chỉ có bộ giải mã** (*decoder-only architecture*)), hoặc một mô hình Transformer bộ mã hóa–bộ giải mã chuẩn để biến đổi một chuỗi đầu vào thành một chuỗi đầu ra. Trong phần còn lại của chương này, phần lớn nội dung không phụ thuộc vào ứng dụng cụ thể, và sẽ chủ yếu tập trung vào kiến trúc bộ mã hóa–bộ giải mã. Ở Mục 6.5, ta sẽ thấy các ứng dụng của kiến trúc chỉ có bộ mã hóa và chỉ có bộ giải mã.

6.1.2 Mã hóa vị trí

Ở dạng ban đầu, cả FFN lẫn các mô hình chú ý dùng trong Transformer đều bỏ qua một khía cạnh quan trọng của việc mô hình hóa chuỗi, đó là thứ tự của các từ đóng vai trò then chốt trong việc biểu đạt ý nghĩa của chuỗi. Điều này có nghĩa là bộ mã hóa và bộ giải mã không nhạy với thông tin vị trí của các từ đầu vào. Một cách đơn giản để khắc phục vấn đề này là cộng thêm các mã hóa vị trí (*positional encodings*) vào biểu diễn của

mỗi từ trong chuỗi. Chính xác hơn, một từ x_j có thể được biểu diễn bằng một vectơ d chiều

$$\mathbf{e}_j \;=\; \mathbf{x}_j + \mathrm{PE}(j). \tag{6.13}$$

Ở đây $\mathbf{x}_j \in \mathbb{R}^d$ là vectơ nhúng từ, có thể thu được bằng các mô hình nhúng từ tiêu chuẩn như đã mô tả ở Chương 3. $\mathrm{PE}(j) \in \mathbb{R}^d$ là biểu diễn của vị trí j. Transformer nguyên bản dùng một mô hình mã hóa vị trí dạng sin, được viết dưới dạng

$$\mathrm{PE}(j, 2k) = \sin\left(j \cdot \frac{1}{10000^{2k/d}}\right), \tag{6.14}$$

$$\mathrm{PE}(j, 2k+1) = \cos\left(j \cdot \frac{1}{10000^{2k/d}}\right), \tag{6.15}$$

trong đó $\mathrm{PE}(j, k)$ là phần tử thứ k của $\mathrm{PE}(j)$. Ý tưởng của mã hóa vị trí là phân biệt các vị trí khác nhau bằng các hàm liên tục. Ở đây ta dùng các hàm sin và cos với những tần số khác nhau. Bạn đọc quan tâm có thể xem Chương 4 để thấy rằng phương pháp này có thể được hiểu như một hệ đếm theo vị trí. Vì việc mã hóa dựa trên từng vị trí riêng lẻ, nó còn được gọi là **mã hóa vị trí tuyệt đối** (*absolute positional encoding*). Ở Mục 6.3.1, ta sẽ thấy một cải tiến cho phương pháp này.

Chú giải của người dịch: Tính tay mã hóa vị trí sin/cos

Lấy $d = 4$, khi đó $k \in \{0, 1\}$ và có hai "tần số": $\omega_0 = 1/10000^{0/4} = 1$ và $\omega_1 = 1/10000^{2/4} = 1/100 = 0.01$. Vectơ vị trí 4 chiều là $\mathrm{PE}(j) = [\sin(j\omega_0) \;\; \cos(j\omega_0) \;\; \sin(j\omega_1) \;\; \cos(j\omega_1)]$ (góc tính bằng radian):

$$\begin{aligned}
&\mathrm{PE}(0) = [0 \;\; 1 \;\; 0 \;\; 1],\\
&\mathrm{PE}(1) \approx [0.841 \;\; 0.540 \;\; 0.010 \;\; 1.000],\\
&\mathrm{PE}(2) \approx [0.909 \;\; -0.416 \;\; 0.020 \;\; 1.000].
\end{aligned}$$

Hai chiều đầu "quay nhanh" (như kim giây), hai chiều sau "quay chậm" (như kim giờ). Giống như đọc giờ trên đồng hồ, tổ hợp của các kim quay với tốc độ khác nhau cho phép phân biệt rất nhiều vị trí. Mọi giá trị đều nằm trong $[-1, 1]$, nên vectơ vị trí không "lấn át" vectơ nhúng từ khi cộng vào theo Phương trình (6.13).

Khi đã có các vectơ nhúng này, chuỗi $\mathbf{e}_1 \cdots \mathbf{e}_m$ được dùng làm đầu

vào cho bộ mã hóa Transformer, tức là

$$\mathbf{H}^0 = \begin{bmatrix} \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_m \end{bmatrix}. \tag{6.16}$$

Tương tự, ta có thể định nghĩa đầu vào ở phía bộ giải mã.



(a) RNN (b) CNN ($r = 3$) (c) Self-attention

Hình 6.2: Các luồng thông tin trong mô hình hồi quy (a, RNN), tích chập (b, CNN với độ rộng cửa sổ $r = 3$) và tự chú ý (c), được biểu diễn bằng các mũi tên nối giữa các vị trí.

6.1.3 Tự chú ý đa đầu

Việc dùng tự chú ý có lẽ là một trong những tiến bộ quan trọng nhất của các mô hình chuỗi-sang-chuỗi. Nó cố gắng học và tận dụng các tương tác trực tiếp giữa mọi cặp đầu vào. Từ góc nhìn học biểu diễn, các mô hình tự chú ý giả định rằng biểu diễn học được tại vị trí i (ký hiệu là $\mathbf{c}_i$) là một tổng có trọng số của các đầu vào trên toàn chuỗi. Do đó đầu ra $\mathbf{c}_i$ được cho bởi

$$\mathbf{c}_i = \sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j, \tag{6.17}$$

trong đó $\alpha_{i,j}$ là trọng số chú ý (*attention weight*) gán cho $\mathbf{h}_j$ khi tính biểu diễn tại vị trí i. Như vậy, ta có thể xem $\mathbf{c}_i$ là một biểu diễn của ngữ cảnh toàn cục tại vị trí i. $\alpha_{i,j}$ có thể được định nghĩa theo nhiều cách khác nhau tùy vào mô hình chú ý cụ thể. Ở đây ta dùng hàm chú ý tích vô hướng có co giãn (*scaled dot-product attention*) để tính $\alpha_{i,j}$ như sau

$$\begin{aligned} \alpha_{i,j} &= \text{Softmax}\left(\mathbf{h}_i \mathbf{h}_j^\top / \beta\right) \\ &= \frac{\exp(\mathbf{h}_i \mathbf{h}_j^\top / \beta)}{\sum_{k=1}^{m} \exp(\mathbf{h}_i \mathbf{h}_k^\top / \beta)}, \end{aligned} \tag{6.18}$$

trong đó hệ số co giãn β thường được đặt bằng $\sqrt{d}$.

Chú giải của người dịch: Tự chú ý bằng số: một câu 3 từ

$\mathbf{h}_i\mathbf{h}_j^\top$ là *tích vô hướng* của hai vectơ hàng (nhân từng cặp phần tử rồi cộng lại), giống $\vec{u}\cdot\vec{v} = u_1v_1 + u_2v_2$ ở lớp 10: hai vectơ càng "cùng hướng" thì tích càng lớn, tức hai từ càng "liên quan". Lấy $d = 2$ (nên $\beta = \sqrt{2} \approx 1.41$) và ba từ $\mathbf{h}_1 = [1\ \ 0]$, $\mathbf{h}_2 = [0\ \ 1]$, $\mathbf{h}_3 = [1\ \ 1]$. Tính cho vị trí $i = 1$: các tích vô hướng là $\mathbf{h}_1\mathbf{h}_1^\top = 1$, $\mathbf{h}_1\mathbf{h}_2^\top = 0$, $\mathbf{h}_1\mathbf{h}_3^\top = 1$; chia cho 1.41 được $[0.71\ \ 0\ \ 0.71]$. Softmax: $e^{0.71} \approx 2.03$, $e^0 = 1$, tổng ≈ 5.06, nên $[\alpha_{1,1}\ \alpha_{1,2}\ \alpha_{1,3}] \approx [0.40\ \ 0.20\ \ 0.40]$. Theo Phương trình (6.17): $\mathbf{c}_1 \approx 0.40[1\ \ 0] + 0.20[0\ \ 1] + 0.40[1\ \ 1] = [0.80\ \ 0.60]$. Biểu diễn mới của từ 1 là một "hỗn hợp" của cả ba từ, trong đó các từ giống nó nhiều hơn (từ 1 và từ 3) đóng góp nhiều hơn. Chia cho β để các tích vô hướng không quá lớn khi d lớn (nếu không, Softmax sẽ dồn gần hết trọng số vào một từ).

So với các mô hình hồi quy và tích chập thông thường, một ưu điểm của mô hình tự chú ý là chúng rút ngắn "khoảng cách" tính toán giữa hai đầu vào. Hình 6.2 minh họa luồng thông tin trong các mô hình này. Ta thấy rằng, với đầu vào tại vị trí i, mô hình tự chú ý có thể truy cập trực tiếp bất kỳ đầu vào nào khác. Ngược lại, mô hình hồi quy và tích chập có thể cần hai bước nhảy trở lên mới "nhìn thấy" được toàn bộ chuỗi.

Ta có thể nhìn tự chú ý từ một góc độ tổng quát hơn bằng cách dùng mô hình chú ý QKV. Giả sử ta có một chuỗi gồm κ truy vấn (*queries*) $\mathbf{Q} = \begin{bmatrix}\mathbf{q}_1\\ \vdots \\ \mathbf{q}_\kappa\end{bmatrix}$, và một chuỗi gồm ψ cặp khóa (*key*)–giá trị (*value*) ($\mathbf{K} = \begin{bmatrix}\mathbf{k}_1\\ \vdots \\ \mathbf{k}_\psi\end{bmatrix}$, $\mathbf{V} = \begin{bmatrix}\mathbf{v}_1\\ \vdots \\ \mathbf{v}_\psi\end{bmatrix}$). Đầu ra của mô hình là một chuỗi vectơ, mỗi vectơ ứng với một truy vấn. Dạng của chú ý QKV được cho bởi

$$\text{Att}_{\text{qkv}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}. \tag{6.19}$$

Ta có thể viết đầu ra của mô hình chú ý QKV dưới dạng một chuỗi

các vectơ hàng

$$\begin{aligned}\mathbf{C} &= \begin{bmatrix}\mathbf{c}_1\\ \vdots \\ \mathbf{c}_\kappa\end{bmatrix}\\ &= \mathrm{Att}_{\mathrm{qkv}}(\mathbf{Q},\mathbf{K},\mathbf{V}).\end{aligned} \tag{6.20}$$

Để áp dụng phương trình này cho tự chú ý, ta chỉ cần đặt

$$\mathbf{H}^q = \mathbf{H}\mathbf{W}^q, \tag{6.21}$$
$$\mathbf{H}^k = \mathbf{H}\mathbf{W}^k, \tag{6.22}$$
$$\mathbf{H}^v = \mathbf{H}\mathbf{W}^v, \tag{6.23}$$

trong đó $\mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v \in \mathbb{R}^{d\times d}$ biểu diễn các phép biến đổi tuyến tính của $\mathbf{H}$.

Xét Phương trình (6.1), ta thu được

$$\begin{aligned}\mathbf{C} &= \mathrm{Att}_{\mathrm{self}}(\mathbf{H})\\ &= \mathrm{Att}_{\mathrm{qkv}}(\mathbf{H}^q,\mathbf{H}^k,\mathbf{H}^v)\\ &= \mathrm{Softmax}\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\right)\mathbf{H}^v.\end{aligned} \tag{6.24}$$

Ở đây $\mathrm{Softmax}(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}})$ là một ma trận $m \times m$, trong đó mỗi hàng biểu diễn một phân phối trên tất cả các vị trí đầu vào, tức là

$$\text{hàng } i \;=\; \begin{bmatrix}\alpha_{i,1} & \cdots & \alpha_{i,m}\end{bmatrix}. \tag{6.25}$$

Chú giải của người dịch: Đọc Phương trình (6.24) theo kích thước ma trận

Nhân ma trận. Nếu $\mathbf{A}$ cỡ $m \times n$ và $\mathbf{B}$ cỡ $n \times p$ thì $\mathbf{AB}$ cỡ $m \times p$, và phần tử hàng i, cột j của nó là tích vô hướng của *hàng* i của $\mathbf{A}$ với *cột* j của $\mathbf{B}$. Ví dụ $\begin{bmatrix}1 & 2\\ 0 & 1\end{bmatrix}\begin{bmatrix}3\\ 1\end{bmatrix} = \begin{bmatrix}1\cdot 3 + 2\cdot 1\\ 0\cdot 3 + 1\cdot 1\end{bmatrix} = \begin{bmatrix}5\\ 1\end{bmatrix}$. **Chuyển vị** $\mathbf{A}^\top$ là đổi hàng thành cột.

Với m từ và d chiều: $\mathbf{H}^q, \mathbf{H}^k, \mathbf{H}^v$ đều cỡ $m\times d$ (mỗi hàng là truy vấn/khóa/giá trị của một từ). $[\mathbf{H}^k]^\top$ cỡ $d \times m$, nên $\mathbf{H}^q[\mathbf{H}^k]^\top$ cỡ $m \times m$: ô (i, j) là tích vô

hướng giữa truy vấn của từ i và khóa của từ j — tức "từ i quan tâm đến từ j bao nhiêu". Softmax được lấy *theo từng hàng*, nên mỗi hàng thành một phân phối xác suất (Phương trình (6.25)). Cuối cùng nhân với $\mathbf{H}^v$ ($m \times m$ nhân $m \times d$) ra $\mathbf{C}$ cỡ $m \times d$: hàng i của $\mathbf{C}$ chính là $\sum_j \alpha_{i,j}\mathbf{v}_j$, đúng như Phương trình (6.17) nhưng tính cho mọi vị trí cùng một lúc. Hình ảnh đời thường: truy vấn là "câu hỏi" của một từ, khóa là "nhãn" trên mỗi ngăn tủ, giá trị là "đồ vật" trong ngăn; từ i so câu hỏi với từng nhãn rồi lấy về một hỗn hợp đồ vật theo mức độ khớp.

Ta có thể cải tiến mô hình tự chú ý trên bằng một kỹ thuật gọi là **chú ý đa đầu** (*multi-head attention*). Phương pháp này có thể được lý giải từ góc độ học từ nhiều không gian con (*subspaces*) đặc trưng có số chiều thấp hơn. Nó chiếu đầu vào vào nhiều không gian con và học các biểu diễn riêng trong từng không gian. Cụ thể, ta chiếu toàn bộ không gian đầu vào vào τ không gian con (gọi là các đầu (*heads*)). Ví dụ, ta biến đổi $\mathbf{H} \in \mathbb{R}^{m\times d}$ thành τ ma trận cỡ $m \times \frac{d}{\tau}$, ký hiệu là $\{\mathbf{H}_1^{\text{head}}, \ldots, \mathbf{H}_\tau^{\text{head}}\}$. Mô hình chú ý sau đó được áp dụng τ lần, mỗi lần cho một đầu. Cuối cùng, các đầu ra của những lần chạy này được ghép nối lại và biến đổi bằng một phép chiếu tuyến tính. Quy trình này có thể được biểu diễn bằng

$$\mathbf{C} \;=\; \text{Merge}(\mathbf{C}_1^{\text{head}}, \cdots, \mathbf{C}_\tau^{\text{head}})\mathbf{W}_c. \tag{6.26}$$

Với mỗi đầu h,

$$\mathbf{C}_h^{\text{head}} = \text{Softmax}\left(\frac{\mathbf{H}_h^q[\mathbf{H}_h^k]^\top}{\sqrt{d}}\right)\mathbf{H}_h^v, \tag{6.27}$$

$$\mathbf{H}_h^q = \mathbf{H}\mathbf{W}_h^q, \tag{6.28}$$

$$\mathbf{H}_h^k = \mathbf{H}\mathbf{W}_h^k, \tag{6.29}$$

$$\mathbf{H}_h^v = \mathbf{H}\mathbf{W}_h^v. \tag{6.30}$$

Ở đây $\text{Merge}(\cdot)$ là hàm ghép nối, và $\text{Att}_{\text{qkv}}(\cdot)$ là hàm chú ý đã mô tả ở Phương trình (6.20). $\mathbf{W}_h^q, \mathbf{W}_h^k, \mathbf{W}_h^v \in \mathbb{R}^{d\times\frac{d}{\tau}}$ là các tham số của các phép chiếu từ không gian d chiều sang không gian $\frac{d}{\tau}$ chiều cho truy vấn, khóa và giá trị. Do đó, $\mathbf{H}_h^q$, $\mathbf{H}_h^k$, $\mathbf{H}_h^v$ và $\mathbf{C}_h^{\text{head}}$ đều là các ma trận $m \times \frac{d}{\tau}$. $\text{Merge}(\mathbf{C}_1^{\text{head}}, \cdots, \mathbf{C}_\tau^{\text{head}})$ tạo ra một ma trận $m \times d$. Ma trận này sau đó được biến đổi bằng một ánh xạ tuyến tính $\mathbf{W}_c \in \mathbb{R}^{d\times d}$, cho ra kết quả cuối cùng $\mathbf{C} \in \mathbb{R}^{m\times d}$.

Dù ký hiệu có vẻ rườm rà, việc cài đặt các mô hình đa đầu lại rất đơn giản với các bộ công cụ học sâu hiện đại. Một cách làm phổ biến trong

các hệ thống dựa trên Transformer là lưu đầu vào của tất cả các đầu trong những cấu trúc dữ liệu gọi là tensor (*tensors*), nhờ đó tận dụng được tài nguyên tính toán song song để tăng hiệu quả. Một thảo luận tổng quát hơn về chú ý QKV và chú ý đa đầu có thể xem ở Chương 5.

Chú giải của người dịch: Đa đầu: chia nhỏ rồi ghép lại

Ví dụ $d = 512$, $\tau = 8$: mỗi đầu làm việc trong không gian $512/8 = 64$ chiều. Đầu thứ h tạo ra $\mathbf{C}_h^{\text{head}}$ cỡ $m \times 64$; Merge đặt 8 ma trận này cạnh nhau theo chiều ngang thành $m \times (8 \cdot 64) = m \times 512$, rồi nhân với $\mathbf{W}_c$ (512×512). Ý tưởng: mỗi đầu có bộ $\mathbf{W}_h^q, \mathbf{W}_h^k, \mathbf{W}_h^v$ riêng nên có thể "chú ý" theo một kiểu khác nhau (đầu này để ý quan hệ ngữ pháp, đầu kia để ý từ đứng gần…), giống một nhóm 8 người cùng đọc một câu, mỗi người tìm một loại manh mối, rồi gộp ý kiến lại. Vì mỗi đầu nhỏ hơn, tổng khối lượng tính toán xấp xỉ một đầu đầy đủ d chiều.

6.1.4 Chuẩn hóa tầng

Chuẩn hóa tầng là một cách đơn giản và hiệu quả để làm cho việc huấn luyện mạng nơ-ron ổn định hơn, bằng cách chuẩn hóa các giá trị kích hoạt của các tầng ẩn (*hidden layers*) theo từng tầng. Như Ba và cộng sự [2016] đề xuất, với đầu ra của một tầng $\mathbf{h} \in \mathbb{R}^d$, phương pháp chuẩn hóa tầng tính một đầu ra đã chuẩn hóa $\text{LNorm}(\mathbf{h}) \in \mathbb{R}^d$ bằng

$$\text{LNorm}(\mathbf{h}) \;=\; \boldsymbol{\alpha} \odot \frac{\mathbf{h} - \mu}{\sigma + \epsilon} + \boldsymbol{\beta}. \tag{6.31}$$

Ở đây $\mu \in \mathbb{R}$ và $\sigma \in \mathbb{R}$ lần lượt là trung bình và độ lệch chuẩn (là các số vô hướng) của các giá trị kích hoạt. Gọi h_k là chiều thứ k của $\mathbf{h}$. Các số vô hướng μ và σ được định nghĩa là

$$\mu = \frac{1}{d} \sum_{k=1}^{d} h_k, \tag{6.32}$$

$$\sigma = \sqrt{\frac{1}{d} \sum_{k=1}^{d} (h_k - \mu)^2}. \tag{6.33}$$

Ở đây $\boldsymbol{\alpha} \in \mathbb{R}^d$ và $\boldsymbol{\beta} \in \mathbb{R}^d$ là các tham số co giãn và dịch chuyển. Chúng có thể được coi là tham số của chuẩn hóa tầng, với giá trị được học cùng

với các tham số khác của mô hình Transformer. Việc cộng ϵ vào σ nhằm đảm bảo ổn định số học. Nói chung, ϵ được chọn là một số rất nhỏ.

Ta minh họa phương pháp chuẩn hóa tầng cho các trạng thái ẩn của một bộ mã hóa bằng ví dụ sau (giả sử $m = 4$, $d = 3$, $\boldsymbol{\alpha} = \mathbf{1}$, $\boldsymbol{\beta} = \mathbf{0}$ và $\epsilon = 0.1$).

$$\begin{array}{c} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \\ \mathbf{h}_4 \end{array} \begin{bmatrix} 1 & 1 & 2 \\ 0.9 & 0.9 & 0 \\ 0.7 & 0.8 & 0 \\ 3 & 1 & 7 \end{bmatrix} \begin{array}{l} \mu = 1.3,\ \sigma = 0.5 \\ \mu = 0.6,\ \sigma = 0.4 \\ \mu = 0.5,\ \sigma = 0.4 \\ \mu = 3.7,\ \sigma = 2.5 \end{array} \Longrightarrow \begin{bmatrix} \frac{1-1.3}{0.5+0.1} & \frac{1-1.3}{0.5+0.1} & \frac{2-1.3}{0.5+0.1} \\ \frac{0.9-0.6}{0.4+0.1} & \frac{0.9-0.6}{0.4+0.1} & \frac{0-0.6}{0.4+0.1} \\ \frac{0.7-0.5}{0.4+0.1} & \frac{0.8-0.5}{0.4+0.1} & \frac{0-0.5}{0.4+0.1} \\ \frac{3-3.7}{2.5+0.1} & \frac{1-3.7}{2.5+0.1} & \frac{7-3.7}{2.5+0.1} \end{bmatrix}$$

Chú giải của người dịch: Chuẩn hóa tầng giống "quy về điểm chuẩn"

Đây chính là phép chuẩn hóa quen thuộc trong thống kê mô tả: trừ đi trung bình rồi chia cho độ lệch chuẩn, áp dụng cho *từng hàng* (từng từ) một cách riêng rẽ. Tính hàng $\mathbf{h}_1 = [1 \quad 1 \quad 2]$: $\mu = 4/3 \approx 1.33$; $\sigma = \sqrt{\frac{1}{3}\left((-0.33)^2 + (-0.33)^2 + 0.67^2\right)} \approx \sqrt{0.22} \approx 0.47$ (sách làm tròn thành 0.5). Kết quả hàng 1 $\approx [-0.5 \quad -0.5 \quad 1.17]$. Sau chuẩn hóa, các số của mỗi hàng có trung bình xấp xỉ 0 và độ phân tán xấp xỉ 1, dù ban đầu hàng $\mathbf{h}_4 = [3 \quad 1 \quad 7]$ có các số lớn hơn hẳn các hàng khác — nhờ vậy giá trị trong mạng không "phình to" hay "co lại" quá mức qua nhiều tầng. Ký hiệu $\odot$ là nhân theo từng phần tử; $\boldsymbol{\alpha}$ và $\boldsymbol{\beta}$ cho phép mô hình tự co giãn, dịch chuyển lại nếu cần.

Như đã bàn ở Mục 6.1.1, khối chuẩn hóa tầng trong mỗi tầng con được dùng để chuẩn hóa đầu ra của một khối residual. Ở đây ta mô tả một cách viết tổng quát hơn cho cấu trúc này. Giả sử $F(\cdot)$ là một tầng con của mạng nơ-ron. Khi đó, cấu trúc post-norm của $F(\cdot)$ được cho bởi:

$$\mathbf{H}_{\text{out}} = \text{LNorm}(F(\mathbf{H}_{\text{in}}) + \mathbf{H}_{\text{in}}), \tag{6.34}$$

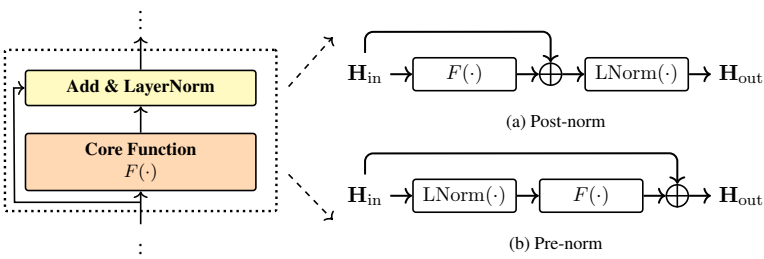
trong đó $\mathbf{H}_{\text{in}}$ và $\mathbf{H}_{\text{out}}$ là đầu vào và đầu ra của tầng con này. Rõ ràng, Phương trình (6.3) là một trường hợp cụ thể của phương trình này.

Một cách khác để đưa chuẩn hóa tầng và kết nối tắt vào mô hình là thực hiện hàm $\text{LNorm}(\cdot)$ ngay trước hàm $F(\cdot)$, và thiết lập một ánh xạ đồng nhất (*identity mapping*) từ đầu vào tới đầu ra của toàn bộ tầng con. Cấu trúc này, gọi là cấu trúc **pre-norm** (*chuẩn hóa trước*), có thể được

viết dưới dạng

$$\mathbf{H}_{\text{out}} \;=\; F(\text{LNorm}(\mathbf{H}_{\text{in}})) + \mathbf{H}_{\text{in}}. \tag{6.35}$$

Cả mô hình Transformer post-norm lẫn pre-norm đều được dùng rộng rãi trong các hệ thống NLP. Xem Hình 6.3 để so sánh hai cấu trúc này. Nói chung, kết nối tắt được xem là một cách hiệu quả để việc huấn luyện các mạng nơ-ron nhiều tầng trở nên dễ dàng hơn. Theo nghĩa này, Transformer pre-norm có vẻ hứa hẹn vì nó tuân theo quy ước rằng một kết nối tắt được tạo ra để đi vòng qua toàn bộ mạng, và ánh xạ đồng nhất từ đầu vào tới đầu ra giúp tối ưu các mô hình sâu dễ hơn. Tuy nhiên, khi xét tới khả năng biểu đạt của mô hình, việc dùng Transformer post-norm có thể có lợi thế về mặt mô hình hóa, vì nó không phụ thuộc quá nhiều vào kết nối tắt và buộc việc học biểu diễn phải được mô hình hóa tinh vi hơn. Ở Mục 6.3.2, ta sẽ bàn thêm về vấn đề này.



Hình 6.3: Cấu trúc post-norm (a) và pre-norm (b). $F(\cdot)$ = hàm lõi (*Core Function*), LNorm$(\cdot)$ = chuẩn hóa tầng, và $\oplus$ = kết nối tắt.

Chú giải của người dịch: Post-norm và pre-norm khác nhau ở đâu?

Hãy so hai dòng: post-norm LNorm$(F(\mathbf{x})+\mathbf{x})$ và pre-norm $F(\text{LNorm}(\mathbf{x}))+\mathbf{x}$. Ở pre-norm, đầu vào $\mathbf{x}$ đi "thẳng" lên đầu ra mà không bị biến đổi gì (ánh xạ đồng nhất); xếp chồng nhiều tầng thì có một "đường cao tốc" từ đáy lên đỉnh mạng, nên mạng rất sâu vẫn dễ huấn luyện. Ở post-norm, mọi thứ (kể cả phần $\mathbf{x}$) đều phải đi qua LNorm ở mỗi tầng, nên không còn đường đi thẳng như vậy — khó huấn luyện hơn khi mạng sâu, nhưng có thể biểu đạt tốt hơn.

6.1.5 Mạng nơ-ron truyền thẳng

Việc dùng FFN trong Transformer một phần xuất phát từ thực tế là các đầu ra phức tạp có thể được tạo ra bằng cách biến đổi đầu vào qua các phép phi tuyến. Dù bản thân mô hình tự chú ý đã có một chút tính phi tuyến (trong Softmax$(\cdot)$), cách phổ biến hơn để đưa tính phi tuyến vào là thêm các tầng có hàm kích hoạt phi tuyến và các phép biến đổi tuyến tính. Với đầu vào $\mathbf{H}_{\text{in}} \in \mathbb{R}^{m\times d}$ và đầu ra $\mathbf{H}_{\text{out}} \in \mathbb{R}^{m\times d}$, hàm $\mathbf{H}_{\text{out}} = \text{FFN}(\mathbf{H}_{\text{in}})$ trong Transformer có dạng

$$\mathbf{H}_{\text{out}} = \mathbf{H}_{\text{hidden}}\mathbf{W}_f + \mathbf{b}_f, \tag{6.36}$$

$$\mathbf{H}_{\text{hidden}} = \text{ReLU}(\mathbf{H}_{\text{in}}\mathbf{W}_h + \mathbf{b}_h), \tag{6.37}$$

trong đó $\mathbf{H}_{\text{hidden}} \in \mathbb{R}^{m\times d_{\text{ffn}}}$ biểu diễn các trạng thái ẩn (*hidden states*), còn $\mathbf{W}_h \in \mathbb{R}^{d\times d_{\text{ffn}}}$, $\mathbf{b}_h \in \mathbb{R}^{d_{\text{ffn}}}$, $\mathbf{W}_f \in \mathbb{R}^{d_{\text{ffn}}\times d}$ và $\mathbf{b}_f \in \mathbb{R}^d$ là các tham số. Đây là một FFN hai tầng, trong đó tầng thứ nhất (hay tầng ẩn) đưa vào tính phi tuyến thông qua ReLU$(\cdot)$[2] và tầng thứ hai chỉ là một phép biến đổi tuyến tính. Trong Transformer, người ta thường dùng kích thước tầng ẩn lớn hơn. Ví dụ, một lựa chọn phổ biến là $d_{\text{ffn}} = 4d$, tức là kích thước của mỗi biểu diễn ẩn gấp 4 lần kích thước của đầu vào.

Chú giải của người dịch: FFN: "mở rộng rồi thu hẹp"

Với $d = 512$, $d_{\text{ffn}} = 2048$: mỗi hàng (mỗi từ) 1×512 được nhân với $\mathbf{W}_h$ (512×2048) để "mở rộng" thành 2048 số, cho qua ReLU (giữ số dương, biến số âm thành 0, ví dụ ReLU$([-2\ \ 3]) = [0\ \ 3]$), rồi nhân với $\mathbf{W}_f$ (2048×512) để "thu hẹp" về 512 số. Khác với tự chú ý, FFN xử lý từng từ *riêng rẽ* (các hàng không trộn lẫn với nhau), nhưng cùng một bộ tham số dùng chung cho mọi vị trí. Số tham số của FFN: $512\cdot2048+2048+2048\cdot512+512 \approx 2{,}1$ triệu, nhiều gấp đôi khoảng $4\cdot512^2 \approx 1{,}05$ triệu của tầng chú ý.

Lưu ý rằng việc dùng một tầng con FFN rộng đã chứng tỏ giá trị thực tiễn lớn trong nhiều hệ thống tiên tiến nhất (*state-of-the-art*). Tuy nhiên, hệ quả là số lượng tham số của mô hình bị chi phối bởi FFN. Bảng 6.1 cho thấy số tham số và độ phức tạp thời gian của các mô-đun khác nhau trong một hệ thống Transformer chuẩn. Ta thấy FFN chiếm phần lớn kích thước mô hình khi d_{ffn} lớn, dù chúng không phải là thành phần tốn thời

[2] ReLU$(x) = \max\{0, x\}$.

gian nhất. Do đó, với các mô hình Transformer rất lớn, ta mong muốn giải quyết vấn đề này để xây dựng những hệ thống hiệu quả hơn.

6.1.6 Chú ý phía bộ giải mã

Một tầng giải mã gồm hai tầng con chú ý: tầng thứ nhất là tầng con tự chú ý, tầng thứ hai là tầng con chú ý chéo. Các tầng con này dựa trên cấu trúc post-norm hoặc pre-norm, nhưng khác nhau ở cách định nghĩa hàm chú ý. Chẳng hạn, xét cấu trúc post-norm mô tả ở Phương trình (6.34). Ta có thể định nghĩa tầng con chú ý chéo và tầng con tự chú ý của một tầng giải mã là

$$\begin{aligned}\mathbf{S}_{\text{cross}} &= \text{Layer}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) \\ &= \text{LNorm}(\text{Att}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) + \mathbf{S}_{\text{self}}), \qquad (6.38)\\ \mathbf{S}_{\text{self}} &= \text{Layer}_{\text{self}}(\mathbf{S}) \\ &= \text{LNorm}(\text{Att}_{\text{self}}(\mathbf{S}) + \mathbf{S}), \qquad (6.39)\end{aligned}$$

trong đó $\mathbf{S} \in \mathbb{R}^{n\times d}$ là đầu vào của tầng con tự chú ý, $\mathbf{S}_{\text{cross}} \in \mathbb{R}^{n\times d}$ và $\mathbf{S}_{\text{self}} \in \mathbb{R}^{n\times d}$ là đầu ra của các tầng con, còn $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{m\times d}$ là đầu ra của bộ mã hóa[3].

Bảng 6.1: Số tham số và độ phức tạp thời gian của các mô-đun Transformer khác nhau trong các thiết lập khác nhau. m = độ dài chuỗi nguồn, n = độ dài chuỗi đích, d = số chiều mặc định của một tầng ẩn, d_{ffn} = số chiều của tầng ẩn FFN, τ = số đầu trong các mô hình chú ý, và L = số tầng mã hóa hoặc giải mã. Cột $\times$ cho biết số lần một mô hình con được áp dụng ở phía bộ mã hóa hoặc bộ giải mã. Độ phức tạp thời gian được ước lượng bằng cách đếm số phép nhân dấu phẩy động.

	Mô hình con	Số tham số	Độ phức tạp thời gian	$\times$
Bộ mã hóa	Tự chú ý đa đầu	$4d^2$	$O(m^2 \cdot d)$	L
	Mạng truyền thẳng	$2d \cdot d_{\text{ffn}} + d + d_{\text{ffn}}$	$O(m \cdot d \cdot d_{\text{ffn}})$	L
	Chuẩn hóa tầng	$2d$	$O(m \cdot d)$	$2L$
Bộ giải mã	Tự chú ý đa đầu	$4d^2$	$O(n^2 \cdot d)$	L
	Chú ý chéo đa đầu	$4d^2$	$O(m \cdot n \cdot d)$	L
	Mạng truyền thẳng	$2d \cdot d_{\text{ffn}} + d + d_{\text{ffn}}$	$O(n \cdot d \cdot d_{\text{ffn}})$	L
	Chuẩn hóa tầng	$2d$	$O(n \cdot d)$	$3L$

[3] Với một bộ mã hóa có L tầng mã hóa, $\mathbf{H}_{\text{enc}} = \mathbf{H}^L$.

Chú giải của người dịch: Đọc Bảng 6.1

Ký hiệu $O(\cdot)$ ("bậc của") cho biết khối lượng tính toán tăng theo các kích thước như thế nào, bỏ qua hằng số. Ví dụ, tự chú ý tốn $O(m^2 \cdot d)$ vì ma trận $\mathbf{H}^q[\mathbf{H}^k]^\top$ có $m \times m$ ô, mỗi ô là một tích vô hướng d chiều (cần d phép nhân): câu dài gấp đôi thì phần này tốn gấp bốn. Số tham số $4d^2$ của tự chú ý đến từ bốn ma trận $d \times d$: $\mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v$ và $\mathbf{W}_c$. Chuẩn hóa tầng có $2d$ tham số là $\boldsymbol{\alpha}$ và $\boldsymbol{\beta}$; mỗi tầng mã hóa có 2 khối chuẩn hóa, mỗi tầng giải mã có 3.

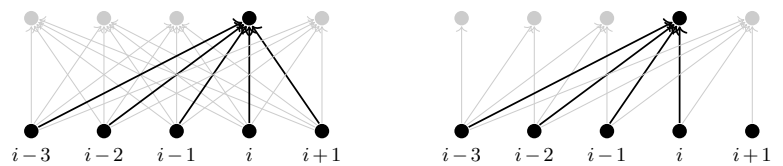
Giống như các mô hình chú ý thông thường, chú ý chéo chủ yếu được dùng để mô hình hóa sự tương ứng giữa chuỗi phía nguồn và chuỗi phía đích. Hàm $\text{Att}_{\text{cross}}(\cdot)$ dựa trên mô hình chú ý QKV, mô hình này sinh đầu ra bằng cách truy vấn một tập các cặp khóa–giá trị. Cụ thể hơn, ta định nghĩa truy vấn, khóa và giá trị là các ánh xạ tuyến tính của $\mathbf{S}_{\text{self}}$ và $\mathbf{H}_{\text{enc}}$ như sau

$$\mathbf{S}^q_{\text{self}} = \mathbf{S}_{\text{self}}\mathbf{W}^q_{\text{cross}}, \tag{6.40}$$

$$\mathbf{H}^k_{\text{enc}} = \mathbf{H}_{\text{enc}}\mathbf{W}^k_{\text{enc}}, \tag{6.41}$$

$$\mathbf{H}^v_{\text{enc}} = \mathbf{H}_{\text{enc}}\mathbf{W}^v_{\text{enc}}, \tag{6.42}$$

trong đó $\mathbf{W}^q_{\text{cross}}, \mathbf{W}^k_{\text{enc}}, \mathbf{W}^v_{\text{enc}} \in \mathbb{R}^{d\times d}$ là tham số của các ánh xạ. Nói cách khác, truy vấn được định nghĩa dựa trên $\mathbf{S}_{\text{self}}$, còn khóa và giá trị được định nghĩa dựa trên $\mathbf{H}_{\text{enc}}$.



(a) Encoder-side Self-attention (b) Decoder-side Self-attention

Hình 6.4: Tự chú ý ở phía bộ mã hóa (a) và phía bộ giải mã (b). Mỗi đường nối một đầu vào với một đầu ra của mô hình tự chú ý, cho biết trạng thái đầu ra phụ thuộc vào trạng thái đầu vào đó. Với tự chú ý của bộ mã hóa, đầu ra tại bất kỳ vị trí nào cũng được tính khi có quyền truy cập toàn bộ chuỗi. Ngược lại, với tự chú ý của bộ giải mã, đầu ra tại vị trí i được tính khi chỉ nhìn thấy các đầu vào ở các vị trí từ i trở về trước.

Khi đó, $\text{Att}_{\text{cross}}(\cdot)$ được định nghĩa là

$$\begin{aligned}\text{Att}_{\text{cross}}(\mathbf{H}_{\text{enc}}, \mathbf{S}_{\text{self}}) &= \text{Att}_{\text{qkv}}(\mathbf{S}^q_{\text{self}}, \mathbf{H}^k_{\text{enc}}, \mathbf{H}^v_{\text{enc}}) \\ &= \text{Softmax}\left(\frac{\mathbf{S}^q_{\text{self}}[\mathbf{H}^k_{\text{enc}}]^\top}{\sqrt{d}}\right)\mathbf{H}^v_{\text{enc}}. \end{aligned} \tag{6.43}$$

Chú giải của người dịch: Chú ý chéo: ma trận $n \times m$

Ở đây truy vấn đến từ phía đích (n hàng) còn khóa và giá trị đến từ phía nguồn (m hàng), nên $\mathbf{S}^q_{\text{self}}[\mathbf{H}^k_{\text{enc}}]^\top$ là ma trận $n \times m$: hàng i cho biết từ đích thứ i "nhìn" vào từng từ nguồn nhiều hay ít. Ví dụ khi dịch "I love cats" thành "Tôi yêu mèo", lúc sinh từ *mèo*, hàng tương ứng sẽ dồn phần lớn trọng số vào *cats*. Kết quả nhân với $\mathbf{H}^v_{\text{enc}}$ ($m \times d$) cho ma trận $n \times d$, cùng kích thước với $\mathbf{S}_{\text{self}}$ nên cộng kết nối tắt được.

Hàm $\text{Att}_{\text{self}}(\cdot)$ có dạng tương tự $\text{Att}_{\text{cross}}(\cdot)$, với các ánh xạ tuyến tính của $\mathbf{S}$ được lấy làm truy vấn, khóa và giá trị, như sau

$$\begin{aligned}\text{Att}_{\text{self}}(\mathbf{S}) &= \text{Att}_{\text{qkv}}(\mathbf{S}^q, \mathbf{S}^k, \mathbf{S}^v) \\ &= \text{Softmax}\Big(\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}} + \mathbf{M}\Big)\mathbf{S}^v, \end{aligned} \tag{6.44}$$

trong đó $\mathbf{S}^q = \mathbf{S}\mathbf{W}^q_{\text{dec}}$, $\mathbf{S}^k = \mathbf{S}\mathbf{W}^k_{\text{dec}}$ và $\mathbf{S}^v = \mathbf{S}\mathbf{W}^v_{\text{dec}}$ là các ánh xạ tuyến tính của $\mathbf{S}$ với tham số $\mathbf{W}^q_{\text{dec}}, \mathbf{W}^k_{\text{dec}}, \mathbf{W}^v_{\text{dec}} \in \mathbb{R}^{d\times d}$.

Dạng này tương tự Phương trình (6.20). Tuy nhiên, một điểm khác biệt then chốt so với tự chú ý của bộ mã hóa là mô hình ở đây phải tuân theo quy tắc sinh từ trái sang phải (xem Hình 6.4). Tức là, với một từ phía đích ở vị trí i, mô hình chỉ được nhìn thấy các từ phía đích trong ngữ cảnh bên trái $y_1, \ldots, y_{i-1}$. Để đảm bảo điều này, ta cộng một biến mặt nạ (*masking variable*) $\mathbf{M}$ vào ma trận trọng số chưa chuẩn hóa $\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}}$. Cả $\mathbf{M}$ và ma trận trọng số đều có kích thước $n \times n$, nên các giá trị âm lớn trong $\mathbf{M}$ sẽ triệt tiêu các điểm chú ý tương ứng. Để ngăn việc chú ý tới ngữ cảnh bên phải (các từ tương lai) tại bước i, $\mathbf{M}$ được định nghĩa là:

$$M(i,j) = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases}, \tag{6.45}$$

trong đó $M(i,j)$ là một số hạng độ lệch cho điểm căn chỉnh giữa vị trí i

và j. Dưới đây là một ví dụ về cách áp dụng biến mặt nạ (giả sử $n = 4$).

$$
\begin{aligned}
&\text{Softmax}\left(\frac{\mathbf{S}^q[\mathbf{S}^k]^\top}{\sqrt{d}} + \mathbf{M}\right) \\
&= \text{Softmax}\left(\begin{bmatrix} 2 & 0.1 & 1 & 1 \\ 0 & 0.9 & 0.9 & 0.9 \\ 0.2 & 0.8 & 0.7 & 2 \\ 0.3 & 1 & 0.3 & 3 \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}\right) \\
&= \text{Softmax}\left(\begin{bmatrix} 2 & -\infty & -\infty & -\infty \\ 0 & 0.9 & -\infty & -\infty \\ 0.2 & 0.8 & 0.7 & -\infty \\ 0.3 & 1 & 0.3 & 3 \end{bmatrix}\right) \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.3 & 0.7 & 0 & 0 \\ 0.2 & 0.4 & 0.4 & 0 \\ 0.05 & 0.1 & 0.05 & 0.8 \end{bmatrix}.
\end{aligned} \tag{6.46}
$$

Chú giải của người dịch: Vì sao cộng $-\infty$ lại "che" được tương lai?

Softmax lấy $e^{(\cdot)}$ của từng số, mà $e^{-\infty} = 0$ (thực tế người ta dùng một số âm rất lớn như -10^9, khi đó $e^{-10^9} \approx 0$). Vì vậy mọi ô bị cộng $-\infty$ đều nhận trọng số 0 sau Softmax, và phần còn lại của hàng tự chia lại cho có tổng bằng 1. Kiểm tra hàng 2: chỉ còn $[0 \;\; 0.9]$, $e^0 = 1$, $e^{0.9} \approx 2.46$, nên trọng số là $1/3.46 \approx 0.29 \approx 0.3$ và $2.46/3.46 \approx 0.71 \approx 0.7$, khớp với Phương trình (6.46). Mặt nạ có dạng "tam giác dưới": hàng i chỉ được mở tới cột i, đúng như học sinh làm bài thi không được xem đáp án các câu sau. Nhờ mặt nạ, khi huấn luyện ta vẫn tính song song cho cả n vị trí cùng lúc mà không vị trí nào "nhìn trộm" từ tương lai.

Như đã nói ở Mục 6.1.3, có thể dễ dàng cải tiến các mô hình này bằng cơ chế chú ý đa đầu. Ngoài ra, vì bộ giải mã thường là thành phần tốn thời gian nhất trong các ứng dụng thực tế, hiệu quả tính toán tổng thể của các hệ thống này phụ thuộc nhiều vào việc tối ưu chú ý phía bộ giải mã.

6.1.7 Huấn luyện và suy luận

Transformer có thể được huấn luyện và sử dụng theo cách tiêu chuẩn. Ví dụ, ta có thể huấn luyện một mô hình Transformer bằng hạ gradient (*gradient descent*) để cực tiểu hóa một hàm mất mát (*loss function*) trên dữ liệu huấn luyện (xem Chương 2), và kiểm tra mô hình đã huấn luyện bằng tìm kiếm chùm (*beam search*) trên dữ liệu chưa thấy (xem Chương 5). Dưới đây chúng tôi trình bày một số kỹ thuật thường được dùng trong huấn luyện và suy luận (*inference*) với mô hình Transformer.

- **Lập lịch tốc độ học** (*Learning Rate Scheduling*). Giống như các mạng nơ-ron tiêu chuẩn, Transformer có thể được huấn luyện trực tiếp bằng lan truyền ngược (*back-propagation*). Quá trình huấn luyện thường được lặp lại rất nhiều lần để mô hình khớp tốt với dữ liệu huấn luyện. Ở mỗi bước huấn luyện, ta cập nhật trọng số của mạng nơ-ron bằng cách dịch chúng một bước nhỏ theo hướng ngược với gradient của hàm mất mát. Có nhiều cách thiết kế quy tắc cập nhật cho huấn luyện. Một lựa chọn phổ biến là phương pháp tối ưu Adam [Kingma và Ba, 2014]. Để điều chỉnh tốc độ học (*learning rate*) trong quá trình huấn luyện, Vaswani và cộng sự [2017] đưa ra một chiến lược lập lịch tốc độ học: tăng tốc độ học tuyến tính trong một số bước rồi giảm dần. Họ thiết kế tốc độ học có dạng

$$\eta \;=\; \eta_0 \cdot \min\left\{ n_{\text{step}}^{-0.5},\; n_{\text{step}} \cdot (n_{\text{warmup}})^{-1.5} \right\}, \tag{6.47}$$

  trong đó η_0 là tốc độ học ban đầu, n_{step} là số bước huấn luyện đã thực hiện, và n_{warmup} là số bước khởi động (*warmup*). Trong n_{warmup} bước đầu tiên, tốc độ học η tăng dần khi quá trình huấn luyện tiến triển. Nó đạt giá trị cao nhất tại thời điểm $n_{\text{step}} = n_{\text{warmup}}$, rồi giảm theo hàm nghịch đảo căn bậc hai (tức là $\eta_0 \cdot n_{\text{step}}^{-0.5}$).

Chú giải của người dịch: Thử số với Phương trình (6.47)

Lấy $n_{\text{warmup}} = 4000$. Hai biểu thức trong min là $1/\sqrt{n_{\text{step}}}$ (giảm dần) và $n_{\text{step}}/4000^{1.5}$ (tăng tuyến tính). Tại $n_{\text{step}} = 1000$: $1/\sqrt{1000} \approx 0.0316$ và $1000/252982 \approx 0.0040$, lấy số nhỏ hơn là 0.0040 (đang ở pha tăng). Tại $n_{\text{step}} = 4000$: cả hai đều ≈ 0.0158 (đỉnh). Tại $n_{\text{step}} = 16000$: $1/\sqrt{16000} \approx 0.0079$, chỉ còn một nửa đỉnh. Lúc đầu tham

số còn ngẫu nhiên, bước đi quá lớn dễ làm huấn luyện "bung"; vì thế ta "khởi động" chậm như khởi động trước khi chạy bộ.

- **Chia lô và đệm** (*Batching and Padding*). Để cân bằng giữa tối ưu toàn cục và sự hội tụ của huấn luyện, người ta thường cập nhật trọng số bằng một tập mẫu tương đối nhỏ, gọi là một **lô nhỏ** (*minibatch*). Vì vậy, ta có thể xét phiên bản theo lô của quá trình tính xuôi và tính ngược, trong đó toàn bộ lô nhỏ được dùng cùng lúc để thu thông tin gradient. Một ưu điểm của chia lô là nó cho phép hệ thống tận dụng các phép toán tensor hiệu quả để xử lý nhiều chuỗi trong một lần chạy. Điều này đòi hỏi mọi chuỗi đầu vào trong một lô nhỏ được lưu trong một khối bộ nhớ duy nhất để có thể được đọc và xử lý đồng thời. Để minh họa ý tưởng này, xét một lô nhỏ gồm bốn mẫu có phía nguồn là:

$$\begin{array}{cccccc} \text{A} & \text{B} & \text{C} & \text{D} & \text{E} & \text{F} \\ \text{M} & \text{N} & & & & \\ \text{R} & \text{S} & \text{T} & & & \\ \text{W} & \text{X} & \text{Y} & \text{Z} & & \end{array}$$

Ta có thể lưu các chuỗi này trong một khối liên tục 4×6, trong đó mỗi "hàng" biểu diễn một chuỗi, như sau

$$\begin{array}{cccccc} \text{A} & \text{B} & \text{C} & \text{D} & \text{E} & \text{F} \\ \text{M} & \text{N} & \square & \square & \square & \square \\ \text{R} & \text{S} & \text{T} & \square & \square & \square \\ \text{W} & \text{X} & \text{Y} & \text{Z} & \square & \square \end{array}$$

Ở đây, các từ đệm (*padding words*) □ được nối vào cuối các chuỗi ngắn hơn để mọi chuỗi được căn thẳng hàng trong bộ nhớ. Thông thường, ta không muốn phần đệm ảnh hưởng tới hoạt động của hệ thống, nên có thể đơn giản định nghĩa □ là một vectơ không (gọi là **đệm không** (*zero padding*)). Mặt khác, trong một số trường hợp ta lại muốn dùng phần đệm để mô tả điều gì đó mà các chuỗi đầu vào không bao quát. Ví dụ, ta có thể thay các từ đệm bằng các từ ngữ cảnh ở bên trái (hoặc bên phải) của một chuỗi, dù điều này có thể đòi hỏi sửa đổi hệ thống để bảo đảm các từ ngữ cảnh mới thêm vào không làm xuất hiện nội dung ngoài ý muốn ở đầu ra.

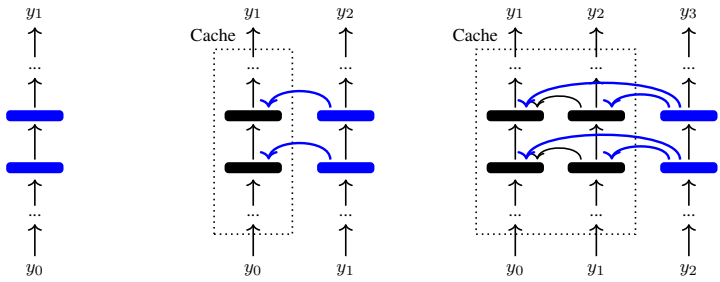
Chú giải của người dịch: Đệm và mặt nạ

Máy tính xử lý nhanh nhất khi dữ liệu là một "bảng" chữ nhật đều đặn, giống như xếp học sinh thành các hàng ghế bằng nhau: hàng nào thiếu người thì đặt ghế trống. Trong thực tế, ngoài việc đặt vectơ đệm bằng 0, người ta còn dùng một mặt nạ (giống $\mathbf{M}$ ở Phương trình (6.45)) với giá trị $-\infty$ tại các cột ứng với $\square$, để không từ thật nào "chú ý" vào ghế trống.

- **Tìm kiếm và bộ nhớ đệm** (*Search and Caching*). Khi kiểm tra, ta cần tìm kiếm trong không gian các giả thuyết ứng viên (hay các chuỗi phía đích ứng viên) để xác định giả thuyết có điểm cao nhất

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}} \; \text{score}(\mathbf{x}, \mathbf{y}), \tag{6.48}$$

trong đó $\text{score}(\mathbf{x}, \mathbf{y})$ là điểm của mô hình cho chuỗi phía đích $\mathbf{y}$ khi biết chuỗi phía nguồn $\mathbf{x}$. Dù có nhiều thuật toán tìm kiếm được thiết kế cho mục đích này, phần lớn có cấu trúc tương tự nhau: chương trình tìm kiếm hoạt động bằng cách mở rộng một tập các chuỗi phía đích ứng viên, mỗi lần thêm một token. Theo cách này, thuật toán thu được có thể được xem là một thủ tục sinh từ trái sang phải. Để thảo luận chi tiết hơn về các thuật toán tìm kiếm và điểm mô hình của các mô hình chuỗi-sang-chuỗi tổng quát, xem Chương 5. Lưu ý rằng mọi thiết kế của $\text{score}(\mathbf{x}, \mathbf{y})$, dù phức tạp đến đâu, đều dựa trên việc tính $\Pr(\mathbf{y}|\mathbf{x})$. Vì các mô hình chú ý dùng trong Transformer đòi hỏi tính tích vô hướng của mọi cặp vectơ đầu vào trong một tầng, độ phức tạp thời gian của thuật toán tìm kiếm là một hàm bậc hai theo độ dài của $\mathbf{y}$. Do đó, việc tính đi tính lại đầu ra của các mô hình chú ý cho những vị trí đã xử lý là không hiệu quả. Vấn đề này có thể được giải quyết bằng cách lưu vào bộ nhớ đệm (*cache*) các trạng thái của mỗi tầng cho những từ đã thấy. Hình 6.5 minh họa việc dùng cơ chế bộ nhớ đệm trong một bước tìm kiếm. Mọi trạng thái của các vị trí $< i$ được lưu giữ và dễ dàng truy cập trong bộ nhớ đệm. Tại vị trí i, ta chỉ cần tính các trạng thái cho từ mới được thêm vào rồi cập nhật bộ nhớ đệm.



(a) decoding step 1 (b) decoding step 2 (c) decoding step 3

Hình 6.5: Minh họa cơ chế bộ nhớ đệm (*Cache*) trong bộ giải mã Transformer, ở các bước giải mã 1, 2, 3 (a–c). Các hình chữ nhật biểu thị trạng thái của các tầng giải mã hoặc tầng con. Tại bước i, mọi trạng thái từ các bước trước được lưu trong bộ nhớ đệm (xem các khung nét chấm), và ta chỉ cần tính các trạng thái cho bước hiện tại (xem các hình chữ nhật và mũi tên màu xanh). Sau đó ta thêm các trạng thái mới sinh vào bộ nhớ đệm và chuyển sang bước $i + 1$.

Chú giải của người dịch: Vì sao bộ nhớ đệm giúp tiết kiệm?

Khi sinh từ thứ i, nhờ mặt nạ, trạng thái của các vị trí $1, \ldots, i-1$ không phụ thuộc vào từ mới, nên chúng y hệt như ở bước trước. Không có bộ nhớ đệm, để sinh câu 100 từ ta phải tính lại $1 + 2 + \cdots + 100 = 5050$ "lượt vị trí"; có bộ nhớ đệm, mỗi bước chỉ tính 1 vị trí mới, tổng cộng 100 lượt (phép chú ý của vị trí mới vẫn phải đọc lại các khóa, giá trị đã lưu). Đây chính là ý tưởng của *bộ nhớ đệm KV* sẽ gặp lại nhiều lần trong các chương sau.

6.2 Mô hình có nhận biết cú pháp

Mặc dù Transformer là các mô hình học sâu không sử dụng một cách tường minh các cấu trúc hay giả định ngôn ngữ học, việc đưa tri thức có sẵn của chúng ta vào những hệ thống như vậy có thể vẫn cần thiết. Điều này một phần là vì các nhà nghiên cứu NLP từ lâu đã tin rằng cần một mức trừu tượng cao hơn trong biểu diễn dữ liệu để phát triển các hệ thống NLP lý tưởng, và đã có rất nhiều hệ thống sử dụng tiên nghiệm cấu trúc (*structural priors*). Tuy nhiên, cấu trúc là một chủ đề rộng. Để thảo luận về các loại cấu trúc khác nhau mà ta có thể xét tới, xem công trình của

See [2018]. Chẳng hạn, các thiên kiến quy nạp (*inductive biases*) dùng trong thiết kế mô hình có thể được xem như một dạng tiên nghiệm cấu trúc, và các mô hình NLP cũng có thể tự học cấu trúc tiềm ẩn của bài toán. Trong tiểu mục này, chúng ta sẽ bàn về một số vấn đề như vậy. Ta sẽ tập trung vào các phương pháp đưa cấu trúc ngôn ngữ học vào mô hình Transformer. Vì Transformer có thể được áp dụng cho nhiều tác vụ NLP với định dạng đầu vào và đầu ra rất khác nhau, chúng tôi sẽ chủ yếu bàn về các sửa đổi cho bộ mã hóa Transformer (gọi là **bộ mã hóa Transformer có nhận biết cú pháp** (*syntax-aware Transformer encoders*)). Tuy vậy, thảo luận của chúng tôi mang tính tổng quát, và các phương pháp này có thể dễ dàng mở rộng cho bộ giải mã Transformer.

Chú giải của người dịch: Cú pháp là gì?

Cú pháp là cách các từ kết hợp với nhau thành cụm và câu theo quy tắc ngữ pháp. Ví dụ trong câu "Con mèo đen ngủ", "con mèo đen" là một *cụm danh từ* (làm chủ ngữ), "ngủ" là *động từ* (làm vị ngữ). Các nhãn như vậy (từ loại, quan hệ chủ–vị…) chính là "tri thức ngôn ngữ học" mà mục này muốn đưa thêm vào Transformer.

6.2.1 Đầu vào và đầu ra có nhận biết cú pháp

Một trong những phương pháp đơn giản nhất để đưa cấu trúc vào hệ thống NLP là sửa đổi chuỗi đầu vào, còn hệ thống giữ nguyên. Lấy một ví dụ đơn giản, xét một câu trong đó mỗi từ x_j được gán một tập gồm κ nhãn cú pháp $\{\text{tag}_j^1, \cdots, \text{tag}_j^\kappa\}$ (chẳng hạn nhãn từ loại (*part-of-speech, POS*) và nhãn quan hệ phụ thuộc (*dependency*)). Ta có thể ghép nối các ký hiệu này để định nghĩa một "từ" mới

$$x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$$

Khi đó, vectơ nhúng của từ này được cho bởi

$$\mathbf{e}_j \;=\; e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) + \text{PE}(j), \tag{6.49}$$

trong đó $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \in \mathbb{R}^d$ là vectơ nhúng của $x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$. Vì $x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa$ là một ký hiệu phức hợp, ta phân rã bài toán học $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa)$ thành các bài toán dễ hơn. Ví dụ, ta có thể xây dựng κ mô hình nhúng, mỗi mô hình sinh vectơ nhúng cho một loại nhãn.

Khi đó ta viết $e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa)$ dưới dạng tổng của vectơ nhúng từ và các vectơ nhúng nhãn

$$e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \;=\; \mathbf{x}_j + e(\text{tag}_j^1) + \cdots + e(\text{tag}_j^\kappa), \qquad (6.50)$$

trong đó $\{e(\text{tag}_j^1), \cdots, e(\text{tag}_j^\kappa)\}$ là các vectơ nhúng của các nhãn. Một cách khác là kết hợp các vectơ nhúng này qua một mạng nơ-ron có dạng

$$e(x_j/\text{tag}_j^1/\cdots/\text{tag}_j^\kappa) \;=\; \text{FFN}_{\text{embed}}(\mathbf{x}_j, e(\text{tag}_j^1), \cdots, e(\text{tag}_j^\kappa)), \quad (6.51)$$

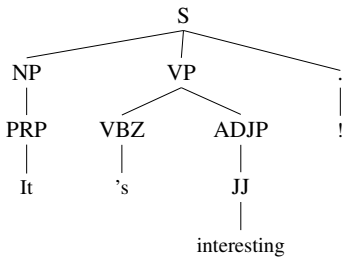
trong đó $\text{FFN}_{\text{embed}}(\cdot)$ là một mạng nơ-ron truyền thẳng có một hoặc hai tầng.

Chú giải của người dịch: Ví dụ về "từ" gắn nhãn

Với từ "cats" có nhãn từ loại NNS (danh từ số nhiều) và nhãn phụ thuộc *dobj* (tân ngữ trực tiếp), "từ" mới là `cats/NNS/dobj`. Nếu học riêng một vectơ cho mỗi tổ hợp như vậy thì số tổ hợp sẽ bùng nổ (mỗi từ $\times$ mỗi nhãn từ loại $\times$ mỗi nhãn phụ thuộc). Phương trình (6.50) tránh điều này: chỉ cần một vectơ cho "cats", một cho "NNS", một cho "dobj" rồi cộng lại, giống cách vectơ vị trí được cộng vào vectơ từ ở Phương trình (6.13).

Ta có thể áp dụng kỹ thuật tương tự ở phía bộ giải mã, coi $y_i/\text{tag}_i^1/\cdots/\text{tag}_i^\kappa$ là một từ đích được bổ sung cú pháp. Tuy nhiên, làm như vậy có thể làm tăng mạnh kích thước bộ từ vựng phía đích, gây ra thách thức tính toán cho cả huấn luyện lẫn suy luận.

Một cách phổ biến khác để biểu diễn một câu là thông qua cây cú pháp (*syntax tree*). Trong ngôn ngữ học, cú pháp của một câu có thể được diễn giải theo nhiều cách khác nhau, dẫn tới nhiều loại ngữ pháp và các biểu diễn dựa trên cây (hoặc đồ thị) tương ứng. Dù các biểu diễn này khác nhau ở dạng cú pháp cụ thể, một cách tiếp cận chung để sử dụng chúng trong mô hình hóa chuỗi là **tuyến tính hóa cây** (*tree linearization*). Xét câu sau đây được chú giải bằng một cây phân tích thành phần (*constituency-based parse tree*):



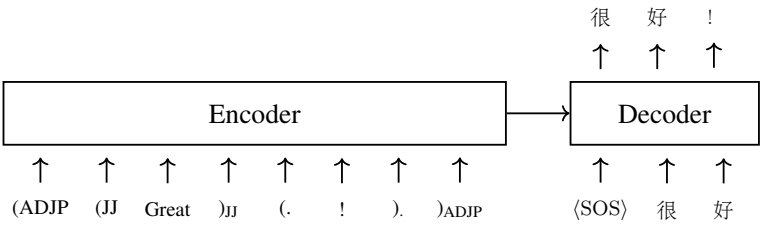
Ta có thể viết cấu trúc cây này thành một chuỗi gồm các từ, nhãn cú pháp và dấu ngoặc bằng một thuật toán duyệt cây, như sau

(S	(NP	(PRP	It	)$_{PRP}$	)$_{NP}$	(VP	(VBZ	's	)$_{VBZ}$
(ADJP	(JJ	interesting	)$_{JJ}$	)$_{ADJP}$	)$_{VP}$	(.	!	).	)$_{S}$

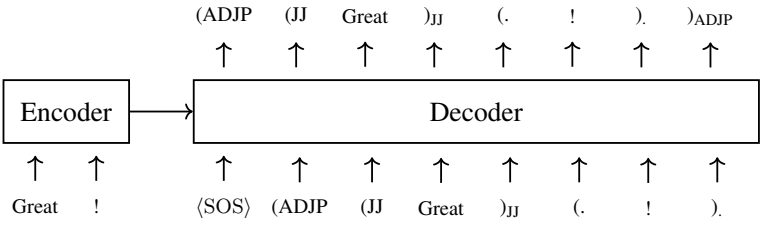
Chú giải của người dịch: Đọc cây cú pháp "It's interesting!"

Câu tiếng Anh "It's interesting!" (Thật thú vị!) được giữ nguyên vì ví dụ minh họa chính cấu trúc tiếng Anh. Các nhãn: S = câu, NP = cụm danh từ, VP = cụm động từ, ADJP = cụm tính từ, PRP = đại từ ("It"), VBZ = động từ ngôi thứ ba số ít ("'s" = "is"), JJ = tính từ, "." = dấu câu. Duyệt cây từ trên xuống, từ trái sang phải: gặp một nút thì viết "(nhãn", đi hết các con của nó thì viết ")nhãn". Giống như viết biểu thức toán có ngoặc lồng nhau, chuỗi ngoặc này chứa đủ thông tin để dựng lại cây.

Chuỗi các token cú pháp này có thể được dùng trực tiếp làm đầu vào cho bộ mã hóa, tức là mỗi token được biểu diễn bằng tổng của vectơ nhúng từ và vectơ nhúng vị trí của nó, và vectơ nhúng kết hợp này được coi như một đầu vào tiêu chuẩn của bộ mã hóa. Một ứng dụng của cây tuyến tính hóa là dịch máy (*machine translation*) cây-sang-chuỗi (*tree-to-string*), trong đó một cây cú pháp ở một ngôn ngữ được dịch thành một chuỗi ở ngôn ngữ khác [Li và cộng sự, 2017b; Currey và Heafield, 2018]. Cây tuyến tính hóa cũng có thể được dùng để sinh cây. Ví dụ, các tác vụ phân tích cú pháp (*parsing*) có thể được phát biểu thành các bài toán chuỗi-sang-chuỗi, ánh xạ một văn bản đầu vào thành biểu diễn tuần tự của cây cú pháp tương ứng [Vinyals và cộng sự, 2015; Choe và Charniak, 2016]. Hình 6.6 minh họa các mô hình này. Cần lưu ý rằng các phương pháp mô tả ở đây không dành riêng cho Transformer và có thể được áp dụng cho nhiều kiến trúc dựa trên chuỗi, chẳng hạn các mô hình dựa trên RNN.



(a) Tree-to-String Machine Translation



(b) Constituency Parsing

Hình 6.6: Minh họa tuyến tính hóa cây ở phía bộ mã hóa hoặc phía bộ giải mã. (a) Với dịch máy cây-sang-chuỗi (*Tree-to-String Machine Translation*), bộ mã hóa nhận một biểu diễn tuần tự của cây phân tích đầu vào, và bộ giải mã cho ra bản dịch tương ứng (ở đây là câu tiếng Trung *hěn hǎo!*, nghĩa là "Rất tốt!", bản dịch của "Great!"). (b) Với phân tích cú pháp (*Constituency Parsing*), bộ mã hóa nhận một câu, và bộ giải mã cho ra cây cú pháp tương ứng.

6.2.2 Mô hình chú ý có nhận biết cú pháp

Với mô hình Transformer, việc dùng cây cú pháp để định hướng quá trình học biểu diễn chuỗi cũng là điều tự nhiên. Ở mục trước, ta đã thấy biểu diễn của một chuỗi có thể được tính bằng cách liên hệ các vị trí khác nhau trong chuỗi đó. Điều này cho phép ta áp đặt cấu trúc lên các phân phối chú ý trên các vị trí. Để làm vậy, ta dùng tự chú ý của bộ mã hóa với một mặt nạ cộng (*additive mask*)

$$\text{AttSyn}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} + \mathbf{M}\Big)\mathbf{H}^v \tag{6.52}$$

hoặc với một mặt nạ nhân (*multiplicative mask*)

$$\text{AttSyn}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} \odot \mathbf{M}\Big)\mathbf{H}^v, \tag{6.53}$$

trong đó $\mathbf{M} \in \mathbb{R}^{m\times m}$ là một ma trận các biến mặt nạ, trong đó giá trị $M(i,j)$ càng lớn thể hiện tương quan cú pháp giữa vị trí i và j càng mạnh. Trong phần mô tả dưới đây, ta chọn Phương trình (6.53) làm dạng cơ bản.

Một cách phổ biến để thiết kế $\mathbf{M}$ là chiếu các quan hệ cú pháp của cấu trúc cây đầu vào thành các ràng buộc trên chuỗi. Ở đây, ta xét cây phân tích thành phần và cây phân tích phụ thuộc (*dependency parse tree*) để minh họa. Nói chung, có hai loại phương pháp mặt nạ được sử dụng.

- **Mặt nạ 0-1** (*0-1 Masking*). Phương pháp này gán $M(i,j) = 1$ nếu các từ ở vị trí i và j được coi là có tương quan cú pháp, và $M(i,j) = 0$ trong trường hợp ngược lại [Zhang và cộng sự, 2020c; Bai và cộng sự, 2021]. Để mô hình hóa quan hệ giữa hai từ trong cây cú pháp, ta có thể xét khoảng cách giữa các nút tương ứng của chúng. Một trong những dạng đơn giản nhất là

$$M(i,j) \;=\; \begin{cases} 1 & \omega(i,j) \le \omega_{\max} \\ 0 & \text{trường hợp khác} \end{cases}, \tag{6.54}$$

  trong đó $\omega(i,j)$ là độ dài đường đi ngắn nhất giữa các nút ứng với các từ ở vị trí i và j. Ví dụ, với một cây phân tích phụ thuộc, $\omega(i,j)$ là số cạnh phụ thuộc trên đường đi giữa hai từ. Với cây phân tích thành phần, mọi từ đều là nút lá. Do đó, $\omega(i,j)$ biểu thị khoảng

cách trên cây giữa hai lá trong cùng một nhánh của cây. $\omega_{\max}$ là một tham số dùng để kiểm soát khoảng cách tối đa giữa hai nút mà vẫn được coi là có tương quan cú pháp. Ví dụ, giả sử có một cây phân tích phụ thuộc với $\omega_{\max} = 1$, Phương trình (6.54) áp đặt ràng buộc rằng điểm chú ý giữa vị trí i và j chỉ được tính nếu chúng có quan hệ cha–con (từ chi phối–từ phụ thuộc)[4].

- **Mặt nạ mềm** (*Soft Masking*). Thay vì coi **M** là một ràng buộc cứng, ta có thể dùng nó như một ràng buộc mềm, co giãn trọng số chú ý giữa vị trí i và j dựa trên mức độ tương quan giữa các từ tương ứng. Một cách tiếp cận là giảm trọng số chú ý khi $\omega(i, j)$ tăng lên. Một phương pháp đơn giản là biến đổi $\omega(i, j)$ sao cho $M(i, j)$ tương quan nghịch với $\omega(i, j)$ và nằm trong đoạn $[0, 1]$:

$$M(i, j) \;=\; \text{DNorm}(\omega(i, j)). \tag{6.55}$$

Có một số thiết kế khác nhau cho DNorm$(\cdot)$. Ví dụ, ta có thể tính điểm chuẩn hóa của $-\omega(i, j)$ bằng cách trừ đi trung bình và chia cho độ lệch chuẩn [Chen và cộng sự, 2018a], hoặc chuẩn hóa $1/\omega(i, j)$ trên mọi j có thể có trong chuỗi [Xu và cộng sự, 2021b]. Trong trường hợp bộ phân tích cú pháp có thể cho ra một điểm số giữa vị trí i và j, điểm số này cũng có thể được dùng để tính $M(i, j)$. Ví dụ, một bộ phân tích phụ thuộc có thể cho ra xác suất để từ ở vị trí i là cha của từ ở vị trí j [Strubell và cộng sự, 2018]. Khi đó ta có thể viết $M(i, j)$ là

$$M(i, j) \;=\; \text{Pr}_{\text{parent}}(i|j) \tag{6.56}$$

hoặc

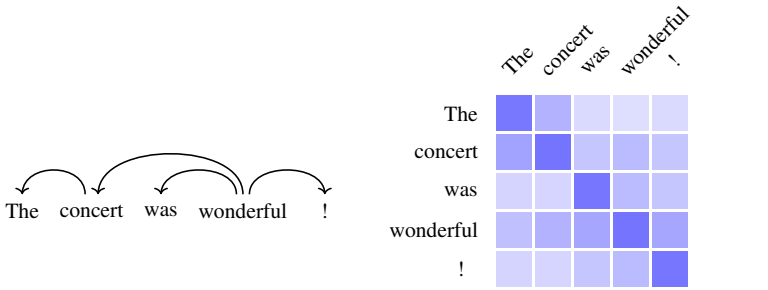
$$M(i, j) \;=\; \max\{\text{Pr}_{\text{parent}}(i|j), \text{Pr}_{\text{parent}}(j|i)\}, \tag{6.57}$$

trong đó $\text{Pr}_{\text{parent}}(i|j)$ và $\text{Pr}_{\text{parent}}(j|i)$ là các xác suất do bộ phân tích cú pháp đưa ra. Xem Hình 6.7 để có ví dụ về cách suy ra một biến mặt nạ mềm từ một cây phân tích phụ thuộc.

[4] Với mặt nạ nhân, $M(i, j) = 0$ không có nghĩa là trọng số chú ý cuối cùng giữa j và i bằng 0, vì đầu vào bằng 0 của hàm Softmax không cho ra xác suất bằng 0. Một cách để "che" hẳn một phần tử của Softmax $\left(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\right)$ là dùng mặt nạ cộng và đặt $M(i, j) = -\infty$ nếu $\omega(i, j) > \omega_{\max}$.

Chú giải của người dịch: Mặt nạ cộng và mặt nạ nhân, bằng số

Giả sử điểm chú ý (trước Softmax) của một hàng là $[2 \;\; 1 \;\; 3]$ và mặt nạ 0-1 là $[1 \;\; 1 \;\; 0]$ (từ thứ ba ở "quá xa" trên cây). Mặt nạ nhân: $[2 \;\; 1 \;\; 3] \odot [1 \;\; 1 \;\; 0] = [2 \;\; 1 \;\; 0]$; Softmax cho $\approx [0.67 \;\; 0.24 \;\; 0.09]$ — từ thứ ba vẫn nhận trọng số 0.09, không phải 0, đúng như chú thích chân trang 4 nói. Mặt nạ cộng với $[0 \;\; 0 \;\; -\infty]$: $[2 \;\; 1 \;\; -\infty]$; Softmax cho $\approx [0.73 \;\; 0.27 \;\; 0]$ — che hẳn. Còn với mặt nạ mềm, các số trong $\mathbf{M}$ là số thực trong $[0, 1]$, chẳng hạn $[1 \;\; 0.5 \;\; 0.2]$: từ càng xa trên cây thì điểm bị "thu nhỏ" càng nhiều.



(a) Dependency Parse Tree (b) Mask $\mathbf{M}$ (darker color means larger value)

Hình 6.7: Các tiên nghiệm suy ra từ một cây phân tích phụ thuộc: (a) cây phân tích phụ thuộc của câu "The concert was wonderful!" (Buổi hòa nhạc thật tuyệt vời!); (b) mặt nạ $\mathbf{M}$ (màu càng đậm nghĩa là giá trị càng lớn). Hàng i của ma trận $\mathbf{M}$ biểu diễn một phân phối mô tả ta có thể gán bao nhiêu trọng số cho $M(i, j)$ theo khoảng cách cú pháp giữa i và j.

6.2.3 Mô hình nhiều nhánh

Việc tích hợp cú pháp vào các hệ thống NLP đặt ra một số thách thức. Điều này một phần là vì các cây phân tích tự động có thể chứa lỗi, và một phần là vì việc dùng cú pháp có thể dẫn tới những giả định mạnh về cấu trúc tiềm ẩn của câu. Thay vì kết hợp thông tin cú pháp và thông tin từ vào một mô hình nguyên khối duy nhất, có thể linh hoạt và hiệu quả hơn nếu xây dựng một mô hình để mã hóa thông tin cú pháp và một mô hình khác để mã hóa chuỗi từ. Một cách để đạt được điều này là dùng nhiều mạng nơ-ron (gọi là các **nhánh** (*branches*) hay **đường** (*paths*)), mỗi mạng xử lý

một loại đầu vào. Đầu ra của các nhánh này sau đó được kết hợp để tạo ra đầu ra cuối cùng [Xie và cộng sự, 2017; Fan và cộng sự, 2020; Lin và cộng sự, 2022b]. Vì vậy, nhiều phương pháp khác nhau đã được dùng để kết hợp các loại đầu vào khác nhau cho các mô hình nơ-ron như Transformer.

Một cách tiếp cận thường dùng là xây dựng hai bộ mã hóa riêng biệt, trong đó một mô hình được huấn luyện để mã hóa đầu vào cú pháp (ký hiệu là $\mathbf{t}$), và mô hình kia được huấn luyện để mã hóa đầu vào văn bản tiêu chuẩn (ký hiệu là $\mathbf{x}$). Hình 6.8(a) minh họa kiến trúc nhiều bộ mã hóa này. Bộ mã hóa cú pháp $\text{Encode}_{\text{syn}}(\mathbf{t})$ dựa trên các mô hình trình bày ở Mục 6.2.1 và 6.2.2, còn bộ mã hóa văn bản $\text{Encode}_{\text{text}}(\mathbf{x})$ là một bộ mã hóa Transformer tiêu chuẩn. Các biểu diễn do những bộ mã hóa này sinh ra sau đó được đưa vào mô hình kết hợp làm đầu vào và được kết hợp thành một biểu diễn lai:

$$\begin{aligned}\mathbf{H}_{\text{hybrid}} &= \text{Combine}(\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}) \\ &= \text{Combine}(\text{Encode}_{\text{syn}}(\mathbf{t}), \text{Encode}_{\text{text}}(\mathbf{x})).\end{aligned} \tag{6.58}$$

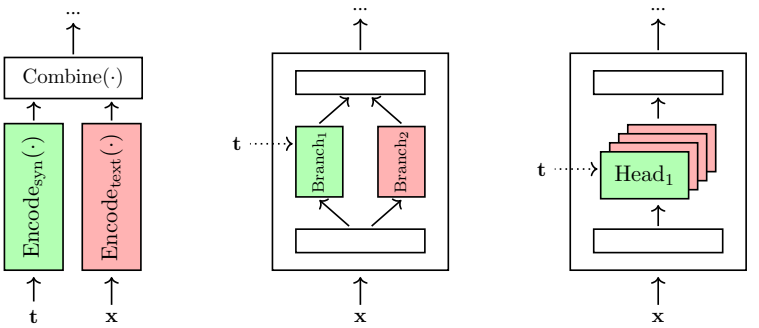
Có một số thiết kế cho $\text{Combine}(\cdot)$, tùy theo tác vụ mà các bộ mã hóa được áp dụng. Ví dụ, nếu ta muốn xây dựng một bộ phân loại (*classifier*) văn bản, $\text{Combine}(\cdot)$ có thể là một mạng gộp (*pooling*) đơn giản. Với các tác vụ phức tạp hơn, như dịch máy, $\text{Combine}(\cdot)$ cũng có thể là một bộ mã hóa Transformer, và ta có thể hợp nhất thông tin từ các nguồn khác nhau bằng cách thực hiện tự chú ý trên $[\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}]$.

Chú giải của người dịch: Ghép rồi tự chú ý

Nếu $\mathbf{H}_{\text{syn}}$ có 12 hàng (12 token cú pháp) và $\mathbf{H}_{\text{text}}$ có 5 hàng (5 từ), cùng d cột, thì $[\mathbf{H}_{\text{syn}}, \mathbf{H}_{\text{text}}]$ ở đây được hiểu là xếp chồng chúng thành một ma trận $17 \times d$. Tự chú ý trên ma trận này cho phép mỗi từ "nhìn" vào cả các token cú pháp và ngược lại, nhờ vậy hai nguồn thông tin được trộn lẫn. Còn "gộp" (*pooling*) đơn giản là, chẳng hạn, lấy trung bình các hàng để được một vectơ duy nhất đại diện cho cả câu.

Mặc dù trong mục này chúng ta tập trung chủ yếu vào các mô hình cú pháp, kiến trúc nhiều bộ mã hóa tổng quát có thể được dùng trong nhiều tác vụ cần đầu vào từ các nguồn bổ sung. Ví dụ, ta có thể dùng một bộ mã hóa để biểu diễn một câu và một bộ mã hóa khác để biểu diễn câu đứng trước nó trong cùng văn bản. Kết hợp hai bộ mã hóa này sẽ cho một

mô hình có nhận biết ngữ cảnh [Voita và cộng sự, 2018; Li và cộng sự, 2020a]. Hơn nữa, kiến trúc của các bộ mã hóa không nhất thiết bị giới hạn ở Transformer. Ta có thể chọn các mô hình khác nhau cho các nhánh khác nhau. Chẳng hạn, trong một kiến trúc mã hóa hai nhánh được dùng rộng rãi, ta có thể dùng một bộ mã hóa dựa trên CNN để mô hình hóa ngữ cảnh cục bộ và một bộ mã hóa Transformer để mô hình hóa ngữ cảnh toàn cục [Wu và cộng sự, 2020b].



(a) Multi-encoder (b) Multi-branch as a Sub-model (c) Multi-head Attention

Hình 6.8: Các kiến trúc nhiều nhánh: (a) nhiều bộ mã hóa (*Multi-encoder*); (b) nhiều nhánh trong vai trò một mô hình con (*Multi-branch as a Sub-model*); (c) chú ý đa đầu (*Multi-head Attention*). Có hai đầu vào: một câu (ký hiệu là **x**) và cây cú pháp của câu (ký hiệu là **t**). Trong kiến trúc nhiều bộ mã hóa (xem hình con (a)), hai bộ mã hóa được xây dựng để lần lượt mã hóa **x** và **t**. Sau đó một mô hình kết hợp nhận đầu ra của các bộ mã hóa và tạo ra một biểu diễn kết hợp của **x** và **t**. Ý tưởng mạng nhiều nhánh có thể được dùng để thiết kế các mô hình con của bộ mã hóa. Một ví dụ đơn giản là ta tạo nhiều đường song song cho một số tầng của bộ mã hóa (xem hình con (b)). Một ví dụ khác là chú ý đa đầu (xem hình con (c)), trong đó ta dùng các đầu khác nhau để học các biểu diễn khác nhau.

Các mô hình con của Transformer cũng có thể là mạng nơ-ron nhiều nhánh. Xem Hình 6.8(b) để có ví dụ gồm hai nhánh tự chú ý. Một nhánh là mạng tự chú ý tiêu chuẩn $\text{Att}_{\text{self}}(\mathbf{H})$, nhánh kia là mạng tự chú ý có nhận biết cú pháp $\text{AttSyn}_{\text{self}}(\mathbf{H})$. Đầu ra của mô hình tự chú ý là một tổ hợp tuyến tính của đầu ra hai nhánh này [Xu và cộng sự, 2021b]:

$$\mathbf{H}_{\text{self}} = \alpha \cdot \text{Att}_{\text{self}}(\mathbf{H}) + (1-\alpha) \cdot \text{AttSyn}_{\text{self}}(\mathbf{H}), \tag{6.59}$$

trong đó α là hệ số kết hợp. $\mathbf{H}_{\text{self}}$ có thể được dùng như thường lệ bằng

cách áp dụng hàm chuẩn hóa tầng và thêm kết nối tắt, nhờ đó giữ cho kiến trúc tổng thể giống hệt các mô hình Transformer tiêu chuẩn.

Mạng chú ý đa đầu cũng có thể được xem là một dạng mô hình nhiều nhánh. Do đó, ta có thể đưa hướng dẫn từ cú pháp vào chỉ một số đầu, còn các đầu còn lại giữ nguyên [Strubell và cộng sự, 2018]. Cách tiếp cận này được minh họa ở Hình 6.8(c), trong đó chỉ một đầu của tầng con tự chú ý dùng cây cú pháp để tính trọng số chú ý.

6.2.4 Mô hình đa tỉ lệ

Trong ngôn ngữ học, cú pháp nghiên cứu cách câu được xây dựng từ các thành phần nhỏ hơn. Các cấp độ khác nhau của những thành phần này thường được tổ chức thành một cấu trúc phân cấp, gọi là **phân cấp cú pháp** (*syntactic hierarchy*). Vì vậy, có thể dùng nhiều cấp độ thành phần cú pháp để giải thích cùng một câu. Ví dụ, các từ cho thấy câu được xây dựng từ những đơn vị nhỏ có nghĩa như thế nào, còn các cụm từ cho thấy câu được xây dựng từ những đơn vị ngôn ngữ lớn hơn như thế nào.

Transformer đa tỉ lệ (*multi-scale*) tận dụng các mức trừu tượng khác nhau trong dữ liệu để biểu diễn một câu bằng các đặc trưng ở nhiều tỉ lệ. Một cách tiếp cận phổ biến là viết một câu dưới nhiều dạng khác nhau rồi kết hợp chúng bằng một mạng nhiều nhánh [Hao và cộng sự, 2019]. Ví dụ, xét câu sau ("Cơ sở nấu bia lâu đời nhất được phát hiện ở Trung Quốc"; giữ nguyên tiếng Anh vì ví dụ minh họa việc tách từ và cụm từ):

The oldest beer-making facility was discovered in China.

Ta có thể tách nó thành một chuỗi các từ, ký hiệu là

$$\mathbf{x}_{\text{words}} = \text{The oldest beer-making facility was discovered in China .}$$

Hoặc, ta có thể viết nó thành một chuỗi các cụm từ nhờ một bộ phân tích cú pháp, ký hiệu là

$$\mathbf{x}_{\text{phrases}} = [\text{The oldest beer-making facility}]_{\text{NP}} [\text{was discovered in China}]_{\text{VP}} [.].$$

Cách đơn giản nhất để xây dựng một mô hình đa tỉ lệ là mã hóa $\mathbf{x}_{\text{words}}$ và $\mathbf{x}_{\text{phrases}}$ bằng hai bộ mã hóa Transformer riêng biệt. Đầu ra của các bộ

mã hóa này sau đó được kết hợp theo một cách nào đó. Điều này dẫn tới cùng dạng với Phương trình (6.58), và ta có thể xem mô hình này là một trường hợp của kiến trúc nhiều bộ mã hóa tổng quát.

Cả $\mathbf{x}_{\text{words}}$ và $\mathbf{x}_{\text{phrases}}$ đều có thể được xem là các chuỗi token. Ví dụ, $\mathbf{x}_{\text{words}}$ có chín token dựa trên từ, còn $\mathbf{x}_{\text{phrases}}$ có ba token dựa trên cụm từ[5]. Tuy nhiên, liệt kê mọi cụm từ có thể có sẽ dẫn tới một bộ từ vựng khổng lồ. Do đó, ta cần một phương pháp để biểu diễn mỗi cụm từ thành một vectơ nhúng theo cách hiệu quả về tính toán. Bằng cách coi việc nhúng cụm từ là một bài toán mô hình hóa chuỗi, ta có thể dễ dàng học biểu diễn của các chuỗi con bằng các mô hình chuỗi đã mô tả ở các chương trước. Khi đó ta có một quá trình học hai giai đoạn. Ở giai đoạn thứ nhất, ta học vectơ nhúng của các đơn vị đầu vào ở các tỉ lệ khác nhau bằng các mô hình riêng biệt. Ở giai đoạn thứ hai, ta học cách mã hóa các chuỗi ở các tỉ lệ khác nhau bằng một mô hình nhiều nhánh.

Tổng quát hơn, trong học biểu diễn đa tỉ lệ, ta không cần tự giới hạn ở các đơn vị có ý nghĩa ngôn ngữ học. Ví dụ, ta có thể học cách tách từ con (subword) (*sub-word*) từ dữ liệu và biểu diễn một câu đầu vào thành một chuỗi các từ con. Điều này cho ta một biểu diễn phân cấp của câu (ví dụ: từ con $\rightarrow$ từ $\rightarrow$ cụm từ). Dù các từ con học được có thể không có ý nghĩa ngôn ngữ học, chúng mang lại những góc nhìn mới cho việc mô hình hóa từ và cụm từ, cũng như một tỉ lệ đặc trưng mới. Ngoài ra, ta không nhất thiết phải xây dựng nhiều bộ mã hóa cho mô hình hóa đa tỉ lệ. Một cách tiếp cận khác là đưa các biểu diễn ở các tỉ lệ khác nhau trực tiếp vào các mô-đun tự chú ý đa đầu, giúp dễ mô hình hóa tương tác giữa các tỉ lệ khác nhau hơn [Guo và cộng sự, 2020; Li và cộng sự, 2022b].

Tuy nhiên, một vấn đề của các cách tiếp cận trên là các biểu diễn (hoặc ma trận trọng số chú ý) học được ở các tỉ lệ khác nhau có kích thước khác nhau. Ví dụ, trong các tình huống trên, biểu diễn học từ $\mathbf{x}_{\text{words}}$ là một ma trận $9 \times d$, trong khi biểu diễn học từ $\mathbf{x}_{\text{phrases}}$ là một ma trận $3 \times d$. Một giải pháp đơn giản cho vấn đề này là thực hiện tăng mẫu (*upsampling*) biểu diễn dựa trên cụm từ để mở rộng nó thành một ma trận $9 \times d$. Tương tự, ta có thể thực hiện giảm mẫu (*downsampling*) biểu diễn dựa trên từ để thu nhỏ nó thành một ma trận $3 \times d$. Khi đó mô hình kết hợp $\text{Combine}(\cdot)$ có thể được cài đặt như mô tả ở Mục 6.2.3.

[5] $\mathbf{x}_{\text{phrases}}$ gồm ba token: *The oldest beer-making facility*, *was discovered in China* và .

Chú giải của người dịch: Tăng mẫu và giảm mẫu, cụ thể

Cụm thứ nhất "The oldest beer-making facility" gồm 4 từ, cụm thứ hai "was discovered in China" gồm 4 từ, cụm thứ ba "." gồm 1 từ: tổng 9. Một cách tăng mẫu đơn giản là *chép* hàng của mỗi cụm cho từng từ thuộc cụm đó: hàng 1 chép 4 lần, hàng 2 chép 4 lần, hàng 3 giữ 1 lần, được ma trận $9 \times d$. Ngược lại, một cách giảm mẫu là lấy *trung bình* các hàng của những từ cùng cụm, đưa ma trận $9 \times d$ về $3 \times d$. Sau đó hai biểu diễn có cùng kích thước nên có thể cộng hoặc ghép với nhau.

Đáng lưu ý là mô hình hóa đa tỉ lệ được bàn đến rộng rãi trong nhiều lĩnh vực. Chẳng hạn, trong thị giác máy tính (*computer vision*), mô hình hóa đa tỉ lệ thường được hiểu là quá trình học một chuỗi các bản đồ đặc trưng (*feature maps*) từ ảnh đầu vào [Fan và cộng sự, 2021; Li và cộng sự, 2022f]. Khác với các mô hình nhiều nhánh trình bày ở đây, các mô hình Transformer thị giác đa tỉ lệ tận dụng bản chất phân cấp của đặc trưng khi biểu diễn ảnh. Các hệ thống loại này thường dựa trên một chồng các tầng, trong đó mỗi tầng học đặc trưng ở một tỉ lệ lớn hơn (ví dụ, dung lượng kênh cao hơn) từ các đặc trưng do tầng trước sinh ra.

6.2.5 Transformer như một bộ học cú pháp

Cho đến nay, chúng ta đã bàn về cây cú pháp như những ràng buộc hay tiên nghiệm cho quá trình mã hóa, để có thể tận dụng các biểu diễn ngôn ngữ học khi học mạng nơ-ron. Một câu hỏi tự nhiên là liệu các mô hình nơ-ron này có thể học trực tiếp từ dữ liệu một số tri thức về cấu trúc ngôn ngữ, mà không cần các chú giải ngôn ngữ học do con người thiết kế hay không. Điều này phản ánh một mục tiêu nền tảng của NLP: học tri thức ngôn ngữ từ dữ liệu và mã hóa nó một cách ngầm định bên trong mô hình.

Để khám phá các tính chất ngôn ngữ mà hệ thống NLP đã học được, một phương pháp đơn giản là xem xét hành vi cú pháp trong đầu ra của hệ thống. Ví dụ, ta có thể kiểm tra xem đầu ra của các hệ thống sinh ngôn ngữ có chứa lỗi ngữ pháp hay không. Một cách tiếp cận khác là yêu cầu các hệ thống này thực hiện những tác vụ có ý nghĩa về mặt ngôn ngữ học, dù chúng không được huấn luyện tường minh cho việc đó [Brown và cộng sự, 2020]. Tuy nhiên, chỉ xem xét cách các dự đoán của mô hình thể hiện năng lực cú pháp là chưa đủ để trả lời trọn vẹn câu hỏi. Cũng có thể mạng

nơ-ron học được tri thức ngôn ngữ nhưng không sử dụng nó khi dự đoán [Clark và cộng sự, 2019a]. Do đó, ta cần nghiên cứu những gì được mô hình hóa và học được bên trong chính mạng nơ-ron.

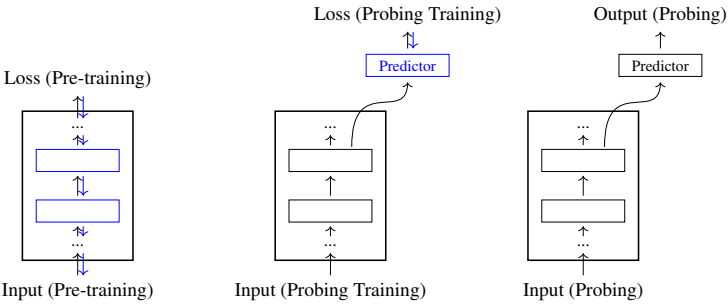
Một cách tiếp cận để xem xét cấu trúc ngôn ngữ tiềm ẩn trong mô hình Transformer là xây dựng các phép thăm dò (*probes*) nhằm xác định xem các mô hình này có nắm bắt các khái niệm ngôn ngữ học như quan hệ phụ thuộc và từ loại hay không, và ở mức độ nào. Một cách thăm dò tổng quát là trích xuất các biểu diễn bên trong của mô hình rồi thăm dò chúng để tìm các hiện tượng ngôn ngữ. Với Transformer, việc này thường được thực hiện bằng cách xem xét các bản đồ chú ý (*attention maps*) và/hoặc đầu ra của các tầng chú ý. Sau đó ta xây dựng một bộ dự đoán thăm dò (*probing predictor*) (hay bộ phân loại thăm dò (*probing classifier*)) nhận các biểu diễn bên trong này làm đầu vào và cho ra các khái niệm ngôn ngữ học tương ứng [Belinkov, 2022]. Bộ dự đoán thăm dò có thể dựa trên các quy tắc kinh nghiệm đơn giản hoặc trên các mô hình có tham số được tối ưu riêng cho tác vụ thăm dò. Các công trình gần đây cho thấy các mô hình ngôn ngữ quy mô lớn dựa trên Transformer thể hiện những năng lực mạnh, gọi là **năng lực đột sinh** (*emergent abilities*), trên nhiều tác vụ thăm dò khác nhau. Chúng tôi sẽ không bàn chi tiết về các hệ thống mô hình hóa ngôn ngữ này trong chương này mà để dành cho các chương sau. Dù vậy, ở đây ta giả sử có một bộ mã hóa Transformer đã được huấn luyện đầy đủ trên dữ liệu không gán nhãn và sẵn sàng để thăm dò. Hình 6.9 minh họa quá trình thăm dò này.

Nhiều phương pháp thăm dò đã được dùng trong các công trình gần đây nhằm phân tích và hiểu những gì được học trong các bộ mã hóa nơ-ron. Dưới đây chúng tôi mô tả một số phương pháp phổ biến.

- **Cây** (*Trees*). Với một bộ mã hóa Transformer đã huấn luyện, có thể dễ dàng xác định khả năng hai từ trong một câu có quan hệ ngôn ngữ với nhau bằng cách tính trọng số chú ý giữa chúng. Ta có thể dùng đại lượng này để định nghĩa một độ đo khoảng cách cú pháp giữa hai từ ở vị trí i và j

$$d_s(i,j) \;=\; 1 - \alpha(i,j). \tag{6.60}$$

  Với độ đo này, có thể dễ dàng xây dựng **cây khung nhỏ nhất** (*minimum-spanning tree*) cho câu. Tức là, ta nối tất cả các từ thành một cấu trúc cây có tổng khoảng cách nhỏ nhất. Cấu trúc cây này



(a) Training the Transformer Model (b) Training the Probing Predictor (c) Probing on New Data

Hình 6.9: Tổng quan về thăm dò cho các mô hình dựa trên Transformer. Cho một mô hình Transformer (ví dụ, một mô hình ngôn ngữ dựa trên Transformer), trước hết ta tối ưu tham số mô hình trên một lượng dữ liệu không gán nhãn (xem hình con (a): huấn luyện mô hình Transformer, với *Loss/Input (Pre-training)* = hàm mất mát/đầu vào của tiền huấn luyện). Sau đó, ta xây dựng một bộ dự đoán (*Predictor*) nhận trạng thái của một tầng ẩn của mô hình Transformer và sinh đầu ra cho một tác vụ thăm dò. Bộ dự đoán có thể được huấn luyện như thường lệ, trong đó chỉ tham số của bộ dự đoán được tối ưu còn tham số của mô hình Transformer được giữ cố định (xem hình con (b): huấn luyện bộ dự đoán thăm dò). Mô hình Transformer và bộ dự đoán được dùng cùng nhau để đưa ra dự đoán trên dữ liệu mới nhằm thăm dò (xem hình con (c): thăm dò trên dữ liệu mới).

có thể được xem là một biểu diễn cây tiềm ẩn của câu được suy ra từ mạng nơ-ron. Dù cấu trúc giống cây phụ thuộc này có thể là một nguồn thông tin cú pháp đã học cho các tác vụ hạ nguồn, nó lại cho biết rất ít về tri thức cú pháp thực sự của mô hình. Một cách để căn chỉnh các biểu diễn trong bộ mã hóa với cấu trúc ngôn ngữ là học cách sinh ra các cây cú pháp nhất quán với chú giải của con người. Để làm được điều này, ta cần xây dựng một bộ dự đoán thăm dò được huấn luyện trên dữ liệu có chú giải cây. Giả sử ta có một cây phụ thuộc do con người chú giải cho một câu. Với mỗi cặp từ, ta có thể thu được một khoảng cách $\omega(i, j)$ bằng cách đếm số cạnh giữa chúng. Sau đó, ta có thể học một độ đo khoảng cách dựa trên các biểu diễn bên trong của bộ mã hóa để xấp xỉ $\omega(i, j)$. Một dạng đơn giản của độ đo như vậy dựa trên khoảng cách Euclid [Manning và cộng sự, 2020]. Gọi $\mathbf{A} \in \mathbb{R}^{d \times k_s}$ là một ma trận tham số. Bình phương khoảng cách Euclid được cho bởi

$$d_s^2(i, j) \;=\; \|(\mathbf{h}_i - \mathbf{h}_j)\mathbf{A}\|_2^2, \tag{6.61}$$

trong đó $\mathbf{h}_i$ và $\mathbf{h}_j$ là các biểu diễn do một tầng mã hóa sinh ra tại vị trí i và j[6]. Với một tập S các câu có chú giải cây, ta có thể tối ưu mô hình bằng

$$\hat{\mathbf{A}} \;=\; \arg\min_{\mathbf{A}} \sum_{s \in S} \frac{1}{|s|^2} \sum_{i \in s, j \in s} |\omega(i, j) - d_s^2(i, j)|, \tag{6.62}$$

trong đó $|s|$ là độ dài của câu s, và (i, j) chỉ một cặp từ trong s. Mô hình đã tối ưu sau đó được dùng để phân tích cú pháp các câu kiểm tra bằng thuật toán cây khung nhỏ nhất, và ta có thể so sánh các cây phân tích này với các cây do con người chú giải. Để thu được cây có hướng, vốn là dạng chuẩn của cú pháp phụ thuộc, có thể cập nhật mô hình trên bằng cách xét khoảng cách tương đối của một từ tới gốc. Chi tiết hơn có thể xem trong công trình của Manning và cộng sự [2020]. Ở đây, bộ dự đoán thăm dò hoạt động tương tự một bộ phân tích cú pháp nơ-ron, được huấn luyện để dự đoán cây cú pháp dựa trên một biểu diễn của câu đầu vào. Ý tưởng này có thể được mở rộng cho các dạng cấu trúc cú pháp khác, chẳng hạn cây cấu trúc cụm từ [Shi và cộng sự, 2016].

[6]Nói chung, $\mathbf{h}_i$ và $\mathbf{h}_j$ là đầu ra của tầng cuối cùng của bộ mã hóa. Ngoài ra, chúng cũng có thể là tổng có trọng số của đầu ra tất cả các tầng.

Chú giải của người dịch: Thăm dò cây bằng khoảng cách

Ý tưởng: nếu bộ mã hóa thật sự "hiểu" cú pháp, thì sau một phép biến đổi tuyến tính **A** phù hợp, khoảng cách giữa hai vectơ từ sẽ gần bằng số cạnh giữa hai từ trên cây. Ví dụ, trong "The concert was wonderful" với cây ở Hình 6.7, "The" nối với "concert" ($\omega = 1$), "concert" nối với "wonderful" ($\omega = 1$), nên $\omega(\text{The}, \text{wonderful}) = 2$. Phương trình (6.62) tìm **A** sao cho d_s^2 khớp với các số $1, 2, \ldots$ này nhất có thể (lấy trung bình sai số tuyệt đối trên mọi cặp từ, chia cho $|s|^2$ là số cặp). **Cây khung nhỏ nhất**: trong số mọi cách nối n từ bằng $n-1$ cạnh thành một cây, chọn cách có tổng "chiều dài" cạnh nhỏ nhất — giống bài toán nối điện cho các làng với tổng chiều dài dây ngắn nhất. Vì **A** là phép biến đổi tuyến tính đơn giản, nếu thăm dò thành công thì thông tin cú pháp hẳn đã có sẵn trong các vectơ **h**, chứ không phải do bộ thăm dò tự "học" ra.

- **Nhãn cú pháp và ngữ nghĩa** (*Syntactic and Semantic Labels*). Nhiều tác vụ phân tích cú pháp và ngữ nghĩa có thể được phát biểu thành bài toán dự đoán nhãn ngôn ngữ học khi biết một câu hoặc các đoạn của nó. Một ví dụ đơn giản là gán nhãn từ loại (*part-of-speech tagging*), trong đó mỗi từ của câu được gán một lớp từ. Một bộ thăm dò cho gán nhãn từ loại có thể là một bộ phân loại, mỗi lần nhận một biểu diễn $\mathbf{h}_j$ và cho ra lớp từ tương ứng. Một cách thăm dò tổng quát cho các bài toán này là **thăm dò cạnh** (*edge probing*) [Tenney và cộng sự, 2019b;a]. Cho một câu, một cạnh có nhãn được định nghĩa là một bộ

$$(\text{span}_1, \text{span}_2, \text{label}),$$

trong đó span_1 là một đoạn $[i_1, j_1]$, span_2 là một đoạn thứ hai tùy chọn $[i_2, j_2]$, và label là nhãn tương ứng. Mục tiêu của ta là học một bộ thăm dò để dự đoán label khi biết span_1 và span_2. Ví dụ, với gán nhãn từ loại, span_1 là đoạn đơn vị $[j, j]$ cho mỗi vị trí j, span_2 là đoạn rỗng, và label là nhãn từ loại ứng với từ thứ j của câu. Với phân tích phụ thuộc và phân giải đồng tham chiếu (*coreference resolution*), span_1 và span_2 là hai từ hoặc hai thực thể, và label là quan hệ giữa chúng. Với phân tích thành phần, span_1 là một đoạn từ, span_2 là đoạn rỗng, và label là phạm trù cú pháp của nút cây sinh ra span_1. Trong các trường hợp đơn giản, mô hình thăm dò có thể là

một mạng nơ-ron truyền thẳng nhiều tầng với tầng đầu ra Softmax. Như thường lệ, mô hình này được huấn luyện trên dữ liệu có nhãn rồi kiểm tra trên dữ liệu mới.

Chú giải của người dịch: Ví dụ về thăm dò cạnh

Câu "Lan nói cô ấy sẽ đến": với đồng tham chiếu, span_1 = "Lan" (đoạn $[1, 1]$), span_2 = "cô ấy" (đoạn $[3, 4]$), label = *cùng chỉ một người*. Với gán nhãn từ loại, $\text{span}_1 = [2, 2]$ ("nói"), không có span_2, label = *động từ*. Cùng một khuôn dạng "(đoạn, đoạn, nhãn)" nên một bộ thăm dò chung có thể kiểm tra nhiều loại tri thức ngôn ngữ khác nhau.

- **Hình thức bề mặt của từ và câu** (*Surface Forms of Words and Sentences*). Các tác vụ thăm dò cũng có thể được thiết kế để kiểm tra xem các biểu diễn có mã hóa thông tin bề mặt của câu hay của từ hay không [Adi và cộng sự, 2016; Conneau và cộng sự, 2018]. Một tác vụ thăm dò mức câu đơn giản là **dự đoán độ dài câu** (*sentence length prediction*). Để làm điều này, trước hết ta biểu diễn câu bằng một vectơ duy nhất $\mathbf{h}$[7], rồi xây dựng một bộ phân loại để xếp $\mathbf{h}$ vào khoảng độ dài tương ứng. Tương tự, có thể xây dựng các bộ thăm dò để dự đoán xem hai từ ở vị trí i và j có bị đảo thứ tự trong câu hay không khi biết $\mathbf{h}_i$ và $\mathbf{h}_j$. Ta cũng có thể xây dựng các bộ thăm dò cho những bài toán truyền thống của hình thái học (*morphology*). Ví dụ, ta khôi phục lại từ ở vị trí j hoặc dự đoán nghĩa của nó từ biểu diễn $\mathbf{h}_j$. Ngoài ra, các tác vụ thăm dò có thể tập trung vào những vấn đề ngôn ngữ cụ thể, chẳng hạn khả năng xử lý số [Wallace và cộng sự, 2019] và hư từ (*function words*) [Kim và cộng sự, 2019].

- **Điền vào chỗ trống** (*Cloze*). Tất nhiên, ta có thể thăm dò các mô hình nơ-ron cho những bài toán vượt ra ngoài cú pháp và hình thái học. Một góc nhìn về các mô hình Transformer tiền huấn luyện (*pre-trained*) quy mô lớn là xem chúng như những cơ sở tri thức chứa các sự kiện về thế giới. Vì vậy, thật hấp dẫn khi thử xem liệu ta có thể dùng chúng để kiểm tra tri thức sự kiện hay không. Một phương pháp đơn giản là yêu cầu bộ thăm dò khôi phục thành phần bị thiếu của một câu [Petroni và cộng sự, 2019]. Ví dụ, nếu ta có một câu

[7] $\mathbf{h}$ có thể được tính bằng cách thực hiện một phép gộp trên $\{\mathbf{h}_1, \cdots, \mathbf{h}_m\}$.

điền vào chỗ trống

Shiji was written by ____. (*Sử ký* được viết bởi ____.)

ta mong bộ thăm dò đưa ra câu trả lời *Sima Qian* (Tư Mã Thiên) vì có một sự kiện dạng chủ thể–quan hệ–đối tượng (Shiji, written-by, Sima Qian). Bộ thăm dò này đơn giản có thể là một **mô hình ngôn ngữ có mặt nạ** (*masked language model*), loại mô hình được dùng rộng rãi trong học tự giám sát (*self-supervised learning*) của bộ mã hóa Transformer.

Trong NLP, thăm dò có liên hệ chặt chẽ với việc tiền huấn luyện các mô hình ngôn ngữ lớn (*large language models*) (xem Chương 7 và 8). Nói chung, ta có thể xem các tác vụ thăm dò như những ứng dụng hạ nguồn của các mô hình tiền huấn luyện này, dù thăm dò thường được dùng như một công cụ phân tích để nhanh chóng kiểm tra năng lực của mô hình. Lý tưởng nhất, ta muốn xây dựng một bộ thăm dò tận dụng tốt nhất các biểu diễn bên trong để giải một tác vụ cho trước. Tuy nhiên, khi bộ thăm dò phức tạp và được huấn luyện đủ tốt, sẽ khó xác định bài toán được giải nhờ thông tin phong phú trong các biểu diễn hay nhờ năng lực mô hình hóa của chính bộ thăm dò. Một cách phổ biến để tách riêng đóng góp của các biểu diễn là so sánh các bộ thăm dò với những mô hình cơ sở (baseline) (*baselines*) hợp lý, hoặc tiến hành so sánh bằng các tác vụ đối chứng (*control tasks*) [Hewitt và Liang, 2019; Belinkov, 2022].

Chú giải của người dịch: Vì sao bộ thăm dò nên đơn giản?

Hãy hình dung ta muốn biết một học sinh có thật sự hiểu bài không. Nếu cho phép em tra cứu sách và hỏi gia sư trong lúc làm bài (bộ thăm dò quá mạnh), điểm cao chưa chắc phản ánh hiểu biết của chính em. *Tác vụ đối chứng* giống như một bài kiểm tra với "đáp án ngẫu nhiên" vô nghĩa: nếu bộ thăm dò vẫn đạt điểm cao trên đó, nghĩa là nó chỉ đang "học thuộc", và kết quả trên tác vụ thật cần được đánh giá thận trọng.

6.3 Các kiến trúc cải tiến

Trong mục này, chúng ta trình bày một số cải tiến cho mô hình Transformer nguyên bản (*vanilla*). Khác với mục trước, phần lớn các cải

tiến ở đây xuất phát từ góc nhìn của học máy chứ không phải của ngôn ngữ học.

6.3.1 Các mô hình chú ý cục bộ

Các phương pháp tự chú ý (*self-attention*) đã bàn trong Mục 6.1.3 có thể được xem là học các biểu diễn (*representation*) trên toàn bộ chuỗi đầu vào. Mặc dù cơ chế chú ý toàn cục (*global attention*) này có khả năng mạnh trong việc nắm bắt các phụ thuộc xa (*long-distance dependency*), nó lại không mô hình hóa thông tin cục bộ một cách tường minh. Ở đây, ta xét một vài kỹ thuật cố gắng nắm bắt tường minh tính cục bộ trong các biểu diễn.

1. Tri thức tiên nghiệm cho mô hình hóa cục bộ

Một trong những cách đơn giản nhất để đưa việc mô hình hóa ngữ cảnh cục bộ vào Transformer là thêm một số hạng phạt (*penalty term*) vào hàm chú ý, nhờ đó hạn chế các trọng số chú ý lớn giữa những vị trí cách xa nhau. Ở phía bộ mã hóa (*encoder*), điều này dẫn đến một dạng mà ta đã gặp vài lần trong chương này:

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} - \gamma \cdot \mathbf{G}\Big)\mathbf{H}^v, \tag{6.63}$$

trong đó γ là trọng số (hay "nhiệt độ") của số hạng phạt, và $\mathbf{G} \in \mathbb{R}^{m \times m}$ là ma trận phạt. Mỗi phần tử $G(i,j)$ cho biết ta phạt mô hình nặng đến mức nào khi nó chú ý từ vị trí i tới vị trí j. Một lựa chọn đơn giản cho $G(i,j)$ là khoảng cách tuyệt đối giữa i và j, chẳng hạn

$$G(i,j) \;=\; |i-j|. \tag{6.64}$$

Ngoài ra, $G(i,j)$ có thể được định nghĩa bằng một hàm phạt Gauss [Yang và cộng sự, 2018a]

$$G(i,j) \;=\; \frac{(i-j)^2}{2\sigma_i^2}, \tag{6.65}$$

trong đó σ_i là độ lệch chuẩn của phân phối Gauss. Với một i cho trước, cả hai số hạng phạt đều tăng (theo kiểu tuyến tính hoặc theo hàm mũ) khi khoảng cách $|i-j|$ tăng lên.

Chú giải của người dịch: Phạt khoảng cách hoạt động thế nào?

Hàm Softmax biến một dãy điểm số thành các xác suất: điểm càng cao thì trọng số chú ý càng lớn. Trừ đi $\gamma \cdot G(i,j)$ nghĩa là "trừ điểm" những vị trí ở xa. Ví dụ với $\gamma = 1$, $G(i,j) = |i-j|$ và vị trí $i = 3$: điểm chú ý tới các vị trí $j = 1,2,3,4,5$ bị trừ lần lượt $2,1,0,1,2$. Nếu ban đầu mọi điểm đều bằng 0 thì sau khi trừ, Softmax của $(-2,-1,0,-1,-2)$ xấp xỉ $(0.07; 0.18; 0.49; 0.18; 0.07)$: mô hình tập trung vào chính vị trí 3 và các láng giềng gần. Với dạng Gauss $(i-j)^2/(2\sigma_i^2)$, mức phạt tăng theo bình phương khoảng cách, nên các vị trí xa bị "dập" mạnh hơn nhiều; σ_i càng lớn thì "cửa sổ" chú ý càng rộng.

Phương pháp này có thể mở rộng cho mô hình chú ý chéo (*cross-attention*) như sau

$$\text{AttLocal}_{\text{cross}}(\mathbf{H}, \mathbf{S}) \;=\; \text{Softmax}\Big(\frac{\mathbf{S}^q[\mathbf{H}^k]^\top}{\sqrt{d}} - \gamma \cdot \mathbf{G}\Big)\mathbf{H}^v, \tag{6.66}$$

trong đó $\mathbf{G}$ là một ma trận $n \times m$. Mỗi phần tử của $\mathbf{G}$ có thể được định nghĩa là

$$G(i,j) \;=\; \frac{(\mu_i - j)^2}{2\sigma_i^2}, \tag{6.67}$$

trong đó μ_i là kỳ vọng (giá trị trung bình) của phân phối Gauss trên các vị trí phía nguồn. Cả μ_i và σ_i đều có thể được xác định bằng các quy tắc kinh nghiệm (*heuristics*). Một cách khác là ta xây dựng thêm các mạng nơ-ron để mô hình hóa chúng, học các tham số của chúng đồng thời với phần còn lại của mô hình. Ví dụ, ta có thể dùng một mạng nơ-ron truyền thẳng (*feed-forward neural network*) để dự đoán μ_i khi biết $\mathbf{s}_i$.

Một phương án thay thế cho Phương trình (6.63) (hoặc Phương trình (6.66)) là coi số hạng phạt như một phân phối riêng và kết hợp tuyến tính nó với mô hình chú ý ban đầu. Chẳng hạn, ta có thể định nghĩa mô hình tự chú ý là

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \;=\; \Big((1-\beta)\cdot \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}}\Big) + \beta \cdot \text{Softmax}(-\gamma \cdot \mathbf{G})\Big)\mathbf{H}^v, \tag{6.68}$$

trong đó $\beta \in [0,1]$ là hệ số kết hợp tuyến tính. Lưu ý rằng, để tránh phải tinh chỉnh thủ công siêu tham số (*hyperparameter*) β, ta có thể dùng một mạng cổng (*gating network*) để dự đoán β một cách động và huấn luyện nó theo kiểu đầu-cuối (*end-to-end*).

Một phương án khác nữa là dùng một mặt nạ nhân (*multiplicative mask*) để đưa tri thức tiên nghiệm vào mô hình, tương tự Phương trình (6.53). Công thức như sau

$$\text{AttLocal}_{\text{self}}(\mathbf{H}) \;=\; \text{Softmax}\Big(\frac{\mathbf{H}^q[\mathbf{H}^k]^\top}{\sqrt{d}} \odot \mathbf{G}'\Big)\mathbf{H}^v. \tag{6.69}$$

Ở đây, $\mathbf{G}' \in [0,1]^{m\times m}$ là một ma trận có các phần tử nằm trong $[0,1]$. Đại lượng vô hướng $G'(i,j)$ đạt giá trị lớn nhất bằng 1 khi $i=j$ và giảm dần khi j càng xa i. $G'(i,j)$ có thể thu được bằng cách chuẩn hóa $-G(i,j)$ trên mọi j, hoặc bằng cách dùng các hàm suy giảm khác.

Chú giải của người dịch: Ba cách đưa "tính cục bộ" vào cơ chế chú ý

- Phương trình (6.63): *trừ* điểm phạt trước khi qua Softmax (phạt cộng).
- Phương trình (6.68): *trộn* hai phân phối — phân phối chú ý thông thường và một phân phối chỉ phụ thuộc khoảng cách — theo tỉ lệ $(1-\beta):\beta$. Ví dụ $\beta = 0.3$ nghĩa là 70% "nghe theo" nội dung, 30% "nghe theo" khoảng cách.
- Phương trình (6.69): *nhân* từng phần tử ($\odot$) điểm chú ý với một hệ số trong $[0,1]$, bằng 1 ở đường chéo và nhỏ dần khi xa đường chéo.

Cả ba đều là ràng buộc "mềm": vị trí xa không bị cấm hẳn, chỉ bị giảm ưu tiên.

2. Chú ý cục bộ

Thuật ngữ *chú ý cục bộ (local attention)* được dùng rộng rãi để mô tả nhiều loại bài toán khác nhau và để chỉ nhiều mô hình khác nhau trong các tài liệu NLP. Các phương pháp nêu ở trên là những phương pháp áp đặt ràng buộc mềm (*soft constraint*) lên mô hình chú ý. Thật ra, chú ý cục bộ bắt nguồn từ những nỗ lực giới hạn phạm vi của mô hình chú ý vì các lý do về mô hình hóa và tính toán [Luong và cộng sự, 2015]. Các nghiên cứu trong hướng này thường tập trung vào việc đưa vào các ràng buộc cứng (*hard constraint*), để mô hình thu được có thể tập trung vào một phần của đầu vào và bỏ qua phần còn lại. Ví dụ, ta có thể dự đoán một khoảng các vị trí phía nguồn cần chú ý tới, khi biết một vị trí phía đích [Sperber và cộng sự, 2018; Yang và cộng sự, 2018a; Sukhbaatar và cộng sự, 2019]. Ngoài ra, các khoảng chú ý cũng có thể được suy ra từ cây cú pháp; chẳng hạn,

cấu trúc cây con cú pháp có thể giúp thu hẹp phạm vi chú ý. Do đó, nhiều mô hình có ràng buộc cú pháp có thể được xem là các trường hợp cụ thể của chú ý cục bộ (xem Mục 6.2.4). Hơn nữa, khái niệm chú ý cục bộ có thể được khái quát hóa cho một tập hợp phong phú các mô hình, chẳng hạn các **mô hình chú ý thưa** (*sparse attention models*), mặc dù những mô hình này thường được bàn tới trong bối cảnh các phương pháp học máy hiệu quả. Ta sẽ thấy một vài ví dụ ở Mục 6.4.

Trong học sâu (*deep learning*), một trong những kiến trúc được dùng rộng rãi nhất để học đặc trưng từ một vùng giới hạn của đầu vào là CNN. Vì vậy, sẽ rất thú vị nếu xét các phương pháp kết hợp CNN với mô hình Transformer để có được ưu điểm của cả hai cách tiếp cận, chẳng hạn CNN xử lý các phụ thuộc ngắn hạn, còn mô hình tự chú ý xử lý các phụ thuộc dài hạn. Một cách tiếp cận là xây dựng một mô hình chuỗi hai nhánh, trong đó một nhánh dựa trên CNN và nhánh kia dựa trên mô hình tự chú ý [Wu và cộng sự, 2020b]. Một cách tiếp cận khác là tích hợp các tầng CNN vào các khối Transformer theo cách cho phép mô hình học được cả biểu diễn cục bộ lẫn toàn cục [Wu và cộng sự, 2019; Gulati và cộng sự, 2020].

Chú giải của người dịch: Ràng buộc mềm và ràng buộc cứng

Ràng buộc mềm giống như "khuyến khích" học sinh hỏi bạn ngồi cạnh: vẫn được hỏi bạn ở xa nhưng ít khi làm vậy. *Ràng buộc cứng* giống như quy định "chỉ được trao đổi với bạn cùng bàn": những vị trí ngoài cửa sổ bị loại hẳn (trọng số chú ý bằng 0). Ràng buộc cứng còn giúp tiết kiệm tính toán, vì ta không cần tính điểm chú ý cho các cặp vị trí bị loại.

3. Nhúng vị trí tương đối

Nhúng vị trí tương đối (*relative positional embedding*), còn được gọi là **biểu diễn vị trí tương đối** (*relative positional representation*) (**RPR**), là một cải tiến của phương pháp nhúng vị trí tuyệt đối (*absolute positional embedding*) được dùng trong Transformer chuẩn [Shaw và cộng sự, 2018; Huang và cộng sự, 2018]. Ý tưởng của RPR là mô hình hóa khoảng cách giữa hai vị trí trong một chuỗi, thay vì gán cho mỗi vị trí một biểu diễn cố định. Kết quả là ta có một biểu diễn theo cặp $\text{PE}(i, j)$ cho hai vị trí i và j. Một cách đơn giản để định nghĩa $\text{PE}(i, j)$ là coi nó như một bảng tra cứu (*lookup table*) cho mọi cặp i và j. Cụ thể hơn, gọi $\mathbf{u}_\pi$ là một biểu diễn d chiều ứng với một khoảng cách π cho trước. Dạng của $\text{PE}(i, j)$ trong phương pháp RPR nguyên bản là

$$\text{PE}(i, j) \;=\; \mathbf{u}_{\text{clip}(j-i, k_{\text{rpr}})}, \tag{6.70}$$

trong đó $\text{clip}(x, k_{\text{rpr}})$ là hàm "cắt" x vào trong đoạn $[-k_{\text{rpr}}, k_{\text{rpr}}]$

$$\text{clip}(x, k_{\text{rpr}}) \;=\; \max\{-k_{\text{rpr}}, \min\{x, k_{\text{rpr}}\}\}. \tag{6.71}$$

Như vậy, ta có một mô hình với các tham số

$$\mathbf{U}_{\text{rpr}} \;=\; \begin{bmatrix} \mathbf{u}_{-k_{\text{rpr}}} \\ \vdots \\ \mathbf{u}_0 \\ \vdots \\ \mathbf{u}_{k_{\text{rpr}}} \end{bmatrix}. \tag{6.72}$$

Mặc dù cách viết này hơi không chính thức, ta có thể xem $\mathbf{U}_{\text{rpr}}$ là một ma trận $\in \mathbb{R}^{(2k_{\text{rpr}}+1)\times d}$, và chọn hàng ứng với $\text{clip}(j - i, k_{\text{rpr}})$ khi tính RPR cho i và j cho trước.

Chú giải của người dịch: Ví dụ tính hàm clip

Lấy $k_{\text{rpr}} = 3$. Khi đó chỉ có $2{\cdot}3{+}1 = 7$ vectơ $\mathbf{u}_{-3}, \ldots, \mathbf{u}_3$. Với $i = 1, j = 2$: $j - i = 1$, $\text{clip}(1, 3) = 1$, nên $\text{PE}(1, 2) = \mathbf{u}_1$. Với $i = 1, j = 5$: $j - i = 4$, nhưng $\text{clip}(4, 3) = \max\{-3, \min\{4, 3\}\} = 3$, nên $\text{PE}(1, 5) = \mathbf{u}_3$. Với $i = 5, j = 1$: $j - i = -4$, $\text{clip}(-4, 3) = -3$, nên $\text{PE}(5, 1) = \mathbf{u}_{-3}$. Nghĩa là mọi khoảng cách từ 3 trở lên được "gộp chung" một vectơ. Bạn có thể đối

chiếu với bảng nhỏ ở bên phải Hình 6.10(b). Điểm mấu chốt: biểu diễn chỉ phụ thuộc vào *hiệu* $j - i$ (cách nhau bao xa, về phía nào), không phụ thuộc vào việc i, j đứng ở đâu trong câu.

Dùng phương pháp trên, ta có thể định nghĩa ba mô hình RPR $\text{PE}^q(i,j)$, $\text{PE}^k(i,j)$ và $\text{PE}^v(i,j)$ lần lượt cho truy vấn (*query*), khóa (*key*) và giá trị (*value*). Khi đó, theo dạng của Phương trình (6.17), đầu ra của mô hình tự chú ý tại vị trí i có thể được viết là

$$\begin{aligned}\mathbf{c}_i &= \sum_{j=1}^{m} \alpha_{i,j} \left[\mathbf{h}_j^v + \text{PE}^v(i,j)\right] \\ &= \sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j^v + \sum_{j=1}^{m} \alpha_{i,j} \text{PE}^v(i,j),\end{aligned} \tag{6.73}$$

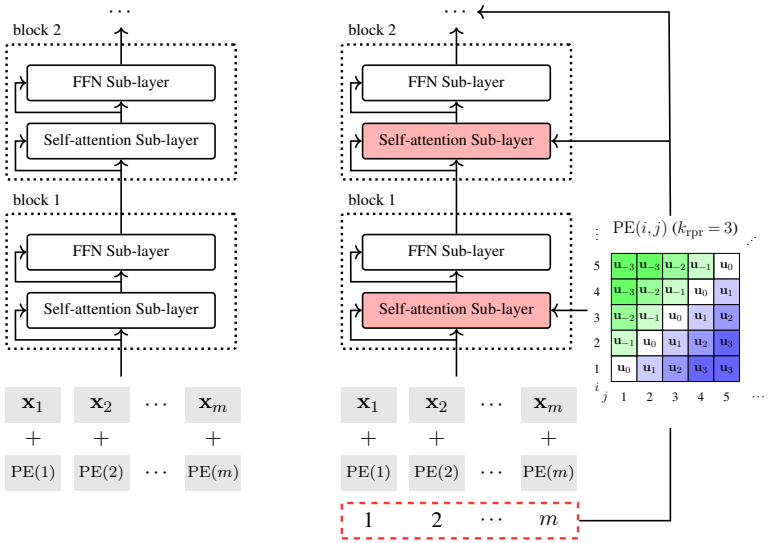
trong đó $\mathbf{h}_j^v$ là vectơ hàng thứ j của $\mathbf{H}^v$. Biểu diễn này gồm hai thành phần: $\sum_{j=1}^{m} \alpha_{i,j} \mathbf{h}_j^v$ là biểu diễn cơ bản, còn $\sum_{j=1}^{m} \alpha_{i,j} \text{PE}^v(i,j)$ là biểu diễn vị trí.

Trọng số chú ý $\alpha_{i,j}$ được tính theo cách thông thường, nhưng có thêm các số hạng $\text{PE}^q(i,j)$ và $\text{PE}^k(i,j)$ được cộng vào mỗi truy vấn và khóa:

$$\alpha_{i,j} = \text{Softmax}\left(\frac{[\mathbf{h}_i^q + \text{PE}^q(i,j)][\mathbf{h}_j^k + \text{PE}^k(i,j)]^\top}{\sqrt{d}}\right). \tag{6.74}$$

Hình 6.10 cho thấy kiến trúc bộ mã hóa Transformer có và không có RPR. Khi dùng RPR, $\text{PE}^q(i,j)$, $\text{PE}^k(i,j)$, $\text{PE}^v(i,j)$ được đưa trực tiếp vào từng tầng con tự chú ý, nhờ vậy ta có thể tận dụng tốt hơn thông tin vị trí cho việc mô hình hóa chuỗi. Lưu ý rằng việc dùng hàm cắt (xem Phương trình (6.71)) làm cho việc mô hình hóa trở nên đơn giản, vì ta không cần phân biệt các khoảng cách tương đối trong các trường hợp $|j - i| \geq k_{\text{rpr}}$. Mô hình dựa trên khoảng cách đã bị cắt này có thể giúp mô hình hóa tốt hơn trong các cửa sổ ngữ cảnh cục bộ.

Các Phương trình (6.73) và (6.74) đưa ra một cách tiếp cận tổng quát cho việc mô hình hóa chuỗi có nhạy với vị trí. Mô hình này có nhiều biến thể. Trong công trình đầu tiên về RPR của Shaw và cộng sự [2018], các biểu diễn vị trí cho truy vấn bị loại bỏ, và mô hình chỉ làm việc với



(a) Transformer without RPR (b) Transformer with RPR

Hình 6.10: Minh họa bộ mã hóa Transformer không có và có biểu diễn vị trí tương đối (RPR). Trong RPR, mỗi cặp vị trí được biểu diễn bằng một vectơ PE(i, j) nhờ một mô hình được tham số hóa bởi $\mathbf{U}_{\text{rpr}}$. PE(i, j) được đưa vào từng tầng con tự chú ý, nhờ đó ta có thể tận dụng thông tin vị trí ở các bước trung gian của quá trình học biểu diễn. (Nhãn trong hình: *block* = khối; *FFN Sub-layer* = tầng con FFN; *Self-attention Sub-layer* = tầng con tự chú ý; (a) Transformer không có RPR; (b) Transformer có RPR.)

$\text{PE}^k(i,j)$ và $\text{PE}^v(i,j)$, như sau

$$\alpha_{i,j} = \text{Softmax}\Big(\frac{\mathbf{h}_i^q[\mathbf{h}_j^k + \text{PE}^k(i,j)]^\top}{\sqrt{d}}\Big). \tag{6.75}$$

Ngược lại, một số biến thể cố gắng cải tiến mô hình RPR khi tính trọng số chú ý nhưng bỏ qua $\text{PE}^v(i,j)$ khi tính biểu diễn giá trị [Dai và cộng sự, 2019; He và cộng sự, 2021]. Thay vì coi RPR là một số hạng cộng thêm vào từng biểu diễn, các nhà nghiên cứu cũng đã khám phá những cách khác để đưa RPR vào Transformer [Huang và cộng sự, 2020b; Raffel và cộng sự, 2020]. Chúng tôi giới thiệu bạn đọc quan tâm tới các bài báo này để biết thêm chi tiết.

6.3.2 Các mô hình sâu

Nhiều hệ thống NLP tiên tiến nhất (*state-of-the-art*) dựa trên các mô hình Transformer sâu. Ví dụ, các mô hình ngôn ngữ lớn (*large language model*) gần đây thường gồm hàng chục tầng Transformer (hay chính xác hơn là hàng trăm tầng), và cho thấy hiệu năng mạnh trên nhiều tác vụ [Ouyang và cộng sự, 2022; Touvron và cộng sự, 2023a]. Bằng cách xếp chồng các tầng Transformer, ta dễ dàng xây dựng được một mô hình sâu. Tuy nhiên, như thường lệ, huấn luyện các mạng nơ-ron rất sâu là việc khó. Một khó khăn xuất phát từ thực tế là mặt sai số (*error surface*) của mạng nơ-ron sâu có tính không lồi (*non-convex*) rất cao và chứa nhiều cực trị địa phương (*local optima*), khiến quá trình huấn luyện dễ bị mắc kẹt. Mặc dù có những thuật toán tối ưu giúp giảm nhẹ vấn đề này, hầu hết các cách tiếp cận thực tế đều dựa vào các phương pháp dựa trên gradient để tối ưu mạng nơ-ron sâu. Kết quả là, việc huấn luyện một mô hình có nhiều tầng Transformer trở nên khó khăn do hiện tượng tiêu biến gradient (*vanishing gradient*) và bùng nổ gradient (*exploding gradient*) trong quá trình lan truyền ngược (*back-propagation*). Ở đây, ta xét một số kỹ thuật để huấn luyện các mô hình Transformer sâu.

Chú giải của người dịch: Tiêu biến và bùng nổ gradient

Khi lan truyền ngược qua L tầng, gradient gần như bị *nhân* liên tiếp với L hệ số (mỗi tầng một hệ số). Nếu mỗi hệ số khoảng 0.9 thì sau 100 tầng còn $0.9^{100} \approx 0.00003$: gradient gần như biến mất, các tầng đầu hầu như

không học được (*tiêu biến*). Nếu mỗi hệ số khoảng 1.1 thì $1.1^{100} \approx 13\,781$: gradient lớn vọt, việc cập nhật tham số trở nên hỗn loạn (*bùng nổ*). Đây cũng giống như hàm mũ a^n ở THPT: với $a < 1$ nó tiến về 0, với $a > 1$ nó tăng rất nhanh.

1. Nhìn lại các kiến trúc Pre-Norm và Post-Norm

Như đã giới thiệu trước đây, một tầng con Transformer là một mạng phần dư (*residual network*), trong đó một "đường tắt" được tạo ra để cộng trực tiếp đầu vào của mạng vào đầu ra của tầng con. Điều này cho phép gradient chảy trực tiếp hơn từ đầu ra về đầu vào, làm giảm nhẹ vấn đề tiêu biến gradient. Nói chung, một kết nối tắt (*residual connection*) trong Transformer được dùng cùng với một đơn vị chuẩn hóa tầng (*layer normalization*) để tạo thành một tầng con. Điều này dẫn tới hai loại kiến trúc, gọi là chuẩn hóa sau (*post-norm*) và chuẩn hóa trước (*pre-norm*). Cụ thể, nhắc lại từ Mục 6.1.4 rằng kiến trúc post-norm có thể được biểu diễn là:

$$\mathbf{z}^l = \text{LNorm}(F^l(\mathbf{z}^{l-1}) + \mathbf{z}^{l-1}), \tag{6.76}$$

trong đó $\mathbf{z}^l$ và $\mathbf{z}^{l-1}$ lần lượt là đầu ra và đầu vào của tầng con l, và $F^l(\cdot)$ là hàm lõi của tầng con này. Kiến trúc pre-norm đưa ánh xạ đồng nhất (*identity mapping*) $\mathbf{z}^{l-1}$ ra ngoài hàm chuẩn hóa tầng, có dạng

$$\mathbf{z}^l = F^l(\text{LNorm}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}. \tag{6.77}$$

Hãy xét sự khác biệt giữa dòng thông tin trong hai kiến trúc này:

- Kiến trúc post-norm ngăn không cho ánh xạ đồng nhất của đầu vào được cộng trực tiếp vào đầu ra cuối cùng của tầng con. Đây không phải một mạng phần dư thực thụ, vì toàn bộ thông tin đều phải đi qua một hàm phi tuyến (tức là đơn vị chuẩn hóa tầng). Do đó, kiến trúc post-norm kém hiệu quả hơn cho lan truyền ngược. Wang và cộng sự [2019a] chỉ ra rằng gradient của hàm mất mát của một mạng Transformer gồm L tầng con theo $\mathbf{z}^l$ được cho bởi

$$\frac{\partial E}{\partial \mathbf{z}^l} = \frac{\partial E}{\partial \mathbf{z}^L} \cdot \prod_{k=l}^{L-1} \frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k} \cdot \prod_{k=l}^{L-1} \left(1 + \frac{\partial F^k(\mathbf{z}^k)}{\partial \mathbf{z}^k}\right), \tag{6.78}$$

  trong đó $\mathbf{z}^L$ là đầu ra của tầng cuối cùng, $\mathbf{v}^k$ là cách viết tắt của $F^k(\mathbf{z}^{k-1})$, và E là sai số được đo bằng một hàm mất mát nào đó.

$\frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k}$ và $\frac{\partial F^k(\mathbf{z}^k)}{\partial \mathbf{z}^k}$ lần lượt là gradient của hàm chuẩn hóa tầng và của hàm lõi. Mặc dù phương trình ở đây trông phức tạp, ta thấy rằng $\prod_{k=l}^{L-1} \frac{\partial \text{LNorm}(\mathbf{v}^k)}{\partial \mathbf{v}^k}$ chỉ đơn giản là tích của $L-l$ thừa số. Điều này có nghĩa là gradient sai số sẽ bị co giãn lặp đi lặp lại khi L lớn lên, dẫn đến nguy cơ tiêu biến hoặc bùng nổ gradient cao hơn ở các mô hình sâu hơn.

- Kiến trúc pre-norm mô tả một mạng nơ-ron phần dư chuẩn, trong đó đầu vào của toàn bộ mạng được cộng trực tiếp vào đầu ra của nó. Ta có thể viết gradient của sai số tại $\mathbf{z}^l$ là:

$$\begin{aligned}\frac{\partial E}{\partial \mathbf{z}^l} &= \frac{\partial E}{\partial \mathbf{z}^L} \cdot \left(1 + \sum_{k=l}^{L-1} \frac{\partial F^k(\text{LNorm}(\mathbf{z}^k))}{\partial \mathbf{z}^k}\right) \\ &= \frac{\partial E}{\partial \mathbf{z}^L} + \frac{\partial E}{\partial \mathbf{z}^L} \cdot \sum_{k=l}^{L-1} \frac{\partial F^k(\text{LNorm}(\mathbf{z}^k))}{\partial \mathbf{z}^k}. \end{aligned} \tag{6.79}$$

Dễ thấy rằng $\frac{\partial E}{\partial \mathbf{z}^l}$ nhận được phản hồi trực tiếp về sai số, vì số hạng đầu tiên ở vế phải (tức là $\frac{\partial E}{\partial \mathbf{z}^L}$) là gradient của đầu ra mô hình, vốn không phụ thuộc vào độ sâu của mạng.

Chú giải của người dịch: Vì sao pre-norm dễ huấn luyện hơn?

Hãy hình dung gradient như một "tín hiệu sửa lỗi" truyền từ tầng trên cùng xuống tầng dưới. Trong Phương trình (6.78) (post-norm), tín hiệu phải đi qua một *tích* gồm $L-l$ thừa số: nếu mỗi thừa số là 0.8 và có 50 thừa số thì tín hiệu chỉ còn $0.8^{50} \approx 0.000014$ lần ban đầu. Trong Phương trình (6.79) (pre-norm), gradient là một *tổng*: số hạng thứ nhất $\frac{\partial E}{\partial \mathbf{z}^L}$ đi thẳng từ đầu ra xuống tầng l mà không bị nhân với gì cả, giống như có một "đường cao tốc" riêng. Vì vậy dù mạng sâu đến đâu, tầng dưới vẫn luôn nhận được một phần tín hiệu nguyên vẹn. (Ký hiệu $\frac{\partial E}{\partial \mathbf{z}}$ là *đạo hàm riêng*: đạo hàm của E theo từng thành phần của vectơ $\mathbf{z}$, khi giữ các thành phần khác cố định.)

Việc dùng kiến trúc pre-norm cũng giúp ích cho quá trình tối ưu trong những bước hạ gradient đầu tiên. Ví dụ, người ta nhận thấy rằng mô hình Transformer pre-norm có thể được huấn luyện với tốc độ học (*learning rate*) lớn hơn ở giai đoạn đầu của huấn luyện, thay vì cần khởi động tốc độ học (*learning rate warmup*) tăng dần từ một giá trị nhỏ [Xiong và cộng sự, 2020].

Mặc dù kiến trúc pre-norm giúp việc tối ưu các mô hình Transformer sâu dễ dàng hơn, ta không thể đơn giản kết luận rằng nó là lựa chọn tốt hơn trong mọi trường hợp so với kiến trúc post-norm. Thực tế, cả hai loại mô hình Transformer post-norm và pre-norm đều đã được dùng thành công trong nhiều ứng dụng. Ví dụ, kiến trúc post-norm được dùng rộng rãi trong các mô hình kiểu BERT, trong khi kiến trúc pre-norm là lựa chọn phổ biến hơn cho các mô hình ngôn ngữ lớn dạng sinh gần đây. Nhìn rộng ra, hai kiến trúc này cung cấp các khuôn mẫu khác nhau để thiết kế mô hình Transformer sâu, mỗi loại có ưu và nhược điểm riêng. Kiến trúc post-norm buộc biểu diễn phải được học qua nhiều hàm phi tuyến hơn, dẫn đến một mô hình có khả năng biểu đạt cao nhưng phức tạp và tương đối khó huấn luyện. Ngược lại, kiến trúc pre-norm giúp việc huấn luyện mô hình Transformer dễ hơn, nhưng có thể kém biểu đạt hơn so với kiến trúc post-norm tương ứng nếu mô hình học được trở nên quá phụ thuộc vào các đường tắt.

Một cải tiến cho các kiến trúc này là kiểm soát mức độ mà ta muốn "bỏ qua" một tầng con. Một cách đơn giản để làm việc này là gán trọng số cho các đường khác nhau thay vì coi chúng như nhau. Ví dụ, có thể đưa vào một hệ số vô hướng cho kết nối tắt để quyết định ta coi trọng kết nối tắt này đến mức nào so với đường đi qua hàm lõi [He và cộng sự, 2016b; Liu và cộng sự, 2020a;b]. Một dạng tổng quát hơn của mô hình này là

$$\mathbf{z}^l \;=\; \text{LNorm}(F^l(\mathbf{z}^{l-1}) + \beta \cdot \mathbf{z}^{l-1}) + \gamma \cdot \mathbf{z}^{l-1}, \tag{6.80}$$

trong đó β là trọng số của ánh xạ đồng nhất bên trong hàm chuẩn hóa tầng, và γ là trọng số của ánh xạ đồng nhất bên ngoài hàm chuẩn hóa tầng. Rõ ràng, cả hai kiến trúc post-norm và pre-norm đều có thể xem là trường hợp đặc biệt của phương trình này. Nghĩa là, nếu $\beta = 1$ và $\gamma = 0$ thì nó trở thành Phương trình (6.76); nếu $\beta = 0$ và $\gamma = 1$ thì nó trở thành Phương trình (6.77)[8]. Mô hình này đưa ra một góc nhìn nhiều nhánh khi xây dựng các khối phần dư. Đầu vào của khối này có thể được tính qua nhiều đường với độ phức tạp mô hình hóa khác nhau. Khi β và γ nhỏ, biểu diễn bị buộc phải học thông qua một mô hình "sâu" với nhiều tầng đơn vị phi tuyến nối tiếp nhau. Ngược lại, khi β và γ lớn, biểu diễn có

[8] **[ND]** Chính xác hơn, với $\beta = 0$, $\gamma = 1$ ta được $\mathbf{z}^l = \text{LNorm}(F^l(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}$, tức là chuẩn hóa tầng đặt sau hàm lõi chứ không phải trước như Phương trình (6.77); tuy vậy, điểm cốt yếu giống pre-norm là ánh xạ đồng nhất $\mathbf{z}^{l-1}$ nằm ngoài hàm chuẩn hóa tầng và được cộng thẳng vào đầu ra.

nhiều khả năng được học bằng một mô hình "nông" với ít tầng hơn. Để xác định các giá trị tối ưu của β và γ, ta có thể gán cho chúng các giá trị cố định dựa trên một số tính chất lý thuyết hoặc hiệu năng của hệ thống trên tập kiểm định (*validation set*), hoặc tính các giá trị này bằng những hàm bổ sung có thể huấn luyện được để làm việc đó [Srivastava và cộng sự, 2015]. Cần nhấn mạnh rằng có thể cân nhắc nhiều loại kiến trúc khác khi thiết kế một tầng con Transformer. Chẳng hạn, có thể đưa thêm nhiều đơn vị chuẩn hóa tầng vào một tầng con [Ding và cộng sự, 2021; Wang và cộng sự, 2022b], hoặc ngược lại, đơn giản là loại bỏ chúng khỏi tầng con [Bachlechner và cộng sự, 2021].

2. Khởi tạo tham số

Cũng như với các mạng nơ-ron sâu khác, có nhiều sự quan tâm tới việc phát triển các phương pháp khởi tạo tham số (*parameter initialization*) cho mô hình Transformer sâu, nhằm giúp quá trình tối ưu diễn ra trong một vùng thuận lợi hơn của không gian tham số. Tuy nhiên, khởi tạo là một chủ đề rộng trong tối ưu các mô hình học máy, và việc bàn tổng quát về chủ đề này nằm ngoài phạm vi của mục này. Ở đây, ta sẽ tập trung vào các phương pháp khởi tạo tham số cụ thể được dùng trong các hệ thống dựa trên Transformer, thay vì các bài toán tối ưu tổng quát.

Mặc dù tham số của một mạng nơ-ron có thể được khởi tạo theo nhiều cách, hầu hết các hệ thống thực tế đều dùng những kỹ thuật đơn giản để đưa ra giá trị ban đầu thích hợp. Chẳng hạn, xét khởi tạo Xavier (*Xavier initialization*) cho một ma trận tham số $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ [Glorot và Bengio, 2010]. Ta định nghĩa một biến η như sau

$$\eta \;=\; \text{gain} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \tag{6.81}$$

trong đó gain là một siêu tham số, mặc định bằng 1. Khi đó, mỗi phần tử của $\mathbf{W}$ có thể được khởi tạo bằng một phân phối đều (*uniform distribution*)

$$W \;\sim\; U(-\eta, \eta) \tag{6.82}$$

hoặc dùng một phân phối Gauss (*Gaussian distribution*)

$$W \;\sim\; \mathcal{N}\big(0, \eta^2\big). \tag{6.83}$$

Chú giải của người dịch: Ví dụ khởi tạo Xavier

Giả sử $d_{\text{in}} = d_{\text{out}} = 512$ và gain $= 1$. Khi đó $\eta = \sqrt{6/1024} \approx 0.077$. Với Phương trình (6.82), mỗi phần tử của $\mathbf{W}$ được chọn ngẫu nhiên, "đều tay" trong khoảng $(-0.077; 0.077)$ (ký hiệu $\sim$ đọc là "được lấy mẫu theo"). Với Phương trình (6.83), mỗi phần tử được lấy theo phân phối chuẩn (hình chuông) có trung bình 0 và độ lệch chuẩn η. Ý tưởng: ma trận càng "to" ($d_{\text{in}} + d_{\text{out}}$ lớn) thì các số khởi tạo càng nhỏ, để tổng của rất nhiều tích không làm giá trị đầu ra phình to hoặc teo nhỏ khi đi qua mỗi tầng.

Phương pháp này có thể dễ dàng điều chỉnh để khởi tạo các mô hình Transformer có số tầng lớn. Một cách tiếp cận phổ biến là tìm một giá trị gain phù hợp hơn, khi tính đến thực tế rằng điều kiện ban đầu tối ưu cho việc tối ưu hóa có thể khác nhau giữa các mạng nơ-ron có độ sâu khác nhau. Ví dụ, ta có thể tăng giá trị gain khi độ sâu của mô hình tăng. Như vậy, gain có thể được định nghĩa là một hàm của độ sâu mạng theo dạng sau

$$\text{gain} \;=\; a \cdot L^b, \tag{6.84}$$

trong đó a là một số vô hướng, và L^b là độ sâu của mạng lũy thừa b. Thông thường, cả a và b đều là số dương, cho thấy xu hướng ưu tiên giá trị tham số ban đầu lớn hơn ở các mô hình sâu hơn. Ví dụ, Wang và cộng sự [2022a] chứng minh rằng bằng cách chọn giá trị phù hợp cho a và b, có thể huấn luyện thành công một mô hình Transformer rất sâu.

Phương trình (6.84) gán cho gain cùng một giá trị cho mọi tầng con. Tuy nhiên, người ta nhận thấy rằng chuẩn (*norm*) của gradient trở nên nhỏ hơn khi tầng con nằm càng xa tầng đầu ra. Việc áp dụng một giá trị gain thống nhất cho toàn bộ mô hình có thể khiến các tầng thấp bị huấn luyện chưa đủ do vấn đề tiêu biến gradient. Vì lý do này, ta có thể phát triển các phương pháp nhạy với vị trí của tầng con trong mạng nơ-ron. Dạng tổng quát của các phương pháp như vậy là

$$\text{gain} \;=\; \frac{a}{l^b}. \tag{6.85}$$

Ở đây l ký hiệu độ sâu của một tầng con. Nếu l lớn hơn (tức là tầng con gần đầu ra hơn), gain sẽ nhỏ hơn và các tham số tương ứng sẽ được đặt giá trị nhỏ hơn. Một ví dụ của phương pháp này có thể tìm thấy trong công trình của Zhang và cộng sự [2019a].

Chú giải của người dịch: So sánh hai cách chọn gain

Lấy $a = 1$, $b = 0.5$. Theo Phương trình (6.84), mạng có $L = 16$ tầng con dùng chung gain $= 16^{0.5} = 4$ cho *mọi* tầng. Theo Phương trình (6.85), mỗi tầng con có gain riêng: tầng $l = 1$ có gain $= 1$, tầng $l = 4$ có gain $= 1/2$, tầng $l = 16$ có gain $= 1/4$. Như vậy tầng càng gần đầu ra thì tham số khởi tạo càng nhỏ.

Tất nhiên, việc áp dụng các phương pháp tổng quát để khởi tạo mạng nơ-ron nhiều tầng sâu cho mô hình Transformer cũng rất đơn giản. Một ví dụ là xét **hằng số Lipschitz** (*Lipschitz constant*) khi khởi tạo tham số; điều này đã được chứng minh giúp cải thiện độ ổn định khi huấn luyện các mô hình sâu [Szegedy và cộng sự, 2014b; Xu và cộng sự, 2020]. Một cách tiếp cận khác là dùng các phương pháp bậc hai (*second-order methods*) để ước lượng giá trị phù hợp của các tham số. Ví dụ, ta có thể tính ma trận Hessian (*Hessian*) của mỗi ma trận tham số để mô hình hóa độ cong của nó [Skorski và cộng sự, 2021].

Chú giải của người dịch: Hằng số Lipschitz và Hessian

Một hàm f có *hằng số Lipschitz* K nếu $|f(x) - f(y)| \leq K\,|x - y|$ với mọi x, y: đầu vào thay đổi một chút thì đầu ra không thể thay đổi quá K lần lượng đó. Ví dụ $f(x) = 3x$ có $K = 3$. Giữ K của mỗi tầng không quá lớn giúp tín hiệu (và gradient) không bị phóng đại dần khi đi qua nhiều tầng. *Ma trận Hessian* là bảng các đạo hàm bậc hai; với hàm một biến nó chính là $f''(x)$ mà bạn đã biết cho biết đồ thị "cong" lên hay xuống và cong nhiều hay ít.

Với các mô hình có số tầng lớn, cũng có thể tiền huấn luyện một số tầng bằng các mô hình nhỏ hơn rồi dùng tham số đã huấn luyện của chúng để khởi tạo mô hình lớn hơn [Chen và cộng sự, 2015]. Nghĩa là, trước tiên ta thu được một ước lượng thô của các tham số theo cách ít tốn kém, rồi tiếp tục quá trình huấn luyện trên toàn bộ mô hình như thường lệ. Các phương pháp này thuộc về một nhóm phương pháp huấn luyện gọi là **tăng trưởng mô hình** (*model growth*) hoặc **tăng trưởng độ sâu** (*depth growth*).

Lấy một ví dụ đơn giản, xét một mô hình Transformer (chẳng hạn bộ mã hóa Transformer) có $2L$ tầng con. Ta có thể huấn luyện mô hình này bằng phương pháp **huấn luyện từ nông đến sâu** (*shallow-to-deep training*) [Li và cộng sự, 2020b]. Trước tiên, ta huấn luyện một mô hình L tầng con (gọi là mô hình nông) theo cách thông thường. Sau đó, ta tạo

một mô hình $2L$ tầng con (gọi là mô hình sâu) bằng cách xếp chồng mô hình nông hai lần, rồi tiếp tục huấn luyện mô hình sâu này. Để xây dựng các mô hình sâu hơn nữa, quy trình này có thể được lặp lại nhiều lần; chẳng hạn, ta bắt đầu với một mô hình L tầng con và thu được một mô hình $L \cdot 2^I$ tầng con sau I lần lặp. Lưu ý rằng nhiều mô hình tiền huấn luyện cũng được dùng theo cách tương tự. Ví dụ, với các phương pháp kiểu BERT, một bộ mã hóa Transformer được huấn luyện trên dữ liệu quy mô lớn, rồi các tham số đã tối ưu được dùng để khởi tạo các hệ thống hạ nguồn (*downstream*).

3. Hợp nhất tầng

Một vấn đề khác khi huấn luyện mô hình Transformer sâu là dự đoán cuối cùng chỉ phụ thuộc vào tầng cuối cùng của mạng nơ-ron. Mặc dù việc dùng kết nối tắt cho phép một tầng cao truy cập trực tiếp vào các tầng thấp hơn, vẫn còn một đường "dài" để truyền thông tin từ dưới lên trên. Một cách đơn giản để giải quyết điều này là tạo ra các kết nối tắt bỏ qua nhiều tầng hơn. Ví dụ, xét một nhóm L tầng con Transformer. Với tầng con ở độ sâu l, ta có thể đưa vào $l-1$ kết nối tắt, mỗi kết nối nối tầng con này với một tầng con phía trước. Bằng cách này, ta xây dựng một mạng kết nối dày đặc (*densely connected network*), trong đó mỗi tầng con nhận đầu vào là đầu ra của tất cả các tầng con trước nó [Huang và cộng sự, 2017a]. Đầu ra của tầng con cuối cùng có thể xem là sự kết hợp của các biểu diễn ở các cấp độ khác nhau.

Theo ký hiệu dùng ở các tiểu mục trước, ta ký hiệu đầu ra của tầng con ở độ sâu l là $\mathbf{z}^l$, và ký hiệu hàm của tầng con là $\text{Layer}^l(\cdot)$. Khi đó, $\mathbf{z}^l$ có thể được biểu diễn là

$$\mathbf{z}^l = \text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{6.86}$$

Ta có thể đơn giản coi $\text{Layer}^l(\cdot)$ là một hàm hợp nhất thông tin từ $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$. Có nhiều lựa chọn khả dĩ cho $\text{Layer}^l(\cdot)$. Ví dụ, một dạng đơn giản của $\text{Layer}^l(\cdot)$ là

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(F^l(\mathbf{Z}^l)), \tag{6.87}$$

$$\mathbf{Z}^l = \phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{6.88}$$

Ở đây $\phi(\cdot)$ nhận các đầu ra của tầng $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ và hợp nhất chúng thành một biểu diễn duy nhất $\mathbf{Z}^l$. Một ví dụ đơn giản của $\phi(\cdot)$ là gộp trung

bình (*average pooling*), tính tổng của $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ rồi chia cho $l-1$. Xem Bảng 6.2 để biết thêm các ví dụ về $\phi(\cdot)$.

Áp dụng một kiến trúc tương tự cho tầng con Transformer, ta cũng có thể xét dạng post-norm

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(\mathbf{Z}^l), \tag{6.89}$$

$$\mathbf{Z}^l = \phi(F^l(\mathbf{z}^{l-1}), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}), \tag{6.90}$$

hoặc dạng pre-norm

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \mathbf{Z}^l, \tag{6.91}$$

$$\mathbf{Z}^l = \phi(\text{LNorm}(F^l(\mathbf{z}^{l-1})), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}). \tag{6.92}$$

Bảng 6.2: Các hàm hợp nhất. FFN$(\cdot)$ = mạng nơ-ron truyền thẳng, $[\cdot]$ = ghép nối các vectơ đầu vào, và $\text{Att}_{\text{self}}(\cdot)$ = hàm tự chú ý. Mọi hàm hợp nhất đều có thể được theo sau bởi một hàm chuẩn hóa tầng; ví dụ, ta có thể viết tổng có trọng số của $\{\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}\}$ là $\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{LNorm}(\sum_{k=1}^{l-1} \text{weight}_k \cdot \mathbf{z}^k)$.

Mục	Hàm
Gộp trung bình	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \frac{1}{l-1}\sum_{k=1}^{l-1} \mathbf{z}^k$
Tổng có trọng số	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \sum_{k=1}^{l-1} \text{weight}_k \cdot \mathbf{z}^k$
Mạng truyền thẳng	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{FFN}([\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}])$
Tự chú ý	$\phi(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) = \text{FFN}([\text{Att}_{\text{self}}(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1})])$

Các mô hình này khá tổng quát. Ví dụ, một tầng con bộ mã hóa post-norm chuẩn có thể thu lại được như một trường hợp đặc biệt của các Phương trình (6.89–6.90), nếu ta bỏ sự phụ thuộc vào các tầng con từ 1 đến $l-2$, và định nghĩa $\phi(\cdot)$ là

$$\phi(F^l(\mathbf{z}^{l-1}), \mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) \;=\; F^l(\mathbf{z}^{l-1}) + \mathbf{z}^{l-1}. \tag{6.93}$$

Chú giải của người dịch: Hợp nhất tầng bằng ví dụ số

Giả sử ta đang ở tầng con $l = 4$ và (cho gọn) mỗi đầu ra chỉ là một số: $z^1 = 2$, $z^2 = 4$, $z^3 = 9$. Gộp trung bình cho $\phi = (2+4+9)/3 = 5$. Tổng có trọng số với weight $= (0.1;\ 0.2;\ 0.7)$ cho $\phi = 0.2 + 0.8 + 6.3 = 7.3$ — tầng gần nhất được coi trọng nhất. Các trọng số này có thể được học trong

quá trình huấn luyện. Nhờ vậy, tầng 4 "nhìn thấy" trực tiếp cả tầng 1 chứ không chỉ tầng 3 như Transformer thông thường.

Các mạng kết nối dày đặc giúp thông tin dễ chảy hơn qua các kết nối trực tiếp giữa các tầng con, nhưng mô hình thu được phức tạp hơn một chút, nhất là khi ta dùng các hàm hợp nhất có tham số. Trong thực tế, ta thường chỉ thêm kết nối dày đặc cho một tập con các tầng con, để toàn bộ mạng không quá dày đặc. Ví dụ, ta có thể chỉ thêm kết nối từ các tầng con thấp tới vài tầng con cuối cùng. Nhờ đó, việc dự đoán có thể được thực hiện với khả năng truy cập trực tiếp vào các cấp độ biểu diễn khác nhau [Wang và cộng sự, 2018a].

4. Chính quy hóa

Trong học máy, chính quy hóa (*regularization*) được dùng để giảm nhẹ hiện tượng quá khớp (*overfitting*) khi huấn luyện các mạng nơ-ron sâu. Vì vậy, việc áp dụng các kỹ thuật chính quy hóa cho mô hình Transformer là điều đương nhiên. Do chủ đề chính quy hóa đã được bàn ở Chương 2, ở đây ta xét một số phương pháp chưa được đề cập trong sách này nhưng đặc biệt hữu ích cho việc huấn luyện mô hình Transformer sâu.

Một cách tiếp cận để chính quy hóa mô hình Transformer sâu là bỏ qua ngẫu nhiên các tầng con hoặc các tầng trong quá trình huấn luyện [Huang và cộng sự, 2016; Pham và cộng sự, 2019]. Trong mỗi lần chạy mô hình, ta giữ lại mỗi tầng con với xác suất ρ và xếp chồng các tầng con được chọn để tạo thành một mô hình mới. Như vậy, về bản chất ta huấn luyện một tập hợp mô hình (*ensemble*) gồm các mạng nơ-ron khác nhau có chung kiến trúc và tham số trên cùng một tập dữ liệu. Bằng cách này, mỗi tầng con học cách hoạt động tương đối độc lập, và hiện tượng quá khớp được giảm bớt nhờ ngăn chặn sự đồng thích nghi (*co-adaptation*) giữa các tầng con. Thực ra, dropout các tầng con (hoặc các tầng) và dropout các nơ-ron về cơ bản là những biến thể của cùng một ý tưởng. Đôi khi, phương pháp mô tả ở đây được gọi là **dropout tầng con** (*sub-layer dropout*) hoặc **dropout tầng** (*layer dropout*).

Tại thời điểm kiểm tra, ta cần kết hợp tất cả các mạng khả dĩ để đưa ra dự đoán. Một cách đơn giản để làm điều này là co giãn lại đầu ra của các thành phần ngẫu nhiên trong mô hình [Li và cộng sự, 2021a]. Ví dụ, giả sử mỗi tầng con có kiến trúc pre-norm. Khi đó, đầu ra của tầng con ở

độ sâu l là

$$\mathbf{z}^l \;=\; \rho \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1}. \tag{6.94}$$

Chú giải của người dịch: Vì sao nhân với ρ khi kiểm tra?

Khi huấn luyện, nhánh $\text{LNorm}(F^l(\cdot))$ chỉ "có mặt" với xác suất ρ; nếu vắng mặt, tầng con chỉ chuyển nguyên $\mathbf{z}^{l-1}$ lên trên. Vậy *trung bình* (kỳ vọng) đóng góp của nhánh này là $\rho \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})) + (1-\rho) \cdot 0$. Khi kiểm tra, ta giữ mọi tầng con nhưng nhân nhánh đó với ρ để đầu ra có cùng "độ lớn trung bình" như lúc huấn luyện. Ví dụ $\rho = 0.8$: nhánh lõi chỉ được tính 80%.

Một ý tưởng khác là chia sẻ tham số giữa các tầng con. Một trong những phương pháp đơn giản nhất là dùng đúng cùng một bộ tham số cho mọi tầng con tương ứng [Dehghani và cộng sự, 2018]. Ví dụ, mọi tầng con FFN có thể dựa trên cùng một mạng truyền thẳng. Cách tiếp cận này có tác dụng chính quy hóa tương tự các phương pháp cộng chuẩn của các ma trận tham số vào hàm mất mát để phạt độ phức tạp của mô hình. Với các hệ thống thực tế, việc dùng kiến trúc chia sẻ có thể mang lại lợi ích đáng kể, vì nó cho phép ta dùng lại cùng một mô hình con để xây dựng một mạng nơ-ron sâu, nhờ đó giảm đáng kể tổng lượng bộ nhớ sử dụng. Ta sẽ bàn thêm về các vấn đề hiệu quả ở Mục 6.4.4.

6.3.3 Các mô hình lấy cảm hứng từ phương pháp số

Một mạng phần dư tính đầu ra của nó bằng tổng của ánh xạ đồng nhất và một phép biến đổi của đầu vào. Mô hình như vậy có thể được hiểu là một rời rạc hóa Euler (*Euler discretization*) của các **phương trình vi phân thường** (*ordinary differential equations*) (**ODE**) [Ee, 2017; Haber và Ruthotto, 2017]. Để minh họa ý tưởng này, ta xét một dạng tổng quát của mạng phần dư

$$\mathbf{z}^l \;=\; f^l\big(\mathbf{z}^{l-1}\big) + \mathbf{z}^{l-1}, \tag{6.95}$$

trong đó $f^l(\mathbf{z}^{l-1})$ ký hiệu một hàm nhận biến đầu vào $\mathbf{z}^{l-1}$ và tạo ra một biến đầu ra trong cùng không gian. Rõ ràng, một tầng con Transformer là trường hợp đặc biệt của phương trình này. Ví dụ, với Transformer pre-norm, ta có $f^l(\cdot) = \text{LNorm}(F^l(\cdot))$.

Để đơn giản ký hiệu, ta viết lại phương trình trên dưới một dạng tương đương

$$\mathbf{z}(l) \;=\; f\big(\mathbf{z}(l-1), l\big) + \mathbf{z}(l-1). \tag{6.96}$$

Ta dùng các ký hiệu $\mathbf{z}(l)$ và $f(\mathbf{z}(\cdot, l))$ để nhấn mạnh rằng $\mathbf{z}(\cdot)$ và $f(\cdot)$ là các hàm của l. Ở đây ta giả sử l là một biến rời rạc. Nếu ta nới lỏng l thành một biến liên tục và $\mathbf{z}(l)$ thành một hàm liên tục của l, thì có thể biểu diễn Phương trình (6.96) là

$$\mathbf{z}(l) \;=\; \triangle l \cdot f\big(\mathbf{z}(l-\triangle l), l\big) + \mathbf{z}(l-\triangle l). \tag{6.97}$$

Phương trình này có thể viết tiếp thành

$$\frac{\mathbf{z}(l) - \mathbf{z}(l-\triangle l)}{\triangle l} \;=\; f\big(\mathbf{z}(l-\triangle l), l\big). \tag{6.98}$$

Lấy giới hạn $\triangle l \to 0$, ta được một ODE

$$\frac{\mathrm{d}\mathbf{z}(l)}{\mathrm{d}l} \;=\; f\big(\mathbf{z}(l), l\big). \tag{6.99}$$

Do đó, ta có thể nói rằng một tầng con Transformer pre-norm (tức các Phương trình (6.96) và (6.95)) là một rời rạc hóa Euler của ODE trên. Đây là một kết quả thú vị! Về bản chất, một tầng con đóng vai trò như một bộ giải ODE (*ODE solver*).

Chú giải của người dịch: Phương trình vi phân và phương pháp Euler

Vế trái của Phương trình (6.98) chính là "tỉ số gia số" $\frac{\triangle y}{\triangle x}$ mà bạn gặp khi học định nghĩa đạo hàm ở lớp 11; cho $\triangle l \to 0$ ta được đạo hàm, đó là Phương trình (6.99). Một *phương trình vi phân* cho biết "tốc độ thay đổi" của một đại lượng thay vì cho chính đại lượng đó. *Phương pháp Euler* giải gần đúng bằng cách đi từng bước nhỏ: giá trị mới = giá trị cũ + bước $\times$ tốc độ. Ví dụ với $\frac{\mathrm{d}z}{\mathrm{d}l} = z, z(0) = 1$ (nghiệm đúng $z = e^l$) và bước 1: $z(1) = 1 + 1 \cdot 1 = 2$, $z(2) = 2 + 1 \cdot 2 = 4$ (nghiệm đúng $e^2 \approx 7.39$; bước càng nhỏ thì càng chính xác). Một tầng con có kết nối tắt làm đúng việc "giá trị cũ $\mathbf{z}^{l-1}$ + thay đổi $f^l(\mathbf{z}^{l-1})$", nên mỗi tầng tương ứng với một bước Euler có độ dài bước bằng 1.

Các Phương trình (6.96) và (6.95) biểu diễn các dạng chuẩn của **phương pháp Euler** (*Euler method*). Phương pháp này tính một ước

lượng mới của nghiệm bằng cách tiến thêm một bước dọc theo l từ ước lượng trước đó. Nói chung, có hai khía cạnh có thể được cân nhắc khi thiết kế các phương pháp số cho ODE.

- **Phương pháp đa bước tuyến tính** (*Linear Multi-step Methods*). Một phương pháp đa bước tuyến tính tính ước lượng hiện tại của nghiệm bằng cách lấy các ước lượng và thông tin đạo hàm từ nhiều bước trước đó. Công thức tổng quát của phương pháp p bước có thể được viết là

$$\mathbf{z}(l) \;=\; \sum_{i=1}^{p} a_i \cdot \mathbf{z}(l-i) + h \sum_{i=1}^{p+1} b_i \cdot f\big(\mathbf{z}(l-i),\, l-i+1\big), \quad (6.100)$$

  trong đó h là độ dài bước ta tiến mỗi lần[9], tức là $\triangle l$ trong các Phương trình (6.97) và (6.98). $\{a_i\}$ và $\{b_i\}$ là các hệ số của các điểm nghiệm và các đạo hàm trong tổ hợp tuyến tính. Với định nghĩa này, ta có thể coi phương pháp Euler là một phương pháp giải ODE một bước, bậc thấp[10].

- **Phương pháp Runge–Kutta (bậc cao)** (*(Higher-order) Runge-Kutta Methods*). Các phương pháp Runge–Kutta (RK) và các biến thể của chúng cho phép tính nghiệm ở bước tiếp theo bằng cách sử dụng các kết quả trung gian khi giải ODE. Kết quả là ta thu được các phương pháp bậc cao hơn nhưng vẫn theo dạng phương pháp một bước, nghĩa là nghiệm ước lượng chỉ phụ thuộc vào $\mathbf{z}(l-1)$ chứ không phụ thuộc vào đầu ra ở nhiều bước trước đó.

Chú giải của người dịch: Ký hiệu $O(h^{p+1})$

$O(h^{p+1})$ đọc là "bậc h^{p+1}": sai số nhỏ cỡ h^{p+1} khi bước h nhỏ. Ví dụ với $h = 0.1$: phương pháp bậc 1 (như Euler) có sai số mỗi bước cỡ $0.1^2 = 0.01$, còn phương pháp bậc 4 có sai số mỗi bước cỡ $0.1^5 = 0.00001$. Bậc càng cao thì giảm bước đi một chút sẽ giảm sai số đi rất nhiều.

[9] Gọi $\{t_0, \cdots, t_i\}$ là các giá trị của biến l tại các bước $\{0, \cdots, i\}$. Trong các phương pháp đa bước tuyến tính, ta giả sử $t_i = t_0 + ih$.

[10] Trong giải tích số, **sai số chặt cụt địa phương** (*local truncation error*) của một phương pháp giải ODE tại một bước được định nghĩa là hiệu giữa nghiệm xấp xỉ tính bằng phương pháp đó và nghiệm đúng. Phương pháp được gọi là có bậc p nếu nó có sai số chặt cụt địa phương là $O(h^{p+1})$.

Thực ra, các phương pháp đa bước tuyến tính, dù không được nhắc tới một cách tường minh, đã được dùng trong các phương pháp hợp nhất tầng bàn ở Mục 6.3.2. Ví dụ, lấy các Phương trình (6.91) và (6.92) cùng một hàm hợp nhất tuyến tính, một tầng con pre-norm có kết nối dày đặc tới mọi tầng con trước đó có thể được biểu diễn là

$$\text{Layer}^l(\mathbf{z}^1, \cdots, \mathbf{z}^{l-1}) \;=\; a_1 \cdot \mathbf{z}^{l-1} + \cdots + a_{l-1} \cdot \mathbf{z}^1 + b_1 \cdot \text{LNorm}(F^l(\mathbf{z}^{l-1})). \tag{6.101}$$

Phương trình này là một trường hợp cụ thể của Phương trình (6.100), trong đó ta đặt $h = 1$ và bỏ đi một số số hạng ở vế phải.

Việc áp dụng các phương pháp Runge–Kutta cho mô hình Transformer cũng rất đơn giản [Li và cộng sự, 2022a]. Cho một ODE như mô tả trong Phương trình (6.99), nghiệm Runge–Kutta hiển bậc p được cho bởi

$$\mathbf{z}(l) = \mathbf{z}(l-1) + \sum_{i=1}^{p} \gamma_i \cdot \mathbf{g}_i, \tag{6.102}$$

$$\mathbf{g}_i = h \cdot f\Big(\mathbf{z}(l-1) + \sum_{j=1}^{i-1} \beta_{i,j} \cdot \mathbf{g}_j,\; l - 1 + \lambda_i \cdot h\Big). \tag{6.103}$$

Ở đây $\mathbf{g}_i$ biểu diễn một bước trung gian, chỉ xuất hiện trong quá trình trên. $\{\gamma_i\}$, $\{\beta_{i,j}\}$ và $\{\lambda_i\}$ là các hệ số được xác định bằng cách dùng chuỗi Taylor (*Taylor series*) của $\mathbf{z}(l)$. Để đơn giản hóa mô hình, ta giả sử cùng một hàm f được dùng cho mọi $\{\mathbf{g}_i\}$. Khi đó, ta bỏ sự phụ thuộc vào số hạng $l - 1 + \lambda_i \cdot h$ trong f, và viết lại Phương trình (6.103) thành

$$\mathbf{g}_i \;=\; h \cdot f\Big(\mathbf{z}(l-1) + \sum_{j=1}^{i-1} \beta_{i,j} \cdot \mathbf{g}_j\Big), \tag{6.104}$$

trong đó $f(\cdot)$ là một hàm không phụ thuộc vào i.

Lấy ví dụ, xét nghiệm Runge–Kutta bậc 4 (RK4)

$$\mathbf{z}(l) = \mathbf{z}(l-1) + \frac{1}{6}(\mathbf{g}_1 + 2\mathbf{g}_2 + 2\mathbf{g}_3 + \mathbf{g}_4), \tag{6.105}$$

$$\mathbf{g}_1 = h \cdot f(\mathbf{z}(l-1)), \tag{6.106}$$

$$\mathbf{g}_2 = h \cdot f(\mathbf{z}(l-1) + \tfrac{1}{2}\mathbf{g}_1), \tag{6.107}$$

$$\mathbf{g}_3 = h \cdot f(\mathbf{z}(l-1) + \tfrac{1}{2}\mathbf{g}_2), \tag{6.108}$$

$$\mathbf{g}_4 = h \cdot f(\mathbf{z}(l-1) + \mathbf{g}_3). \tag{6.109}$$

Các phương trình này định nghĩa một kiến trúc tầng con mới. Ví dụ, bằng cách đặt $h = 1$ và $f(\cdot) = \text{LNorm}(F^l(\cdot))$, ta thu được một tầng con Transformer RK4, như minh họa trong Hình 6.11. Phương pháp này dẫn tới một mô hình sâu, vì mỗi tầng con bao gồm bốn lần tính $f(\cdot)$ nối tiếp nhau. Mặt khác, mô hình thu được lại tiết kiệm tham số, vì ta dùng lại cùng một hàm $f(\cdot)$ bên trong tầng con mà không đưa thêm tham số mới.

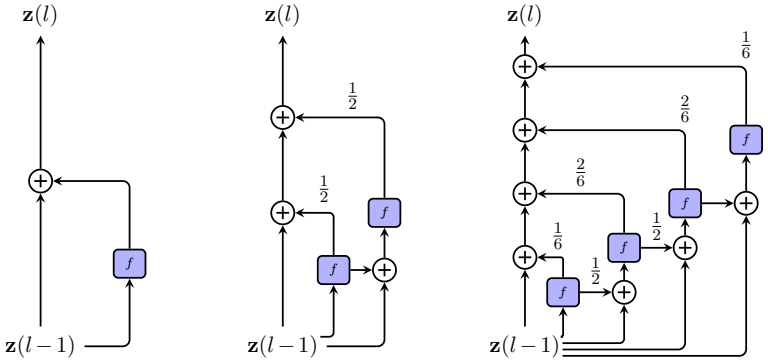
Chú giải của người dịch: RK4 bằng ví dụ số

Lấy lại ví dụ $\frac{\mathrm{d}z}{\mathrm{d}l} = z$, tức $f(z) = z$, với $z(0) = 1$, $h = 1$: $g_1 = 1$; $g_2 = 1 + 0.5 \cdot 1 = 1.5$; $g_3 = 1 + 0.5 \cdot 1.5 = 1.75$; $g_4 = 1 + 1.75 = 2.75$. Suy ra $z(1) = 1 + \frac{1}{6}(1 + 3 + 3.5 + 2.75) \approx 2.708$, rất gần nghiệm đúng $e \approx 2.718$, trong khi một bước Euler chỉ cho 2. Đổi lại, RK4 phải tính f bốn lần cho mỗi bước. Trong Transformer RK4, f là cùng một mạng con được gọi bốn lần, nên số tham số không đổi nhưng lượng tính toán tăng.

Cho đến đây trong tiểu mục này, phần thảo luận của ta tập trung vào thiết kế các kiến trúc Transformer lấy cảm hứng từ các hệ động lực (*dynamical systems*). Mặc dù mô hình ODE cơ bản là liên tục theo độ sâu l, các phương pháp này vẫn theo khuôn khổ chung của mạng nơ-ron, trong đó l được coi là biến rời rạc, và năng lực biểu diễn của mô hình phần lớn được quyết định bởi siêu tham số này. Một cách tiếp cận khác là dùng ODE nơ-ron (*neural ODE*) để nới lỏng "độ sâu" thành một biến liên tục. Bằng cách này, ta có thể thu được một mô hình có độ sâu liên tục để tính nghiệm của ODE. Tuy nhiên, vì thảo luận chi tiết về ODE nơ-ron nằm ngoài phạm vi chương này, chúng tôi giới thiệu bạn đọc quan tâm tới các bài báo liên quan để biết thêm chi tiết [Chen và cộng sự, 2018c; Kidger, 2022; Xiao và cộng sự, 2026].

6.3.4 Các mô hình rộng

Phần lớn các phương pháp ta đã nghiên cứu trong mục này là những ví dụ về việc học và sử dụng mô hình sâu. Một lựa chọn thiết kế khác mà ta thường gặp là xác định độ rộng của mạng nơ-ron. Thông thường, **độ rộng** (*width*) của một mô hình Transformer được định nghĩa là số chiều của biểu diễn tại một vị trí nào đó trong chuỗi đầu vào, tức là tham số d. Tăng độ rộng này là một cách phổ biến để có được một mô hình phức tạp và mạnh hơn. Ví dụ, trong công trình của Vaswani và cộng sự [2017], một



(a) Pre-norm (b) RK2 (c) RK4

Hình 6.11: Minh họa kiến trúc tầng con pre-norm (a) và Runge–Kutta (b và c). $\mathbf{z}(l-1)$ ký hiệu đầu vào của tầng con ở độ sâu l, $\mathbf{z}(l)$ ký hiệu đầu ra của tầng con, và f (trong các hộp màu xanh) ký hiệu hàm $f(\cdot) = \text{LNorm}(F^l(\cdot))$.

mô hình rộng (gọi là Transformer-Big) mang lại cải thiện đáng kể về chất lượng dịch cho các hệ thống dịch máy (*machine translation*). Những mô hình rộng như vậy đã được dùng rộng rãi để nâng cao hiệu năng hệ thống trên các tác vụ quy mô lớn [Lepikhin và cộng sự, 2021; Fedus và cộng sự, 2022b].

Tuy nhiên, việc phát triển các mô hình Transformer rất rộng là khó khăn. Một thách thức lớn là huấn luyện những hệ thống như vậy rất tốn kém về tính toán. Trong khi số tham số của mô hình (kích thước mô hình) tăng tuyến tính theo d, thì độ phức tạp thời gian của mô hình tăng theo bình phương của d (xem Bảng 6.1). Trong một số tác vụ NLP, người ta nhận thấy bằng thực nghiệm rằng công sức huấn luyện cần thiết để đạt hiệu năng thỏa đáng tuân theo một định luật co giãn (*scaling law*) [Kaplan và cộng sự, 2020]. Những kết quả này nhấn mạnh sự cần thiết của các cách cải thiện hiệu quả huấn luyện khi tăng d.

Một phương pháp đơn giản là tăng dần mô hình theo chiều d, thay vì huấn luyện mô hình từ đầu. Giả sử ta có một mô hình ban đầu với ma trận tham số $\mathbf{W}_1$ kích thước $d_1 \times d_1$, chẳng hạn phép biến đổi tuyến tính của mỗi truy vấn hoặc khóa trong một tầng nào đó. Ta có thể huấn luyện mô hình này theo cách thông thường để thu được $\mathbf{W}_1$ đã tối ưu. Sau đó, ta muốn mở rộng mô hình này thành một mô hình rộng hơn, trong đó $\mathbf{W}_1$

được thay bằng ma trận tham số $\mathbf{W}_2$ kích thước $d_2 \times d_2$. Để đơn giản, giả sử $d_2 = kd_1$. Có một vài cách để mở rộng một ma trận $d_1 \times d_1$ thành một ma trận $kd_1 \times kd_1$. Đơn giản nhất có lẽ là dùng $\mathbf{W}_1$ để lấp đầy $\mathbf{W}_2$. Ta có thể viết $\mathbf{W}_2$ dưới dạng

$$\mathbf{W}_2 = \overbrace{\begin{bmatrix} \frac{\mathbf{W}_1}{\rho} & \cdots & \frac{\mathbf{W}_1}{\rho} \\ \vdots & & \vdots \\ \frac{\mathbf{W}_1}{\rho} & \cdots & \frac{\mathbf{W}_1}{\rho} \end{bmatrix}}^{k \text{ lần}} \Bigg\} k \text{ lần}, \tag{6.110}$$

trong đó ρ là một siêu tham số dùng để kiểm soát chuẩn của $\mathbf{W}_2$. Ví dụ, nếu $\rho = k$ thì $\mathbf{W}_2$ sẽ có cùng chuẩn l_1 với $\mathbf{W}_1$. Phương trình trên cung cấp một điểm xuất phát tốt cho việc huấn luyện mô hình rộng, và ta có thể huấn luyện $\mathbf{W}_2$ như thường lệ sau khi khởi tạo. Quy trình này có thể được lặp lại nhiều lần để xây dựng mô hình có độ rộng tùy ý. Cả phương pháp này lẫn phương pháp tăng trưởng độ sâu mô tả ở Mục 6.3.2 đều là những trường hợp cụ thể của phương pháp tổng quát tăng trưởng mô hình. Nói cách khác, ta có thể thu được một mô hình lớn hơn bằng cách mở rộng một mô hình nhỏ theo chiều dọc hoặc chiều ngang, hoặc cả hai. Các phương pháp khác để biến đổi $\mathbf{W}_1$ thành $\mathbf{W}_2$ bao gồm những phương pháp xét tới các tính chất toán học khác của phép biến đổi [Chen và cộng sự, 2015]. Các mô hình này có thể xếp vào nhóm mạng nơ-ron tái sử dụng (*reusable neural networks*), nơi ta quan tâm tới các mô hình và thuật toán để chuyển tham số từ mô hình nhỏ sang mô hình lớn hơn (đáng kể) [Wang và cộng sự, 2023b].

Chú giải của người dịch: Ví dụ mở rộng ma trận

Lấy $k = 2$, $\mathbf{W}_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ ($d_1 = 2$) và $\rho = 2$. Khi đó $\mathbf{W}_2$ là ma trận 4×4 gồm 2×2 khối, mỗi khối là $\mathbf{W}_1/2 = \begin{bmatrix} 0.5 & 1 \\ 1.5 & 2 \end{bmatrix}$. Nếu đầu vào mới cũng là đầu vào cũ được "lặp đôi", ví dụ $[x_1, x_2, x_1, x_2]$, thì mỗi nửa đầu ra của $\mathbf{W}_2$ đúng bằng đầu ra cũ của $\mathbf{W}_1$ (vì hai khối, mỗi khối chia 2, cộng lại). Như vậy mô hình rộng "kế thừa" được những gì mô hình hẹp đã học. (Chuẩn l_1 của ma trận ở đây có thể hiểu là tổng giá trị tuyệt đối lớn nhất theo một cột; mỗi cột của $\mathbf{W}_2$ gồm k khối chia cho $\rho = k$, nên tổng này không đổi.)

Khó khăn thứ hai khi xây dựng một mô hình Transformer rộng là yêu

cầu bộ nhớ lớn. Do mạng truyền thẳng thường có tầng ẩn lớn hơn các phần khác của mô hình, nó cần tương đối nhiều bộ nhớ hơn khi mô hình trở nên rộng hơn. Xét mạng truyền thẳng mô tả ở Mục 6.1.5

$$\begin{aligned}\mathbf{H}_{\text{out}} &= \text{FFN}(\mathbf{H}_{\text{in}}) \\ &= \text{ReLU}(\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h + \mathbf{b}_h) \cdot \mathbf{W}_f + \mathbf{b}_f,\end{aligned} \tag{6.111}$$

trong đó $\mathbf{W}_h \in \mathbb{R}^{d \times d_{\text{ffn}}}$ và $\mathbf{W}_f \in \mathbb{R}^{d_{\text{ffn}} \times d}$ là các tham số của các phép biến đổi tuyến tính. d_{ffn} thường lớn hơn d vài lần. Do đó, $\mathbf{W}_h$ và $\mathbf{W}_f$ sẽ chiếm phần lớn kích thước mô hình nếu d và d_{ffn} có giá trị rất lớn.

Trong một số trường hợp, kích thước của mạng truyền thẳng có thể vượt quá dung lượng bộ nhớ của một thiết bị đơn lẻ. Vấn đề này có thể được giải quyết bằng các mô hình **hỗn hợp chuyên gia** (*mixture-of-experts*) (**MoE**) [Shazeer và cộng sự, 2017]. Một mô hình MoE gồm M mô hình chuyên gia $\{e_1(\cdot), \cdots, e_M(\cdot)\}$. Cho một đầu vào $\mathbf{h}_{\text{in}} \in \mathbb{R}^d$, mỗi mô hình chuyên gia tạo ra một đầu ra $e_k(\mathbf{h}_{\text{in}})$. Đầu ra của mô hình MoE là một tổ hợp tuyến tính của $\{e_1(\mathbf{h}_{\text{in}}), \cdots, e_M(\mathbf{h}_{\text{in}})\}$, cho bởi

$$\mathbf{h}_{\text{out}} \;=\; \sum_{i=1}^{M} g_i(\mathbf{h}_{\text{in}}) \cdot e_i(\mathbf{h}_{\text{in}}), \tag{6.112}$$

trong đó $g(\cdot)$ là một mô hình cổng (*gating model*) (còn gọi là **mô hình định tuyến** (*routing model*)). Đầu ra của nó là một vectơ $g(\mathbf{h}_{\text{in}}) = \begin{bmatrix} g_1(\mathbf{h}_{\text{in}}) & \cdots & g_M(\mathbf{h}_{\text{in}}) \end{bmatrix}$, trong đó mỗi phần tử $g_i(\mathbf{h}_{\text{in}})$ cho biết trọng số của mô hình chuyên gia tương ứng. Trong nhiều ứng dụng, người ta giả sử $g(\mathbf{h}_{\text{in}})$ là một vectơ thưa (*sparse vector*). Điều này có nghĩa là chỉ một số ít mô hình chuyên gia tham gia vào việc tính đầu ra. Một dạng $g(\mathbf{h}_{\text{in}})$ được dùng rộng rãi là dùng tầng Softmax

$$g(\mathbf{h}_{\text{in}}) \;=\; \text{Softmax}(\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g), \tag{6.113}$$

trong đó $\mathbf{W}_g \in \mathbb{R}^{d \times M}$ là ma trận tham số của tầng này. Để buộc $g(\mathbf{h}_{\text{in}})$ thưa, ta có thể chọn k phần tử lớn nhất (top-k) của $g(\mathbf{h}_{\text{in}})$, tức là đặt các phần tử không thuộc top-k bằng 0. Một phương pháp khác là trước tiên chọn top-k trên $\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g$, rồi chuẩn hóa k phần tử đó bằng hàm Softmax.

Gọi π là tập chỉ số của k mô hình chuyên gia được chọn. Mô hình MoE với định tuyến top-k có dạng sau

$$\mathbf{h}_{\text{out}} \;=\; \sum_{i \in \pi} g_i(\mathbf{h}_{\text{in}}) \cdot e_i(\mathbf{h}_{\text{in}}). \tag{6.114}$$

Chú giải của người dịch: MoE bằng ví dụ

Hãy hình dung một bệnh viện có $M = 4$ bác sĩ chuyên khoa (các "chuyên gia") và một quầy tiếp đón (mô hình cổng). Mỗi bệnh nhân (đầu vào) chỉ được chuyển tới $k = 2$ bác sĩ phù hợp nhất, chứ không phải gặp cả bốn. Bằng số: giả sử $\mathbf{h}_{\text{in}} \cdot \mathbf{W}_g = (2; 0; 1; -1)$. Chọn top-2 là chuyên gia 1 và 3 (điểm 2 và 1), nên $\pi = \{1, 3\}$. Chuẩn hóa bằng Softmax trên hai điểm này: $g_1 = \frac{e^2}{e^2+e^1} \approx 0.73$, $g_3 \approx 0.27$. Đầu ra là $0.73 \cdot e_1(\mathbf{h}_{\text{in}}) + 0.27 \cdot e_3(\mathbf{h}_{\text{in}})$; chuyên gia 2 và 4 hoàn toàn không phải chạy, nhờ đó tiết kiệm tính toán dù tổng số tham số rất lớn.

Một ưu điểm của cách tiếp cận này là ta có thể phân bổ các mô hình chuyên gia khác nhau lên các bộ xử lý khác nhau, giúp có thể thực thi các mô hình này trên các máy tính toán song song. Trong mỗi lần chạy mô hình MoE, dù trong huấn luyện hay suy luận (*inference*), ta chỉ cần kích hoạt và sử dụng k mô hình chuyên gia thay vì tất cả. Bằng cách này, cách tiếp cận MoE tự động học một mô hình thưa bằng cách giới hạn số mô hình chuyên gia hoạt động mỗi lần trong huấn luyện và suy luận. Độ thưa được quyết định bởi siêu tham số k; chẳng hạn, giá trị k nhỏ dẫn tới một mô hình thưa, còn giá trị k lớn dẫn tới một mô hình dày đặc.

Hãy quay lại thảo luận về Phương trình (6.111). Việc áp dụng cách tiếp cận MoE cho mạng nơ-ron truyền thẳng là đơn giản. Để đơn giản hóa thảo luận, xét phép biến đổi tuyến tính của tầng thứ nhất như trong Phương trình (6.111), tức là $\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h$. Ta có thể xấp xỉ $\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h$ dưới dạng MoE

$$\begin{aligned}\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h &\approx \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot e_i(\mathbf{H}_{\text{in}}) \\ &= \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot [\mathbf{H}_{\text{in}} \cdot \mathbf{W}_h^i]. \end{aligned} \tag{6.115}$$

Ở đây $\mathbf{W}_h$ được chia thành M lát (hay ma trận con) $\{\mathbf{W}_h^1, \cdots, \mathbf{W}_h^M\}$, viết là

$$\mathbf{W}_h = \begin{bmatrix} \mathbf{W}_h^1 & \cdots & \mathbf{W}_h^M \end{bmatrix}. \tag{6.116}$$

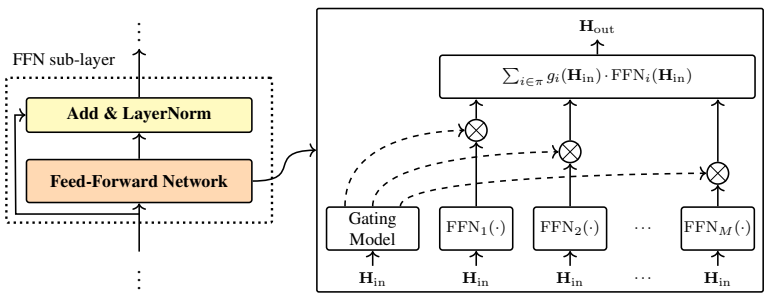
Vì vậy, mỗi mô hình chuyên gia $e_i(\mathbf{H}_{\text{in}}) = \mathbf{H}_{\text{in}} \cdot \mathbf{W}_h^i$ giải một bài toán con của phép ánh xạ tuyến tính ban đầu, và Phương trình (6.115) có thể được xem là một lời giải theo kiểu chia để trị (*divide-and-conquer*) cho bài toán nhân ma trận.

Tất nhiên, ta có thể coi bất kỳ mạng nơ-ron truyền thẳng nào là một

mô hình chuyên gia, dẫn tới mô hình sau

$$\mathbf{H}_{\text{out}} = \sum_{i \in \pi} g_i(\mathbf{H}_{\text{in}}) \cdot \text{FFN}_i(\mathbf{H}_{\text{in}}), \tag{6.117}$$

trong đó $\text{FFN}_i(\cdot)$ là một mạng nơ-ron truyền thẳng "nhỏ" có cùng dạng với Phương trình (6.111). Mô hình này được minh họa bằng một ví dụ trong Hình 6.12. Trong các cài đặt thực tế, mọi mô hình chuyên gia này có thể chạy song song trên các thiết bị khác nhau, nên hệ thống thu được rất hiệu quả.



Hình 6.12: Minh họa mô hình MoE áp dụng cho một tầng con FFN. Có M mạng FFN (gọi là các mô hình chuyên gia) và một mô hình cổng. Mỗi FFN được gán trọng số bởi mô hình cổng. Đầu ra của mô hình là tổng các đầu ra có trọng số của top-k FFN (ký hiệu bởi π). Vì các FFN này hoạt động độc lập và có thể đặt trên các thiết bị tính toán khác nhau, mô hình có thể dễ dàng mở rộng quy mô khi M lớn hơn. (Nhãn trong hình: *FFN sub-layer* = tầng con FFN; *Add & LayerNorm* = cộng và chuẩn hóa tầng; *Feed-Forward Network* = mạng truyền thẳng; *Gating Model* = mô hình cổng.)

Lưu ý rằng, từ góc nhìn học máy, MoE là một cách tiếp cận tổng quát để kết hợp các mạng nơ-ron khác nhau, mỗi mạng được phát triển để giải quyết một khía cạnh khác nhau của bài toán [Yuksel và cộng sự, 2012; Masoudnia và Ebrahimpour, 2014]. Ứng dụng ở đây chỉ là một trường hợp đặc biệt của khuôn khổ tổng quát của MoE. Cách tiếp cận này cũng thường được dùng để cải thiện hiệu năng tổng thể của các bộ dự đoán, điều có thể được bàn trong lĩnh vực học tập hợp (*ensemble learning*) [Zhou, 2012a].

Một khó khăn khác khi phát triển các mô hình Transformer khổng lồ là sự bất ổn định khi huấn luyện (*training instability*). Cũng như nhiều mạng

nơ-ron đặc biệt lớn khác, việc tối ưu trực tiếp một mô hình Transformer có hàng tỉ tham số có thể khiến mô hình bị mắc kẹt ở các cực tiểu địa phương (*local minima*), và đôi khi gây ra những đột biến (*spike*) lớn của hàm mất mát trong quá trình huấn luyện [Lepikhin và cộng sự, 2021; Fedus và cộng sự, 2022b; Chowdhery và cộng sự, 2022]. Ngay cả khi tinh chỉnh cẩn thận siêu tham số, chiến lược huấn luyện và giá trị tham số ban đầu, vẫn thường gặp những tình huống phải khởi động lại việc huấn luyện từ một điểm lưu (*checkpoint*) trước đó để thoát khỏi các vùng tối ưu bất ổn định. Một trong những lý do của khó khăn huấn luyện này là các cài đặt chuẩn của các phép toán đại số tuyến tính, như nhân ma trận, có thể trở nên bất ổn định số (*numerically unstable*) khi thao tác trên các vectơ và ma trận cực lớn. Do đó, điều cốt yếu là cải thiện độ ổn định huấn luyện bằng cách dùng các phương pháp vững về mặt số học và các kỹ thuật độ chính xác hỗn hợp (*mixed precision*).

Chú giải của người dịch: Bất ổn định số và độ chính xác hỗn hợp

Máy tính chỉ lưu số thực với một số chữ số hữu hạn. Ví dụ, với định dạng 16 bit (FP16), số lớn nhất biểu diễn được chỉ khoảng 65 504; cộng dồn hàng triệu tích trong một phép nhân ma trận lớn có thể vượt ngưỡng này (tràn số) hoặc làm mất các chữ số nhỏ (sai số làm tròn). *Độ chính xác hỗn hợp* nghĩa là phần lớn phép tính dùng định dạng ít bit (nhanh, tiết kiệm bộ nhớ), còn những chỗ nhạy cảm (như cộng dồn, cập nhật tham số) dùng định dạng nhiều bit hơn (FP32) để giữ độ chính xác.

6.4 Các mô hình hiệu quả

Hiệu quả là một yếu tố quan trọng cần cân nhắc trong nhiều ứng dụng thực tế của mô hình Transformer. Ví dụ, ta có thể muốn chạy và/hoặc huấn luyện một mô hình Transformer trong giới hạn về bộ nhớ và thời gian. Hiệu quả không phải là một vấn đề đơn lẻ mà bao gồm cả một loạt bài toán. Dù các bài toán này có thể được phân loại theo nhiều cách khác nhau, có hai khía cạnh cơ bản mà ta có thể xem xét trong một bài toán về hiệu quả.

- **Hiệu quả về thời gian và không gian.** Với một bài toán cho trước, ta muốn mô hình nhỏ và nhanh, trong khi vẫn giải bài toán chính

xác nhất có thể. Ví dụ, trong một số ứng dụng dịch máy, ta có thể xây dựng một mô hình với số lượng tham số nhỏ để vừa với bộ nhớ hạn chế, và có thể phát triển một thuật toán tìm kiếm nhanh để đạt được việc dịch với độ trễ (*latency*) thấp. Một khó khăn thực tế ở đây là việc cải thiện hiệu quả thường dẫn đến dự đoán kém hơn. Trong nhiều trường hợp, ta cần tìm một sự đánh đổi giữa hiệu quả và độ chính xác.

- **Khả năng mở rộng (*scalability*).** Khi quy mô bài toán tăng lên, ta mong muốn chi phí tính toán phát sinh thêm là nhỏ nhất. Ví dụ, việc huấn luyện một mạng nơ-ron được gọi là hiệu quả nếu thời gian cần để tối ưu nó vẫn ngắn ở mức hợp lý khi khối lượng dữ liệu huấn luyện tăng lên. Một thước đo hiệu quả khác là lượng tài nguyên tiêu tốn khi xử lý những đầu vào lớn hơn. Chẳng hạn, một hệ thống dịch máy là kém hiệu quả trong việc dịch các câu dài nếu lượng bộ nhớ chiếm dụng (*memory footprint*) và độ trễ tăng theo hàm mũ với số từ đầu vào.

Trong mục này, chúng ta sẽ không bàn về mọi vấn đề liên quan đến hiệu quả, vốn là một chủ đề rất rộng. Thay vào đó, ta xem xét các phương pháp hiệu quả được dùng rộng rãi cho việc mô hình hóa và sinh chuỗi dựa trên Transformer; một số trong đó là những cải tiến về kiến trúc mô hình, còn một số là các phương pháp không phụ thuộc kiến trúc và cũng có thể dùng trong các hệ thống khác. Phần lớn nội dung ở đây tập trung vào việc phát triển các mô hình Transformer gọn nhẹ và nhanh, tương đối vững vàng trước các chuỗi đầu vào và đầu ra dài.

Nói chung, cùng một phương pháp tối ưu có thể được áp dụng cho các mô-đun khác nhau của một hệ thống dựa trên Transformer. Để đơn giản hóa thảo luận, trong mục này chúng ta chủ yếu xét các tầng con tự chú ý và các tầng con FFN. Tuy vậy, thảo luận của chúng ta mang tính tổng quát và các phương pháp được trình bày ở đây có thể áp dụng cho các phần khác của một mô hình Transformer, chẳng hạn các tầng con chú ý chéo.

Chú giải của người dịch: Vì sao "hiệu quả" lại quan trọng đến vậy?

Một mô hình ngôn ngữ lớn có thể có hàng chục tỷ tham số. Nếu mỗi tham số lưu bằng 2 byte, thì riêng việc chứa 10 tỷ tham số đã cần khoảng 20 GB bộ nhớ, chưa kể bộ nhớ cho các phép tính trung gian. Vì vậy, các kỹ thuật làm mô hình "nhẹ hơn" (tốn ít bộ nhớ) và "nhanh hơn" (ít phép tính hơn) quyết định việc mô hình có chạy được trên một máy cụ thể hay không, và người dùng phải chờ bao lâu để nhận câu trả lời.

6.4.1 Chú ý thưa

Trên thực tế, các cơ chế chú ý dùng trong Transformer rất tốn thời gian, đặc biệt khi chuỗi đầu vào dài. Để minh họa, xét một bộ giải mã Transformer dự đoán một phân phối trên bộ từ vựng tại mỗi bước thời gian khi đã biết các từ đứng trước. Giả sử chuỗi do bộ giải mã sinh ra có độ dài n và đầu vào của một tầng con tự chú ý là một ma trận $n \times d$ là $\mathbf{S}$. Trước hết, $\mathbf{S}$ được biến đổi tuyến tính để thu được các truy vấn $\mathbf{S}^q \in \mathbb{R}^{n\times d}$, các khóa $\mathbf{S}^k \in \mathbb{R}^{n\times d}$ và các giá trị $\mathbf{S}^v \in \mathbb{R}^{n\times d}$. Để đơn giản ký hiệu trong tiểu mục này, ta dùng $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ để lần lượt biểu diễn $\mathbf{S}^q$, $\mathbf{S}^k$ và $\mathbf{S}^v$.

Khi đó, đầu ra của tầng con tự chú ý có thể được tính bằng

$$\text{Att}_{\text{self}}(\mathbf{S}) \;=\; \mathbf{AV}, \tag{6.118}$$

trong đó $\mathbf{A}$ là một ma trận chú ý (*attention matrix*) kích thước $n \times n$, hay bản đồ chú ý (*attention map*)

$$\mathbf{A} \;=\; \text{Softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \mathbf{M}\right). \tag{6.119}$$

Ở đây, $\mathbf{M}$ là một ma trận mặt nạ dùng để ngăn mô hình chú ý tới ngữ cảnh tương lai tại mỗi vị trí. Cụ thể, với vị trí i, $M(i,j) = 0$ nếu $j \le i$, và $M(i,j) = -\infty$ trong trường hợp ngược lại. Cả độ phức tạp thời gian lẫn độ phức tạp không gian của tầng con tự chú ý đều tăng theo bình phương của n[11]. Do đó, cơ chế tự chú ý chuẩn trở nên quá tốn kém về tính toán khi n lớn.

[11] Chính xác hơn, lượng bộ nhớ chiếm dụng là $n^2 + n \cdot d$, trong đó số hạng n^2 chiếm ưu thế khi $n \gg d$.

Chú giải của người dịch: Vì sao chi phí tăng theo n^2?

Ma trận $\mathbf{A}$ có n hàng và n cột: mỗi phần tử $A(i,j)$ cho biết vị trí i "chú ý" tới vị trí j bao nhiêu. Với chuỗi 1 000 token, ta phải tính và lưu $1\,000 \times 1\,000 = 10^6$ trọng số; với chuỗi 100 000 token thì con số này là 10^{10} — gấp 10 000 lần, dù chuỗi chỉ dài gấp 100 lần. Đây chính là ý nghĩa của "tăng theo bình phương". Ma trận mặt nạ $\mathbf{M}$ cộng $-\infty$ vào các ô $j > i$; sau Softmax, $e^{-\infty} = 0$, nên các vị trí tương lai nhận trọng số bằng 0.

Cách cài đặt chuẩn của mô hình trên dựa vào các phép tính trên ma trận dày đặc, chẳng hạn các phép nhân trong Phương trình (6.118) và (6.119). Một cách tiếp cận phổ biến để giảm cả lượng bộ nhớ chiếm dụng lẫn số phép toán dấu phẩy động là làm thưa (*sparsification*). Bằng cách giả sử $\mathbf{A}$ là một ma trận thưa (*sparse matrix*), chỉ một phần $\varrho \cdot n^2$ trong số các phần tử của nó còn khác 0, trong đó ϱ là **tỷ lệ thưa** (*sparsity ratio*). Việc dùng các biểu diễn ma trận thưa giảm mạnh yêu cầu về bộ nhớ. Hơn nữa, việc tính $\frac{\mathbf{QK}^\top}{\sqrt{d}}$ và $\mathbf{AV}$ trở nên hiệu quả hơn, vì mô hình chỉ xử lý một tập con nhỏ các vị trí liên quan.

Cho một vị trí i, ta định nghĩa **trường chú ý** (*attention field*) π_i là tập các vị trí được xem xét khi tính biểu diễn tại vị trí này. Do đó, ta chỉ cần tính chú ý tích vô hướng giữa vị trí i đã cho và mỗi vị trí $j \in \pi_i$. Điều này dẫn đến một ma trận chú ý thưa $\mathbf{A}'$, trong đó

$$A'(i,j) \;=\; \begin{cases} a_{i,j} & j \in \pi_i \text{ và } j \le i \\ 0 & \text{trường hợp khác} \end{cases}, \tag{6.120}$$

với $a_{i,j}$ là một trọng số khác 0. Một cách cài đặt đơn giản cho mô hình này chỉ cần sửa đổi nhẹ $\mathbf{M}$, dẫn đến một biến mặt nạ mới $\mathbf{M}'$

$$M'(i,j) \;=\; \begin{cases} 0 & j \in \pi_i \text{ và } j \le i \\ -\infty & \text{trường hợp khác} \end{cases}. \tag{6.121}$$

Trong các cài đặt thực tế, một cách hiệu quả hơn là dùng các phép toán thưa cho $\mathbf{QK}^\top$ và $\mathbf{A}'\mathbf{V}$ bằng cách lần lượt xét $\mathbf{M}'$ và $\mathbf{A}'$. Nghĩa là, ta chỉ tính cho những cặp có trọng số chú ý khác 0 và bỏ qua phần còn lại.

Chú giải của người dịch: Ví dụ nhỏ về trường chú ý

Giả sử $n = 5$ và ta cho mỗi vị trí chỉ được nhìn chính nó và một vị trí liền trước: $\pi_i = \{i-1, i\}$. Khi đó, ở hàng $i = 4$ của $\mathbf{M}'$, chỉ hai ô $j = 3$ và $j = 4$ bằng 0, ba ô còn lại bằng $-\infty$. Mỗi hàng của $\mathbf{A}'$ chỉ có tối đa 2 số khác 0 thay vì tối đa 5 — tổng cộng 9 ô khác 0 thay vì 15 ô của ma trận chú ý "tam giác dưới" đầy đủ. Với n rất lớn, nếu mỗi hàng chỉ giữ một số cố định w ô, tổng số ô cần tính chỉ khoảng $w \cdot n$ thay vì cỡ n^2.

Có thể xét một số cách tiếp cận cho tự chú ý thưa.

- **Chú ý dựa trên khoảng (*span-based attention*)/Chú ý cục bộ (*local attention*).** Như đã bàn ở Mục 6.3.1, trong nhiều trường hợp việc sử dụng ngữ cảnh trong mô hình hóa chuỗi mang tính cục bộ. Ý tưởng cơ bản của chú ý cục bộ là giới hạn sự chú ý trong một vùng cục bộ của chuỗi đầu vào. Khi đó ta có thể viết π_i là

$$\pi_i \ = \ [a_i^l, a_i^r], \tag{6.122}$$

  trong đó a_i^l và a_i^r là đầu trái và đầu phải của π_i. Đại lượng $a_i^r - a_i^l + 1$ xác định kích thước của vùng, nên ta có thể dùng nó để điều khiển độ thưa của mô hình chú ý; chẳng hạn, nếu $a_i^r - a_i^l + 1 \ll n$, mô hình sẽ rất thưa. a_i^l và a_i^r có thể thu được bằng các phương pháp heuristic hoặc các phương pháp học máy. Bạn đọc có thể tham khảo các bài báo liên quan để biết thêm chi tiết [Luong và cộng sự, 2015; Sperber và cộng sự, 2018; Yang và cộng sự, 2018a; Sukhbaatar và cộng sự, 2019]. Xem Hình 6.13 (b) để thấy minh họa về chú ý cục bộ.

- **Chú ý theo khối (*chunked attention*).** Trong phương pháp này, ta chia một chuỗi thành các khối và chạy mô hình chú ý trên từng khối [Parmar và cộng sự, 2018; Qiu và cộng sự, 2020a]. Cho một chuỗi $\{1, \cdots, n\}$, ta định nghĩa $\{\text{chunk}_1, \cdots, \text{chunk}_q\}$ là một cách phân đoạn chuỗi. Một khối có thể được biểu diễn dưới dạng một khoảng

$$\text{chunk}_k \ = \ [c_k^l, c_k^r]. \tag{6.123}$$

  Ở bước chú ý, ta coi mỗi khối là một chuỗi và thực hiện tự chú ý trên nó như thường lệ. Nói cách khác, biểu diễn tại vị trí i được tính chỉ bằng ngữ cảnh trong khối mà i thuộc về. Theo nghĩa này, mô hình

này có thể được xem như một biến thể của chú ý cục bộ. Hình 6.13 (c) minh họa mô hình này. Một vấn đề còn lại là phân đoạn chuỗi như thế nào. Có một vài cách để làm điều này. Ví dụ, như đã bàn ở Mục 6.2.4, ta có thể phân đoạn theo góc nhìn ngôn ngữ học, chia chuỗi thành các đơn vị có cơ sở ngôn ngữ học. Trong các hệ thống thực tế, đôi khi thuận tiện hơn nếu chia chuỗi thành các khối có độ dài bằng nhau. Như vậy, độ thưa của mô hình được điều khiển bởi kích thước các khối; chẳng hạn, dùng các khối nhỏ hơn sẽ cho một mô hình chú ý thưa hơn.

- **Chú ý theo bước trượt (*strided attention*).** Vì phương pháp chú ý theo khối áp đặt một cách phân đoạn cứng lên chuỗi đầu vào, nó có thể mất khả năng học biểu diễn từ các đầu vào ở những khối khác nhau. Một cách khác để đạt được chú ý theo khối là cho phép các khối chồng lấn nhau [Child và cộng sự, 2019; Beltagy và cộng sự, 2020; Ainslie và cộng sự, 2020]. Cách tiếp cận này tương tự họ các phương pháp thường được dùng để áp dụng một mô hình cục bộ lên dữ liệu 1D hoặc 2D nhằm sinh ra đầu ra có cùng hình dạng. Giống như CNN, ta dùng một cửa sổ ngữ cảnh để biểu diễn trường đầu vào của mô hình chú ý. Cửa sổ ngữ cảnh trượt dọc theo chuỗi, mỗi lần tiến lên một bước có kích thước $stride$ (*bước trượt*). Trong trường hợp đặc biệt, nếu $stride$ bằng kích thước cửa sổ ngữ cảnh, mô hình này quy về mô hình chú ý theo khối đã nói ở trên. Nếu $stride$ nhỏ hơn kích thước cửa sổ ngữ cảnh, mô hình chú ý sẽ trở nên dày đặc hơn. Hình 6.13 (d) cho thấy trường hợp $stride = 1$, khi đó sự chồng lấn giữa các khối là lớn nhất. Một cách để đạt được chú ý tương đối thưa hơn là dùng một cửa sổ ngữ cảnh giãn (*dilated context window*). Hình 6.13 (e) cho thấy một ví dụ về mô hình chú ý theo bước trượt có giãn, trong đó cửa sổ ngữ cảnh không liên tục, với các khoảng trống có kích thước 1.

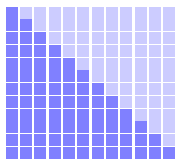
- **Học trường chú ý.** Vì trường chú ý π_i có thể là bất kỳ tập con nào của $\{1, \cdots, n\}$, ta có thể phát triển các mô hình chú ý thưa tổng quát hơn bằng cách xét các bản đồ chú ý vượt ra ngoài các mẫu dựa trên khối. Câu hỏi duy nhất là làm sao xác định mô hình sẽ chú ý tới những vị trí nào đối với một vị trí cho trước. Một cách đơn giản là dùng một mô hình rẻ hơn về tính toán để ước lượng "mức độ quan trọng" của mỗi vị trí. Sau đó, trọng số chú ý chỉ được tính cho một

số vị trí được cho là quan trọng nhất [Zhou và cộng sự, 2021]. Cách thứ hai là gom nhóm: các vị trí được chia thành nhóm, rồi trọng số chú ý chỉ được tính cho các vị trí trong cùng một nhóm. Điều này có thể thực hiện bằng cách phân cụm các khóa và truy vấn. Chẳng hạn, ta có thể phân cụm khóa và truy vấn bằng phân cụm k-means (*k-means clustering*). Các tâm cụm (*centroid*) có thể được coi là các tham số bổ sung của mô hình chú ý, và do đó có thể được học trong quá trình tối ưu [Roy và cộng sự, 2021]. Một lợi ích của việc học trường chú ý là mô hình có thể trải rộng sự chú ý của nó trên chuỗi. Đây là một tính chất hữu ích cho nhiều bài toán NLP vì sự phụ thuộc giữa các từ đôi khi có tầm xa, không bị giới hạn trong một cửa sổ ngữ cảnh cục bộ. Xem Hình 6.13 (f) để thấy ví dụ về bản đồ chú ý học được qua mô hình này. Các cách khác để học cách chú ý là dùng các hàm sắp xếp hoặc hàm băm để gom các vectơ khóa và truy vấn tương tự nhau [Kitaev và cộng sự, 2020; Tay và cộng sự, 2020a]. Các hàm này có thể là những hàm được thiết kế theo heuristic hoặc các mạng nơ-ron với tham số học được. Bằng cách dùng các hàm này, ta có thể sắp xếp lại chuỗi sao cho các đầu vào trong cùng một nhóm nằm cạnh nhau trong chuỗi đã sắp xếp lại. Theo cách này, bản đồ chú ý thu được có dạng theo khối, và mô hình hiệu quả về mặt tính toán nhờ sử dụng cách tiếp cận chú ý theo khối.

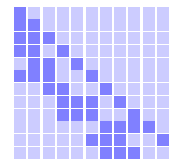
- **Các phương pháp lai.** Ở trên, chúng ta đã bàn về một loạt mô hình chú ý thưa khác nhau. Việc tìm hiểu các phương pháp kết hợp nhiều mô hình với nhau để tận dụng ưu điểm của chúng theo cách nào đó là điều tự nhiên. Một cách đơn giản để làm điều này là kết hợp các trường chú ý của các mô hình khác nhau. Ví dụ, trong hệ thống của Zaheer và cộng sự [2020], bản đồ chú ý được sinh ra bằng cách xét ba mô hình thưa khác nhau, gồm chú ý cục bộ (chú ý theo khối), chú ý toàn cục (*global attention*) và chú ý ngẫu nhiên (*random attention*)[12]. Mô hình thu được vẫn là một mô hình thưa, nhưng có phần vững vàng hơn vì nó bao gồm nhiều mẫu từ các góc nhìn khác nhau của việc mô hình hóa chú ý. Một cách khác để kết

[12] Ở đây, mô hình chú ý toàn cục cho mỗi từ chỉ chú ý tới một từ đặc biệt đại diện cho toàn bộ chuỗi, thường được đặt ở đầu chuỗi. Mô hình chú ý ngẫu nhiên cho mỗi từ chú ý tới một tập ngẫu nhiên các từ trong chuỗi.

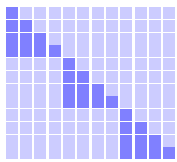
hợp nhiều mô hình chú ý là dùng các mô hình khác nhau cho các đầu khác nhau trong chú ý đa đầu [Child và cộng sự, 2019; Beltagy và cộng sự, 2020]. Ví dụ, có thể dùng một đầu làm mô hình chú ý cục bộ, và dùng một đầu khác làm mô hình chú ý toàn cục (xem Hình 6.13 (g–h)).



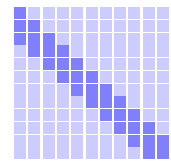
(a) Standard Attention



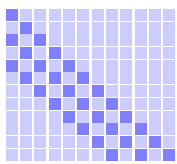
(b) Span-based Attention



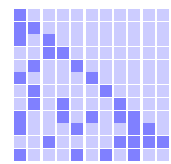
(c) Chunked Attention



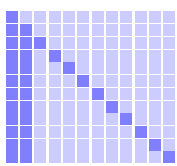
(d) Strided Attention



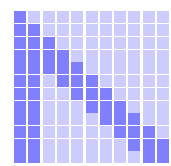
(e) Dilated Strided Attention



(f) Learning Attention Fields



(g) Global Attention



(h) Hybrid Methods

Hình 6.13: Minh họa bản đồ chú ý của các mô hình khác nhau (tự chú ý ở phía bộ giải mã). Ô sẫm màu nghĩa là $A'(i,j) \neq 0$ (tức là i chú ý tới j), và ô sáng màu nghĩa là $A'(i,j) = 0$ (tức là i không chú ý tới j). Trong mọi bản đồ chú ý này, ta giả sử mặc định mỗi vị trí đều chú ý tới chính nó (xem các đường chéo). Các hình con: (a) chú ý chuẩn; (b) chú ý dựa trên khoảng; (c) chú ý theo khối; (d) chú ý theo bước trượt; (e) chú ý theo bước trượt có giãn; (f) học trường chú ý; (g) chú ý toàn cục; (h) các phương pháp lai.

Chú giải của người dịch: Đọc Hình 6.13 thế nào?

Mỗi bản đồ là một bảng ô vuông: hàng thứ i ứng với vị trí đang được tính biểu diễn, cột thứ j ứng với vị trí được "nhìn tới". Ô sẫm ở hàng i, cột j nghĩa là vị trí i có dùng thông tin của vị trí j. Ở (a), cả tam giác dưới đều sẫm: mỗi vị trí nhìn mọi vị trí đứng trước. Ở (c), các ô sẫm tạo thành những khối vuông nhỏ dọc đường chéo: mỗi vị trí chỉ nhìn trong khối của mình. Ở (d), một "dải" hẹp chạy dọc đường chéo: mỗi vị trí nhìn vài vị trí ngay trước nó. Ở (g), cột đầu tiên sẫm: mọi vị trí đều nhìn tới từ đặc biệt ở đầu chuỗi. Càng ít ô sẫm, mô hình càng thưa và càng tiết kiệm.

Một nhược điểm của các mô hình thưa so với các mô hình dày đặc là chúng thường kém hiệu quả hơn trên GPU/CPU do các ràng buộc phần cứng. Dù về lý tưởng các mô hình thưa có thể giảm cả yêu cầu bộ nhớ lẫn yêu cầu tính toán, thông lượng (*throughput*) tính toán của các mô hình thưa chậm hơn nhiều so với các mô hình dày đặc. Trên thực tế, các mô hình thưa khó tiến gần tới FLOPS đỉnh của một GPU hay CPU[13]. Do đó, chúng chủ yếu được dùng để cải thiện hiệu quả bộ nhớ hơn là để tăng tốc độ tính toán thuần túy trên phần cứng hiện nay. Mặt khác, các mô hình thưa vẫn rất hữu ích cho những người làm NLP trong bối cảnh Transformer tiết kiệm bộ nhớ, đặc biệt khi Transformer được dùng để xử lý các chuỗi cực dài.

Chú giải của người dịch: Vì sao "thưa" chưa chắc đã nhanh?

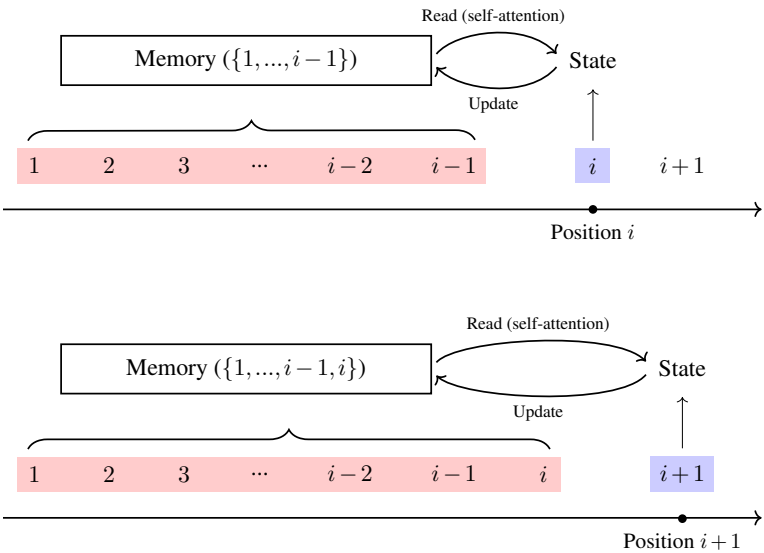
GPU được thiết kế để làm cực nhanh các phép nhân trên những khối số liền mạch, đều đặn (như cả một ma trận đầy). Với ma trận thưa, máy phải "nhảy cóc" tới các ô khác 0 nằm rải rác, đọc thêm thông tin về vị trí của chúng, nên phần cứng không chạy hết công suất. Hình dung như việc chở hàng: chở một xe tải đầy một lần thường nhanh hơn đi giao lẻ từng gói nhỏ tới nhiều địa chỉ, dù tổng số hàng ít hơn.

6.4.2 Mô hình hồi quy và mô hình bộ nhớ

Đối với các bài toán sinh chuỗi, Transformer cũng có thể được xem như các hệ thống bộ nhớ. Xét lại thiết lập tự hồi quy tổng quát, trong đó ta được cho các trạng thái của $i-1$ vị trí trước đó và muốn dự đoán trạng

[13] FLOPS = số phép toán dấu phẩy động mỗi giây (*floating point operations per second*).

thái tiếp theo. Trong tự chú ý, điều này được thực hiện bằng cách dùng truy vấn tại vị trí i (tức là $\mathbf{q}_i$) để truy cập các cặp khóa–giá trị của các vị trí trước đó (tức là $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$). Sau đó, ta chuyển sang vị trí $i+1$ và nối thêm $(\mathbf{k}_i, \mathbf{v}_i)$ vào tập các cặp khóa–giá trị. Quy trình này có thể được diễn giải một cách trực quan bằng các cơ chế bộ nhớ (xem Chương 4). Một mô hình Transformer duy trì một kho bộ nhớ (*memory bank*) lưu giữ thông tin trong quá khứ. Khi mô hình xử lý chuỗi, nó lặp lại cùng một thao tác: sinh đầu ra bằng cách đọc từ bộ nhớ, rồi cập nhật bộ nhớ để đưa thông tin mới vào. Điều này được minh họa trong Hình 6.14.



Hình 6.14: Transformer như một hệ thống bộ nhớ. Tại vị trí i, tập các cặp khóa–giá trị từ các vị trí $\{1, \cdots, i-1\}$ đóng vai trò bộ nhớ về ngữ cảnh quá khứ. Mô hình truy cập bộ nhớ này để sinh ra một đầu ra, rồi sau đó thêm cặp khóa–giá trị của vị trí i vào bộ nhớ. Chuyển sang vị trí tiếp theo, chu trình truy cập và cập nhật bộ nhớ này được lặp lại. (Các nhãn trong hình: *Memory* = bộ nhớ; *State* = trạng thái; *Read (self-attention)* = đọc (bằng tự chú ý); *Update* = cập nhật; *Position* = vị trí.)

Chú giải của người dịch: Bộ nhớ ở đây chính là KV cache

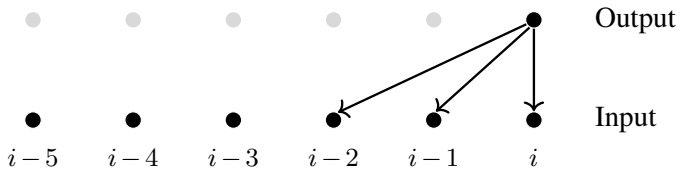
Khi sinh từng token, mô hình cần khóa và giá trị của mọi token đã sinh. Thay vì tính lại chúng ở mỗi bước, ta lưu sẵn trong một "kho" — chính là bộ nhớ đệm KV (KV cache). Mỗi bước: (1) *đọc*: truy vấn mới so khớp với mọi khóa trong kho để lấy thông tin; (2) *ghi*: thêm cặp khóa–giá trị mới vào kho. Vấn đề là kho này cứ lớn dần theo độ dài chuỗi, nên các phương pháp dưới đây tìm cách giữ nó nhỏ hoặc truy cập nó nhanh hơn.

1. Bộ nhớ dựa trên bộ đệm

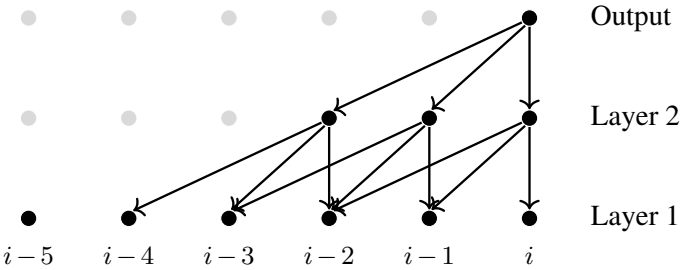
Bộ nhớ ở đây có thể được xem như một kho lưu trữ các vectơ. Từ góc nhìn học máy, đây là một mô hình phi tham số (*non-parametric model*), và chi phí truy cập mô hình tăng lên khi chuỗi dài ra. Rõ ràng, một bộ nhớ có độ dài thay đổi như vậy nói chung sẽ không khả thi nếu mô hình phải xử lý các chuỗi cực dài. Để mô hình hóa các chuỗi có độ dài tùy ý, người ta thường dùng một bộ nhớ có độ dài cố định. Như trong nhiều bài toán NLP, một trong những cách đơn giản nhất để làm điều này là duy trì một bộ đệm (*cache*) chứa thông tin gần đây, tức là ta giới hạn việc mô hình hóa trong một cửa sổ ngữ cảnh. Gọi n_c là kích thước cửa sổ ngữ cảnh. Mô hình theo dõi $n_c - 1$ trạng thái gần nhất đứng trước vị trí hiện tại, để có thể xét các vị trí đứng trước gần nhất ở mỗi bước. Điều này có nghĩa là, với mỗi vị trí, một tầng con tự chú ý chú ý tới $n_c - 1$ vị trí trước đó, như minh họa trong Hình 6.15 (a).

Nếu xếp chồng nhiều tầng con tự chú ý, một cửa sổ ngữ cảnh lớn hơn sẽ được xét đến. Chẳng hạn, một mô hình gồm hai tầng con tự chú ý có cửa sổ ngữ cảnh kích thước $2n_c - 1$, như minh họa trong Hình 6.15 (b).

Vì vậy, ta có thể nắm bắt một ngữ cảnh đủ lớn bằng cách dùng một mô hình Transformer nhiều tầng. Lưu ý rằng mô hình cửa sổ ngữ cảnh ở đây về bản chất giống mô hình chú ý theo bước trượt đã trình bày ở tiểu mục trước. Các hệ thống loại này thường dễ cài đặt: ta trượt một cửa sổ dọc theo chuỗi, và ở mỗi lần di chuyển, ta đưa ra dự đoán tại vị trí cuối cùng của cửa sổ (khi suy luận), hoặc lan truyền ngược sai số (khi huấn luyện).



(a) Cache-based memory with a single self-attention layer.



(b) Expanded context window with multiple self-attention layers.

Hình 6.15: Minh họa các cơ chế bộ nhớ dựa trên bộ đệm trong mô hình Transformer. (a) Bộ nhớ dựa trên bộ đệm với một tầng tự chú ý duy nhất. (b) Cửa sổ ngữ cảnh được mở rộng nhờ nhiều tầng tự chú ý. (*Input* = đầu vào; *Output* = đầu ra; *Layer* = tầng.)

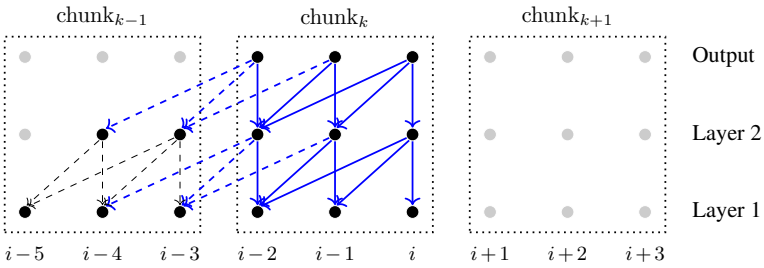
Chú giải của người dịch: Vì sao xếp chồng tầng lại mở rộng ngữ cảnh?

Lấy $n_c = 3$: ở tầng 1, vị trí i chỉ nhìn được $i-2, i-1$ (và chính nó). Nhưng biểu diễn tầng 1 của $i-2$ lại đã "chứa" thông tin của $i-4, i-3$. Vì vậy ở tầng 2, khi i nhìn vào $i-2$ của tầng 1, nó gián tiếp nhận được thông tin từ tận $i-4$. Tổng cộng tầm nhìn là $i-4, \ldots, i$, tức $5 = 2 \cdot 3 - 1$ vị trí. Giống như "tin đồn" lan qua nhiều người: mỗi người chỉ nói với hàng xóm, nhưng qua vài lượt, tin đi rất xa. Với L tầng, tầm nhìn tăng lên khoảng $L(n_c - 1) + 1$ vị trí.

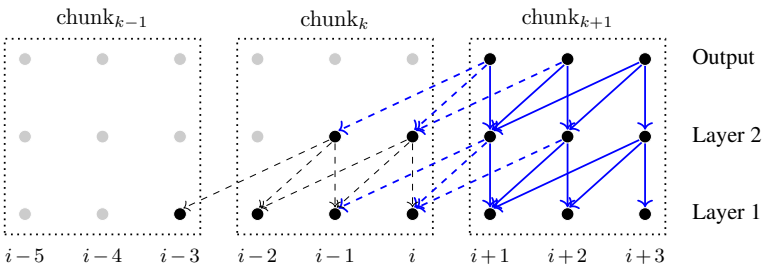
Một chiến lược khác để huấn luyện mô hình cửa sổ ngữ cảnh là chú ý theo khối. Chuỗi được chia một cách tường minh thành các chuỗi con (hay các khối) có độ dài cố định n_c. Các khối này sau đó được coi là những mẫu huấn luyện độc lập. Tuy nhiên, cách tiếp cận này hoàn toàn bỏ qua mối quan hệ giữa các đầu vào ở những khối khác nhau. Một cách để giải quyết vấn đề này là đưa vào sự phụ thuộc giữa các khối. Chẳng hạn, mô hình **Transformer-XL** cho phép mỗi khối truy cập một hoặc nhiều khối đứng trước nó [Dai và cộng sự, 2019]. Trong trường hợp đơn giản nhất, xét một ví dụ trong đó chunk_k có thể thấy khối liền trước nó là chunk_{k-1}. Mỗi vị trí trong chunk_k có thể chú ý tới mọi vị trí đứng trước nó trong cả chunk_k lẫn chunk_{k-1}.

Trong Transformer-XL, cách tiếp cận này được cài đặt dưới một dạng đơn giản hóa. Thứ nhất, mỗi vị trí bị ràng buộc chỉ được chú ý tới $n_c - 1$ vị trí trước đó, sao cho kích thước trường chú ý của một vị trí là như nhau trong giai đoạn huấn luyện và suy luận. Cách làm này đưa bài toán trở về chú ý theo bước trượt, khiến việc cài đặt mô hình chú ý trở nên đơn giản. Tuy nhiên, khác với chú ý theo bước trượt chuẩn, Transformer-XL thực hiện huấn luyện theo từng khối. Khi đã huấn luyện xong trên một khối, ta chuyển thẳng sang khối kế tiếp, thay vì trượt cửa sổ ngữ cảnh tiến lên một bước nhỏ. Thứ hai, dù cách tiếp cận này cho phép có kết nối giữa các khối, các tham số của mạng con ứng với chunk_{k-1} được giữ cố định, và ta chỉ cập nhật tham số của mạng con trên chunk_k ở bước thứ k. Xem Hình 6.16 để thấy minh họa.

Mô hình trên có tinh thần tương tự các mô hình hồi quy, vì cả hai đều yêu cầu phép tính ở một bước phụ thuộc vào trạng thái của các bước trước. Tuy nhiên, nó không ở dạng chuẩn của một mô hình hồi quy, trong đó đầu



(a) Step k of chunk-wise training



(b) Step $k+1$ of chunk-wise training

Hình 6.16: Minh họa việc huấn luyện theo khối trong Transformer-XL [Dai và cộng sự, 2019]. Chuỗi đầu vào được chia thành các khối không chồng lấn có độ dài n_c. Việc huấn luyện tiến hành lần lượt từng khối. Bên trong chunk$_k$, tầm chú ý của mọi token bị giới hạn trong một cửa sổ ngữ cảnh bên trái có kích thước n_c. Cấu trúc này cho phép chú ý xuyên khối. Chẳng hạn, vị trí $i-2$ trong chunk$_k$ có thể chú ý tới các vị trí $i-3$ và $i-4$ nằm trong chunk$_{k-1}$ (xem hình con (a)). Trong quá trình lan truyền ngược, gradient được tính cho mạng con hiện tại (chunk$_k$), còn các giá trị kích hoạt đã lưu đệm từ các khối trước được coi là các hằng số cố định. Các đường nét đứt biểu thị các luồng thông tin của lượt truyền xuôi không nhận gradient. Sau khi hoàn tất bước huấn luyện cho chunk$_k$, quá trình chuyển sang khối tiếp theo và lặp lại. ((a) Bước k và (b) bước $k+1$ của huấn luyện theo khối.)

ra của một đơn vị hồi quy ở một bước là đầu vào ở bước tiếp theo. Thay vào đó, tính "hồi quy" được thể hiện thông qua các kết nối xuyên tầng và xuyên khối, tức là đầu ra của một tầng trong chunk$_{k-1}$ được dùng làm đầu vào của một tầng cao hơn trong chunk$_k$.

2. Mã hóa bộ nhớ dài hạn

Một ý tưởng khác để biểu diễn các trạng thái của một chuỗi là đặt nhiệm vụ này dưới dạng một bài toán mã hóa. Thay vì lưu tất cả các vectơ khóa–giá trị trong quá trình sinh từ trái sang phải, ta nén bộ nhớ của toàn bộ "lịch sử" thành một số lượng cố định các vectơ khóa–giá trị đã mã hóa. Các vectơ đã mã hóa này có thể là một tập con nhỏ của chuỗi gốc $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$, hoặc một tập nhỏ các vectơ mới được sinh ra để mã hóa toàn bộ lịch sử này.

Một cách để thực hiện việc mã hóa này là áp dụng một phép gộp (*pooling*) lên tập $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$ [Rae và cộng sự, 2019a]. Ví dụ, dùng gộp trung bình (*average pooling*), bộ nhớ được rút gọn thành một cặp khóa–giá trị duy nhất $(\bar{\mathbf{k}}, \bar{\mathbf{v}})$:

$$\bar{\mathbf{k}} = \frac{1}{i-1} \sum_{j=1}^{i-1} \mathbf{k}_j, \tag{6.124}$$

$$\bar{\mathbf{v}} = \frac{1}{i-1} \sum_{j=1}^{i-1} \mathbf{v}_j. \tag{6.125}$$

Điều này dẫn tới một mô hình rất hiệu quả, vì ta chỉ cần cập nhật các vectơ tổng hợp $(\bar{\mathbf{k}}, \bar{\mathbf{v}})$ một cách tăng dần [Zhang và cộng sự, 2018a]. Gọi $(\bar{\mathbf{k}}[i], \bar{\mathbf{v}}[i])$ là trạng thái của bộ nhớ tại vị trí i. Một công thức tổng quát hơn được cho dưới dạng đệ quy:

$$\bar{\mathbf{k}}[i] = \text{KMem}(\bar{\mathbf{k}}[i-1], \mathbf{k}_{i-1}), \tag{6.126}$$

$$\bar{\mathbf{v}}[i] = \text{VMem}(\bar{\mathbf{v}}[i-1], \mathbf{v}_{i-1}), \tag{6.127}$$

trong đó $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$ là các hàm cập nhật bộ nhớ bằng cách nhận cả trạng thái của bộ nhớ ở vị trí trước (tức $\bar{\mathbf{k}}[i-1]$ và $\bar{\mathbf{v}}[i-1]$) lẫn các trạng thái mới (tức $\mathbf{k}_{i-1}$ và $\mathbf{v}_{i-1}$). Có nhiều lựa chọn khả dĩ cho các hàm $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$. Chẳng hạn, nếu $\text{KMem}(\cdot)$ và $\text{VMem}(\cdot)$ là các hàm tổng có trọng số, ta có thể suy ra cùng dạng như Phương trình (6.124)

và (6.125). Nếu KMem$(\cdot)$ và VMem$(\cdot)$ là các ô hồi quy (ví dụ các ô RNN hoặc LSTM chuẩn), ta thu được một mô hình bộ nhớ hồi quy.

Chú giải của người dịch: Cập nhật trung bình "tăng dần"

Giống như tính điểm trung bình môn học: nếu đã biết trung bình của m bài kiểm tra là $\bar{x}_m$ và có thêm điểm mới x, ta không cần cộng lại từ đầu mà chỉ cần $\bar{x}_{m+1} = \frac{m\,\bar{x}_m + x}{m+1}$. Ví dụ: trung bình 3 bài là 7, bài thứ tư được 9, thì trung bình mới là $(3 \cdot 7 + 9)/4 = 7{,}5$. Phương trình (6.126) chính là ý tưởng này viết dưới dạng tổng quát: trạng thái mới chỉ phụ thuộc vào trạng thái cũ và dữ liệu mới. Nhờ vậy bộ nhớ luôn có kích thước cố định, dù chuỗi dài bao nhiêu.

Mở rộng khái niệm trên cho các bộ nhớ có nhiều hơn một cặp khóa–giá trị là điều đơn giản. Một cách là biểu diễn mức khối (*chunk-level representation*). Gọi $\{(\bar{\mathbf{k}}_1, \bar{\mathbf{v}}_1), \cdots, (\bar{\mathbf{k}}_\kappa, \bar{\mathbf{v}}_\kappa)\}$ là một bộ nhớ kích thước κ, trong đó mỗi $(\bar{\mathbf{k}}_j, \bar{\mathbf{v}}_j)$ là biểu diễn của một khối có độ dài n_c. Như vậy, bộ nhớ này có thể mã hóa một chuỗi có độ dài tối đa $\kappa \cdot n_c$, bằng cách tính mỗi $(\bar{\mathbf{k}}_j, \bar{\mathbf{v}}_j)$ theo Phương trình (6.126) và (6.127) trên khối tương ứng của nó. Cách thứ hai là tổ chức bộ nhớ thành một hàng đợi ưu tiên (*priority queue*). Một hàm chấm điểm được học, thường là một mạng nơ-ron phụ trợ, đánh giá và gán một điểm ưu tiên cho mỗi cặp khóa–giá trị được sinh ra. Các cặp có điểm cao được đưa vào hàng đợi thông qua một thao tác đẩy (*push*). Theo cách này, bộ nhớ chủ động duy trì một tập động chỉ gồm những cặp khóa–giá trị có giá trị nhất trên toàn chuỗi đầu vào.

Mặc dù biểu diễn bộ nhớ như một tập các vectơ là một lựa chọn thiết kế trực quan cho Transformer, dung lượng của nó bị giới hạn bởi số lượng vectơ. Một mô hình thay thế là **bộ nhớ liên tục** (*continuous memory*). Cách tiếp cận này tận dụng xấp xỉ hàm (*function approximation*), coi các tập khóa $\{\mathbf{k}_1, \cdots, \mathbf{k}_{i-1}\}$ hoặc tập giá trị $\{\mathbf{v}_1, \cdots, \mathbf{v}_{i-1}\}$ như một dãy các điểm dữ liệu cần được khớp bởi một hàm liên tục. Thay vì lưu trữ tường minh các vectơ này, bộ nhớ được tham số hóa bởi chính các hàm đó. Một phương pháp phổ biến là dùng tổ hợp tuyến tính của các hàm cơ sở (*basis function*) đơn giản để xấp xỉ hàm phức tạp được xác định bởi các vectơ khóa hoặc giá trị rời rạc [Martins và cộng sự, 2022].

Chú giải của người dịch: Bộ nhớ liên tục: "vẽ đường cong thay vì lưu từng điểm"

Hãy tưởng tượng bạn có 1 000 điểm đo nhiệt độ trong ngày. Thay vì nhớ cả 1 000 con số, bạn tìm một hàm, chẳng hạn $f(t) = c_1 + c_2 \sin t + c_3 \cos t$, đi gần các điểm đó, rồi chỉ cần nhớ 3 hệ số c_1, c_2, c_3. Các hàm $1, \sin t, \cos t$ gọi là *hàm cơ sở*. Bộ nhớ liên tục làm điều tương tự với các vectơ khóa/giá trị: số hệ số cần lưu cố định, không phụ thuộc chuỗi dài bao nhiêu (đổi lại, thông tin bị xấp xỉ chứ không còn chính xác tuyệt đối).

Cũng dễ dàng dùng đồng thời một bộ nhớ ngắn hạn và một bộ nhớ dài hạn để kết hợp ưu điểm của cả hai. Chẳng hạn, ta dùng một bộ nhớ dựa trên bộ đệm để nắm bắt ngữ cảnh cục bộ, và dùng một bộ nhớ dài hạn hiệu quả mã hóa toàn bộ lịch sử để mô hình hóa các phụ thuộc tầm xa. Ý tưởng này cũng tương tự ý tưởng dùng để kết hợp các mô hình chú ý thưa khác nhau như đã bàn ở tiểu mục trước.

3. Các phương pháp dựa trên truy xuất

Cho tới giờ trong tiểu mục này, chúng ta đã bàn về các cách tiếp cận dựa trên mô hình có độ dài cố định. Cũng có thể phát triển các mô hình bộ nhớ hiệu quả bằng cách cải thiện hiệu quả truy cập bộ nhớ, thay vì giảm cứng dung lượng bộ nhớ. Một cách để làm điều này là lưu các cặp khóa–giá trị trong quá khứ vào một cơ sở dữ liệu (gọi là **cơ sở dữ liệu vectơ** (*vector database*)), và tìm các cặp khóa–giá trị giống nhất khi truy vấn cơ sở dữ liệu. Chính xác hơn, cho một truy vấn $\mathbf{q}$, ta dùng cơ sở dữ liệu để tìm một tập gồm p cặp khóa–giá trị liên quan nhất (ký hiệu Ω_p) bằng cách thực hiện tìm kiếm tương tự (*similarity search*) dựa trên tích vô hướng giữa truy vấn và các vectơ khóa. Sau đó truy vấn $\mathbf{q}$ chú ý tới Ω_p như trong các mô hình tự chú ý chuẩn. Ý tưởng đằng sau phương pháp này là chỉ xét một số ít phần tử đóng góp nhiều nhất vào kết quả chú ý. Do đó, về bản chất, mô hình là một mô hình chú ý thưa hiệu quả về tính toán. Một ưu điểm khác của phương pháp này là nó cho phép tìm kiếm tương tự nhanh trên một tập vectơ rất lớn nhờ các cài đặt được tối ưu hóa cao của cơ sở dữ liệu vectơ. Việc đặt bộ nhớ dưới dạng một hệ thống truy xuất thuộc về khuôn khổ rộng hơn của **sinh văn bản tăng cường truy xuất** (*retrieval-augmented generation*) (**RAG**) hay các cách tiếp cận tăng cường truy xuất. Nó cung cấp một khuôn khổ có khả năng mở rộng để tích hợp bộ nhớ ngoài vào

các kiến trúc nơ-ron như Transformer [Guu và cộng sự, 2020; Lewis và cộng sự, 2020b; Wu và cộng sự, 2021].

Chú giải của người dịch: Truy xuất giống như tra mục lục

Thay vì đọc lại cả cuốn sách mỗi khi cần một thông tin, ta tra mục lục để mở đúng vài trang liên quan nhất. Ở đây, "mục lục" là cơ sở dữ liệu vectơ: nó nhanh chóng tìm ra p khóa có tích vô hướng lớn nhất với truy vấn (tức là "giống" truy vấn nhất), và mô hình chỉ chú ý tới p cặp đó. Vì sau Softmax, các khóa có tích vô hướng nhỏ chỉ nhận trọng số rất gần 0, bỏ qua chúng thường làm kết quả thay đổi không đáng kể.

6.4.3 Các mô hình số chiều thấp

Trong nhiều ứng dụng thực tế, Transformer là các mô hình "nhiều chiều". Điều này không chỉ vì dữ liệu đầu vào và/hoặc đầu ra nằm trong các không gian nhiều chiều, mà còn vì một số biểu diễn trung gian của dữ liệu trong mô hình có số chiều cao. Như đã bàn ở Mục 6.4.1, số chiều cao này một phần đến từ việc tính tổng có trọng số của các vectơ giá trị, như trong Phương trình (6.128) (nhắc lại ở đây cho tiện theo dõi)

$$\text{Att}_{\text{self}}(\mathbf{S}) \;=\; \mathbf{AV} \tag{6.128}$$

và từ việc tính chính ma trận chú ý, như trong Phương trình (6.129)

$$\mathbf{A} \;=\; \text{Softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d}} + \mathbf{M}\right), \tag{6.129}$$

vốn bao gồm các phép nhân ma trận lớn $\mathbf{QK}^\top$ và $\mathbf{AV}$ khi độ dài n và số chiều ẩn d lớn.

Các phép toán $\mathbf{AV}$ và $\mathbf{QK}^\top$ có độ phức tạp thời gian $O(n^2 \cdot d)$ và độ phức tạp không gian $O(n^2 + n \cdot d)$. Trong khi các cách tiếp cận trước đã giảm bớt độ phức tạp này bằng các mô hình thưa, tiểu mục này tập trung vào các phương pháp xấp xỉ những phép toán này bằng tính toán dày đặc. Một ý tưởng đơn giản là chiếu $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ thành các ma trận nhỏ hơn, nhờ đó giảm gánh nặng tính toán của phép nhân ma trận. Vì $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$ đều thuộc $\mathbb{R}^{n \times d}$, điều này có thể đạt được bằng cách giảm chiều độ dài n, giảm chiều ẩn d, hoặc giảm cả hai.

Chú giải của người dịch: Ký hiệu $O(\cdot)$ và đếm phép tính

$O(n^2 \cdot d)$ đọc là "bậc $n^2 d$": số phép tính tăng tỷ lệ với $n^2 d$ khi các đại lượng lớn dần (bỏ qua hằng số). Thật vậy, nhân ma trận $n \times d$ với ma trận $d \times n$ cho ra $n \cdot n$ phần tử, mỗi phần tử là một tích vô hướng của hai vectơ d chiều (khoảng d phép nhân), tổng cộng cỡ $n \cdot n \cdot d$ phép nhân. Với $n = 4\,096$, $d = 1\,024$ thì đó là khoảng $1{,}7 \times 10^{10}$ phép nhân chỉ cho một tầng, một đầu chú ý.

1. Giảm n

Lưu ý rằng đầu ra $\text{Att}_{\text{self}}(\mathbf{S})$ bắt buộc phải là ma trận $n \times d$, nên ta không thể giảm số lượng truy vấn. Thay vào đó, ta xét việc giảm số lượng khóa và giá trị. Giả sử n' là một số nguyên với $n' < n$, và $\mathbf{K}$, $\mathbf{V}$ có thể được biến đổi theo cách nào đó thành các ma trận $n' \times d$ là $\mathbf{K}'$ và $\mathbf{V}'$. Ta có thể thu được một mô hình "nhỏ hơn" đơn giản bằng cách thay $\mathbf{K}$ và $\mathbf{V}$ bởi $\mathbf{K}'$ và $\mathbf{V}'$, cho ta

$$\text{Att}_{\text{self}}(\mathbf{S}) = \mathbf{A}\mathbf{V}' \tag{6.130}$$

$$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}[\mathbf{K}']^\top}{\sqrt{d}} + \mathbf{M}\right). \tag{6.131}$$

Mô hình này có dạng chuẩn của tự chú ý, nhưng có độ phức tạp thời gian và không gian thấp hơn, tức là $O(n' \cdot n \cdot d) < O(n^2 \cdot d)$ và $O(n' \cdot n + n' \cdot d) < O(n^2 + n \cdot d)$. Nếu $n' \ll n$, mô hình thu được sẽ tuyến tính theo n.

Chú giải của người dịch: Kích thước các ma trận sau khi giảm n

$\mathbf{Q}$ vẫn là $n \times d$, còn $[\mathbf{K}']^\top$ là $d \times n'$, nên $\mathbf{A}$ giờ là ma trận $n \times n'$ (mỗi truy vấn chỉ có n' trọng số). Nhân $\mathbf{A}$ ($n \times n'$) với $\mathbf{V}'$ ($n' \times d$) lại cho ra đúng $n \times d$ như yêu cầu. Nếu giữ $n' = 256$ cố định, khi n tăng gấp đôi thì chi phí chỉ tăng gấp đôi (tuyến tính), thay vì gấp bốn như với n^2.

Vấn đề then chốt ở đây là làm sao thu được $\mathbf{K}'$ và $\mathbf{V}'$ theo cách giữ lại phần lớn thông tin trong $\mathbf{K}$ và $\mathbf{V}$. Có một số cách để làm việc này. Một phương pháp đơn giản là chọn những khóa và giá trị được cho là quan trọng. Mức độ quan trọng của một khóa (hoặc giá trị) có thể được tính theo một thước đo rẻ về tính toán nào đó. Ví dụ, ta có thể lấy mẫu một số ít tích vô hướng truy vấn–khóa và ước lượng mức độ quan trọng của một khóa bằng cách tổng hợp các kết quả tích vô hướng này.

Dù đơn giản, phương pháp trên vẫn dựa vào các phép toán thưa như lấy mẫu và tổng hợp. Một cách khác là dùng tính toán dày đặc để chiếu $\mathbf{K}$ và $\mathbf{V}$ xuống $\mathbf{K}'$ và $\mathbf{V}'$. Một cách tiếp cận phổ biến là dùng CNN [Liu và cộng sự, 2018]. Gọi $\text{Conv}(\cdot)$ là một phép tích chập 1D trượt dọc theo chiều n. Khi đó $\mathbf{K}'$ được tính như sau:

$$\mathbf{K}' = \text{Conv}(\mathbf{K}, \mathbf{W}_c, \text{size}_r, \text{stride}), \tag{6.132}$$

trong đó $\mathbf{W}_c$ là ma trận tham số của các bộ lọc, size_r là kích thước của trường tiếp nhận (*receptive field*), và stride là kích thước bước của phép tích chập. Nói chung, ta có thể đạt tỷ lệ nén cao bằng cách chọn các giá trị lớn cho size_r và stride. Tương tự, ta có thể tính $\mathbf{V}'$ bằng một hàm tích chập khác. Đáng chú ý là, nếu tham số n' cố định cho mọi mẫu, việc nén $\mathbf{K}$ và $\mathbf{V}$ dọc theo chiều độ dài về bản chất giống mô hình bộ nhớ có độ dài cố định đã mô tả ở tiểu mục trước. Các phương pháp trình bày ở đây tổng quát hơn và có thể được áp dụng cho các bộ nhớ có độ dài thay đổi.

Chú giải của người dịch: Tích chập 1D để nén chuỗi

Hình dung một "cửa sổ" rộng size_r trượt dọc theo n hàng của $\mathbf{K}$, mỗi lần nhảy stride hàng; tại mỗi vị trí, nó gộp các hàng trong cửa sổ thành một hàng mới (bằng tổ hợp tuyến tính với trọng số học được $\mathbf{W}_c$). Ví dụ $n = 12$, $\text{size}_r = \text{stride} = 3$: cửa sổ đặt ở các hàng 1–3, 4–6, 7–9, 10–12, cho ra $n' = 4$ hàng, tức là nén chuỗi khóa xuống 3 lần.

Ta cũng có thể muốn mô hình hóa hàm chú ý bằng cách coi ma trận chú ý $\mathbf{A}$ là một biểu diễn nhiều chiều của dữ liệu, rồi áp dụng các phương pháp giảm chiều (*dimensionality reduction*) truyền thống. Theo thực nghiệm, trong nhiều bài toán, $\mathbf{A}$ (hay chính xác hơn là $\mathbf{QK}^\top$) được thấy là một ma trận hạng thấp (*low-rank matrix*). Trong trường hợp này, ta có thể nén $\mathbf{A}$ trong khi vẫn giữ lại được nhiều thông tin nhất có thể. Có nhiều cách để làm điều này. Ví dụ, ta có thể dùng tích của các ma trận nhỏ hơn làm xấp xỉ cho $\mathbf{A}$ thông qua kỹ thuật SVD (xem Chương 3). Tuy nhiên, việc dùng SVD gây thêm chi phí tính toán so với mô hình chú ý chuẩn. Một ý tưởng đơn giản hơn là biến đổi trực tiếp $\mathbf{K}$ và $\mathbf{V}$ thành các ma trận kích thước nhỏ hơn bằng các ánh xạ tuyến tính, cho bởi

$$\mathbf{K}' = \mathbf{U}^k \mathbf{K}, \tag{6.133}$$

$$\mathbf{V}' = \mathbf{U}^v \mathbf{V}, \tag{6.134}$$

trong đó $\mathbf{U}^k \in \mathbb{R}^{n' \times n}$ và $\mathbf{U}^v \in \mathbb{R}^{n' \times n}$ là các ma trận tham số. Rõ ràng, điều này dẫn tới một mô hình tương đương với mô hình được mô tả trong Phương trình (6.130) và (6.131). Dù rất trực quan, cách tiếp cận này đã được chứng minh là cho sai số xấp xỉ ϵ đủ nhỏ, miễn là n' tăng tuyến tính theo d/ϵ^2 [Wang và cộng sự, 2020b].

Chú giải của người dịch: Ma trận "hạng thấp" là gì?

Một ma trận có *hạng thấp* khi các hàng của nó "lặp lại" nhau theo nghĩa có thể tạo ra từ một vài hàng gốc bằng cách nhân với số rồi cộng lại. Ví dụ, ma trận $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$ có 9 số nhưng mọi hàng đều là bội của $[1\ 2\ 3]$; nó bằng tích của cột $[1; 2; 3]$ với hàng $[1\ 2\ 3]$, nên chỉ cần lưu 6 số (hạng bằng 1). Nếu ma trận chú ý "gần" hạng thấp như vậy, ta có thể nén nó mà mất rất ít thông tin. SVD là một công cụ đại số tuyến tính tìm cách phân tích như thế một cách tối ưu.

2. Giảm d

Một cách tiếp cận khác để giảm độ phức tạp tính toán là giảm số chiều d. Một trong những phương pháp đơn giản nhất là chiếu tất cả truy vấn và khóa lên một không gian d' chiều ($d' < d$), và tính tích vô hướng giữa các cặp truy vấn–khóa trong không gian mới. Về mặt mô hình hóa, ta chỉ cần thay $\mathbf{Q} \in \mathbb{R}^{n \times d}$ và $\mathbf{K} \in \mathbb{R}^{n \times d}$ bằng các biểu diễn mới $\mathbf{Q}' \in \mathbb{R}^{n \times d'}$ và $\mathbf{K}' \in \mathbb{R}^{n \times d'}$. Ta có thể dễ dàng sửa Phương trình (6.129) để dùng $\mathbf{Q}'$ và $\mathbf{K}'$ khi tính ma trận chú ý

$$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}'[\mathbf{K}']^\top}{\sqrt{d'}} + \mathbf{M}\right). \tag{6.135}$$

$\mathbf{Q}'$ và $\mathbf{K}'$ được cho bởi

$$\mathbf{Q}' = \mathbf{Q}\mathbf{U}^q, \tag{6.136}$$

$$\mathbf{K}' = \mathbf{K}\mathbf{U}^k, \tag{6.137}$$

trong đó $\mathbf{U}^q \in \mathbb{R}^{d \times d'}$ và $\mathbf{U}^k \in \mathbb{R}^{d \times d'}$ là các ma trận tham số của các phép biến đổi tuyến tính.

Cũng có thể khai thác các **phương pháp hạt nhân** (*kernel methods*) để thu được một mô hình chú ý tích vô hướng hiệu quả. Ý tưởng cơ bản

là ánh xạ các điểm dữ liệu (biểu diễn bằng vectơ) từ một không gian sang không gian khác, biến một bài toán khó trong không gian gốc thành một bài toán dễ xử lý hơn trong không gian mới. "Thủ thuật hạt nhân (*kernel trick*)" cho phép ta tính các tích trong như vậy một cách ngầm định thông qua một hàm hạt nhân (*kernel function*), mà không cần xây dựng tường minh ánh xạ $\phi(\cdot)$[14]. Các tích trong như vậy trong không gian đặc trưng thường được biểu diễn bằng các hàm hạt nhân, ký hiệu là $K(\cdot, \cdot)$.

Một điều thú vị là xấp xỉ $\mathbf{A}$ theo cách tương tự $K(\cdot, \cdot)$ trong các phương pháp hạt nhân. Để minh họa, lưu ý rằng trong Phương trình (6.129), $\mathbf{A}$ biểu diễn các trọng số chú ý đã được chuẩn hóa. Tử số có thể được viết dưới dạng

$$\widetilde{\mathbf{A}} \;=\; \text{Mask}\Big(\exp\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\Big)\Big). \tag{6.139}$$

Ở đây $\text{Mask}(\cdot)$ là một hàm có tác dụng giống như việc dùng biến mặt nạ cộng $\mathbf{M}$. Khi đó, $\mathbf{A}$ có thể được biểu diễn là

$$\mathbf{A} \;=\; \mathbf{D}^{-1}\widetilde{\mathbf{A}}, \tag{6.140}$$

trong đó $\mathbf{D}$ là một ma trận đường chéo (*diagonal matrix*) kích thước $n \times n$. Mỗi phần tử trên đường chéo chính bằng tổng các phần tử của hàng tương ứng trong $\widetilde{\mathbf{A}}$, biểu thị hệ số chuẩn hóa của Softmax. Thay phương trình này vào Phương trình (6.129), ta có

$$\text{Att}_{\text{self}}(\mathbf{S}) \;=\; \mathbf{D}^{-1}\widetilde{\mathbf{A}}\mathbf{V}. \tag{6.141}$$

[14]Trong giải tích toán học, tích trong (*inner product*) là một khái niệm tổng quát hóa của tích vô hướng, thường được ký hiệu là $\langle \cdot, \cdot \rangle$. Một định nghĩa hình thức đòi hỏi $\langle \cdot, \cdot \rangle$ thỏa mãn một số tính chất trong một không gian vectơ. Mặc dù tích trong có những dạng khác nhau trong các bối cảnh khác nhau, trong không gian Euclid $\mathbb{R}^d$ nó tương đương với tích vô hướng. Tức là, cho hai vectơ $\mathbf{a} \in \mathbb{R}^d$ và $\mathbf{b} \in \mathbb{R}^d$, ta có

$$\begin{aligned} \langle \mathbf{a}, \mathbf{b} \rangle &= \mathbf{a} \cdot \mathbf{b} \\ &= \sum_{i=1}^{d} a_i \cdot b_i \end{aligned} \tag{6.138}$$

Chú giải của người dịch: Tách Softmax thành "tử số" và "mẫu số"

Nhớ lại Softmax của một hàng $(z_1, \ldots, z_n)$ cho trọng số $e^{z_j} / \sum_{j'} e^{z_{j'}}$. $\widetilde{\mathbf{A}}$ chứa các tử số e^{z_j} (ô bị che thì bằng 0), còn $\mathbf{D}$ là ma trận chỉ có số trên đường chéo, mỗi số là mẫu số $\sum_{j'} e^{z_{j'}}$ của một hàng. Nhân với $\mathbf{D}^{-1}$ (ma trận đường chéo chứa các số nghịch đảo) đơn giản là chia mỗi hàng của $\widetilde{\mathbf{A}}$ cho tổng của hàng đó. Ví dụ hàng $(e^0, e^{\ln 3}) = (1, 3)$ có tổng 4, sau chuẩn hóa thành $(0{,}25;\ 0{,}75)$.

Trong mô hình này, $\widetilde{A}(i, j)$ có thể được xem là một hàm tương tự trên mọi cặp truy vấn–khóa trong một không gian d chiều. Ở đây ta giả sử rằng hàm này, vốn có dạng tích vô hướng của các vectơ, có thể được xấp xỉ bằng một hàm hạt nhân

$$\begin{aligned}\widetilde{A}(i, j) &= K(\mathbf{q}_i, \mathbf{k}_j) \\ &= \langle \phi(\mathbf{q}_i), \phi(\mathbf{k}_j) \rangle.\end{aligned}$$

$\phi(\cdot)$ là một ánh xạ từ $\mathbb{R}^d$ sang $\mathbb{R}^{d'}$. Ta có thể biểu diễn các truy vấn và khóa theo dạng sau

$$\begin{aligned}\mathbf{Q}' &= \phi(\mathbf{Q}) \\ &= \begin{bmatrix} \phi(\mathbf{q}_1) \\ \vdots \\ \phi(\mathbf{q}_n) \end{bmatrix}, \qquad (6.142) \\ \mathbf{K}' &= \phi(\mathbf{K}) \\ &= \begin{bmatrix} \phi(\mathbf{k}_1) \\ \vdots \\ \phi(\mathbf{k}_n) \end{bmatrix}. \qquad (6.143)\end{aligned}$$

Sau đó, ta xây dựng một mô hình chú ý hạt nhân hóa (*kernelized attention model*) bằng cách xấp xỉ trọng số chú ý $\alpha_{i,j}$ dưới dạng

$$\alpha_{i,j} \approx \frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top}. \qquad (6.144)$$

Ý tưởng then chốt đằng sau mô hình chú ý hạt nhân hóa này là ta có thể bỏ hàm Softmax nếu các truy vấn và khóa được ánh xạ sang một không

gian mới. Dùng phép xấp xỉ này, vectơ đầu ra thứ i của mô hình chú ý (tức là vectơ hàng thứ i của $\text{Att}_{\text{self}}(\mathbf{S})$) được cho bởi

$$\begin{aligned}\mathbf{c}_i &= \sum_{j=1}^{n} \alpha_{i,j} \cdot \mathbf{v}_j \\ &\approx \sum_{j=1}^{n} \Big(\frac{\phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top} \cdot \mathbf{v}_j \Big) \\ &= \frac{\sum_{j=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_j)^\top \mathbf{v}_j}{\sum_{j'=1}^{n} \phi(\mathbf{q}_i)\phi(\mathbf{k}_{j'})^\top} \\ &= \frac{\phi(\mathbf{q}_i)\big(\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j\big)}{\phi(\mathbf{q}_i)\big(\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top\big)}. \end{aligned} \tag{6.145}$$

Ở đây $\phi(\mathbf{q}_i) \in \mathbb{R}^{1\times d'}$, còn $\phi(\mathbf{k}_j)^\top \mathbf{v}_j \in \mathbb{R}^{d'\times d}$. Do đó, số hạng bên trong $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j$ là một ma trận $d' \times d$, và tích cuối cùng cho ra một vectơ trong $\mathbb{R}^{1\times d}$.

Dù phương trình trông có vẻ hơi phức tạp, ý tưởng lại đơn giản: thay vì cho truy vấn chú ý tới mọi khóa để thu được trọng số chú ý $\alpha_{i,j}$, ta có thể tính tổng các tích $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j \in \mathbb{R}^{d'\times d}$ rồi nhân nó với truy vấn đã hạt nhân hóa $\phi(\mathbf{q}_i)$. Quay lại ký hiệu dùng trong Phương trình (6.141), ta định nghĩa phần tử đường chéo thứ i của $\mathbf{D}$ là $\phi(\mathbf{q}_i)\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top$. Khi đó, mô hình chú ý có thể được viết lại dưới dạng

$$\begin{aligned}\text{Att}_{\text{self}}(\mathbf{S}) &= \mathbf{D}^{-1}\phi(\mathbf{Q})\phi(\mathbf{K})^\top \mathbf{V} \\ &= \mathbf{D}^{-1}\mathbf{Q}'\mathbf{K}'^\top \mathbf{V} \\ &= \mathbf{D}^{-1}\big(\mathbf{Q}'(\mathbf{K}'^\top \mathbf{V})\big). \end{aligned} \tag{6.146}$$

Ở đây ta đổi thứ tự tính toán từ trái-sang-phải thành phải-sang-trái bằng cách dùng dấu ngoặc. Với $\mathbf{Q}' \in \mathbb{R}^{n\times d'}$ và $\mathbf{K}' \in \mathbb{R}^{n\times d'}$, mô hình này có độ phức tạp thời gian và không gian lần lượt là $O(n \cdot d \cdot d')$ và $O(n \cdot d + n \cdot d' + d \cdot d')$. Do đó, mô hình tuyến tính theo độ dài chuỗi n, và đôi khi được gọi là **mô hình chú ý tuyến tính** (*linear attention model*). Một ưu điểm tính toán của mô hình này là ta chỉ cần tính phép nhân $\mathbf{K}'^\top \mathbf{V}$ (tức là $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j$) và hệ số chuẩn hóa tương ứng (tức là $\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top$) một lần. Các kết quả này sau đó có thể được dùng cho bất kỳ truy vấn nào [Katharopoulos và cộng sự, 2020]. Mô hình cần duy trì $\sum_{j=1}^{n} \phi(\mathbf{k}_j)^\top \mathbf{v}_j$

và $\sum_{j'=1}^{n} \phi(\mathbf{k}_{j'})^\top$, và cập nhật chúng khi có các vectơ khóa và giá trị mới đến.

Chú giải của người dịch: Vì sao đổi thứ tự nhân lại tiết kiệm đến vậy?

Phép nhân ma trận có tính kết hợp: $(\mathbf{XY})\mathbf{Z} = \mathbf{X}(\mathbf{YZ})$, kết quả như nhau nhưng chi phí có thể rất khác. Tính $(\mathbf{Q}'\mathbf{K}'^\top)\mathbf{V}$ trước tiên tạo ra ma trận $n \times n$ (cỡ $n^2 d'$ phép nhân). Còn tính $\mathbf{K}'^\top \mathbf{V}$ trước chỉ tạo ma trận nhỏ $d' \times d$ (cỡ $n\, d' d$ phép nhân), rồi nhân $\mathbf{Q}'$ với nó (thêm cỡ $n\, d' d$). Với $n = 100\,000$, $d = d' = 64$: cách thứ nhất cần khoảng $6{,}4 \times 10^{11}$ phép nhân, cách thứ hai chỉ khoảng 8×10^8 — ít hơn gần 800 lần. Mẹo này chỉ dùng được khi đã bỏ Softmax, vì Softmax phải áp dụng lên từng hàng của ma trận $n \times n$ nên buộc ta tính nó trước.

Dù vậy, mô hình hạt nhân hóa này còn một số hạn chế, chẳng hạn làm thế nào để xây dựng ánh xạ đặc trưng (*feature map*) $\phi(\cdot)$ sao cho xấp xỉ tốt mô hình chú ý chuẩn. Bạn đọc quan tâm có thể tham khảo công trình của Choromanski và cộng sự [2020] để biết thêm chi tiết.

Ý tưởng thứ hai để giảm d là dùng các mô hình không gian con (*subspace model*), trong đó một bài toán trong không gian d chiều được biến đổi thành các bài toán con trong những không gian có số chiều thấp hơn, và lời giải của bài toán gốc được xấp xỉ bằng một tổ hợp nào đó của các lời giải của những bài toán con này. Trong một mô hình không gian con tổng quát, một vectơ khóa d chiều $\mathbf{k}$ có thể được ánh xạ thành một tập các vectơ d' chiều $\{\mathbf{K}'_1, \cdots, \mathbf{K}'_\eta\}$. Để đơn giản hóa việc mô hình hóa, ta có thể làm điều này bằng cách phân đoạn vectơ, tức là chia $\mathbf{k}$ thành η vectơ con, mỗi vectơ có $d' = \frac{d}{\eta}$ chiều. Ta có thể biến đổi mọi vectơ truy vấn và giá trị theo cùng cách đó. Sau đó, mô hình chú ý được áp dụng trong từng không gian con này.

Tuy nhiên, phương pháp này không làm giảm tổng lượng tính toán. Như được trình bày trong công trình của Lample và cộng sự [2019], thay vào đó ta có thể xấp xỉ chú ý tích vô hướng trên một tập các cặp khóa–giá trị bằng cách xét p ứng viên hàng đầu trong mỗi không gian con. Chính xác hơn, ta tìm p cặp khóa–giá trị tốt nhất trong mỗi không gian con, việc này rẻ hơn về tính toán. Tích Descartes (*Cartesian product*) của các tập p khóa tốt nhất này gồm p^η khóa tích (*product key*). Tương tự, ta thu được p^η giá trị tích. Phần việc còn lại rất đơn giản: các truy vấn d chiều chú ý tới

những khóa tích và giá trị tích d chiều này. Một khác biệt thú vị giữa mô hình không gian con này và mô hình trong không gian d chiều là các khóa tích và giá trị tích được sinh ra có thể khác với bất kỳ cặp khóa–giá trị gốc nào trong $\{(\mathbf{k}_1, \mathbf{v}_1), \cdots, (\mathbf{k}_{i-1}, \mathbf{v}_{i-1})\}$. Điều này mang lại một cách để học các biểu diễn mới của thông tin trong quá khứ.

Chú giải của người dịch: Khóa tích: ghép các "nửa" tốt nhất

Lấy $\eta = 2$: mỗi khóa được cắt làm hai nửa. Ở nửa thứ nhất, ta tìm $p = 3$ nửa-khóa khớp nhất với nửa đầu của truy vấn, ví dụ $\{a_1, a_2, a_3\}$; ở nửa thứ hai tìm được $\{b_1, b_2, b_3\}$. Ghép mọi cặp (tích Descartes, giống việc đếm số cách chọn một bộ "áo–quần" theo quy tắc nhân) cho $3 \times 3 = 3^2 = 9$ khóa đầy đủ $[a_s; b_t]$. Lưu ý một khóa ghép như $[a_1; b_3]$ có thể chưa từng xuất hiện nguyên vẹn trong dữ liệu — đó là lý do sách nói mô hình có thể tạo ra "biểu diễn mới".

Cho tới giờ, chúng ta đã bàn về các cách tiếp cận giảm chiều theo chiều n hoặc chiều d. Có thể dễ dàng kết hợp chúng để xây dựng một mô hình "ít chiều hơn". Ví dụ, giả sử ta có phép giảm $n \to n'$ cho khóa và giá trị, và phép giảm $d \to d'$ cho truy vấn và khóa. Mô hình có dạng

$$\begin{aligned} \mathrm{Att}_{\mathrm{self}}(\mathbf{S}) &= \mathbf{A}\mathbf{V}', \\ \mathbf{A} &= \mathrm{Softmax}\left(\frac{\mathbf{Q}'\mathbf{K}'^{\top}}{\sqrt{d'}} + \mathbf{M}\right), \end{aligned} \tag{6.147}$$

trong đó $\mathbf{Q}' \in \mathbb{R}^{n \times d'}$, $\mathbf{K}' \in \mathbb{R}^{n' \times d'}$ và $\mathbf{V}' \in \mathbb{R}^{n' \times d}$ là các biểu diễn ít chiều của truy vấn, khóa và giá trị. Như thường lệ, ta có thể dễ dàng thu được các biểu diễn này thông qua các ánh xạ tuyến tính của $\mathbf{Q}$, $\mathbf{K}$ và $\mathbf{V}$. Độ phức tạp thời gian và không gian của mô hình này là $O(n' \cdot n \cdot d')$ và $O(n' \cdot n + n' \cdot d')$.

6.4.4 Chia sẻ tham số và giá trị kích hoạt

Sự dư thừa (*redundancy*) là hiện tượng phổ biến ở hầu hết các mạng nơ-ron quy mô lớn. Kết quả là nhiều mô hình trong số này bị thừa tham số (*over-parameterized*), khiến việc huấn luyện và suy luận kém hiệu quả hơn. Một cách tiếp cận phổ biến để giảm dư thừa là đơn giản hóa mô hình bằng cách loại bỏ các thành phần không cần thiết. Chẳng hạn, ta có thể tỉa (*prune*) một mô hình phức tạp hoặc chia sẻ các mô-đun con giữa các

phần khác nhau của mô hình để thu được một mô hình gọn hơn. Trong tiểu mục này, chúng ta bàn về các phương pháp chia sẻ tham số và trạng thái trung gian trong các mô hình Transformer. Việc thảo luận về chuyển giao mô hình và tỉa mô hình được để lại cho Mục 6.4.7.

Các kiến trúc chia sẻ tham số được dùng rộng rãi trong các hệ thống dựa trên mạng nơ-ron. Các ví dụ nổi tiếng gồm CNN và RNN, trong đó cùng một bộ tham số (hoặc các tầng) được áp dụng trên những vùng khác nhau của đầu vào. Điều này tạo ra một mạng nơ-ron lớn mà nhiều phần của nó có cùng kiến trúc và cùng các tham số được chia sẻ. Đối với Transformer cũng như các mô hình chuỗi khác, cơ chế chia sẻ có thể được áp dụng ở các mức mô hình hóa khác nhau. Một ví dụ đơn giản là chia sẻ vectơ nhúng (*embedding sharing*). Trong dịch máy, một chiến lược điển hình để xử lý các từ thuộc hai ngôn ngữ là xây dựng hai mô hình nhúng riêng biệt. Một cách khác là dùng một mô hình nhúng duy nhất cho cả hai ngôn ngữ. Khi đó, các tham số của mô hình được học trong quá trình huấn luyện cả mạng phía nguồn lẫn mạng phía đích. Chiến lược như vậy cũng thường được áp dụng trong các mô hình chuỗi đa ngôn ngữ, chẳng hạn các mô hình ngôn ngữ có khả năng xử lý văn bản bằng nhiều ngôn ngữ khác nhau.

Đối với các mạng nơ-ron nhiều tầng, một phương pháp phổ biến là chia sẻ theo tầng (*layer-wise sharing*). Giả sử có một chồng các tầng, tất cả đều có cùng dạng

$$\mathbf{S}^l \;=\; \text{Layer}(\mathbf{S}^{l-1}; \theta^l). \tag{6.148}$$

Ta có thể ràng buộc tham số cho một số hoặc tất cả các tầng này. Chẳng hạn, cho một tập các tầng $\{l_1, l_2, \cdots, l_n\}$, ta áp đặt ràng buộc $\theta^{l_1} = \theta^{l_2} = \cdots = \theta^{l_n}$, để có thể thu được một mô hình nhỏ hơn và việc tối ưu mô hình có thể dễ dàng hơn. Trên thực tế, mô hình chia sẻ tầng này rất có lợi nếu có nhiều tầng tham gia, vì ta có thể lặp lại cùng một quá trình nhiều lần để xây dựng một mạng nơ-ron rất sâu [Dehghani và cộng sự, 2018]. Ví dụ, việc chia sẻ một tầng con FFN duy nhất cho tất cả các tầng của bộ mã hóa được thấy là hiệu quả trong việc giảm dư thừa ở các hệ thống dịch máy [Pires và cộng sự, 2023].

Chú giải của người dịch: Chia sẻ tầng tiết kiệm bao nhiêu?

Giả sử mỗi tầng có 10 triệu tham số và mô hình có 12 tầng: không chia sẻ thì cần $12 \times 10 = 120$ triệu tham số. Nếu cả 12 tầng dùng chung một bộ θ, mô hình chỉ còn 10 triệu tham số, nhưng vẫn "chạy" qua 12 lượt biến đổi — giống như dùng cùng một chiếc máy ép 12 lần thay vì mua 12 chiếc máy. Lưu ý: số tham số (bộ nhớ) giảm mạnh, nhưng số phép tính khi chạy thì không giảm.

Đối với Transformer, việc chia sẻ cũng có thể được thực hiện trong chú ý đa đầu. Một ví dụ là **chú ý đa truy vấn** (*multi-query attention*) [Shazeer, 2019]. Nhớ lại từ Mục 6.1.3 rằng đầu ra của một đầu h trong tự chú ý đa đầu chuẩn có thể được viết là

$$\begin{aligned}\mathbf{C}_h^{\text{head}} &= \text{Att}_{\text{qkv}}(\mathbf{S}_h^q, \mathbf{S}_h^k, \mathbf{S}_h^v) \\ &= \text{Att}_{\text{qkv}}(\mathbf{S}\mathbf{W}_h^q, \mathbf{S}\mathbf{W}_h^k, \mathbf{S}\mathbf{W}_h^v).\end{aligned} \tag{6.149}$$

Ở đây $\mathbf{S}_h^q = \mathbf{S}\mathbf{W}_h^q$, $\mathbf{S}_h^k = \mathbf{S}\mathbf{W}_h^k$ và $\mathbf{S}_h^v = \mathbf{S}\mathbf{W}_h^v$ là các truy vấn, khóa và giá trị, thu được bằng cách biến đổi tuyến tính đầu vào $\mathbf{S}$ với các ma trận tham số riêng biệt $\mathbf{W}_h^q$, $\mathbf{W}_h^k$ và $\mathbf{W}_h^v$. Trong chú ý đa truy vấn, ta chia sẻ cùng các khóa và giá trị cho tất cả các đầu, nhưng dùng các truy vấn khác nhau cho các đầu khác nhau. Dạng của mô hình này được cho bởi

$$\mathbf{C}_h^{\text{head}} = \text{Att}_{\text{qkv}}(\mathbf{S}\mathbf{W}_h^q, \mathbf{S}\mathbf{W}_0^k, \mathbf{S}\mathbf{W}_0^v). \tag{6.150}$$

Vì các khóa $\mathbf{S}\mathbf{W}_0^k$ và giá trị $\mathbf{S}\mathbf{W}_0^v$ không phụ thuộc vào h, chúng chỉ được tính và lưu một lần, thay vì lặp lại một cách dư thừa cho mỗi đầu. Điều này giúp tiết kiệm đáng kể về tính toán và băng thông bộ nhớ (*memory bandwidth*) trong quá trình suy luận, đặc biệt khi số đầu chú ý lớn. Nhờ tính hiệu quả của nó, chú ý đa truy vấn (và biến thể của nó, chú ý truy vấn theo nhóm (*grouped-query attention*)) đã được tích hợp thành công vào các mô hình ngôn ngữ lớn hiện đại, bao gồm Llama 2 [Touvron và cộng sự, 2023b] và Falcon[15].

[15] `https://falconllm.tii.ae/index.html`

Chú giải của người dịch: Đa truy vấn và theo nhóm

Với 32 đầu chú ý, chú ý đa đầu chuẩn phải lưu 32 bộ khóa–giá trị cho mỗi token trong bộ nhớ đệm KV; chú ý đa truy vấn chỉ lưu 1 bộ dùng chung, tức là nhỏ hơn 32 lần. Chú ý truy vấn theo nhóm là phương án ở giữa: chia 32 đầu thành, chẳng hạn, 8 nhóm, mỗi nhóm 4 đầu dùng chung một bộ khóa–giá trị, nên chỉ cần lưu 8 bộ. Cách này cân bằng giữa tiết kiệm bộ nhớ và giữ chất lượng mô hình.

Mở rộng ý tưởng chia sẻ sang các tình huống tổng quát hơn, về nguyên tắc, các trạng thái trung gian có thể được chia sẻ trong khắp một mạng nơ-ron. Chẳng hạn, việc tái sử dụng giá trị kích hoạt của nơ-ron cho phép các thành phần con được tính nhiều lần một cách hiệu quả. Trong Transformer, phương pháp này có thể được áp dụng trực tiếp cho các ma trận tự chú ý. Các nghiên cứu đã cho thấy bằng thực nghiệm rằng các bản đồ chú ý ở những tầng kề nhau thường có tương quan cao trong một số tác vụ NLP [Xiao và cộng sự, 2019]. Do đó, một chiến lược hiệu quả về tính toán là tính bản đồ chú ý dày đặc một lần và tái sử dụng nó ở các tầng tiếp theo.

Nói rộng hơn, cơ chế chia sẻ có thể được xem như một quá trình trong đó ta tái sử dụng các kết quả đã tính trước đó thay vì tính lại chúng ngay tại chỗ. Do đó, có thể tái sử dụng thông tin giữa các lần chạy khác nhau của một mạng nơ-ron. Một ví dụ liên quan là mạng phần dư khả nghịch (*reversible residual network*), trong đó giá trị kích hoạt của một tầng có thể được khôi phục từ giá trị kích hoạt của tầng ngay sau nó [Gomez và cộng sự, 2017]. Vì vậy, ta chỉ giữ đầu ra của tầng mới nhất trong lượt truyền xuôi. Sau đó, trong lượt truyền ngược của quá trình huấn luyện, ta tái tạo đầu ra của mỗi tầng từ tầng kế sau nó. Một ưu điểm của thiết kế khả nghịch này là thông tin sinh ra trong lượt truyền xuôi được chia sẻ một cách ngầm định, và mô hình tiết kiệm bộ nhớ [Kitaev và cộng sự, 2020].

Chú giải của người dịch: "Khả nghịch" nghĩa là gì?

Một phép biến đổi là khả nghịch nếu từ đầu ra ta suy ngược được đầu vào, như hàm $y = 2x + 1$ có hàm ngược $x = (y - 1)/2$. Bình thường, khi huấn luyện, ta phải lưu đầu ra của *mọi* tầng để dùng lúc lan truyền ngược, rất tốn bộ nhớ khi mạng sâu. Với các tầng khả nghịch, ta chỉ cần lưu đầu ra tầng cuối, rồi lần lượt "tính ngược" ra đầu ra của các tầng trước khi cần — đổi một ít phép tính lấy rất nhiều bộ nhớ.

6.4.5 Các phương án thay thế cho tự chú ý

Ta đã thấy rằng việc dùng tự chú ý (*self-attention*) là một trong những nguồn chính gây ra chi phí tính toán và bộ nhớ cao trong Transformer. Một câu hỏi tự nhiên là liệu có những phương án thay thế hiệu quả cho các mô hình tự chú ý hay không. Ở đây, chúng tôi trình bày ngắn gọn một số biến thể của Transformer, trong đó các tầng con tự chú ý được thay bằng những loại kiến trúc mạng nơ-ron khác.

1. CNN thay thế cho tự chú ý

Mạng nơ-ron tích chập (*convolutional neural network*) (CNN) là loại mạng nơ-ron đơn giản và được dùng rộng rãi, và được coi là phương án tiềm năng để thay thế các mô hình tự chú ý. Để áp dụng CNN vào Transformer, tất cả những gì ta cần là xây dựng một tầng con tích chập để thay cho tầng con tự chú ý trong một khối Transformer. Mặc dù một bộ lọc CNN có trường tiếp nhận (*receptive field*) hạn chế và do đó chỉ nhận đầu vào từ một cửa sổ ngữ cảnh "cục bộ", ta vẫn có thể dễ dàng mô hình hóa ngữ cảnh rộng bằng cách xếp chồng nhiều tầng con tích chập. Một ưu điểm chính của CNN là độ phức tạp tính toán của nó tăng tuyến tính theo độ dài chuỗi n, so với mức tăng bậc hai của các mô hình tự chú ý. Hơn nữa, việc có sẵn các cài đặt CNN đã được tối ưu hóa rất tốt khiến chúng dễ áp dụng cho mô hình hóa chuỗi trong các hệ thống thực tế. Để cải thiện thêm hiệu quả bộ nhớ, ta có thể dùng các biến thể CNN gọn nhẹ, chẳng hạn CNN theo chiều sâu (*depthwise CNN*) [Wu và cộng sự,

2018a][16].

Chú giải của người dịch: Vì sao CNN theo chiều sâu tiết kiệm tham số?

Lấy ví dụ nhỏ: cửa sổ $k = 3$ từ, mỗi từ có $d = 4$ đặc trưng. Một bộ lọc chuẩn nhìn cả khối $3 \times 4 = 12$ số, nhân từng số với một trọng số rồi cộng lại thành *một* số đầu ra. Muốn đầu ra cũng có 4 đặc trưng thì cần 4 bộ lọc như vậy: $4 \times 12 = 48$ tham số ($= d^2 \cdot k$). Với CNN theo chiều sâu, mỗi "kênh" (mỗi cột đặc trưng) chỉ được trộn *với chính nó* dọc theo 3 vị trí, nên mỗi kênh chỉ cần 3 trọng số: tổng cộng $4 \times 3 = 12$ tham số ($= d \cdot k$). Khi d lớn (ví dụ $d = 1024$), mức tiết kiệm là khoảng d lần.

2. Chú ý tuyến tính

Cũng như nhiều cách tiếp cận thực tế cho mô hình hóa chuỗi, có sự quan tâm đáng kể tới việc phát triển các mô hình tuyến tính nhằm tăng tốc xử lý chuỗi dài. Tuy có nhiều cách định nghĩa một mô hình tuyến tính, một dạng tổng quát thường được dùng trong các mô hình chuỗi là

$$\mathbf{z}_i \;=\; f(a \cdot \mathbf{z}_{i-1} + b \cdot \mathbf{s}_i). \tag{6.152}$$

Ở đây $\mathbf{s}_i$ biểu diễn một số trạng thái trung gian của mô hình tại bước i, và $\mathbf{z}_i$ biểu diễn bản tóm tắt các trạng thái quá khứ tính đến bước i. Dễ thấy đây là một mô hình hồi quy: đầu ra tại bước i chỉ phụ thuộc vào đầu vào ở bước hiện tại và đầu ra ở bước trước đó. Giống như các lựa chọn

[16]Nhắc lại từ Chương 2 rằng trong CNN, một bộ lọc (hoặc một tập bộ lọc) ánh xạ các biến đầu vào nằm trong trường tiếp nhận của nó thành một biến đầu ra (hoặc một tập biến đầu ra) thông qua một ánh xạ tuyến tính. Giả sử đầu vào và đầu ra của một bài toán được biểu diễn dưới dạng chuỗi các vectơ đặc trưng. Cho một bộ lọc chuẩn có trường tiếp nhận kích thước $k \times d$, ta trượt nó dọc theo chuỗi. Ở mỗi bước, bộ lọc nhận $k \times d$ đặc trưng đầu vào và tạo ra một đặc trưng đầu ra duy nhất. Quá trình này thường được biểu diễn như sau:

$$y \;=\; \text{ReduceSum}(\mathbf{x} \odot \mathbf{W}), \tag{6.151}$$

trong đó $\mathbf{x} \in \mathbb{R}^{k \times d}$ là biểu diễn ma trận của đầu vào, $y \in \mathbb{R}$ là đặc trưng đầu ra, và $\mathbf{W} \in \mathbb{R}^{k \times d}$ là ma trận trọng số. Hàm $\text{ReduceSum}(\cdot)$ tính tổng của tất cả các tích theo từng phần tử giữa $\mathbf{x}$ và $\mathbf{W}$. Nếu muốn đầu vào và đầu ra có cùng số đặc trưng d, ta phải áp dụng d bộ lọc khác nhau, dẫn đến tổng cộng $d \cdot (k \cdot d) = d^2 \cdot k$ tham số. Trong CNN theo chiều sâu, ta tách phép tích chập theo không gian khỏi các chiều đặc trưng. Thay vì áp dụng các bộ lọc đầy đủ $k \times d$, mỗi kênh trong d kênh đặc trưng đầu vào được tích chập độc lập với bộ lọc không gian 1 chiều riêng có kích thước k. Nhờ đó, số tham số riêng biệt giảm mạnh xuống còn $d \cdot k$ (mỗi kênh trong d kênh ứng với một bộ lọc riêng chứa k tham số riêng biệt).

thiết kế phổ biến trong các hệ thống dựa trên mạng nơ-ron, phần tuyến tính được nối tiếp bởi một phép biến đổi $f(\cdot)$, có thể là một hàm kích hoạt hoặc một mạng nơ-ron truyền thẳng. Lưu ý rằng Phương trình (6.152) chỉ định nghĩa một mô hình tuyến tính chuẩn khi $f(\cdot)$ là hàm tuyến tính. Việc dùng $f(\cdot)$ mang lại sự linh hoạt lớn hơn trong mô hình hóa bài toán, mặc dù thuật ngữ *mô hình tuyến tính* có thể không còn áp dụng được nếu $f(\cdot)$ có dạng phi tuyến.

Công thức trên mô tả một mô hình có cấu trúc tuyến tính, có thể được xem như một trường hợp của một họ mô hình toán học tổng quát. Thông thường, nó có thể được biểu diễn như một cấu trúc chuỗi, hay một tập các nút có thứ tự. Mô hình lặp lại cùng một quá trình tính toán từ nút đầu tiên đến nút cuối cùng, mỗi lần lấy thông tin từ bước hiện tại và bước trước, rồi tạo ra một vectơ đầu ra được dùng ở các bước thời gian tiếp theo. Do đó, chi phí bộ nhớ và thời gian của mô hình tăng tuyến tính theo độ dài của chuỗi.

Ta có thể mở rộng Phương trình (6.152) thành một mô hình RNN chuẩn chỉ bằng cách áp dụng một phép biến đổi tuyến tính cho đầu vào hiện tại và trạng thái trước đó, tức là $\mathbf{z}_i = f(\mathbf{z}_{i-1} \cdot \mathbf{W}_z + \mathbf{s}_i \cdot \mathbf{W}_s)$. Do vậy, việc áp dụng RNN và các biến thể của nó vào Transformer để thu được một mô hình lai là khá đơn giản. Ví dụ, ta có thể dùng LSTM và GRU để xây dựng một số tầng Transformer, nhằm kết hợp ưu điểm của cả mô hình hồi quy lẫn mô hình tự chú ý [Chen và cộng sự, 2018b]. Vì các mô hình hồi quy truyền thống đã được bàn kỹ ở Chương 2, chúng tôi bỏ qua phần thảo luận về chúng ở đây.

Chú giải của người dịch: Mô hình "hồi quy" tuyến tính hiểu thế nào?

Phương trình (6.152) giống một *dãy số truy hồi* quen thuộc ở lớp 11: số hạng sau được tính từ số hạng trước. Chẳng hạn với $f(x) = x$, $a = 0.5$, $b = 1$ và các đầu vào vô hướng $s_1 = 2, s_2 = 4, s_3 = 6$, bắt đầu từ $z_0 = 0$: $z_1 = 2$, $z_2 = 0.5 \cdot 2 + 4 = 5$, $z_3 = 0.5 \cdot 5 + 6 = 8.5$. Mỗi bước chỉ cần nhớ *một* giá trị z_{i-1}, nên đi hết chuỗi dài n chỉ tốn n bước — chi phí tăng tuyến tính. Ngược lại, tự chú ý cho mỗi vị trí "nhìn" lại tất cả vị trí khác, nên cần khoảng $n \times n$ phép so sánh.

Thực ra, ta có thể quan tâm nhiều hơn tới việc phát triển các mô hình chú ý tuyến tính (*linear attention*), để có được một hệ thống hiệu quả mà

vẫn giữ được lợi ích của việc mô hình hóa chuỗi bằng chú ý toàn cục. Một phần khó khăn khi làm điều này là dạng của tự chú ý không tuyến tính. Hãy dành một chút thời gian để xem khó khăn này nảy sinh như thế nào. Nhắc lại rằng kết quả của tự chú ý có thể được viết dưới dạng sau

$$\begin{aligned}\text{Att}_{\text{self}} &= \mathbf{A} \cdot \mathbf{V} \\ &= \psi(\mathbf{Q} \cdot \mathbf{K}^\top) \cdot \mathbf{V}.\end{aligned} \tag{6.153}$$

Ở đây $\psi(\cdot)$ là một hàm được ghép từ các phép co giãn, lấy mũ, che (masking) và chuẩn hóa (tức là $\psi(\mathbf{a}) = \text{Normalize}(\text{Mask}(\exp(\frac{\mathbf{a}}{\sqrt{d}})))$). Vì $\psi(\cdot)$ là một hàm phi tuyến phức tạp, không có cách trực tiếp nào để đơn giản hóa phép tính, và ta thường phải tính riêng hai phép nhân ma trận (một bên trong $\psi(\cdot)$ và một bên ngoài $\psi(\cdot)$). Hệ quả là ta cần lưu trữ tường minh tất cả các cặp khóa–giá trị, và duyệt qua từng cặp với mỗi truy vấn. Không có gì ngạc nhiên khi điều này dẫn tới một mô hình có chi phí tính toán tăng theo bình phương độ dài chuỗi n.

Mặc dù trong tự chú ý, khóa và giá trị gắn liền với nhau, chúng được dùng ở các bước riêng biệt. Một giải pháp tinh tế là tách truy vấn khỏi tương tác khóa–giá trị, sao cho thông tin ngữ cảnh có thể được mã hóa độc lập với bất kỳ truy vấn cụ thể nào. Ý tưởng then chốt là bỏ qua hàm Softmax phi tuyến bằng cách áp dụng một ánh xạ đặc trưng (*feature map*) $\phi(\cdot)$ cho các truy vấn và khóa. Như đã giới thiệu ở Mục 6.4.3, ta biến đổi $\mathbf{Q}$ và $\mathbf{K}$ thành $\mathbf{Q}' = \phi(\mathbf{Q}) \in \mathbb{R}^{n \times d'}$ và $\mathbf{K}' = \phi(\mathbf{K}) \in \mathbb{R}^{n \times d'}$. Khi đó cơ chế chú ý có thể được viết lại thành:

$$\begin{aligned}\text{Att}_{\text{self}} &\equiv \psi'(\mathbf{Q}' \cdot \mathbf{K}'^\top) \cdot \mathbf{V} \\ &= \mathbf{D}^{-1}(\mathbf{Q}' \cdot \mathbf{K}'^\top) \cdot \mathbf{V} \\ &= \mathbf{D}^{-1}\Big(\mathbf{Q}' \cdot (\mathbf{K}'^\top \cdot \mathbf{V})\Big),\end{aligned} \tag{6.154}$$

trong đó $\mathbf{D}$ là ma trận chuẩn hóa dạng đường chéo, và $\psi'(\mathbf{Z}) = \mathbf{D}^{-1}\mathbf{Z}$[17]. Trong không gian đã biến đổi này, tích vô hướng truy vấn–khóa không cần đến chuẩn hóa Softmax dạng mũ. Nó tương ứng với việc chuẩn hóa bằng cách nhân với $\mathbf{D}^{-1}$. Vì phép toán cốt lõi giờ chỉ còn là phép nhân ma trận thuần túy, ta có thể khai thác tính kết hợp của phép nhân ma trận để thay đổi thứ tự tính toán.

[17] Nếu $\mathbf{D}$ là ma trận đường chéo, đôi khi chúng tôi ký hiệu $\mathbf{D}^{-1}\mathbf{A}$ là $\frac{\mathbf{A}}{\mathbf{D}}$.

Chú giải của người dịch: Đổi thứ tự nhân để tiết kiệm

Phép nhân ma trận có tính kết hợp: $(\mathbf{XY})\mathbf{Z} = \mathbf{X}(\mathbf{YZ})$, giống $(2 \cdot 3) \cdot 4 = 2 \cdot (3 \cdot 4)$, nhưng *khối lượng tính toán* của hai cách có thể rất khác nhau. Với $\mathbf{Q}', \mathbf{K}'$ cỡ $n \times d'$ và $\mathbf{V}$ cỡ $n \times d$: tính $\mathbf{Q}'\mathbf{K}'^\top$ trước tạo ra ma trận $n \times n$ (khoảng $n^2 d'$ phép nhân), còn tính $\mathbf{K}'^\top \mathbf{V}$ trước chỉ tạo ra ma trận nhỏ $d' \times d$ (khoảng $nd'd$ phép nhân). Ví dụ $n = 10\,000$, $d = d' = 64$: cách đầu cần cỡ 6.4 tỷ phép nhân cho bước này, cách sau chỉ cỡ 41 triệu. Điều này chỉ làm được khi bỏ Softmax, vì Softmax "kẹp" giữa $\mathbf{Q}$ và $\mathbf{K}$ không cho ta đổi thứ tự.

Điều này dẫn tới một cách biểu diễn tự hồi quy mạnh mẽ: các khóa và giá trị trước hết được gộp lại qua $\mathbf{K}'^\top \cdot \mathbf{V}$, rồi các truy vấn chú ý tới biểu diễn toàn cục này. Vì $\mathbf{K}'^\top \cdot \mathbf{V} = \sum_{j=1}^{n} \mathbf{k}_j'^\top \cdot \mathbf{v}_j$, ta có thể viết $\mathbf{K}'^\top \cdot \mathbf{V}$ dưới dạng của Phương trình (6.152) như sau

$$\boldsymbol{\mu}_j \;=\; \boldsymbol{\mu}_{j-1} + \mathbf{k}_j'^\top \cdot \mathbf{v}_j. \tag{6.155}$$

Ở đây $\boldsymbol{\mu}_j \in \mathbb{R}^{d' \times d}$ là một biến được cộng dồn thêm $\mathbf{k}_j'^\top \cdot \mathbf{v}_j$ mỗi lần. Tương tự, ta có thể định nghĩa một biến khác $\boldsymbol{\nu}_j \in \mathbb{R}^{d'}$

$$\boldsymbol{\nu}_j \;=\; \boldsymbol{\nu}_{j-1} + \mathbf{k}_j'^\top. \tag{6.156}$$

Khi đó, đầu ra của tự chú ý cho truy vấn thứ j có thể được viết là (xem thêm Phương trình (6.145))

$$\mathrm{Att}_{\mathrm{self},j} \;=\; \frac{\mathbf{q}_j' \cdot \boldsymbol{\mu}_n}{\mathbf{q}_j' \cdot \boldsymbol{\nu}_n}. \tag{6.157}$$

Rõ ràng, cách này tạo ra một mô hình có thời gian tuyến tính, vì $\boldsymbol{\mu}_n$ và $\boldsymbol{\nu}_n$ là tuyến tính theo n. Trong các cài đặt đơn giản của mô hình này, chỉ $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ được lưu giữ. Mỗi khi gặp một truy vấn mới, ta cập nhật $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ bằng Phương trình (6.155) và (6.156), rồi tính $\mathrm{Att}_{\mathrm{self},j} = \frac{\mathbf{q}_j' \cdot \boldsymbol{\mu}_j}{\mathbf{q}_j' \cdot \boldsymbol{\nu}_j}$[18].

[18]Trong sinh văn bản tự hồi quy, ta sinh chuỗi từ trái sang phải. Trong trường hợp này, ta không cần xét các khóa và giá trị ở các vị trí $> j$.

Chú giải của người dịch: Ý nghĩa của $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$

Hãy coi $\boldsymbol{\mu}_j$ là một "cuốn sổ" kích thước cố định ($d' \times d$ số) ghi tổng dồn mọi cặp khóa–giá trị đã gặp, còn $\boldsymbol{\nu}_j$ ghi tổng dồn các khóa (dùng để chuẩn hóa, giống như chia cho tổng trọng số khi tính trung bình có trọng số). Khi một từ mới đến, ta chỉ *cộng thêm* một mục vào sổ, không phải đọc lại toàn bộ lịch sử. Tử số $\mathbf{q}'_j \cdot \boldsymbol{\mu}_j$ là "tổng có trọng số của các giá trị", mẫu số $\mathbf{q}'_j \cdot \boldsymbol{\nu}_j$ là "tổng các trọng số"; thương của chúng là một trung bình có trọng số — đúng tinh thần của cơ chế chú ý, nhưng với chi phí mỗi bước không đổi.

Một mở rộng đơn giản của mô hình chú ý tuyến tính là cho phép Phương trình (6.155) và (6.156) kết hợp các số hạng khác nhau với trọng số khác nhau. Ví dụ, ta có thể định nghĩa lại $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ là

$$\boldsymbol{\mu}_j = a \cdot \boldsymbol{\mu}_{j-1} + (1-a) \cdot \mathbf{k}_j'^{\top} \cdot \mathbf{v}_j, \tag{6.158}$$

$$\boldsymbol{\nu}_j = a \cdot \boldsymbol{\nu}_{j-1} + (1-a) \cdot \mathbf{k}_j'^{\top} \tag{6.159}$$

và huấn luyện tham số a như thường lệ. Ngoài ra, ta có thể coi a như một cổng và dùng một mạng nơ-ron khác để tính a [Peng và cộng sự, 2021]. Một thiết kế mô hình khác là thêm nhiều số hạng hơn vào Phương trình (6.155) và (6.156) để có một công thức biểu đạt mạnh hơn cho cách tiếp cận chú ý tuyến tính [Bello, 2020; Schlag và cộng sự, 2021].

Ta đã thấy ý tưởng chung của việc thiết kế các mô hình tuyến tính cho cơ chế chú ý. Lựa chọn thiết kế then chốt của những mô hình như vậy là loại bỏ phép chuẩn hóa dựa trên Softmax, nhờ đó thu được các dạng biểu diễn tuyến tính dựa trên những trạng thái trung gian khác nhau của mô hình. Điều này thúc đẩy một số phương án thay thế tự chú ý được phát triển gần đây, trong đó các hệ thống suy luận hiệu quả được xây dựng trên nền các mô hình hồi quy cho mô hình hóa chuỗi [Peng và cộng sự, 2023; Sun và cộng sự, 2023]. Mặc dù các hệ thống này có kiến trúc khác nhau, các mô hình bên dưới có dạng tương tự nhau, như mô tả trong Phương trình (6.152). Lưu ý rằng khi dùng công thức tổng quát của các mô hình hồi quy, ta không bị giới hạn trong cơ chế chú ý QKV chuẩn. Thay vào đó, ta có thể gán ý nghĩa và hình thức mới cho các truy vấn, khóa và giá trị.

Phần thảo luận ở đây cũng liên quan đến các mô hình bộ nhớ đã bàn ở Mục 6.4.2. Từ góc độ bộ nhớ, các khóa và giá trị có thể được xem như các mã hóa của ngữ cảnh. Vì vậy, trong mô hình chú ý tuyến tính ở trên,

ta có một hệ thống bộ nhớ trong đó hai biến đơn giản $\boldsymbol{\mu}_j$ và $\boldsymbol{\nu}_j$ được dùng để biểu diễn toàn bộ thông tin ngữ cảnh tính đến vị trí j. Điều này tạo ra một bộ nhớ có độ dài cố định, rất hữu ích trong thực tế. Cũng có những cách tiếp cận tuyến tính khác để mã hóa chuỗi dài. Chẳng hạn, ta có thể xem mô hình **trung bình trượt** (*moving average*) như một trường hợp của Phương trình (6.152), và lấy trung bình (có trọng số hoặc không) một dãy các vectơ trạng thái của mô hình Transformer.

3. Mô hình không gian trạng thái

Trong các hệ thống điều khiển, **mô hình không gian trạng thái** (*state-space model*) (**SSM**) là cách biểu diễn một hệ thống mà đầu vào và đầu ra liên hệ với nhau qua một số **biến trạng thái** (*state variable*) (gọi tắt là *trạng thái*), và động lực học của hệ được mô tả bằng các phương trình vi phân bậc nhất của các trạng thái này. Để lấy ví dụ đơn giản, ta xét một hệ tuyến tính bất biến theo thời gian, liên tục theo thời gian, được cho dưới dạng biểu diễn không gian trạng thái. Trong suốt mục này, ta dùng quy ước vectơ hàng: mọi biến đều được biểu diễn dưới dạng vectơ hàng và các phép biến đổi tuyến tính được áp dụng ở bên phải:

$$\frac{d\mathbf{z}(t)}{dt} = \mathbf{z}(t) \cdot \mathbf{A} + \mathbf{s}(t) \cdot \mathbf{B}, \tag{6.160}$$

$$\mathbf{o}(t) = \mathbf{z}(t) \cdot \mathbf{C} + \mathbf{s}(t) \cdot \mathbf{D}. \tag{6.161}$$

Ở đây, $\mathbf{s}(t)$, $\mathbf{o}(t)$ và $\mathbf{z}(t)$ lần lượt biểu diễn giá trị của biến đầu vào, biến đầu ra và biến trạng thái tại thời điểm t[19]. Trong trường hợp tổng quát, các biến này có thể có số chiều khác nhau. Để đơn giản, ta giả sử $\mathbf{s}(t), \mathbf{o}(t) \in \mathbb{R}^d$ và $\mathbf{z}(t) \in \mathbb{R}^{d_z}$[20]. Phương trình (6.160) được gọi là **phương trình trạng thái** (*state equation*), trong đó $\mathbf{A} \in \mathbb{R}^{d_z \times d_z}$ là ma trận trạng thái và $\mathbf{B} \in \mathbb{R}^{d \times d_z}$ là ma trận đầu vào. Phương trình (6.161) được gọi là **phương trình đầu ra** (*output equation*), trong đó $\mathbf{C} \in \mathbb{R}^{d_z \times d}$ là ma trận đầu ra và $\mathbf{D} \in \mathbb{R}^{d \times d}$ là ma trận truyền thẳng.

[19] Chúng tôi dùng chữ in đậm để nhấn mạnh rằng các biến này là vectơ.

[20] Trong lý thuyết không gian trạng thái tổng quát, các biến và ma trận này được định nghĩa trên trường số phức. Vì các mô hình định nghĩa trên trường số phức bao hàm cả không gian trạng thái giá trị thực, để đơn giản chúng tôi giới hạn thảo luận trong trường số thực nhiều chiều.

Chú giải của người dịch: Phương trình vi phân $\frac{dz}{dt}$ nói gì?

$\frac{d\mathbf{z}(t)}{dt}$ là đạo hàm của trạng thái theo thời gian, tức *tốc độ thay đổi* của trạng thái. Phương trình (6.160) nói rằng trạng thái thay đổi nhanh hay chậm tùy vào trạng thái hiện tại (qua **A**) và đầu vào hiện tại (qua **B**). Ví dụ đời thường, một chiều: lượng nước z trong bể có lỗ rò, vòi bơm vào với lưu lượng $s(t)$: $\frac{dz}{dt} = -0.1\,z + s(t)$ (nước càng nhiều thì rò càng nhanh). "Trạng thái" z chính là *trí nhớ* của hệ về toàn bộ lịch sử đầu vào. Đầu ra (Phương trình (6.161)) là thứ ta quan sát được, tính từ trạng thái và đầu vào.

Các phương trình này mô tả một ánh xạ liên tục từ đầu vào $\mathbf{s}(t)$ đến đầu ra $\mathbf{o}(t)$ theo thời gian. Vì vậy chúng thường được dùng để xử lý dữ liệu chuỗi thời gian liên tục. Để áp dụng mô hình này cho mô hình hóa chuỗi, ta cần sửa các phương trình trên để có dạng rời rạc của biểu diễn không gian trạng thái. Giả sử $\{\mathbf{s}_0, \mathbf{s}_1, \cdots, \mathbf{s}_n\}$ là một chuỗi các điểm dữ liệu đầu vào được lấy mẫu từ $\mathbf{s}(t)$ với bước thời gian Δt. Tương tự, ta định nghĩa $\{\mathbf{z}_0, \mathbf{z}_1, \cdots, \mathbf{z}_n\}$ và $\{\mathbf{o}_0, \mathbf{o}_1, \cdots, \mathbf{o}_n\}$ là các chuỗi vectơ trạng thái và vectơ đầu ra. Với ký hiệu này, ta có một phiên bản rời rạc hóa của SSM, được viết là

$$\mathbf{z}_t = \mathbf{z}_{t-1} \cdot \overline{\mathbf{A}} + \mathbf{s}_t \cdot \overline{\mathbf{B}}, \tag{6.162}$$

$$\mathbf{o}_t = \mathbf{z}_t \cdot \overline{\mathbf{C}} + \mathbf{s}_t \cdot \overline{\mathbf{D}}. \tag{6.163}$$

Công thức SSM này định nghĩa một RNN có kết nối tắt (residual). Nói chính xác hơn, Phương trình (6.162) mô tả một đơn vị hồi quy đọc đầu vào tại bước t và trạng thái tại bước $t-1$, mà không dùng bất kỳ hàm kích hoạt nào. Phương trình (6.163) mô tả một tầng đầu ra cộng các phép biến đổi tuyến tính của trạng thái $\mathbf{z}_t$ và ánh xạ tắt $\mathbf{s}_t$.

Các tham số $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$, $\overline{\mathbf{C}}$ và $\overline{\mathbf{D}}$ có thể được suy ra từ **A**, **B**, **C** và **D** theo nhiều cách khác nhau, tùy vào việc Phương trình (6.160) được xấp xỉ bằng Phương trình (6.162) như thế nào[21]. Một cách rời rạc hóa theo thời

[21] Quá trình rời rạc hóa có thể được hiểu là giải số phương trình vi phân bên dưới. Lưu ý rằng Phương trình (6.160) là một phương trình vi phân thường (ODE):

$$\frac{d\mathbf{z}(t)}{dt} = g(\mathbf{z}(t), t), \tag{6.164}$$

trong đó

$$g(\mathbf{z}(t), t) = \mathbf{z}(t) \cdot \mathbf{A} + \mathbf{s}(t) \cdot \mathbf{B}. \tag{6.165}$$

Có nhiều phương pháp số để xấp xỉ nghiệm của một ODE. Ví dụ, phương pháp Euler để giải

gian là **biến đổi song tuyến tính** (*bilinear transform*) (còn gọi là phương pháp Tustin). Một cách khác là dùng phép rời rạc hóa **giữ bậc không** (*Zero-Order-Hold*) (**ZOH**). Thảo luận chi tiết về các cách tiếp cận này nằm ngoài phạm vi của cuốn sách, và chúng tôi giới thiệu bạn đọc quan tâm tới các giáo trình chuẩn về lý thuyết điều khiển để biết thêm chi tiết [Åström và Wittenmark, 2013].

Chú giải của người dịch: Rời rạc hóa bằng phương pháp Euler

Ý tưởng giống hệt định nghĩa đạo hàm ở lớp 11: với Δt nhỏ, $\frac{dz}{dt} \approx \frac{z_t - z_{t-1}}{\Delta t}$, nên $z_t \approx z_{t-1} + \Delta t \cdot \frac{dz}{dt}$. Với ví dụ bể nước một chiều $\frac{dz}{dt} = -0.1z + s$ và $\Delta t = 1$: $z_t = z_{t-1}(1 - 0.1) + 1 \cdot s_t$, tức $\overline{A} = 0.9$, $\overline{B} = 1$. Vậy hệ liên tục đã biến thành một công thức truy hồi từng bước — đúng dạng RNN tuyến tính ở Phương trình (6.162). Ma trận $\mathbf{I}$ trong Phương trình (6.168) là *ma trận đơn vị*, đóng vai trò như số 1 trong phép nhân ma trận.

Dạng hồi quy của Phương trình (6.162) giúp dễ dàng tính các trạng thái và đầu ra trên một chuỗi các bước thời gian rời rạc. Ta có thể trải $\mathbf{z}_t$ và $\mathbf{o}_t$ ra theo kiểu truyền thẳng

$$
\begin{array}{l|l}
\mathbf{z}_0 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} & \mathbf{o}_0 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_0 \cdot \overline{\mathbf{D}} \\
\mathbf{z}_1 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} & \mathbf{o}_1 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{D}} \\
\mathbf{z}_2 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^2 + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} + \mathbf{s}_2 \cdot \overline{\mathbf{B}} & \mathbf{o}_2 = \mathbf{s}_0 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^2 \cdot \overline{\mathbf{C}} + \mathbf{s}_1 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}} \cdot \overline{\mathbf{C}} + \\
 & \quad \mathbf{s}_2 \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} + \mathbf{s}_2 \cdot \overline{\mathbf{D}} \\
\ldots\ldots & \ldots\ldots
\end{array}
$$

ODE có thể được biểu diễn là (xem Mục 6.3.3)

$$\mathbf{z}_t = \mathbf{z}_{t-1} + \Delta t \cdot g(\mathbf{z}_{t-1}, t). \tag{6.166}$$

Thay Phương trình (6.165) vào Phương trình (6.166), ta được

$$
\begin{aligned}
\mathbf{z}_t &= \mathbf{z}_{t-1} + \Delta t(\mathbf{z}_{t-1} \cdot \mathbf{A} + \mathbf{s}_t \cdot \mathbf{B}) \\
&= \mathbf{z}_{t-1} \cdot (\mathbf{I} + \Delta t \cdot \mathbf{A}) + \mathbf{s}_t \cdot (\Delta t \cdot \mathbf{B}).
\end{aligned} \tag{6.167}
$$

Điều này cho ta một trong những dạng đơn giản nhất của phương trình trạng thái rời rạc hóa [Gu và cộng sự, 2022b], tức là

$$\overline{\mathbf{A}} = \mathbf{I} + \Delta t \cdot \mathbf{A}, \tag{6.168}$$

$$\overline{\mathbf{B}} = \Delta t \cdot \mathbf{B} \tag{6.169}$$

Dễ dàng viết được

$$\mathbf{z}_t = \sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i}, \tag{6.170}$$

$$\mathbf{o}_t = \sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i} \cdot \overline{\mathbf{C}} + \mathbf{s}_t \cdot \overline{\mathbf{D}}. \tag{6.171}$$

Rõ ràng, vế phải của Phương trình (6.171) có thể được hiểu là tổng của một số hạng tích chập và một số hạng tuyến tính. Vì

$$\sum_{i=0}^{t} \mathbf{s}_i \cdot \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-i} \cdot \overline{\mathbf{C}} = \begin{bmatrix} \mathbf{s}_0 & \mathbf{s}_1 & \cdots & \mathbf{s}_t \end{bmatrix} \cdot \begin{bmatrix} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t} \cdot \overline{\mathbf{C}} & \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{t-1} \cdot \overline{\mathbf{C}} & \cdots & \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} \end{bmatrix}^{\top}, \tag{6.172}$$

ta định nghĩa một bộ lọc tích chập với các tham số

$$\mathbf{W}_{\text{ssm}} = \begin{bmatrix} \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{n_{\max}} \cdot \overline{\mathbf{C}} & \overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^{n_{\max}-1} \cdot \overline{\mathbf{C}} & \cdots & \overline{\mathbf{B}} \cdot \overline{\mathbf{C}} \end{bmatrix}, \tag{6.173}$$

trong đó $n_{\max}$ là độ dài tối đa của chuỗi[22]. Khi đó, đầu ra của mô hình không gian trạng thái cho một chuỗi $\mathbf{S} = \begin{bmatrix} \mathbf{s}_0 \\ \vdots \\ \mathbf{s}_n \end{bmatrix}$ có thể được biểu diễn là

$$\mathbf{O} = \text{Conv}(\mathbf{S}, \mathbf{W}_{\text{ssm}}) + \text{Linear}(\mathbf{S}, \overline{\mathbf{D}}), \tag{6.174}$$

trong đó $\text{Conv}(\cdot)$ là phép tích chập, và $\text{Linear}(\cdot)$ là phép biến đổi tuyến tính. Cách xử lý mô hình không gian trạng thái như vậy cho phép cài đặt hệ thống một cách hiệu quả bằng các thuật toán tích chập song song nhanh.

Chú giải của người dịch: Một mô hình, hai cách tính

Xét trường hợp một chiều với $\overline{A} = 0.5$, $\overline{B} = \overline{C} = 1$, $\overline{D} = 0$ và đầu vào $s_0 = 4, s_1 = 2, s_2 = 6$. *Cách hồi quy* (từng bước): $z_0 = 4$, $z_1 = 0.5 \cdot 4 + 2 = 4$, $z_2 = 0.5 \cdot 4 + 6 = 8$. *Cách tích chập* (Phương trình (6.170)):

[22] Ở đây $\mathbf{W}_{\text{ssm}}$ có thể được biểu diễn dưới dạng một tensor kích thước $n_{\max} \times d \times d$.

$z_2 = s_0 \cdot 0.5^2 + s_1 \cdot 0.5 + s_2 = 1 + 1 + 6 = 8$ — cùng kết quả. Cách thứ hai cho thấy mỗi đầu ra là tổng có trọng số của các đầu vào với "bộ lọc" $[0.25,\ 0.5,\ 1]$ cố định, nên mọi vị trí có thể tính *đồng thời* (song song) khi huấn luyện; còn cách thứ nhất chỉ cần nhớ một trạng thái, rất tiện khi sinh từng token lúc suy luận.

Tuy nhiên, các cài đặt ngây thơ của mô hình này thường hoạt động kém. Như với nhiều kiến trúc mạng nơ-ron sâu khác, việc khởi tạo tham số cẩn thận là rất quan trọng. Cụ thể, việc giới hạn ma trận trạng thái $\mathbf{A}$ vào những cấu trúc đặc biệt (ví dụ ma trận HiPPO) đã được chứng minh là thiết yếu để ghi nhớ và tổng quát hóa trên các chuỗi dài [Gu và cộng sự, 2022a].

Một nút thắt tính toán khác là phép nhân ma trận lặp lại cần thiết để tính $\overline{\mathbf{A}}^n$. Khi n lớn, phép toán này tốn kém về tính toán và không ổn định về mặt số học. Một cách tiếp cận phổ biến trong các SSM hiện đại là **chéo hóa** (*diagonalization*). Ý tưởng là áp dụng một phép đổi cơ sở cho không gian trạng thái sao cho $\mathbf{A}$ (hoặc $\overline{\mathbf{A}}$) trở thành ma trận đường chéo. Cho một SSM được tham số hóa bởi $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$, ta có thể định nghĩa một mô hình tương đương $(\mathbf{UAU}^{-1}, \mathbf{BU}^{-1}, \mathbf{UC}, \mathbf{D})$ thông qua một ma trận biến đổi khả nghịch $\mathbf{U}$. Có thể chứng minh rằng hai cách biểu diễn này tương đương về mặt toán học dưới một phép đổi cơ sở[23].

Bằng cách áp dụng phép biến đổi này, và giả sử $\overline{\mathbf{A}}$ có thể được chéo hóa về dạng chính tắc $\mathbf{P}^{-1}\Lambda\mathbf{P}$ (trong đó Λ là ma trận đường chéo), ta áp đặt ràng buộc rằng ma trận trạng thái là đường chéo, từ đó có các **mô hình không gian trạng thái đường chéo** (*diagonal state-space model*). Để minh họa, hãy xét bộ lọc được dùng trong biểu diễn tích chập (Phương trình (6.172)). Thay $\overline{\mathbf{A}} = \mathbf{P}^{-1}\Lambda\mathbf{P}$, ta có thể viết lại số hạng $\overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}}$ thành:

$$\begin{aligned}\overline{\mathbf{B}} \cdot \overline{\mathbf{A}}^t \cdot \overline{\mathbf{C}} &= \overline{\mathbf{B}} \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P})^t \cdot \overline{\mathbf{C}} \\ &= \overline{\mathbf{B}} \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdot (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdots (\mathbf{P}^{-1}\Lambda\mathbf{P}) \cdot \overline{\mathbf{C}} \\ &= (\overline{\mathbf{B}} \cdot \mathbf{P}^{-1}) \cdot \Lambda^t \cdot (\mathbf{P} \cdot \overline{\mathbf{C}}). \end{aligned} \tag{6.175}$$

Vì Λ là ma trận đường chéo, ta có thể tính Λ^t một cách hiệu quả bằng cách đơn giản nâng tất cả các phần tử của Λ lên lũy thừa t. Khi đó ta có

[23] Một phép biến đổi không gian trạng thái ánh xạ các biến trạng thái vào một hệ tọa độ mới: $\mathbf{z}'(t) = \mathbf{z}(t) \cdot \mathbf{U}^{-1}$.

một mô hình rẻ hơn về mặt tính toán, trong đó

$$\overline{\mathbf{A}}' = \Lambda, \tag{6.176}$$

$$\overline{\mathbf{B}}' = \overline{\mathbf{B}} \cdot \mathbf{P}^{-1}, \tag{6.177}$$

$$\overline{\mathbf{C}}' = \mathbf{P} \cdot \overline{\mathbf{C}}, \tag{6.178}$$

$$\overline{\mathbf{D}}' = \overline{\mathbf{D}}. \tag{6.179}$$

Các thảo luận chi tiết hơn về mô hình không gian trạng thái đường chéo trong mô hình hóa chuỗi có thể được tìm thấy trong công trình của Gu và cộng sự [2021].

Chú giải của người dịch: Vì sao ma trận đường chéo dễ lũy thừa?

Ma trận đường chéo chỉ có số khác 0 trên đường chéo chính. Khi nhân hai ma trận đường chéo, ta chỉ việc nhân các phần tử tương ứng trên đường chéo, nên $\begin{bmatrix}2 & 0\\ 0 & 3\end{bmatrix}^t = \begin{bmatrix}2^t & 0\\ 0 & 3^t\end{bmatrix}$ — chỉ cần d_z phép lũy thừa số thông thường, thay vì nhân ma trận đầy đủ t lần. Ở dòng thứ hai của Phương trình (6.175), các cặp $\mathbf{P}\mathbf{P}^{-1}$ đứng cạnh nhau triệt tiêu (bằng ma trận đơn vị), chỉ còn $\mathbf{P}^{-1}\Lambda^t\mathbf{P}$.

Việc áp dụng mô hình không gian trạng thái vào Transformer khá đơn giản. Trong trường hợp này, mỗi tầng con tự chú ý được thay bằng một tầng con SSM như mô tả trong Phương trình (6.162) và (6.163). Như ta đã thấy, có mối liên hệ chặt chẽ giữa mô hình không gian trạng thái với cả CNN và RNN. Đối với mô hình hóa chuỗi, ta có thể xử lý một chuỗi token hoặc tuần tự như trong RNN, hoặc song song như trong CNN. Điều này dẫn tới một mô thức mới kết hợp cả góc nhìn tuần tự lẫn góc nhìn song song đối với bài toán mô hình hóa chuỗi — khi huấn luyện, hệ thống hoạt động như CNN để tận dụng các thuật toán huấn luyện song song nhanh; khi dự đoán, bài toán được chuyển thành bài toán cập nhật tuần tự, có thể giải hiệu quả bằng các mô hình kiểu RNN.

Mặc dù hình thức luận của mô hình không gian trạng thái khác với những gì ta đã thảo luận trong chương này, nó cung cấp một khuôn khổ thống nhất cho mô hình hóa chuỗi. Bằng cách nhìn bài toán qua các góc nhìn bổ trợ nhau (song song và tuần tự), ta có thể tối ưu cho các ràng buộc khác nhau về phần cứng và độ trễ. Một số kiến trúc gần đây được thúc đẩy bởi tính đối ngẫu này, tạo ra các hệ thống kết hợp hiệu quả khả năng song song hóa của Transformer với hiệu quả suy luận của RNN [Orvieto

và cộng sự, 2023; Sun và cộng sự, 2023].

6.4.6 Tính toán có điều kiện

Cho đến giờ, trong thảo luận về các mô hình Transformer hiệu quả, ta đã giả định rằng kiến trúc mô hình được xác định trước khi huấn luyện và giữ cố định trong suốt quá trình. Bây giờ ta chuyển sang trường hợp học các kiến trúc mô hình hiệu quả. Ta có thể viết một mô hình dưới dạng

$$\mathbf{y} \;=\; \text{Model}(\mathbf{x}, g(\mathbf{x})), \tag{6.180}$$

trong đó $\mathbf{x}$ và $\mathbf{y}$ là đầu vào và đầu ra của mô hình. $g(\mathbf{x})$ là một hàm trả về kiến trúc mô hình và các tham số tương ứng với đầu vào $\mathbf{x}$ cho trước. Nói chung, ta theo quy ước phổ biến trong các bài toán học là dùng một kiến trúc mô hình cố định và chỉ học các tham số, chẳng hạn $g(\mathbf{x}) = \theta$. Trong trường hợp này, mục tiêu là học các giá trị tham số tối ưu khi đã biết kiến trúc và dữ liệu huấn luyện. Trên dữ liệu kiểm tra, ta đưa ra dự đoán bằng cùng kiến trúc mô hình đó cùng với các tham số đã được tối ưu.

Một mở rộng tự nhiên của cách tiếp cận này là xét việc học cả kiến trúc mô hình lẫn tham số. Trong học kiến trúc, ta muốn tìm một hàm $\hat{g}(\mathbf{x})$ tạo ra kiến trúc mô hình và giá trị tham số tối ưu cho đầu vào $\mathbf{x}$. Tuy nhiên, việc tìm kiếm trên toàn bộ không gian giả thuyết gồm mọi tổ hợp kiến trúc và tham số có thể có là bất khả thi, nên cần đến các phương pháp thực tế. Có hai lớp phương pháp có thể áp dụng:

- **Tìm kiếm kiến trúc mạng nơ-ron** (*Neural Architecture Search*) (**NAS**). Trong **học máy tự động** (*automated machine learning*) (**AutoML**), tìm kiếm kiến trúc mạng nơ-ron là quá trình khám phá một không gian các mạng nơ-ron để tìm ra mạng phù hợp nhất với một số tiêu chí [Zoph và Le, 2016; Elsken và cộng sự, 2019b]. Khi đã xác định được mạng nơ-ron tối ưu, các tham số của nó được huấn luyện như thường lệ, rồi áp dụng cho dữ liệu mới. Để việc tìm kiếm khả thi, người ta thường dùng thêm một số kỹ thuật, chẳng hạn cắt tỉa không gian tìm kiếm và các thuật toán tìm kiếm nhanh. Việc áp dụng tìm kiếm kiến trúc mạng nơ-ron vào phát triển các mạng nơ-ron hiệu quả khá đơn giản [Howard và cộng sự, 2019; Tan và Le, 2019]. Ta chỉ cần đưa các thước đo hiệu quả vào việc ước lượng hiệu năng của mạng nơ-ron; ví dụ, việc tìm kiếm có thể

được dẫn dắt bởi một tiêu chí phạt những mạng nơ-ron có độ trễ cao hoặc đòi hỏi bộ nhớ quá lớn.

- **Mạng nơ-ron động** (*Dynamic Neural Networks*). Ý tưởng chính của mạng nơ-ron động là điều chỉnh mạng nơ-ron một cách linh hoạt theo các đầu vào khác nhau [Gupta và cộng sự, 2004; Han và cộng sự, 2021b]. Lý tưởng nhất, ta muốn học $\hat{g}(\cdot)$, rồi với mọi đầu vào $\mathbf{x}_{\text{new}}$, ta áp dụng mô hình $\text{Model}(\mathbf{x}_{\text{new}}, \hat{g}(\mathbf{x}_{\text{new}}))$. Kết quả là, ở thời điểm kiểm tra, ta có thể có cấu trúc mô hình và/hoặc tham số khác nhau cho các đầu vào khác nhau. Tuy nhiên, không khả thi để xây dựng một hàm $\hat{g}(\cdot)$ có thể mô hình hóa các mạng nơ-ron tùy ý. Trong thực tế, $\hat{g}(\cdot)$ thường được xem là biểu diễn một họ các mạng con của một siêu mạng. Do đó, bài toán được đặt lại thành một bài toán đơn giản hơn: học cách chọn mạng con nào được dùng cho một đầu vào cho trước.

Từ góc độ học máy, các phương pháp tìm kiếm kiến trúc mạng nơ-ron mang tính tổng quát và có thể áp dụng cho bất kỳ mạng nơ-ron nào. Mặt khác, từ góc độ thực tiễn, vẫn khó tìm được một mạng nơ-ron hiệu quả mà đủ mạnh và tổng quát hóa tốt. Mặc dù tìm kiếm kiến trúc mạng nơ-ron mang lại những ý tưởng thú vị cho việc phát triển các mô hình Transformer hiệu quả, chúng tôi không bàn sâu về nó ở đây. Thay vào đó, bạn đọc có thể tham khảo các bài báo trên để có ý niệm chung, và tham khảo các công trình của So và cộng sự [2019], Wang và cộng sự [2020a], và Hu và cộng sự [2021] về ứng dụng của nó cho Transformer.

Trong tiểu mục này, chúng tôi tập trung vào một họ phương pháp cụ thể của mạng nơ-ron động, gọi là **tính toán có điều kiện** (*conditional computation*). Khái niệm này ban đầu xuất phát từ việc lựa chọn động các nơ-ron của một mạng nơ-ron [Bengio và cộng sự, 2013; 2015]. Gần đây hơn, nó thường được dùng để chỉ quá trình lựa chọn động các phần của một mạng nơ-ron. Một cách nhìn hẹp về tính toán có điều kiện là coi $g(\cdot)$ như một mạng nơ-ron thích nghi, có thể giảm hoặc tăng số đơn vị tính toán (như nơ-ron và tầng) một cách linh hoạt. Nhờ đó, việc tính toán có thể thích ứng với các điều kiện thay đổi, và ta có thể tìm một sự đánh đổi tốt giữa độ chính xác và độ trễ thông qua cơ chế thích nghi này.

Một cách phổ biến để đạt được điều này là học cách bỏ qua một số bước tính toán, để ta chỉ làm việc với một tập con của mạng [Xu và Mcauley,

2023]. Một trong những phương pháp đơn giản nhất, đôi khi được gọi là **thoát sớm** (*early exiting*), là dừng tính toán tại một thời điểm nào đó trong khi mã hóa hoặc giải mã một chuỗi. Kỹ thuật này thường được dùng trong các ứng dụng sinh chuỗi thực tế đòi hỏi độ trễ thấp. Giả sử $y_1 \cdots y_{n_{\max}}$ là chuỗi dài nhất mà hệ thống có thể sinh ra, và $\mathbf{s}_1 \cdots \mathbf{s}_{n_{\max}}$ là chuỗi trạng thái ẩn tương ứng của tầng Transformer trên cùng. Khi đó ta xây dựng một mô hình $f_{\text{stop}}(\cdot)$ nhận lần lượt từng trạng thái ẩn $\mathbf{s}_i$ và tạo ra phân phối của một biến nhị phân $c \in \{\text{stop}, \text{nonstop}\}$ (dừng, không dừng):

$$\Pr(c|\mathbf{s}_i) \;=\; f_{\text{stop}}(\mathbf{s}_i). \tag{6.181}$$

Quá trình sinh kết thúc nếu $\Pr(\text{stop}|\mathbf{s}_i)$ đủ lớn, chẳng hạn

$$\Pr(\text{stop}|\mathbf{s}_i) \;\geq\; \Pr(\text{nonstop}|\mathbf{s}_i) + \theta_{\text{stop}}, \tag{6.182}$$

trong đó θ_{stop} là biên tối thiểu để phân biệt hai hành động[24]. Công thức này cũng liên quan đến bài toán tiêu chí dừng thường được bàn đến trong các thuật toán tìm kiếm cho sinh chuỗi (xem Chương 5). Hàm $f_{\text{stop}}(\cdot)$ có thể được thiết kế theo nhiều cách khác nhau. Ví dụ, trong nhiều ứng dụng thực tế, tiêu chí dừng dựa trên các quy tắc kinh nghiệm đơn giản. Ngoài ra, ta có thể định nghĩa hàm $f_{\text{stop}}(\cdot)$ là một mạng nơ-ron và huấn luyện nó bằng dữ liệu có nhãn.

Chú giải của người dịch: Biên θ_{stop}

Vì chỉ có hai khả năng nên $\Pr(\text{stop}) + \Pr(\text{nonstop}) = 1$. Thay $\Pr(\text{nonstop}) = 1 - \Pr(\text{stop})$ vào Phương trình (6.182) ta được $2\Pr(\text{stop}) \geq 1 + \theta_{\text{stop}}$, chính là dạng trong chú thích chân trang. Ví dụ $\theta_{\text{stop}} = 0.4$ thì mô hình chỉ dừng khi $\Pr(\text{stop}) \geq 0.7$: biên càng lớn, mô hình càng "thận trọng", chỉ dừng khi thật sự chắc chắn.

Cách tiếp cận trên có thể dễ dàng mở rộng cho những tình huống trong đó một số token bị bỏ qua. Cách tiếp cận *học cách bỏ qua* này thường được dùng ở giai đoạn mã hóa, khi mọi token đầu vào đều đã có sẵn từ trước. Gọi $\mathbf{h}_1 \cdots \mathbf{h}_m$ là các biểu diễn mức thấp của chuỗi $x_1 \cdots x_m$. Tương tự Phương trình (6.181), ta có thể xây dựng một mô hình $\Pr(c|\mathbf{s}_i)$ ($c \in \{\text{skip}, \text{nonskip}\}$, tức bỏ qua/không bỏ qua) để xác định xem token x_i có thể bị bỏ qua hay không. Hình 6.17 (a) và (b) minh họa việc thoát sớm

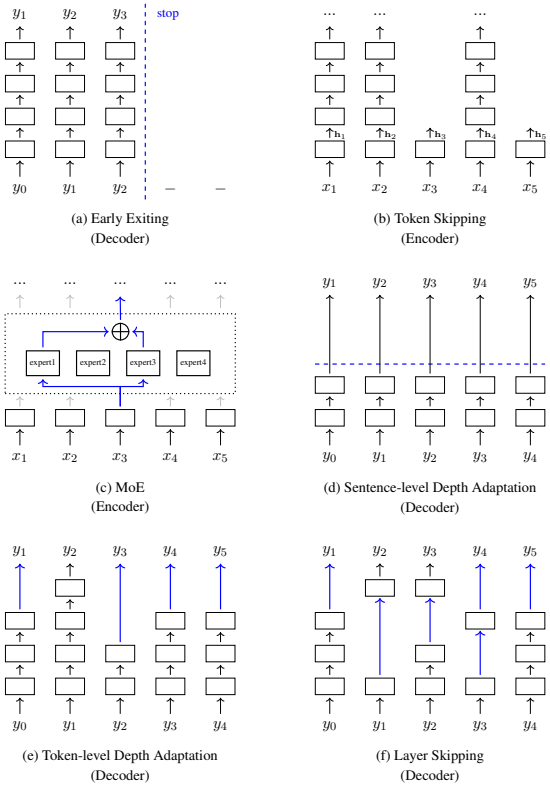
[24] Một dạng tương đương của Phương trình (6.182) là $\Pr(\text{stop}|\mathbf{s}_i) \geq \frac{1+\theta_{\text{stop}}}{2}$.

và bỏ qua token. Lưu ý rằng phương pháp học cách bỏ qua có phần trùng lặp với các hướng nghiên cứu khác về huấn luyện mạng nơ-ron. Ví dụ, việc xóa bớt một số token đầu vào khi huấn luyện được thấy là hữu ích để giúp mô hình Transformer tổng quát hóa tốt hơn [Shen và cộng sự, 2020a; Kim và Cho, 2021]. Phương pháp này cũng liên quan đến các phương pháp giảm mẫu sẽ được thảo luận ở Mục 6.4.8.

Cách tiếp cận thứ hai cho tính toán có điều kiện là dùng các **mô hình chuyên gia thưa** (*sparse expert model*), hoặc trường hợp phổ biến của nó — MoE [Yuksel và cộng sự, 2012]. Trong học sâu, một mô hình loại này thường được xây dựng từ một số chuyên gia, là các mạng nơ-ron có cùng cấu trúc nhưng khác tham số. Bằng cách này, ta có thể tạo ra một mô hình lớn chỉ bằng cách tăng số lượng chuyên gia. Khi chạy mô hình, dù là lúc huấn luyện hay dự đoán, ta chỉ kích hoạt một số ít chuyên gia nhờ các thuật toán định tuyến (xem Hình 6.17 (c)). Mô hình MoE là một mạng thích nghi, vì mỗi khi có đầu vào mới, mô hình định tuyến nó tới các chuyên gia khác nhau. Ở Mục 6.3.4, chúng tôi đã trình bày dạng cơ bản của MoE và cho thấy cách mở rộng quy mô các mô hình Transformer bằng phương pháp thưa này. Để có cái nhìn tổng quan về các tiến bộ gần đây của MoE, chúng tôi giới thiệu bạn đọc quan tâm tới công trình của Fedus và cộng sự [2022a].

Cách tiếp cận thứ ba có thể dùng để điều chỉnh mô hình Transformer theo đầu vào thay đổi là thu gọn số tầng một cách động. Nhiều phương pháp đã được đề xuất để làm việc này nhằm cải thiện hiệu quả suy luận. Đơn giản nhất trong số đó là thoát ra tại một số tầng ẩn mà ở đó ta vẫn có thể đưa ra dự đoán chính xác cho đầu vào (xem Hình 6.17 (d) và (e)). Để làm điều này, ta có thể hoặc xác định độ sâu phù hợp cho toàn bộ chuỗi (gọi là **mô hình thích nghi độ sâu mức câu** (*sentence-level depth-adaptive model*)), hoặc dùng độ sâu thích nghi cho từng token (gọi là **mô hình thích nghi độ sâu mức token** (*token-level depth-adaptive model*)). Ở đây ta xét các mô hình thích nghi độ sâu mức token, nhưng các phương pháp này có thể dễ dàng mở rộng cho mô hình thích nghi độ sâu mức chuỗi.

Ghi chú nhãn hình (người dịch): (a) *Early Exiting (Decoder)*: thoát sớm (bộ giải mã), *stop*: dừng; (b) *Token Skipping (Encoder)*: bỏ qua token (bộ mã hóa); (c) MoE (bộ mã hóa), *expert*: chuyên gia; (d) *Sentence-level Depth Adaptation*: thích nghi độ sâu mức câu; (e) *Token-level Depth*



Hình 6.17: Các phương pháp tính toán có điều kiện, gồm thoát sớm, bỏ qua token, MoE, thích nghi độ sâu mức câu, thích nghi độ sâu mức token, và bỏ qua tầng. Mặc dù các phương pháp này được minh họa bằng quá trình mã hóa hoặc giải mã, phần lớn chúng có thể áp dụng cho cả bộ mã hóa lẫn bộ giải mã Transformer.

Adaptation: thích nghi độ sâu mức token; (f) *Layer Skipping*: bỏ qua tầng.

Giả sử có L tầng xếp chồng tại vị trí i[25]. Lý tưởng nhất, ta muốn tìm một tầng trong chồng tầng đó, có thể dùng làm tầng ẩn cuối cùng để đưa ra dự đoán, và có độ sâu thấp nhất có thể. Tuy nhiên, ta không thể đơn giản dùng tầng thứ L của chồng làm "đáp án chuẩn" (oracle) cho bài toán này, vì ta không bao giờ biết trước tầng cuối cùng sẽ sinh ra gì trong lúc suy luận. Thay vào đó, ta cần xác định xem mạng có nên ngừng tăng thêm tầng ở độ sâu i hay không, dựa trên các tầng đã sinh ra cho đến lúc đó.

Bây giờ giả sử ta có một bộ giải mã Transformer tạo ra một phân phối trên bộ từ vựng V ở mỗi bước. Như thường lệ, ta ký hiệu đầu ra của tầng thứ l tại bước i là $\mathbf{s}_i^l$. Với mỗi $\mathbf{s}_i^l$, ta tạo một tầng đầu ra sinh ra phân phối $\mathbf{p}_i^l$ trên bộ từ vựng (gọi là **bộ phân loại thoát sớm** (*early exit classifier*)), cho bởi

$$\mathbf{p}_i^l \;=\; \text{Softmax}(\mathbf{s}_i^l \cdot \mathbf{W}_{\text{o}}^l), \tag{6.183}$$

trong đó $\mathbf{W}_{\text{o}}^l \in \mathbb{R}^{d\times|V|}$ là ma trận tham số. Như vậy ta có thêm $L-1$ tầng đầu ra, mỗi tầng ứng với một tầng ẩn từ độ sâu 1 đến $L-1$. Khi huấn luyện, ta xét các hàm mất mát entropy chéo của $\{\mathbf{p}_i^1, \cdots, \mathbf{p}_i^{L-1}\}$, và huấn luyện các tầng này cùng với mô hình Transformer. Khi kiểm tra, độ sâu của mạng tăng dần như thường lệ, và ta dùng $\{\mathbf{p}_i^1, \cdots, \mathbf{p}_i^l\}$ và/hoặc $\{\mathbf{s}_i^1, \cdots, \mathbf{s}_i^l\}$ để xác định xem có nên thoát ở tầng thứ l hay không. Có một số tiêu chí thoát, chẳng hạn:

- Các tiêu chí phổ biến dựa trên thước đo độ tin cậy của dự đoán. Một phương pháp đơn giản là tính entropy của $\mathbf{p}_i^l$, và thoát nếu entropy này vượt quá một giá trị định trước.
- Ngoài ra, có thể xem xác suất lớn nhất trong các phần tử của $\mathbf{p}_i^l$ là độ tin cậy của dự đoán.
- Thay vì xét đầu ra của một tầng duy nhất, ta cũng có thể xem xét sự thay đổi của đầu ra hoặc trạng thái ẩn qua một số tầng. Ví dụ, ta có thể đo độ tương đồng giữa $\mathbf{p}_i^{l-1}$ và $\mathbf{p}_i^l$ hoặc giữa $\mathbf{s}_i^{l-1}$ và $\mathbf{s}_i^l$. Nếu độ tương đồng vượt quá một ngưỡng cho trước, ta nói rằng đầu ra của mạng nơ-ron có xu hướng hội tụ và số tầng có thể ngừng tăng.
- Các phương pháp trên có thể được mở rộng để xem xét sự thay đổi trong các dự đoán của những bộ phân loại gắn với các tầng. Ví dụ,

[25] Một tầng ở đây là một khối Transformer chuẩn gồm vài tầng con.

mô hình có thể chọn thoát nếu dự đoán của các bộ phân loại không thay đổi qua một số tầng.

Chú giải của người dịch: Entropy như thước đo "độ do dự"

Entropy của phân phối $\mathbf{p} = (p_1, \ldots, p_{|V|})$ là $H = -\sum_k p_k \log p_k$. Nó *nhỏ* khi mô hình chắc chắn và *lớn* khi mô hình phân vân. Ví dụ với 2 từ: $\mathbf{p} = (0.99,\ 0.01)$ cho $H \approx 0.056$ (dùng ln), còn $\mathbf{p} = (0.5,\ 0.5)$ cho $H = \ln 2 \approx 0.69$. Do đó trong thực tế, khi dùng entropy làm thước đo độ tin cậy, người ta thường cho mô hình thoát khi entropy đủ *thấp* (tức độ tin cậy đủ cao); tương đương, khi "độ tin cậy" vượt một ngưỡng định trước.

Thảo luận về các tiêu chí này có thể tìm thấy trong các bài báo liên quan [Xin và cộng sự, 2020; Zhou và cộng sự, 2020; Schuster và cộng sự, 2022]. Có nhiều cách để cải thiện các phương pháp thoát sớm này. Một cách là khám phá các dạng dự đoán khác cho mỗi tầng. Ví dụ, ta có thể xây dựng một mô hình dự đoán trực tiếp cần bao nhiêu tầng để mô hình hóa đầu vào [Elbayad và cộng sự, 2020]. Một hướng nghiên cứu khác về thoát sớm tập trung vào việc huấn luyện các mô hình này tốt hơn, chẳng hạn ta có thể xét các hàm mất mát khác nhau để huấn luyện các bộ phân loại [Schwartz và cộng sự, 2020; Schuster và cộng sự, 2022]. Ngoài ra, cũng có sự quan tâm tới việc học cách kết hợp đầu ra của nhiều tầng, để có thể đưa ra dự đoán bằng cách sử dụng nhiều mức biểu diễn [Zhou và cộng sự, 2020; Liao và cộng sự, 2021].

Một vấn đề của các mô hình thích nghi độ sâu mức token là biểu diễn ở một số độ sâu nhất định có thể không tồn tại ở các bước trước. Khi đó, tự chú ý chuẩn không áp dụng trực tiếp được, vì ta có thể không chú ý được tới các token trước ở cùng mức biểu diễn. Khi huấn luyện, vấn đề này có thể giải quyết bằng cách dùng đủ L tầng của mô hình đầy đủ. Khi suy luận, ta có thể hoặc nhân bản tầng mà ta thoát ra để lấp đầy chồng tầng, hoặc sửa mô hình tự chú ý để nó có thể chú ý tới biểu diễn của các token trước ở các độ sâu khác nhau.

Cũng có thể chọn một tập con bất kỳ của các tầng để xây dựng một mạng nông. Do đó, các mô hình thích nghi có thể được tổng quát hóa thành các mô hình bỏ qua (xem Hình 6.17 (f)). Giống như bài toán thoát sớm, bài toán bỏ qua có thể được đặt thành một bài toán học, trong đó một bộ phân loại được huấn luyện để quyết định xem một tầng có nên bị bỏ đi hay không. Bài toán học cách bỏ qua đã được nghiên cứu trong lĩnh

vực thị giác máy tính [Wang và cộng sự, 2018b; Wu và cộng sự, 2018b]. Tuy nhiên, việc học một mô hình bỏ qua cho các mạng nơ-ron sâu, quy mô lớn là khó. Với các hệ thống thực tế, có vẻ vẫn hợp lý khi dùng các quy tắc kinh nghiệm hoặc các mô hình rẻ để có được một mạng nơ-ron với các tầng bị bỏ qua, điều này đã được bàn đến trong các mô hình NLP tiền huấn luyện gần đây [Wang và cộng sự, 2022c; Del Corro và cộng sự, 2023].

6.4.7 Chuyển giao và cắt tỉa mô hình

Nhiều mô hình Transformer lớn đã được phát triển thành công để giải quyết các bài toán NLP. Một câu hỏi thường gặp là: liệu ta có thể biến đổi một mô hình lớn, đã được huấn luyện tốt thành một mô hình nhỏ hơn để suy luận hiệu quả hơn không? Ở mức khái quát, điều này có thể được coi là một bài toán **học chuyển giao** (*transfer learning*), trong đó tri thức được chuyển từ mô hình này sang mô hình khác. Tuy nhiên, chúng tôi sẽ không bàn về chủ đề tổng quát này, vốn trải rộng trên nhiều vấn đề và mô hình, phần lớn nằm ngoài phạm vi chương này. Thay vào đó, chúng tôi thu hẹp thảo luận vào hai cách tiếp cận được dùng rộng rãi để huấn luyện các mạng nơ-ron nhỏ hơn từ các mạng lớn hơn.

1. Chưng cất tri thức

Chưng cất tri thức (*knowledge distillation*) là quá trình nén tri thức trong một mạng nơ-ron lớn (hoặc một tổ hợp các mạng nơ-ron) vào một mạng nơ-ron nhỏ hơn [Hinton và cộng sự, 2015]. Trong học có giám sát các mạng nơ-ron, hàm mục tiêu thường được thiết kế để định lượng sai số giữa câu trả lời dự đoán và câu trả lời đúng. Do đó, bằng cách cực tiểu hóa hàm mất mát này, mô hình được huấn luyện để đưa ra câu trả lời đúng. Mặc dù mô hình thường được tối ưu trên dữ liệu huấn luyện theo cách này, điều ta thực sự muốn là tổng quát hóa tốt trên dữ liệu chưa thấy. Tuy nhiên, điều này khó, vì chỉ huấn luyện với các nhãn đúng không cung cấp tín hiệu giám sát trực tiếp nào về khả năng tổng quát hóa. Trong chưng cất tri thức, thay vì ép mô hình khớp với đầu ra đúng, ta huấn luyện nó bắt chước hành vi (hay phân phối đầu ra) của một mô hình lớn hơn. Để làm được điều này, ta chuyển trực tiếp tri thức của một mô hình đã tiền huấn luyện sang mô hình mà ta muốn huấn luyện.

Một cách tiếp cận thường dùng cho chưng cất tri thức là **huấn luyện thầy–trò** (*teacher-student training*). Mô hình *thầy* thường là một mạng nơ-ron tương đối lớn, đã được huấn luyện và có khả năng tổng quát hóa tốt. Mô hình *trò* là một mạng nơ-ron nhỏ hơn, chẳng hạn có ít tầng hơn, mà ta chuyển tri thức sang. Một cách đơn giản để chưng cất tri thức từ mô hình thầy vào mô hình trò là dùng đầu ra của mô hình thầy làm câu trả lời "đúng" để huấn luyện mô hình trò. Giả sử ta có một mô hình Transformer thầy có thể sinh ra một chuỗi các phân phối $\{\Pr(\cdot|y_0, \mathbf{x}), \cdots, \Pr(\cdot|y_0 \cdots y_{n-1}, \mathbf{x})\}$ cho đầu vào $\mathbf{x}$. Để ký hiệu đơn giản, ta ký hiệu phân phối $\Pr(\cdot|y_0 \cdots y_{i-1}, \mathbf{x})$ là $\widetilde{\mathbf{p}}_i$. Tương tự, ta ký hiệu đầu ra của mô hình Transformer trò với cùng đầu vào là $\mathbf{p}_i$. Như thường lệ, ta xét một hàm mất mát $\text{Loss}(\widetilde{\mathbf{p}}_i, \mathbf{p}_i)$ (chẳng hạn hàm entropy chéo) để tính khoảng cách giữa $\widetilde{\mathbf{p}}_i$ và $\mathbf{p}_i$. Khi đó, ta có thể định nghĩa hàm mất mát trên toàn bộ chuỗi là

$$\mathcal{L}(\mathbf{x}, \theta) \;=\; \frac{1}{n} \sum_{i=1}^{n} \text{Loss}(\widetilde{\mathbf{p}}_i, \mathbf{p}_i), \tag{6.184}$$

trong đó θ là tham số của mô hình trò[26]. Dùng hàm mất mát này, ta có thể tối ưu θ cho một tập chuỗi nguồn bất kỳ $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$ bằng cách cực tiểu hóa hàm mục tiêu $\sum_{k=1}^{K} \mathcal{L}(\mathbf{x}_k, \theta)$.

Chú giải của người dịch: Vì sao học từ "thầy" lại tốt hơn học từ nhãn?

Nhãn đúng chỉ cho biết *một* đáp án, ví dụ từ tiếp theo là "mèo" (phân phối $(1, 0, 0)$ trên ba từ "mèo", "chó", "xe"). Mô hình thầy lại cho một phân phối "mềm" như $(0.7,\ 0.25,\ 0.05)$: nó ngầm nói rằng "chó" cũng khá hợp lý, còn "xe" thì không. Thông tin về *mức độ gần đúng* của các phương án sai giúp mô hình trò học được nhiều hơn từ mỗi ví dụ — giống như học sinh học từ lời giảng của thầy chứ không chỉ từ đáp án cuối sách.

Một số mở rộng của phương pháp cơ bản này đã được phát triển để mô hình hóa bài toán chuyển giao tri thức giữa hai mô hình. Một cách đơn giản là dùng các trạng thái ẩn thay vì xác suất đầu ra làm mục tiêu huấn luyện [Romero và cộng sự, 2014]. Trong trường hợp này, mục tiêu là cực tiểu hóa sự khác biệt giữa một số trạng thái ẩn của mô hình thầy và các

[26] Chúng tôi bỏ qua tham số của mô hình thầy vì chúng được giữ cố định trong suốt quá trình huấn luyện.

trạng thái tương ứng của mô hình trò. Thay vì dùng đầu ra của các tầng khác nhau làm mục tiêu huấn luyện mô hình trò, một kỹ thuật khác là mô hình hóa quan hệ giữa các mẫu và huấn luyện mô hình trò bằng cách cực tiểu hóa sự khác biệt giữa các mã hóa quan hệ của mô hình thầy và mô hình trò [Park và cộng sự, 2019; Peng và cộng sự, 2019]. Ví dụ, ta có thể xây dựng một mô hình mã hóa quan hệ dựa trên kiến trúc Transformer. Mục tiêu khi đó là tối ưu mô hình trò sao cho mã hóa quan hệ tương ứng của nó trên một nhóm mẫu càng gần với mã hóa của mô hình thầy càng tốt.

Đối với các bài toán sinh chuỗi, một trường hợp đặc biệt của chưng cất tri thức, có thể được xem như một cách **tăng cường dữ liệu** (*data augmentation*), thường được dùng để phát triển các mô hình gọn nhẹ [Kim và Rush, 2016]. Ví dụ, xét bài toán chuyển khả năng dịch của một mô hình dịch máy đã phát triển tốt (tức mô hình thầy) sang một mô hình mới (tức mô hình trò). Cho một tập câu phía nguồn $\{\mathbf{x}_1, \cdots, \mathbf{x}_K\}$, ta có thể dùng mô hình thầy để dịch mỗi $\mathbf{x}_k$ thành một câu phía đích $\widetilde{\mathbf{y}}_k$. Sau đó, bằng cách coi $\mathbf{x}_k$ và $\widetilde{\mathbf{y}}_k$ là các cặp câu, ta thu được một tập dữ liệu song ngữ gồm $\{(\mathbf{x}_1, \widetilde{\mathbf{y}}_1), \cdots, (\mathbf{x}_K, \widetilde{\mathbf{y}}_K)\}$. Ta có thể dùng tập dữ liệu song ngữ này làm tập dữ liệu có nhãn để huấn luyện mô hình trò như thường lệ. Một ưu điểm của phương pháp tăng cường dữ liệu này là nó không phụ thuộc vào kiến trúc. Nó không đòi hỏi truy cập vào kiến trúc bên trong của mô hình thầy hay mô hình trò. Do đó, nó có thể được áp dụng ngay cả khi mô hình thầy được dùng như một hộp đen. Các thảo luận chi tiết hơn về chưng cất tri thức có thể tìm thấy trong các bài tổng quan của Gou và cộng sự [2021] và Wang và Yoon [2021].

2. Cắt tỉa có cấu trúc

Cắt tỉa là một trong những phương pháp nén mô hình phổ biến nhất và đã được áp dụng cho rất nhiều hệ thống. Một cách tiếp cận thông dụng là **cắt tỉa phi cấu trúc** (*unstructured pruning*), trong đó ta chỉ giữ lại một tập con thưa các kết nối giữa các nơ-ron. Tuy nhiên, như với hầu hết các mô hình thưa, mạng được cắt tỉa theo cách này thường đòi hỏi các cài đặt chuyên biệt và sự hỗ trợ của phần cứng, điều này lại hạn chế mức tăng hiệu quả trong một số ứng dụng. Một phương án đơn giản hơn nhưng mạnh tay hơn là **cắt tỉa có cấu trúc** (*structured pruning*). Trong học sâu, cắt tỉa có cấu trúc là kỹ thuật loại bỏ đồng thời cả một nhóm nơ-ron hoặc kết nối. Ví dụ, ta có thể loại bỏ toàn bộ một tầng nơ-ron khỏi mạng để thu được một mô hình nông hơn. Với kiến trúc nhiều tầng và nhiều đầu, Transformer rất phù hợp với cắt tỉa có cấu trúc, và ta có thể cắt tỉa một mạng Transformer theo nhiều cách khác nhau. Chẳng hạn, ta có thể cắt bỏ một số đầu cụ thể trong cơ chế chú ý đa đầu [Voita và cộng sự, 2019; Michel và cộng sự, 2019], hoặc toàn bộ một số tầng trong chồng tầng [Hou và cộng sự, 2020; Kim và Awadalla, 2020].

Một cách hình thức, ta có thể biểu diễn một mạng nơ-ron như một tập các nhóm tham số $\{\theta_1, \cdots, \theta_R\}$, mỗi nhóm tương ứng với một thành phần hay mô-đun con cụ thể của mạng. Mục tiêu của ta là tìm một tập con của $\{\theta_1, \cdots, \theta_R\}$ cho phép xây dựng một mô hình có hiệu năng tốt trong khi vẫn giữ độ phức tạp tính toán thấp hơn đáng kể. Tuy nhiên, tìm kiếm vét cạn một tập con tối ưu như vậy là bất khả thi, vì số ứng viên có thể có tăng theo cấp số tổ hợp, và việc đánh giá tất cả chúng tốn kém quá mức về mặt tính toán.

Chú giải của người dịch: Bao nhiêu cách cắt tỉa?

Với R nhóm tham số, mỗi nhóm có thể giữ hoặc bỏ, nên có 2^R tập con (kiến thức tổ hợp lớp 11). Một Transformer 24 tầng, mỗi tầng 16 đầu chú ý, có $R = 384$ đầu; 2^{384} là con số có hơn 115 chữ số — không thể thử hết, dù mỗi lần thử chỉ mất một phần tỷ giây.

Một cách tiếp cận cắt tỉa có cấu trúc là loại bỏ ngẫu nhiên các thành phần của mô hình. Ta có thể lặp lại quá trình cắt tỉa ngẫu nhiên này nhiều lần để tạo ra một tập các mô hình ứng viên, rồi chọn mô hình có hiệu năng tốt nhất. Một cách tiếp cận khác dựa vào các quy tắc kinh nghiệm để xác

định thành phần nào là dư thừa và có thể loại bỏ an toàn. Các thước đo thông dụng để đánh giá tầm quan trọng của một nhóm tham số θ_r bao gồm nhiều chỉ số dựa trên chuẩn (norm) của trọng số hoặc gradient của nó [Santacroce và cộng sự, 2023]. Khi đó ta có thể cắt bỏ θ_r nếu các chỉ số này thấp hơn hoặc vượt quá các ngưỡng định trước. Cách tiếp cận thứ ba đặt bài toán cắt tỉa thành một bài toán tối ưu liên tục bằng cách đưa vào các cổng có thể huấn luyện, cho biết sự có mặt hay vắng mặt của các thành phần khác nhau [McCarley và cộng sự, 2019; Wang và cộng sự, 2020d; Lagunas và cộng sự, 2021]. Mô hình cắt tỉa cuối cùng có thể được suy ra trực tiếp từ các cổng đã huấn luyện này. Đáng chú ý là, trong nhiều trường hợp, cắt tỉa không còn chỉ là một bước hậu xử lý cho một mô hình đã huấn luyện xong, mà là một phần không thể tách rời của chính quá trình huấn luyện.

6.4.8 Nén chuỗi

Trong các bài toán mô hình hóa và sinh chuỗi, độ phức tạp thời gian và không gian chịu ảnh hưởng mạnh bởi độ dài của chuỗi đầu vào hoặc đầu ra. Vì vậy, chuỗi ngắn hơn thường được ưa chuộng hơn. Điều này đặc biệt quan trọng với các mô hình Transformer, vì độ phức tạp thời gian và không gian của chúng tăng theo bình phương độ dài chuỗi, khiến lượng bộ nhớ chiếm dụng và độ trễ trở thành những nút thắt tính toán đáng kể với các chuỗi rất dài. Trong khi các tiểu mục trước bàn về những thay đổi trong kiến trúc Transformer để xử lý chuỗi dài, ở mục này chúng tôi tìm hiểu các phương pháp nén chuỗi về những độ dài dễ xử lý hơn.

Một cách tiếp cận đơn giản là ánh xạ chuỗi đầu vào thành một biểu diễn có kích thước cố định. Ví dụ, dùng các mô hình hồi quy đã bàn ở Mục 6.4.2, ta có thể mã hóa một chuỗi vectơ thành một vectơ duy nhất. Phương pháp này có thể dễ dàng được mở rộng để tạo ra một biểu diễn lớn hơn, nhờ đó biểu diễn này có thể giữ lại nhiều thông tin hơn từ đầu vào ban đầu. Ví dụ, ta có thể chọn một số lượng cố định các trạng thái ẩn trên chuỗi để tạo thành một chuỗi mới có độ dài cố định. Một cách khác để biểu diễn một chuỗi có độ dài thay đổi thành một chuỗi có độ dài cố định là thực hiện chú ý chéo (*cross-attention*) giữa các vectơ đầu vào và một tập các biểu diễn ẩn có thể học được. Trong công trình của Jaegle và cộng sự [2021], điều này được thực hiện bằng cách đưa vào r biểu diễn ẩn

$\{\mathbf{u}_1, \cdots, \mathbf{u}_r\}$, rồi áp dụng chú ý chéo giữa các vectơ đầu vào $\mathbf{x}_1, \cdots, \mathbf{x}_m$ và các biểu diễn ẩn này. Mô hình chú ý có thể là một mô hình chú ý QKV chuẩn, trong đó ta coi $\{\mathbf{u}_1, \cdots, \mathbf{u}_r\}$ là các truy vấn và $\{\mathbf{x}_1, \cdots, \mathbf{x}_m\}$ là các khóa và giá trị. Đầu ra của mô hình này là một chuỗi gồm r vectơ, có thể dùng làm đầu vào độ dài cố định cho các hệ thống phía sau.

Chú giải của người dịch: Vì sao đầu ra luôn có r vectơ?

Trong chú ý, mỗi *truy vấn* tạo ra đúng một vectơ đầu ra (một trung bình có trọng số của các giá trị). Ở đây truy vấn là r vectơ $\mathbf{u}$ cố định, còn đầu vào chỉ đóng vai khóa và giá trị. Vì thế dù câu dài $m = 10$ hay $m = 10\,000$ token, đầu ra vẫn luôn là r vectơ (ví dụ $r = 64$) — giống như 64 "phóng viên" mỗi người đọc toàn bộ văn bản rồi viết một bản tóm tắt theo góc nhìn riêng.

Cách tiếp cận thứ hai là dùng giảm mẫu (*downsampling*) để nén chuỗi thành một chuỗi ngắn hơn. Một phương pháp giảm mẫu điển hình là tích chập có bước nhảy (*strided convolution*), đã được dùng rộng rãi trong thị giác máy tính và xử lý tiếng nói. Ví dụ, giả sử có một chuỗi m vectơ trong $\mathbb{R}^d$. Ta có thể định nghĩa một bộ lọc có độ rộng 2 và bước nhảy 2. Khi nhận chuỗi làm đầu vào, bộ lọc tạo ra một chuỗi gồm $\frac{m}{2}$ vectơ mới trong $\mathbb{R}^d$, nên ta có hệ số rút gọn là 2. Ngoài ra, ta có thể xếp chồng nhiều tầng tích chập hoặc tầng gộp (pooling) để đạt được mức rút gọn độ dài mong muốn, gọi là **giảm mẫu lũy tiến** (*progressive downsampling*). Tuy nhiên, có vẻ không thể tránh khỏi việc giảm mẫu sẽ dẫn đến mất mát thông tin [Han và cộng sự, 2020; Burchi và Vielzeuf, 2021]. Ta cần cân nhắc sự đánh đổi giữa độ dài chuỗi đã nén và hiệu năng của các hệ thống phía sau [Xu và cộng sự, 2023b].

Trong NLP, bài toán nén chuỗi cũng liên quan chặt chẽ đến bài toán tách token cho chuỗi đầu vào. Do đó, tách token là một cách tiếp cận thực tế có thể dùng để giải quyết vấn đề độ dài. Chia một chuỗi ký tự thành các token nhỏ (chẳng hạn ký tự) nói chung làm giảm độ thưa của dữ liệu, giúp việc học vectơ nhúng của các token này dễ hơn, nhưng các cách làm như vậy thường dẫn đến chuỗi dài. Ngược lại, ta sẽ có chuỗi ngắn hơn nếu chia chuỗi đầu vào thành các đơn vị lớn hơn, nhưng khi đó có thể gặp vấn đề dữ liệu thưa. Trong các phương pháp tách token tất định, vốn tạo ra kết quả tách token dựa trên thống kê thu thập từ toàn bộ tập dữ liệu, độ dài chuỗi có thể được kiểm soát ở một mức độ nào đó bằng cách điều chỉnh các siêu tham số cụ thể; ví dụ, trong mã hóa cặp byte (byte pair encoding)

[Sennrich và cộng sự, 2016b], tăng kích thước bộ từ vựng nói chung làm giảm số token thu được. Một cách khác để có được một chuỗi token phù hợp là dùng một mô hình để lựa chọn giữa các ứng viên tách token [Kudo, 2018; Provilkov và cộng sự, 2020]. Giống như nhiều mô hình xác suất cho sinh văn bản, trong trường hợp này ta có thể đưa các tri thức tiên nghiệm (prior) vào tiêu chí chọn cách tách token, để thể hiện sự ưu tiên cho chuỗi ngắn hơn so với chuỗi dài hơn.

Cách tiếp cận thứ tư[27] cho nén chuỗi là loại bỏ một số token trong chuỗi. Chẳng hạn, trong nhiều ứng dụng thực tế, chuỗi đơn giản bị cắt cụt khi độ dài vượt quá một ngưỡng định trước. Ta có thể liên hệ điều này với các cách tiếp cận thoát sớm và bỏ qua trong tính toán có điều kiện. Vì vậy, các phương pháp đã bàn ở Mục 6.4.6 có thể áp dụng trực tiếp. Các phương pháp loại bỏ token cũng có thể được xem là các phương pháp cắt tỉa, gọi là **cắt tỉa token** (*token pruning*). Bằng cách loại bỏ những token ít quan trọng đối với việc biểu diễn toàn bộ chuỗi, cắt tỉa token có thể giảm đáng kể độ dài chuỗi mà vẫn duy trì hiệu năng của các hệ thống NLP trên các tác vụ phía sau [Kim và cộng sự, 2023].

6.4.9 Các phương pháp tính toán hiệu năng cao

Cho đến giờ trong mục này, chúng ta đã bàn về các mô hình Transformer hiệu quả từ góc độ học sâu. Tuy nhiên, ta chưa xét đến hiệu quả phần cứng của chúng. Vì phần cứng hiện đại cung cấp nhiều mô thức thực thi khác nhau, mức tiết kiệm thời gian và bộ nhớ trên thực tế thường phụ thuộc vào thông số kỹ thuật của hệ thống phần cứng. Một hướng nghiên cứu về sử dụng hiệu quả tài nguyên tính toán là khám phá các phương pháp tính toán song song. Đã có nhiều nỗ lực phát triển các mô hình Transformer quy mô lớn bằng cách dùng một cụm máy. Thông thường, việc mở rộng Transformer lên các mô hình có hàng tỷ hoặc thậm chí hàng chục tỷ tham số đòi hỏi thiết kế cẩn thận các chiến lược song song để phân mảnh các mạng lớn. Cài đặt hiệu quả hơn cho những hệ thống như vậy còn đòi hỏi xem xét chi phí truyền thông trong cụm, cũng như tận dụng các mô hình thưa chỉ kích hoạt một tập con nhỏ tham số cho mỗi mẫu, nhờ đó có thể dùng những mô hình rất lớn. Phần lớn các phương pháp này đã được

[27] **[ND]** Sách gốc viết "thứ tư" (*fourth*) dù chỉ nêu tường minh ba cách tiếp cận trước đó; có thể hiểu cách dùng tách token là cách thứ ba.

nghiên cứu sâu rộng trong các tài liệu [Lepikhin và cộng sự, 2021; Barham và cộng sự, 2022; Fedus và cộng sự, 2022b]. Kết quả của các nghiên cứu này là nền tảng cho nhiều công trình tiếp theo khảo sát các **định luật co giãn** (*scaling law*) của mô hình ngôn ngữ lớn [Brown và cộng sự, 2020; Chowdhery và cộng sự, 2022]. Vì các mô hình phân tán quy mô lớn mang tính tổng quát và không chuyên biệt cho Transformer, chúng tôi bỏ qua phần thảo luận về chúng ở đây. Bạn đọc quan tâm có thể tham khảo các bài báo trên để biết thêm chi tiết.

Trong tiểu mục này, chúng tôi xét các phương pháp có tính đến phần cứng nhằm đạt hiệu quả tính toán cao hơn cho mô hình Transformer. Trước hết, ta xét một phương pháp đơn giản nhưng được dùng rộng rãi, nhằm biểu diễn và thực thi mạng nơ-ron bằng các định dạng độ chính xác thấp hơn hoặc độ chính xác hỗn hợp [Gholami và cộng sự, 2022]. Các mạng nơ-ron thông thường thường dựa trên biểu diễn dấu phẩy động độ chính xác đơn và/hoặc độ chính xác kép cho dữ liệu. Mặc dù kiểu dữ liệu dấu phẩy động độ chính xác đơn cung cấp một cách đủ chính xác để biểu diễn tham số và trạng thái trung gian trong hầu hết trường hợp, ở một số ứng dụng chúng là không cần thiết. Thay vào đó, có thể dùng định dạng độ chính xác một nửa (hoặc thậm chí thấp hơn) để biểu diễn số thực dấu phẩy động trong mạng nơ-ron. Kích thước của mô hình thu được vì thế chỉ bằng một nửa mô hình ban đầu. Một ưu điểm của việc dùng biểu diễn dấu phẩy động độ chính xác một nửa là, mặc dù xử lý các kiểu dữ liệu này đòi hỏi các API mới cho các phép toán đại số tuyến tính và sự hỗ trợ của phần cứng, nó không làm thay đổi kiến trúc mô hình, chỉ cần sửa đổi nhỏ trong hệ thống. Ví dụ, biểu diễn dấu phẩy động độ chính xác một nửa có thể được áp dụng cho huấn luyện hoặc suy luận của Transformer, hoặc cả hai.

Chú giải của người dịch: Độ chính xác đơn, kép, một nửa

Máy tính lưu số thực theo dạng "dấu phẩy động", gần giống cách viết khoa học 6.02×10^{23}: một phần chữ số có nghĩa và một phần số mũ. Độ chính xác *kép* dùng 64 bit (khoảng 15–16 chữ số có nghĩa), *đơn* dùng 32 bit (khoảng 7 chữ số), *một nửa* dùng 16 bit (khoảng 3–4 chữ số). Một mô hình 7 tỷ tham số ở dạng 32 bit chiếm khoảng 28 GB; chuyển sang 16 bit chỉ còn khoảng 14 GB. Mạng nơ-ron thường chịu được sai số nhỏ như vậy mà vẫn cho kết quả tốt.

Gần đây, việc triển khai các mô hình Transformer lớn được cải thiện

thêm nhờ lượng tử hóa (*quantization*) các mô hình này. Trong xử lý tín hiệu, lượng tử hóa là quá trình ánh xạ các giá trị liên tục (tức biểu diễn dấu phẩy động) thành một tập giá trị rời rạc (tức biểu diễn dấu phẩy tĩnh). Quá trình này thường được thực hiện bằng một hệ thống gọi là *bộ lượng tử hóa*. Trong ngữ cảnh mạng nơ-ron, một bộ lượng tử hóa gồm hai hàm — hàm lượng tử hóa và hàm giải lượng tử hóa. Hàm lượng tử hóa ánh xạ một số dấu phẩy động thành một số nguyên (với ít bit hơn). Một hàm lượng tử hóa đơn giản được cho bởi

$$Q(x) \;=\; \left\lfloor \frac{x}{s} \right\rceil, \tag{6.185}$$

trong đó $\lfloor \cdot \rceil$ là hàm làm tròn[28], x là đầu vào giá trị thực, và s là kích thước bước lượng tử hóa, điều khiển mức độ lượng tử hóa. Hàm lượng tử hóa đi kèm với một hàm giải lượng tử hóa

$$D(r) \;=\; s \cdot r. \tag{6.186}$$

Với ký hiệu này, bộ lượng tử hóa có thể được biểu diễn là $D(Q(x)) = s \cdot \lfloor \frac{x}{s} \rceil$. Hiệu giữa $D(Q(x))$ và x được gọi là *sai số lượng tử hóa*. Giá trị s nhỏ hơn thường làm giảm sai số lượng tử hóa. Tuy nhiên, trong thực tế, ta muốn chọn một giá trị s phù hợp để trải đều các giá trị có thể có của $Q(x)$ trên toàn bộ miền biểu diễn số nguyên. Ví dụ, $s = \frac{\max\{D(r)\}}{2^p - 1}$, trong đó p là số bit dùng để biểu diễn một số nguyên và $\max\{D(r)\}$ là giá trị lớn nhất có thể biểu diễn sau khi giải lượng tử hóa. Các phương trình trên cho thấy một trong những trường hợp đơn giản nhất của lượng tử hóa. Các thảo luận tổng quát hơn về lượng tử hóa có thể tìm thấy trong các sách về xử lý tín hiệu số và các bài tổng quan liên quan [Oppenheim và Schafer, 1975; Rabiner và Gold, 1975; Gray, 1998].

Chú giải của người dịch: Ví dụ lượng tử hóa bằng số

Giả sử các trọng số nằm trong khoảng $[0;\, 2.55]$ và ta dùng $p = 8$ bit (số nguyên từ 0 đến 255). Chọn $s = \frac{2.55}{2^8 - 1} = 0.01$. Với $x = 1.2345$: $Q(x) = \lfloor 123.45 \rceil = 123$ (lưu bằng 1 byte thay vì 4 byte), và $D(123) = 1.23$. Sai số lượng tử hóa là $|1.23 - 1.2345| = 0.0045$, không vượt quá $s/2$. Nếu chọn s nhỏ hơn, sai số giảm nhưng khi đó các số nguyên cần thiết có thể vượt quá 255, tức cần nhiều bit hơn.

[28] $\lfloor a \rceil$ trả về số nguyên gần a nhất.

Áp dụng lượng tử hóa cho Transformer tương đối đơn giản. Ý tưởng là ta lượng tử hóa đầu vào và tham số mô hình bằng $Q(x)$ rồi đưa chúng vào một mô hình Transformer đã lượng tử hóa, trong đó mọi tầng đều hoạt động trên các tensor giá trị nguyên. Nói cách khác, ta cài đặt mô hình chỉ bằng số học số nguyên. Tuy nhiên, cái giá phải trả cho mô hình nén này, như với nhiều cách tiếp cận xấp xỉ khác trong học sâu, là các dự đoán của nó kém chính xác hơn so với Transformer chuẩn. Dùng phép toán số nguyên để xấp xỉ phép toán giá trị liên tục nói chung gây ra sai số xấp xỉ. Các sai số này sẽ tích lũy nếu mạng nơ-ron lượng tử hóa sâu. Hơn nữa, mô hình Transformer có những thành phần (như các tầng con tự chú ý) đòi hỏi các phép toán đại số tuyến tính tương đối phức tạp. Chỉ đơn giản áp dụng lượng tử hóa cho các mô-đun này sẽ làm giảm độ chính xác đáng kể. Một giải pháp là đơn giản hóa kiến trúc mô hình và phát triển các mô hình con mới phù hợp hơn với lượng tử hóa. Ngoài ra, một mô thức phổ biến hơn trong các mạng nơ-ron lượng tử hóa là thêm các hàm giải lượng tử hóa vào mạng, sao cho đầu ra của một tầng là các tensor dấu phẩy động và có thể dùng như thường lệ ở các bước tiếp theo. Xét một ví dụ đơn giản, trong đó ta nhân một ma trận đầu vào giá trị thực $\mathbf{a}$ với một ma trận tham số giá trị thực $\mathbf{A}$. Trước tiên ta lượng tử hóa $\mathbf{a}$ và $\mathbf{A}$, rồi nhân chúng bằng phép nhân ma trận số nguyên. Kết quả sau đó được giải lượng tử hóa thành một ma trận giá trị thực. Bằng cách này, ta thu được một xấp xỉ $D(Q(\mathbf{a}) \cdot Q(\mathbf{A}))$ cho $\mathbf{a} \cdot \mathbf{A}$ với chi phí tính toán rất thấp.

Tuy nhiên, việc kẹp mỗi tầng giữa $Q(\cdot)$ và $D(\cdot)$ sẽ dẫn đến chi phí phụ thêm cho việc chạy $Q(\cdot)$ và $D(\cdot)$. Trong một số ứng dụng thực tế, chi phí tính toán phát sinh do $Q(\cdot)$ và $D(\cdot)$ thậm chí còn lớn hơn thời gian tiết kiệm được từ việc thực hiện các phép toán số nguyên. Nói chung, lợi ích của việc lượng tử hóa mạng nơ-ron chỉ lớn hơn chi phí khi mạng nơ-ron đủ lớn. Vì vậy, trong thực tế, người ta thường chỉ thực hiện tính toán lượng tử hóa cho những phép toán có chi phí tính toán cao. Ví dụ, trong các mô hình ngôn ngữ lớn gần đây, lượng tử hóa chủ yếu được áp dụng cho phép nhân các ma trận lớn, mang lại mức tiết kiệm đáng kể về thời gian và bộ nhớ.

Mặc dù các cách tiếp cận lượng tử hóa có thể được dùng cả khi huấn luyện lẫn suy luận, một cách tiếp cận được dùng rộng rãi là lượng tử hóa mô hình Transformer sau khi huấn luyện (gọi là **lượng tử hóa sau huấn luyện** (*post-training quantization*)). Trong cách tiếp cận này, lượng tử hóa

được thực hiện trên các mạng nơ-ron dấu phẩy động đã được huấn luyện tốt, nhờ đó ít sai số liên quan đến lượng tử hóa hơn. Tuy nhiên, các sai số này không dễ được bù đắp vì chúng xuất hiện sau khi quá trình huấn luyện đã hoàn tất. Một cách tiếp cận hứa hẹn hơn là đưa lượng tử hóa vào trong quá trình huấn luyện, để mô hình có thể học cách bù đắp các sai số liên quan đến lượng tử hóa [Jacob và cộng sự, 2018; Nagel và cộng sự, 2021]. Đã có một số nỗ lực áp dụng huấn luyện có tính đến lượng tử hóa cho Transformer [Bondarenko và cộng sự, 2021; Stock và cộng sự, 2021; Yang và cộng sự, 2023b]. Ngoài hiệu quả tính toán, một yếu tố quan trọng khác đối với các hệ thống hiệu năng cao là các ràng buộc của hệ phân cấp bộ nhớ. Nói chung, thiết kế hệ thống tốt hơn đòi hỏi phải xem xét tốc độ và dung lượng của các cấp bộ nhớ khác nhau. Vấn đề còn phức tạp hơn khi huấn luyện các mô hình Transformer lớn trên phần cứng hiện đại, nơi cả GPU và CPU đều được sử dụng. Một nguyên tắc chung của thiết kế hệ thống là cần giảm thiểu việc truyền dữ liệu giữa các cấp bộ nhớ khác nhau. Mặc dù lý tưởng nhất là có một bộ nhớ cấp cao, dung lượng lớn để lưu mọi dữ liệu cần xử lý, trong nhiều tình huống thực tế, kích thước của bộ nhớ nhanh trên chip nhỏ hơn tổng kích thước dữ liệu nhiều bậc độ lớn. Trong trường hợp này, ta có thể sắp xếp lại thứ tự truy cập bộ nhớ trong các thuật toán sao cho dữ liệu dùng ở các bước tính toán liền kề có thể được nạp vào bộ nhớ tốc độ cao cùng một lúc. Ý tưởng này thúc đẩy sự phát triển của nhiều thư viện đại số tuyến tính nhanh. Ví dụ, có những thuật toán nhân ma trận được tối ưu hóa cao cho các hình dạng ma trận đầu vào khác nhau.

Việc dùng các thuật toán đại số tuyến tính đã tối ưu này để xây dựng một hệ thống dựa trên Transformer là tương đối đơn giản, nhưng các mô-đun của hệ thống này không được tối ưu hóa đồng thời về hiệu quả. Ví dụ, một tầng con tự chú ý gồm một loạt các phép co giãn, chuẩn hóa và nhân ma trận. Mặc dù mỗi phép toán này đều được cài đặt trong các thư viện đại số tuyến tính hiệu quả, việc gọi chúng liên tiếp vẫn đòi hỏi nhiều lần truyền dữ liệu bộ nhớ khi chuyển từ phép toán này sang phép toán khác. Trong thực tế, một cách tốt hơn là giữ một số trạng thái trung gian trong bộ nhớ trên chip và dùng lại chúng ở các bước tính toán tiếp theo thay vì nạp lại từ bộ nhớ chậm hơn. Ví dụ, trên GPU hiện đại, một cách đơn giản để làm điều này là gộp nhiều phép toán thành một phép toán duy nhất, một kỹ thuật được gọi là **hợp nhất nhân** (*kernel fusion*). Với mô hình

Transformer, ý tưởng chung là thiết kế các chiến lược phân chia và bố trí dữ liệu sao cho tối đa hóa lượng tính toán trên mỗi khối dữ liệu được nạp vào bộ nhớ hiệu năng cao, đồng thời giảm thiểu việc truyền dữ liệu bộ nhớ. Đã có một số nỗ lực dùng các chiến lược này để cải thiện các mô hình chú ý trong Transformer [Ivanov và cộng sự, 2021; Pope và cộng sự, 2023]. Một số phương pháp trong đó, như FlashAttention và PagedAttention, đã được tích hợp thành công vào các mô hình ngôn ngữ lớn gần đây [Dao và cộng sự, 2022; Kwon và cộng sự, 2023].

Chú giải của người dịch: Hệ phân cấp bộ nhớ, ví von đời thường

Hãy hình dung bạn nấu ăn: mặt bàn bếp (bộ nhớ trên chip) nhỏ nhưng lấy đồ cực nhanh; tủ lạnh (bộ nhớ GPU chính) to hơn nhưng mỗi lần mở tủ tốn thời gian; siêu thị (bộ nhớ CPU/ổ đĩa) còn xa hơn nữa. Nấu hiệu quả là lấy đủ nguyên liệu cho vài bước liên tiếp ra bàn một lần, làm xong hết rồi mới cất, thay vì mỗi bước lại mở tủ. "Hợp nhất nhân" chính là gộp các bước co giãn, Softmax, nhân ma trận thành một lượt làm việc trên mặt bàn; FlashAttention áp dụng ý tưởng này cho cơ chế chú ý.

6.5 Ứng dụng

Transformer có rất nhiều ứng dụng trong các tác vụ NLP. Mặc dù mô hình Transformer do Vaswani và cộng sự [2017] giới thiệu dựa trên một kiến trúc bộ mã hóa (*encoder*)–bộ giải mã (*decoder*) chuẩn, nó chủ yếu được dùng theo ba cấu hình khác nhau:

- **Mô hình chỉ có bộ giải mã** (*Decoder-only Models*). Nếu bỏ các tầng con chú ý chéo, bộ giải mã trở thành một mô hình ngôn ngữ (*language model*) chuẩn. Do đó, mô hình chỉ có bộ giải mã này có thể được áp dụng cho các bài toán sinh văn bản. Ví dụ, cho một chuỗi token ngữ cảnh bên trái, ta dùng mô hình để dự đoán các token tiếp theo.

- **Mô hình chỉ có bộ mã hóa** (*Encoder-only Models*). Bộ mã hóa Transformer có thể được xem là mô hình chuỗi nhận một chuỗi token làm đầu vào và tạo ra một chuỗi biểu diễn, mỗi biểu diễn ứng với một token đầu vào. Các biểu diễn này có thể được xem như một dạng mã hóa của chuỗi đầu vào, và thường được dùng làm đầu vào

cho một mô hình dự đoán. Kiến trúc bộ mã hóa–bộ dự đoán này là nền tảng của nhiều hệ thống NLP, chẳng hạn các hệ thống phân loại câu và gán nhãn chuỗi (*sequence labeling*). Bộ mã hóa Transformer đã tiền huấn luyện cũng có thể được dùng để ánh xạ các văn bản vào cùng một không gian vectơ, để ta có thể tính khoảng cách hay độ tương đồng giữa hai văn bản bất kỳ.

- **Mô hình bộ mã hóa–bộ giải mã** (*Encoder-Decoder Models*). Mô hình bộ mã hóa–bộ giải mã thường được dùng để mô hình hóa các bài toán chuỗi-sang-chuỗi (*sequence-to-sequence*). Ứng dụng bao gồm nhiều tác vụ trong NLP và các lĩnh vực liên quan, như dịch máy (*machine translation*) và sinh chú thích ảnh.

Lưu ý rằng mặc dù phần lớn các hệ thống dựa trên Transformer có thể được xếp vào ba loại trên, cùng một bài toán NLP nói chung có thể được giải quyết bằng các loại mô hình khác nhau. Ví dụ, các mô hình chỉ có bộ giải mã gần đây đã thể hiện hiệu năng mạnh trên rất nhiều bài toán bằng cách đặt chúng thành các tác vụ sinh văn bản, dù trước đây nhiều bài toán trong số đó được giải quyết bằng mô hình bộ mã hóa–bộ giải mã hoặc mô hình chỉ có bộ mã hóa. Để minh họa cách áp dụng các mô hình trên, mục này xét một vài ứng dụng trong đó Transformer được chọn làm mô hình xương sống.

6.5.1 Mô hình hóa ngôn ngữ

Mô hình hóa ngôn ngữ là một tác vụ NLP, trong đó ta dự đoán token tiếp theo khi biết các token đứng trước nó. Bài toán này thường được phát biểu là ước lượng phân phối của các token tại vị trí $i+1$ khi biết các token tại các vị trí $0 \sim i$. Ở đây ta ký hiệu phân phối này là $\Pr(\cdot \mid x_0, \ldots, x_i)$, trong đó $x_0, \ldots, x_i$ là các token quan sát được. Token dự đoán thu được bằng cách cực đại hóa xác suất:

$$\hat{x}_{i+1} = \underset{x_{i+1} \in V}{\arg\max} \Pr(x_{i+1} | x_0, \cdots, x_i), \tag{6.187}$$

trong đó V là bộ từ vựng. Việc dự đoán có thể được mở rộng cho các token theo sau $\hat{x}_{i+1}$

$$\hat{x}_{k+1} = \underset{x_{k+1} \in V}{\arg\max} \Pr(x_{k+1} | x_0, \cdots, x_i, \hat{x}_{i+1}, \cdots, \hat{x}_k). \tag{6.188}$$

Mô hình này là nền tảng của nhiều hệ thống sinh văn bản: cho các token ngữ cảnh $x_0 \cdots x_i$, ta sinh các token còn lại $\hat{x}_{i+1} \cdots \hat{x}_{k+1}$ để chuỗi trở nên hoàn chỉnh và mạch lạc.

Chú giải của người dịch: Sinh văn bản từng token một

Phương trình (6.187–6.188) mô tả cách "viết tiếp": với ngữ cảnh "Hôm nay trời", mô hình cho xác suất các từ tiếp theo, ví dụ "đẹp" 0.4, "mưa" 0.3, "nắng" 0.2…; arg max chọn từ có xác suất lớn nhất ("đẹp"). Rồi "Hôm nay trời đẹp" trở thành ngữ cảnh mới để đoán từ kế tiếp, cứ thế cho tới khi đủ câu. Dấu mũ ˆ nhắc rằng đây là token do mô hình *tự đoán*, không phải token cho sẵn.

Như đã bàn ở Mục 6.1.1, bộ giải mã Transformer về bản chất là mô hình ngôn ngữ. Một khác biệt then chốt giữa bài toán giải mã trong Transformer bộ mã hóa–bộ giải mã và bài toán mô hình hóa ngôn ngữ là bộ giải mã Transformer đưa ra dự đoán có điều kiện trên các token ngữ cảnh ở cả phía bộ mã hóa lẫn phía bộ giải mã, thay vì chỉ có điều kiện trên các token đứng trước. Để sửa bộ giải mã Transformer thành một mô hình ngôn ngữ chuẩn, ta chỉ cần bỏ các tầng con chú ý chéo, và một khối giải mã Transformer có thể được biểu diễn là

$$\mathbf{S}^l = \text{Layer}_{\text{ffn}}(\mathbf{S}^l_{\text{self}}), \tag{6.189}$$

$$\mathbf{S}^l_{\text{self}} = \text{Layer}_{\text{self}}(\mathbf{S}^{l-1}). \tag{6.190}$$

Ở đây, $\mathbf{S}^l$ ký hiệu đầu ra của khối ở độ sâu l. $\text{Layer}_{\text{self}}(\cdot)$ ký hiệu tầng con tự chú ý, và $\text{Layer}_{\text{ffn}}(\cdot)$ ký hiệu tầng con FFN. Ta thấy khối giải mã này có cùng dạng với một khối mã hóa. Khác biệt giữa khối giải mã và khối mã hóa nằm ở chiến lược che được dùng khi huấn luyện: khối giải mã che sự chú ý từ một vị trí i tới mọi vị trí ngữ cảnh bên phải $k > i$, còn khối mã hóa không có ràng buộc này. Một tầng Softmax được đặt lên trên khối cuối cùng và được dùng để tạo ra phân phối trên bộ từ vựng tại mỗi vị trí. Khi suy luận, bộ giải mã Transformer hoạt động theo kiểu tự hồi quy, như mô tả trong Phương trình (6.188).

Việc huấn luyện các mô hình này theo một mô thức chuẩn. Ta học mô hình bằng cách cập nhật tham số nhiều lần, dựa trên gradient của hàm mất mát trên các mẫu huấn luyện. Mô thức này có thể được mở rộng cho việc huấn luyện các mô hình ngôn ngữ lớn dựa trên Transformer, vốn đã được áp dụng rộng rãi trong AI tạo sinh. Tuy nhiên, huấn luyện mô hình

Transformer ở quy mô lớn, bao gồm mô hình chỉ có bộ giải mã, chỉ có bộ mã hóa và bộ mã hóa–bộ giải mã, có thể dẫn đến những khó khăn mới, chẳng hạn sự bất ổn định khi huấn luyện. Chúng tôi sẽ bàn thêm về những vấn đề này trong các chương sau, nơi tiền huấn luyện (*pre-training*) quy mô lớn là trọng tâm chính.

6.5.2 Mã hóa văn bản

Với nhiều bài toán NLP, một mô thức được dùng rộng rãi là trước hết học một biểu diễn của chuỗi đầu vào, rồi đưa ra dự đoán cho các tác vụ phía sau dựa trên biểu diễn này. Nhờ đó, ta tách việc biểu diễn chuỗi khỏi các tác vụ NLP cụ thể. Một ưu điểm của mô thức này là ta có thể huấn luyện một mô hình chuỗi không chuyên biệt cho bài toán cụ thể nào, nhờ đó nó có thể tổng quát hóa tốt.

Rõ ràng, bộ mã hóa Transformer là một loại mô hình chuỗi và có thể được dùng làm bộ mã hóa văn bản. Xét một bộ mã hóa Transformer với L tầng mã hóa. Đầu ra của tầng mã hóa cuối cùng có thể được xem là biểu diễn của chuỗi. Ở đây, ta thêm một token đặc biệt x_0 vào đầu mỗi chuỗi, cho biết điểm bắt đầu của chuỗi (thường được viết là $\langle\text{SOS}\rangle$ hoặc [CLS]). Cho một chuỗi $m+1$ token đầu vào $x_0, x_1, \cdots, x_m$, đầu ra của bộ mã hóa sẽ là một chuỗi $m+1$ vectơ $\mathbf{h}_0^L, \mathbf{h}_1^L, \cdots, \mathbf{h}_m^L$. Vì x_0 không ứng với một từ thực sự nào và có vectơ nhúng vị trí cố định, nó đóng vai trò như một token biểu diễn toàn cục, thu thập thông tin từ các vị trí khác thông qua cơ chế tự chú ý. Do đó, $\mathbf{h}_0^L$ đóng vai trò biểu diễn của toàn bộ chuỗi, mà không thiên lệch về bất kỳ token hay vị trí cụ thể nào. Trong nhiều trường hợp, ta cần một biểu diễn duy nhất của chuỗi để làm đầu vào cho các thành phần phía sau của hệ thống. Ví dụ, ta có thể xây dựng một hệ thống phân loại câu dựa trên một vectơ duy nhất được sinh ra từ $\{\mathbf{h}_0^L, \ldots, \mathbf{h}_m^L\}$. Trong trường hợp này, ta có thể đơn giản dùng $\mathbf{h}_0^L$ làm biểu diễn của chuỗi. Một cách tổng quát hơn là thêm một tầng gộp (pooling) vào bộ mã hóa, cho phép ta thử các phương pháp gộp khác nhau để sinh vectơ nhúng của chuỗi từ $\{\mathbf{h}_0^L, \cdots, \mathbf{h}_m^L\}$.

Trong mã hóa văn bản, các chuỗi token được biểu diễn bằng các vectơ giá trị thực, thường được gọi là biểu diễn câu (*sentence representation*) hay vectơ nhúng câu (*sentence embedding*), có thể được hiểu như các điểm trong một không gian nhiều chiều [Hill và cộng sự, 2016]. Do đó, một ứng

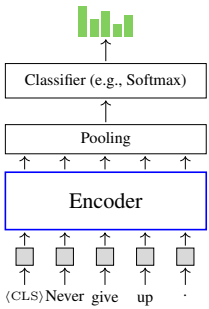
dụng khác của mã hóa văn bản là tính độ tương đồng ngữ nghĩa hoặc cú pháp giữa các chuỗi token dựa trên mức độ gần nhau tương đối của chúng trong không gian này. Một phương pháp trực tiếp cho việc này là tính khoảng cách Euclid hoặc độ tương đồng cosin giữa các vectơ nhúng của chuỗi. Khoảng cách giữa hai chuỗi càng ngắn thì chúng được coi là càng giống nhau. Có nhiều độ đo khoảng cách khác nhau, và có thể kết hợp chúng để có một thước đo độ tương đồng chuỗi vững chắc hơn. Những phép tính độ tương đồng như vậy được áp dụng trong các lĩnh vực như suy diễn văn bản (textual entailment), truy xuất thông tin và đánh giá bản dịch [Cer và cộng sự, 2018; Reimers và Gurevych, 2019]. Ngoài ra, chúng cũng thường được dùng để đánh giá chất lượng nội tại của các mô hình mã hóa văn bản.

Chú giải của người dịch: Độ tương đồng cosin

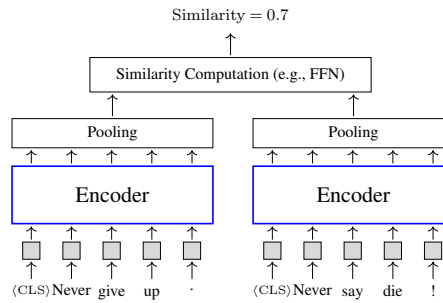
Ở lớp 10 ta đã biết $\cos(\vec{u}, \vec{v}) = \frac{\vec{u}\cdot\vec{v}}{|\vec{u}|\,|\vec{v}|}$. Công thức y hệt áp dụng cho vectơ nhiều chiều. Hai câu có nghĩa gần nhau ("Never give up." — "Không bao giờ bỏ cuộc." và "Never say die!" — "Đừng bao giờ nản chí!") có vectơ nhúng gần cùng hướng, cosin gần 1; hai câu chẳng liên quan cho cosin gần 0. Ví dụ với $\vec{u} = (3; 4)$, $\vec{v} = (4; 3)$: $\cos = \frac{24}{5\cdot 5} = 0.96$, rất giống nhau.

Mã hóa văn bản cũng là một thành phần then chốt của các mô hình chuỗi-sang-chuỗi. Trong các tác vụ chuỗi-sang-chuỗi, ta có thể phát triển một bộ mã hóa Transformer riêng để mô hình hóa chuỗi phía nguồn trong một hệ thống bộ mã hóa–bộ giải mã (xem Hình 6.18). Chẳng hạn, ta có thể tiền huấn luyện một bộ mã hóa Transformer trên một kho ngữ liệu lớn gồm văn bản phía nguồn, rồi dùng nó làm bộ mã hóa trong một mô hình bộ mã hóa–bộ giải mã ở phía sau. Đáng chú ý là, mặc dù bộ mã hóa dựa trên kiến trúc Transformer, bộ giải mã không bị giới hạn trong kiến trúc này. Sự linh hoạt này cho phép ta đưa các bộ mã hóa Transformer đã tiền huấn luyện vào các kiến trúc chuỗi-sang-chuỗi lai, chẳng hạn các hệ thống kết hợp một bộ mã hóa Transformer với một bộ giải mã LSTM.

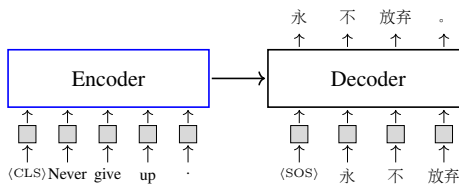
Ghi chú nhãn hình (người dịch): (a) *Classification*: phân loại; *Classifier (e.g., Softmax)*: bộ phân loại (ví dụ Softmax); *Pooling*: gộp; (b) *Similarity Computation (e.g., FFN)*: tính độ tương đồng (ví dụ FFN), *Similarity* $= 0.7$: độ tương đồng bằng 0.7; (c) *Sequence-to-Sequence Modeling*: mô hình hóa chuỗi-sang-chuỗi. Câu ví dụ "Never give up." (Không bao giờ bỏ cuộc.), "Never say die!" (Đừng bao giờ nản chí!), và bản dịch tiếng



(a) Classification



(b) Similarity Computation



(c) Sequence-to-Sequence Modeling

Hình 6.18: Minh họa việc tích hợp bộ mã hóa Transformer làm thành phần của các hệ thống khác nhau. Một cách phổ biến là đưa đầu ra của bộ mã hóa (sau khi gộp) vào một bộ phân loại để có một hệ thống phân loại chuỗi. Một cách khác để sử dụng bộ mã hóa Transformer là tính độ tương đồng giữa hai chuỗi. Ta dùng cùng một bộ mã hóa để biểu diễn hai chuỗi, rồi xây dựng một mạng nơ-ron trên hai biểu diễn đó để tạo ra điểm tương đồng giữa chúng. Ngoài ra, bộ mã hóa Transformer cũng có thể được dùng trong các hệ thống bộ mã hóa–bộ giải mã để mô hình hóa các bài toán chuỗi-sang-chuỗi.

Trung 永不放弃。(“không bao giờ từ bỏ”).

Trong các tình huống học có giám sát, việc huấn luyện một bộ mã hóa Transformer khá đơn giản. Ta có thể coi nó là một thành phần thông thường của mô hình đích và huấn luyện mô hình này trên dữ liệu có nhãn của tác vụ cụ thể. Tuy nhiên, cách này đòi hỏi bộ mã hóa phải được tối ưu trên từng tác vụ, và bộ mã hóa thu được không phải lúc nào cũng tổng quát hóa tốt sang các tác vụ khác, nhất là khi dữ liệu có nhãn khan hiếm trong hầu hết các trường hợp. Một cách tiếp cận phổ biến hơn là đặt việc huấn luyện bộ mã hóa văn bản thành một tác vụ độc lập, trong đó tín hiệu giám sát được lấy hoàn toàn từ văn bản thô. Điều này dẫn các nhà nghiên cứu tới việc phát triển các bộ mã hóa Transformer tự giám sát (*self-supervised*), như BERT, vốn tận dụng lượng lớn văn bản không nhãn, và các bộ mã hóa này được thấy là tổng quát hóa tốt trên nhiều tác vụ phía sau. Thảo luận thêm về các bộ mã hóa Transformer tiền huấn luyện có thể tìm thấy ở Chương 7.

6.5.3 Dịch tiếng nói

Như đã minh họa ở Mục 6.1, mô hình Transformer bộ mã hóa–bộ giải mã chuẩn được đề xuất để mô hình hóa các bài toán chuỗi-sang-chuỗi. Ở đây ta xét bài toán dịch tiếng nói trong một ngôn ngữ thành văn bản trong một ngôn ngữ khác — một bài toán theo truyền thống được giải quyết bằng cả kỹ thuật nhận dạng tiếng nói (*speech recognition*) tự động (ASR) lẫn dịch máy. Thay vì nối tiếp một hệ thống nhận dạng tiếng nói tự động với một hệ thống dịch máy, ta có thể dùng mô hình Transformer để xây dựng một hệ thống dịch tiếng nói sang văn bản (S2T) đầu-cuối (end-to-end), dịch trực tiếp tiếng nói đầu vào thành văn bản đầu ra.

Để đơn giản hóa thảo luận, ta giả sử đầu vào của một hệ thống dịch S2T là một chuỗi các vectơ đặc trưng âm học phía nguồn, ký hiệu là $\mathbf{a}_1 \cdots \mathbf{a}_m$, và đầu ra của hệ thống là một chuỗi token phía đích, ký hiệu là $y_1 \cdots y_n$[29]. Ánh xạ $\mathbf{a}_1 \cdots \mathbf{a}_m$ thành $y_1 \cdots y_n$ là một bài toán chuỗi-sang-chuỗi. Vì vậy, có thể dễ dàng mô hình hóa bài toán bằng một mô hình Transformer bộ

[29]Để có chuỗi đầu vào cho hệ thống, ta cần rời rạc hóa tiếng nói liên tục thành các tín hiệu được biểu diễn bằng vectơ đặc trưng. Quá trình này thường không đơn giản, đòi hỏi hoặc một bộ trích xuất đặc trưng dựa trên nhiều phép xử lý tín hiệu, hoặc một mạng nơ-ron học ánh xạ đặc trưng theo cách đầu-cuối. Nhưng chúng tôi sẽ không đi sâu vào chi tiết các phương pháp này mà đơn giản coi bộ trích xuất đặc trưng đầu vào là một hệ thống phía trước.

mã hóa–bộ giải mã, và việc huấn luyện cũng như suy luận của mô hình này là chuẩn, như trong dịch máy nơ-ron.

Tuy nhiên, trong dịch S2T, ta phải xử lý các ánh xạ chuỗi vượt qua cả ranh giới phương thức (âm thanh – văn bản) lẫn ngôn ngữ. Điều này đặt ra những thách thức mới so với các bài toán dịch máy thông thường và ảnh hưởng đến thiết kế của các mô hình dịch S2T. Đã có một số cải tiến cho Transformer để nó phù hợp hơn với các tác vụ dịch S2T. Một số cải tiến tập trung vào thiết kế các tầng Transformer [Di Gangi và cộng sự, 2019]. Ví dụ, trong hệ thống của Gulati và cộng sự [2020], một tầng con CNN và vectơ nhúng vị trí tương đối được tích hợp vào mỗi tầng Transformer, giúp mô hình nắm bắt hiệu quả cả đặc trưng cục bộ lẫn toàn cục.

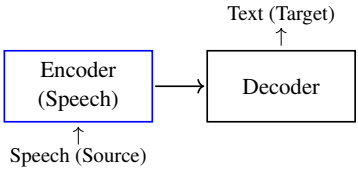
Một hướng nghiên cứu khác về dịch S2T tập trung vào cải tiến kiến trúc bộ mã hóa–bộ giải mã. Việc này bao gồm sửa đổi bộ mã hóa, bộ giải mã, hoặc cả hai. Để minh họa, Hình 6.19 trình bày kiến trúc của ba mô hình dịch S2T. Tất cả đều dựa trên Transformer nhưng có kiến trúc bộ mã hóa khác nhau. Như thấy trong hình, kiến trúc bộ mã hóa–bộ giải mã chuẩn có một bộ mã hóa Transformer để đọc đầu vào phía nguồn $\mathbf{a}_1, \cdots, \mathbf{a}_m$ và một bộ giải mã Transformer để tạo ra đầu ra phía đích $y_1, \cdots, y_n$. Ngược lại, mô hình bộ mã hóa tách rời phân rã bộ mã hóa thành hai bộ mã hóa xếp chồng — một cho mô hình hóa âm học (gọi là **bộ mã hóa tiếng nói** (*speech encoder*)) và một cho mô hình hóa văn bản (gọi là **bộ mã hóa văn bản** (*text encoder*)) [Liu và cộng sự, 2020d; Xu và cộng sự, 2021a]. Thiết kế này phản ánh một cấu trúc phân cấp trong mô hình hóa, trong đó các biểu diễn ở những mức khác nhau của mạng phụ trách những khía cạnh khác nhau của bài toán. Ví dụ, bộ mã hóa tiếng nói mô hình hóa các đặc trưng mức thấp khi ánh xạ các vectơ nhúng âm học thành các đơn vị ngôn ngữ lớn hơn, còn bộ mã hóa văn bản mô hình hóa các đặc trưng ngữ nghĩa hoặc cú pháp khi biểu diễn toàn bộ chuỗi đầu vào. Một ưu điểm của việc tách riêng bộ mã hóa văn bản là quá trình mã hóa tuân theo tri thức tiên nghiệm của ta: trước hết cần phiên âm tiếng nói đầu vào thành văn bản, rồi dịch bản phiên âm sang ngôn ngữ đích. Do đó, ta có thể huấn luyện bộ mã hóa tiếng nói tương tự như cách huấn luyện một hệ thống ASR. Điều này cho phép ta tiền huấn luyện bộ mã hóa tiếng nói và bộ mã hóa văn bản trên dữ liệu không nhãn, rồi sau đó đưa các bộ mã hóa đã tiền huấn luyện vào hệ thống dịch S2T.

Một kiến trúc mã hóa khác là kiến trúc hai luồng, như trong

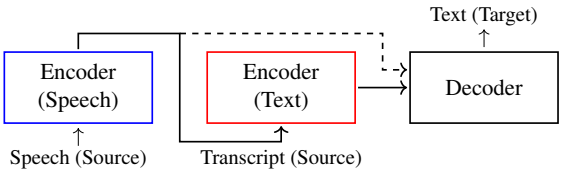
Hình 6.19(c). Giống kiến trúc bộ mã hóa tách rời, kiến trúc này có một bộ mã hóa tiếng nói và một bộ mã hóa văn bản, nhưng hai bộ mã hóa hoạt động song song thay vì nối tiếp [Ye và cộng sự, 2021]. Bộ mã hóa tiếng nói nhận đặc trưng âm học làm đầu vào, còn bộ mã hóa văn bản nhận các token (hoặc vectơ nhúng của chúng) làm đầu vào. Một bộ mã hóa thứ ba, gọi là **bộ mã hóa dùng chung** (*shared encoder*), tích hợp đầu ra của cả bộ mã hóa tiếng nói lẫn bộ mã hóa văn bản, hợp nhất các biểu diễn từ hai phương thức. Kiến trúc hai luồng này linh hoạt vì nó cung cấp nhiều cách để huấn luyện mô hình dịch S2T. Một cách phổ biến là huấn luyện riêng từng nhánh. Ví dụ, nếu ta che bộ mã hóa tiếng nói, mô hình sẽ trở thành một mô hình dịch máy có thể huấn luyện bằng văn bản song ngữ. Ngược lại, nếu che bộ mã hóa văn bản, ta có thể huấn luyện mô hình như một mô hình dịch S2T chuẩn. Khi suy luận, có thể bỏ bộ mã hóa văn bản, và đầu vào tiếng nói được mô hình hóa bằng bộ mã hóa tiếng nói và bộ mã hóa dùng chung.

Ghi chú nhãn hình (người dịch): Encoder (Speech)/(Text): bộ mã hóa (tiếng nói)/(văn bản); *Shared Encoder*: bộ mã hóa dùng chung; *Decoder*: bộ giải mã; *Speech (Source)*: tiếng nói (nguồn); *Transcript (Source)*: bản phiên âm (nguồn); *Text (Target)*: văn bản (đích). (a) Một bộ mã hóa + một bộ giải mã; (b) Bộ mã hóa tách rời + một bộ giải mã; (c) Bộ mã hóa hai luồng + một bộ giải mã.

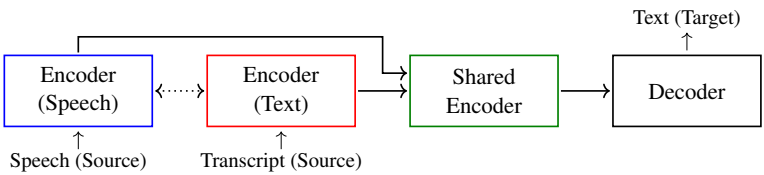
Trong học sâu, huấn luyện thường gắn liền với thiết kế kiến trúc. Vì ta xử lý dữ liệu ở hai phương thức và hai ngôn ngữ, ta có thể xây dựng nhiều tín hiệu giám sát cho việc học đa nhiệm của các mô hình dịch S2T. Một phương pháp được dùng rộng rãi là đưa một hàm mất mát liên quan đến ASR vào việc huấn luyện bộ mã hóa tiếng nói. Ví dụ, trong mô hình bộ mã hóa tách rời, có thể xây dựng một bộ phân loại dựa trên đầu ra của bộ mã hóa tiếng nói. Bằng cách cực tiểu hóa hàm mất mát phân loại thời gian kết nối (*connectionist temporal classification*) (CTC) cho bộ phân loại này, bộ mã hóa tiếng nói có thể được tối ưu theo cách tương tự ASR. Nói chung, huấn luyện mô hình dịch S2T là khó vì dữ liệu tiếng nói–văn bản được căn chỉnh rất khan hiếm. Các giải pháp điển hình cho thách thức này gồm tăng cường dữ liệu, tiền huấn luyện, chưng cất tri thức với dịch máy, v.v. Tuy nhiên, thảo luận sâu về các phương pháp này vượt ra ngoài phạm vi của mục về Transformer này. Bạn đọc quan tâm có thể tham khảo một bài tổng quan gần đây về dịch tiếng nói để biết thêm chi tiết [Xu và



(a) Single Encoder + Single Decoder



(b) Decoupled Encoder + Single Decoder



(c) Two-stream Encoder + Single Decoder

Hình 6.19: Sơ đồ minh họa các mô hình dịch tiếng nói sang văn bản dựa trên Transformer. Ngoài kiến trúc bộ mã hóa–bộ giải mã chuẩn, ta có thể mô hình hóa tường minh thông tin âm học và thông tin văn bản (ngữ nghĩa) bằng hai bộ mã hóa riêng, gọi là bộ mã hóa tiếng nói và bộ mã hóa văn bản. Trong kiến trúc bộ mã hóa tách rời, hai bộ mã hóa được xếp chồng, tức là việc mã hóa văn bản là quá trình diễn ra sau mã hóa tiếng nói. Trong kiến trúc bộ mã hóa hai luồng, hai bộ mã hóa hoạt động song song, và đầu ra của chúng được hợp nhất bằng một bộ mã hóa bổ sung, gọi là bộ mã hóa dùng chung. Đường chấm thể hiện khả năng tương tác giữa hai bộ mã hóa. Ví dụ, ta có thể định nghĩa một hàm mất mát để cực tiểu hóa sự khác biệt giữa đầu ra của chúng, từ đó hướng mô hình tới các biểu diễn được căn chỉnh tốt hơn.

cộng sự, 2023a].

6.5.4 Mô hình thị giác

Mặc dù Transformer lần đầu được dùng trong NLP, việc ứng dụng nó sang các lĩnh vực khác đã trở thành một chủ đề nghiên cứu nổi bật. Ví dụ, trong thị giác máy tính, có một xu hướng đáng chú ý là chuyển từ CNN sang Transformer làm mô hình xương sống. Trong tiểu mục này, ta xét **Vision Transformer** (*Vision Transformer*) (**ViT**) — một ứng dụng thú vị của Transformer vào phân loại ảnh [Dosovitskiy và cộng sự, 2021]. Vision Transformer là một mô hình mang tính cột mốc, mở đường cho các mô hình thị giác thuần Transformer. Ở đây, ta xét cấu trúc cơ bản của Vision Transformer để giữ mục này tập trung và mạch lạc, dù đã có rất nhiều tài liệu về Vision Transformer và các biến thể của nó. Các thảo luận chi tiết hơn về mô hình thị giác có thể tìm thấy trong các bài tổng quan gần đây [Han và cộng sự, 2022; Liu và cộng sự, 2023e].

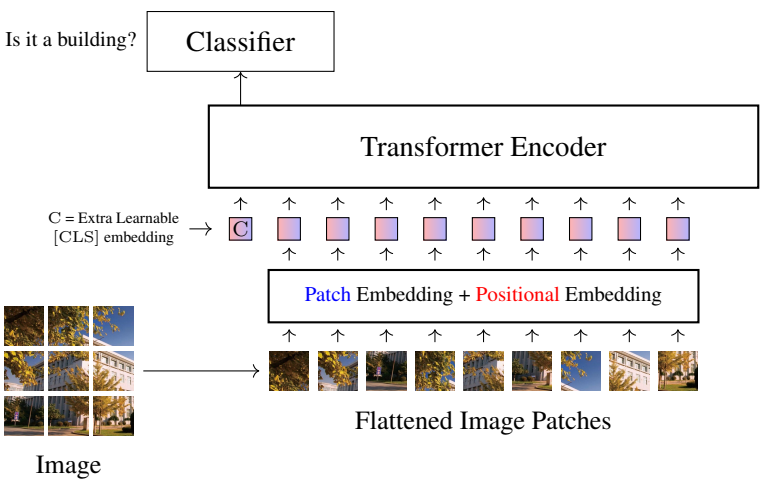
Ý tưởng cốt lõi của Vision Transformer là biến một bức ảnh thành một chuỗi các token thị giác, rồi đưa chuỗi này vào một bộ mã hóa Transformer để sinh ra biểu diễn của ảnh. Bộ mã hóa Transformer là chuẩn, nên chúng tôi không bàn lại ở đây, vì đã giới thiệu Transformer trong chương này. Ánh xạ một ảnh 2 chiều thành một chuỗi token đòi hỏi thêm một số xử lý. Giả sử ta có một ảnh được biểu diễn dưới dạng một bản đồ đặc trưng $H \times W \times C$, trong đó H là chiều cao, W là chiều rộng, và C là số kênh. Bước đầu tiên là chia ảnh này thành một tập các **mảnh** (*patch*). Giả sử mọi mảnh đều là hình vuông cạnh P. Khi đó, các mảnh thu được có thể được biểu diễn bằng các bản đồ đặc trưng có hình dạng $P \times P \times C$. Bằng cách sắp xếp các mảnh này theo một thứ tự cụ thể, ta thu được một chuỗi gồm $\frac{HW}{P^2}$ mảnh, mỗi mảnh được coi như một "token".

Chú giải của người dịch: Ví dụ đếm số mảnh

Một ảnh màu 224×224 điểm ảnh có $C = 3$ kênh (đỏ, lục, lam). Chia thành các mảnh 16×16 thì mỗi chiều có $224/16 = 14$ mảnh, tổng cộng $14 \times 14 = 196$ mảnh, đúng bằng $\frac{HW}{P^2} = \frac{224 \cdot 224}{256}$. "Làm phẳng" một mảnh nghĩa là xếp $16 \times 16 \times 3 = 768$ con số của nó thành một hàng dài, rồi nhân với một ma trận để được vectơ nhúng d chiều. Như vậy bức ảnh trở thành một "câu" gồm 196 "từ".

Với chuỗi mảnh này, các bước tiếp theo khá đơn giản. Với mảnh ở mỗi vị trí, ta thu được một vectơ nhúng d chiều thông qua một phép biến đổi tuyến tính của mảnh đã được làm phẳng. Đầu vào của bộ mã hóa Transformer là một chuỗi vectơ d chiều, mỗi vectơ là tổng của vectơ nhúng mảnh tương ứng và vectơ nhúng vị trí. Hình 6.20 minh họa các bước chia mảnh và nhúng trong Vision Transformer.

Khi đã có một chuỗi vectơ biểu diễn ảnh, ta có thể dùng bộ mã hóa Transformer để mã hóa chuỗi. Quá trình mã hóa hoàn toàn giống như trong mã hóa văn bản, như đã bàn ở Mục 6.5.2. Với các bài toán phân loại, ta chỉ cần một biểu diễn duy nhất của đầu vào. Cách tiện lợi là lấy đầu ra của bộ mã hóa tại vị trí 0 (ký hiệu $\mathbf{h}_0^L$) và đưa nó vào một bộ phân loại. Vì token đầu tiên ([CLS]) đóng vai trò một token toàn cục đặc biệt được mọi token khác chú ý tới, $\mathbf{h}_0^L$ có thể được xem là một biểu diễn tổng hợp của toàn bộ chuỗi.



Hình 6.20: Minh họa Vision Transformer cho phân loại ảnh [Dosovitskiy và cộng sự, 2021]. Có ba bước. Ở bước thứ nhất, ảnh đầu vào được chia thành các mảnh, sau đó các mảnh được làm phẳng và ánh xạ thành vectơ nhúng. Ở bước thứ hai, một bộ mã hóa Transformer được dùng để xử lý chuỗi vectơ nhúng, biểu diễn ảnh dưới dạng một vectơ giá trị thực (ví dụ đầu ra của bộ mã hóa tại vị trí đầu tiên). Ở bước cuối, một bộ phân loại được xây dựng trên biểu diễn ảnh này.

Ghi chú nhãn hình (người dịch): Image: ảnh; *Flattened Image Patches*:

các mảnh ảnh đã làm phẳng; *Patch Embedding + Positional Embedding*: nhúng mảnh + nhúng vị trí; *C = Extra Learnable [CLS] embedding*: C là vectơ nhúng [CLS] bổ sung, học được; *Classifier*: bộ phân loại; *Is it a building?*: Đây có phải tòa nhà không?

Một cách chuẩn để huấn luyện Vision Transformer là cực tiểu hóa một hàm mất mát theo tác vụ trên dữ liệu có nhãn, chẳng hạn ImageNet. Gần đây hơn, lấy cảm hứng từ học tự giám sát trong các mô hình kiểu BERT, đã có những nỗ lực thành công trong việc huấn luyện các bộ mã hóa ảnh dựa trên Transformer trên dữ liệu không nhãn quy mô lớn [Caron và cộng sự, 2021; Bao và cộng sự, 2021; He và cộng sự, 2022]. Lưu ý rằng một trong những đóng góp quan trọng nhất của Vision Transformer là nó thống nhất các mô hình biểu diễn cho các phương thức khác nhau. Điều này gợi ý rằng nếu một đối tượng, dù là ảnh hay văn bản, được biểu diễn dưới dạng một chuỗi vectơ nhúng, thì nó có thể được mô hình hóa dễ dàng bằng kiến trúc Transformer.

6.5.5 Mô hình đa phương thức

Thảo luận ở trên về Vision Transformer mở ra khả năng thống nhất các biểu diễn từ nhiều phương thức bằng cùng một kiến trúc Transformer. Thực tế, nhiều hệ thống đa phương thức gần đây lấy cảm hứng rất lớn từ Transformer [Xu và cộng sự, 2023c]. Các hệ thống như vậy chuyển các đối tượng thuộc những phương thức khác nhau thành các chuỗi vectơ và đưa các vectơ này vào một mô hình Transformer duy nhất. Đầu ra là một biểu diễn hợp nhất của mọi đầu vào, có thể được dùng trong các hệ thống phía sau.

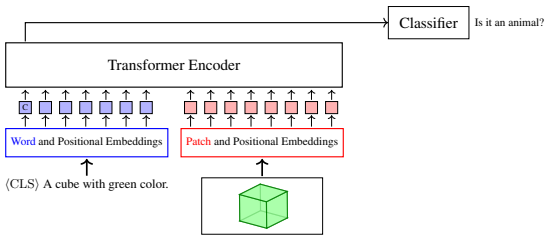
Để lấy ví dụ đơn giản, xét tác vụ mã hóa một cặp gồm một văn bản và bức ảnh tương ứng. Trước hết, ta biểu diễn cả văn bản và ảnh dưới dạng các chuỗi vectơ nhúng có cùng số chiều. Đây là một bước thông thường trong mô hình hóa chuỗi mà ta đã gặp nhiều lần. Ta có thể làm điều này bằng một mô hình nhúng đơn giản (ví dụ mô hình nhúng từ hoặc nhúng mảnh) hoặc một mô hình đã tiền huấn luyện (ví dụ một mô hình thị giác). Sau đó, hai chuỗi này được nối thành một chuỗi dài gồm cả vectơ nhúng văn bản lẫn vectơ nhúng thị giác. Bước tiếp theo là chuẩn: một bộ mã hóa Transformer nhận chuỗi vectơ nhúng đã nối làm đầu vào và tạo ra biểu diễn của văn bản và ảnh ở đầu ra. Lưu ý rằng nối chuỗi văn bản và chuỗi

thị giác là một trong những phương pháp đơn giản nhất cho mô hình hóa thị giác–văn bản. Có một số cách khác để hợp nhất thông tin từ các phương thức khác nhau. Ví dụ, ta có thể đưa các biểu diễn thị giác vào các tầng chú ý của một bộ mã hóa hoặc bộ giải mã văn bản [Li và cộng sự, 2022e; Alayrac và cộng sự, 2022].

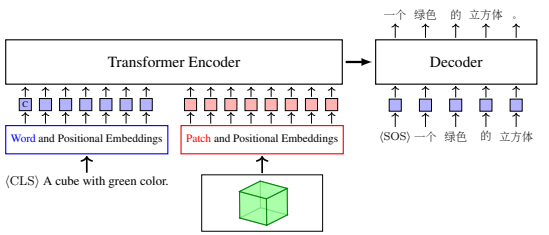
Bộ mã hóa đa phương thức ở trên có thể được dùng trong cả hệ thống chỉ có bộ mã hóa lẫn hệ thống bộ mã hóa–bộ giải mã. Với hệ thống chỉ có bộ mã hóa, xét ví dụ: cho một bức ảnh và mô tả của nó, ta dự đoán lớp của ảnh bằng một bộ phân loại xây dựng trên bộ mã hóa [Kim và cộng sự, 2021]. Với hệ thống bộ mã hóa–bộ giải mã, ta ghép bộ mã hóa với một bộ giải mã, như trong mô hình hóa chuỗi-sang-chuỗi [Cho và cộng sự, 2021]. Ví dụ, ta có thể dùng một bộ giải mã Transformer để sinh văn bản dựa trên đầu ra của bộ mã hóa. Một ứng dụng phổ biến của kiến trúc này là **hỏi đáp thị giác** (*visual question answering*) (**VQA**), trong đó hệ thống được cho một bức ảnh và một câu hỏi về bức ảnh, và có nhiệm vụ sinh ra câu trả lời [Antol và cộng sự, 2015]. Kiến trúc của các mô hình này được minh họa trong Hình 6.21 (a–b).

Các mô hình ngôn ngữ lớn cũng đã được mở rộng để xử lý cả văn bản và các phương thức khác như ảnh, video và âm thanh, dẫn tới những tiến bộ mới trong xử lý đa phương thức [Liu và cộng sự, 2023a; Yin và cộng sự, 2023]. Bằng cách biểu diễn mọi đầu vào dưới dạng một chuỗi vectơ nhúng token, bài toán trở nên đơn giản: ta dự đoán token tiếp theo khi biết ngữ cảnh của nó. Điều này có thể thực hiện bằng các hệ thống chỉ có bộ giải mã, như trong Hình 6.21 (c).

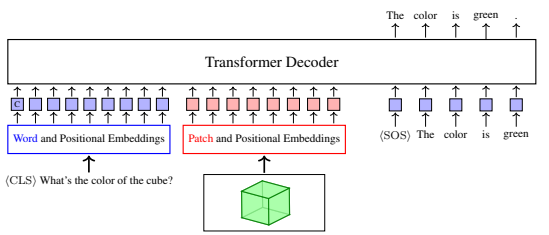
Ghi chú nhãn hình (người dịch): Word and Positional Embeddings: nhúng từ và nhúng vị trí; *Patch and Positional Embeddings*: nhúng mảnh và nhúng vị trí. (a) *Multi-modal Encoder + Classifier*: bộ mã hóa đa phương thức + bộ phân loại; văn bản "A cube with green color." (Một khối lập phương màu xanh lục.), câu hỏi "Is it an animal?" (Đây có phải con vật không?). (b) *Multi-modal Encoder + Text Decoder (Translation)*: bộ mã hóa đa phương thức + bộ giải mã văn bản (dịch), đầu ra tiếng Trung 一个绿色的立方体。("một khối lập phương màu xanh lục"). (c) *Multi-modal Decoder (Language Modeling)*: bộ giải mã đa phương thức (mô hình hóa ngôn ngữ); câu hỏi "What's the color of the cube?" (Khối lập phương màu gì?), câu trả lời "The color is green." (Màu xanh lục.).



(a) Multi-modal Encoder + Classifier



(b) Multi-modal Encoder + Text Decoder (Translation)

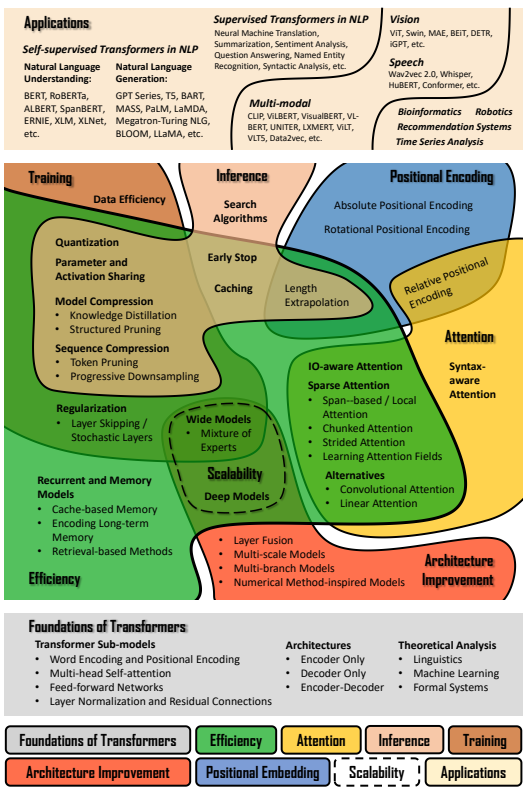


(c) Multi-modal Decoder (Language Modeling)

Hình 6.21: Minh họa các mô hình thị giác–văn bản. Các ô màu xanh lam biểu diễn vectơ nhúng từ + vị trí, còn các ô màu đỏ biểu diễn vectơ nhúng mảnh ảnh + vị trí.

6.6 Tóm tắt

Mô hình Transformer đã được áp dụng rộng rãi kể từ khi kiến trúc này lần đầu được Vaswani và cộng sự [2017] đề xuất. Sự phổ biến này đã thúc đẩy sự phát triển của rất nhiều thuật toán, hệ thống và khái niệm. Một thảo luận thấu đáo về Transformer đòi hỏi bao quát rất rộng, nên không thể đề cập mọi chủ đề hay đưa ra một danh sách tài liệu tham khảo đầy đủ. Hình 6.22 đưa ra một cái nhìn tổng quan ở mức cao về bức tranh Transformer. Lưu ý rằng các mô hình và kỹ thuật liên quan này có thể được phân loại theo nhiều cách, và chúng tôi trình bày một góc nhìn như vậy. Tóm lại, chúng tôi nhấn mạnh các điểm sau:



Hình 6.22: Tổng quan về Transformer.

Ghi chú nhãn hình (người dịch): Các vùng màu ứng với các nhóm chủ đề: *Foundations of Transformers* (nền tảng của Transformer), *Efficiency* (hiệu quả), *Attention* (cơ chế chú ý), *Inference* (suy luận), *Training* (huấn luyện), *Architecture Improvement* (cải tiến kiến trúc), *Positional Embedding* (nhúng vị trí), *Scalability* (khả năng mở rộng quy mô), *Applications* (ứng dụng). Bên trong là các kỹ thuật đã bàn trong chương, ví dụ *Quantization* (lượng tử hóa), *Knowledge Distillation* (chưng cất tri thức), *Structured Pruning* (cắt tỉa có cấu trúc), *Sparse Attention* (chú ý thưa), *Linear Attention* (chú ý tuyến tính), *Mixture of Experts* (hỗn hợp chuyên gia), *Early Stop* (dừng sớm), *Caching* (bộ nhớ đệm), *Length Extrapolation* (ngoại suy độ dài), cùng tên các mô hình tiêu biểu.

- **Nền tảng của Transformer**. Mặc dù tác động của Transformer mang tính cách mạng, chúng không phải là mô hình hoàn toàn "mới". Từ góc độ học sâu, Transformer được ghép từ những khối xây dựng thông dụng, bao gồm nhúng từ (*word embedding*) và nhúng vị trí [Bengio và cộng sự, 2003a; Mikolov và cộng sự, 2013c; Gehring và cộng sự, 2017b], cơ chế chú ý [Bahdanau và cộng sự, 2014; Luong và cộng sự, 2015], kết nối tắt (*residual connection*) [He và cộng sự, 2016b], chuẩn hóa tầng (*layer normalization*) [Ba và cộng sự, 2016], v.v. Nhiều thành phần trong số này đã xuất hiện ở các hệ thống trước đó; ví dụ, những ý tưởng tương tự chú ý QKV có thể thấy trong mạng bộ nhớ [Sukhbaatar và cộng sự, 2015] và mạng chú ý phân cấp [Yang và cộng sự, 2016]. Transformer mang đến một cách tiếp cận mới để tích hợp các thành phần này, tạo ra một kiến trúc độc đáo. Ví dụ, trong Transformer, sự kết hợp giữa chú ý đa đầu (*multi-head attention*) và chú ý QKV dạng tích vô hướng, cùng với việc đưa vào chuẩn hóa tầng và kết nối tắt, đã tạo ra một khối mạng nơ-ron đặc trưng, cụ thể là tầng con tự chú ý. Thiết kế này từ đó đã trở thành tiêu chuẩn trên thực tế trong nhiều hệ thống mô hình hóa chuỗi về sau.

- **Mô hình chú ý**. Thành công của Transformer trong các tác vụ NLP phần lớn được quy cho việc sử dụng tự chú ý đa đầu cho mô hình hóa chuỗi. Điều này dẫn tới sự quan tâm bùng nổ đối với việc cải tiến cơ chế chú ý trong Transformer. Dù không thể trình bày chi tiết mọi mô hình chú ý, có một số hướng nghiên cứu đáng chú ý. Một hướng nổi bật là sửa đổi dạng của chú ý QKV và chú ý đa

đầu để cải thiện hiệu năng. Phạm vi của hướng này rất rộng, vì có vô số khía cạnh cần xem xét khi cải tiến Transformer [Lin và cộng sự, 2022a]. Ví dụ, ta có thể thêm các thành phần mới vào các tầng con tự chú ý để thích ứng chúng với các tác vụ cụ thể, tạo ra nhiều biến thể Transformer khác nhau. Hướng thứ hai là đưa tri thức tiên nghiệm vào thiết kế mô hình chú ý. Điều này hợp lý, vì phần lớn trọng tâm của NLP truyền thống là dùng hiểu biết ngôn ngữ học để định hướng thiết kế hệ thống, và nói chung ta muốn các hệ thống NLP có thể giải thích được về mặt ngôn ngữ học. Ví dụ, nhiều hệ thống dựa trên Transformer nhận cây phân tích cú pháp làm đầu vào dưới nhiều dạng và tận dụng cú pháp trong mô hình hóa chuỗi. Hướng thứ ba là phát triển các mô hình chú ý hiệu quả [Tay và cộng sự, 2020b]. Tự chú ý từ lâu đã bị chỉ trích vì độ phức tạp thời gian bậc hai và việc mỗi token mới phụ thuộc vào mọi token trước đó. Để khắc phục, nhiều nhà nghiên cứu tập trung đơn giản hóa cấu trúc của tự chú ý, hoặc xấp xỉ nó bằng các mô hình thưa hay mô hình hồi quy. Mối quan tâm về hiệu quả này cũng thúc đẩy việc phát triển các phương án thay thế cho tự chú ý, chẳng hạn các mô hình chú ý có độ phức tạp thời gian tuyến tính. Bên cạnh việc khám phá các mô hình chú ý mạnh hơn và hiệu quả hơn, một điều tự nhiên là xem xét những tri thức nào được các mô hình đó học được. Thú vị là, các nhà nghiên cứu đã phát hiện rằng cấu trúc nền tảng của ngôn ngữ có thể được học bởi các mô hình tự chú ý đa đầu, dù các mô hình này không được huấn luyện để biểu diễn tri thức đó [Manning và cộng sự, 2020].

- **Nhúng từ và nhúng vị trí**. Transformer biểu diễn mỗi từ (hay token) đầu vào bằng sự kết hợp của vectơ nhúng từ và vectơ nhúng vị trí. Việc học các vectơ nhúng từ này không phải là điều riêng có của Transformer; ta có thể dùng các vectơ nhúng đã tiền huấn luyện, như Word2Vec hay GloVe, hoặc coi chúng là các tham số học được trong mô hình. Một vấn đề liên quan mật thiết là tách token cho chuỗi đầu vào. Nói chung, cách tách token quyết định số token thu được và ảnh hưởng đến độ khó của việc học các vectơ nhúng tương ứng. Do đó, chọn một phương pháp tách token phù hợp là rất quan trọng với nhiều ứng dụng. Hơn nữa, nhúng vị trí đóng vai trò thiết yếu trong Transformer, vì cơ chế chú ý bên dưới về thiết

kế là không nhạy với thứ tự [Dufter và cộng sự, 2022]. Mặc dù mã hóa vị trí là một chủ đề nghiên cứu chung, nhiều nỗ lực đã được dành để điều chỉnh nó cho Transformer, dẫn tới nhiều sửa đổi kiến trúc khác nhau [Shaw và cộng sự, 2018; Huang và cộng sự, 2018]. Ngoài ra, các nghiên cứu cho thấy với những chuỗi dài hơn đáng kể so với chuỗi gặp khi huấn luyện, có thể đạt khả năng ngoại suy tốt hơn bằng cách thay vectơ nhúng hình sin bằng vectơ nhúng vị trí xoay (RoPE) hoặc bằng cách co giãn trọng số chú ý bằng các hệ số vô hướng theo vị trí [Raffel và cộng sự, 2020; Su và cộng sự, 2021; Press và cộng sự, 2021].

- **Huấn luyện và mở rộng quy mô mô hình**. Trong kỷ nguyên học sâu, các hệ thống hiệu năng cao thường được xây dựng bằng mạng nơ-ron quy mô lớn. Một cách trực tiếp để tăng năng lực của Transformer là xếp chồng thêm tầng hoặc mở rộng kích thước biểu diễn ẩn. Thực vậy, các mô hình Transformer sâu và rộng luôn vượt trội các phiên bản nhỏ hơn. Tuy nhiên, huấn luyện các mô hình cực lớn đặt ra những thách thức đáng kể, nhất là khi áp dụng hạ gradient (*gradient descent*) trên các tập dữ liệu khổng lồ, đòi hỏi tài nguyên tính toán rất lớn. Các giải pháp kỹ thuật bao gồm phân tán việc huấn luyện trên các cụm máy tính [Lepikhin và cộng sự, 2021; Chowdhery và cộng sự, 2022]. Mặc dù huấn luyện phân tán là một mô thức chung không giới hạn ở Transformer, nó ảnh hưởng mạnh tới thiết kế kiến trúc; chẳng hạn, các mô hình hỗn hợp chuyên gia (MoE) thưa có thể tạo điều kiện huấn luyện với lượng tham số khổng lồ và là nền tảng cho nhiều hệ thống quy mô lớn. Việc mở rộng quy mô cũng cho phép nghiên cứu các định luật co giãn — định lượng cách hiệu năng thay đổi theo kích thước mô hình, khối lượng dữ liệu và ngân sách tính toán [Hestness và cộng sự, 2017; Kaplan và cộng sự, 2020]. Việc mở rộng quy mô này đôi khi đi kèm hiện tượng "trỗi sinh (*emergence*)" [Wei và cộng sự, 2022b], trong đó mô hình bộc lộ những năng lực mới không lường trước được. Trong các nghiên cứu gần đây, việc có được những năng lực trỗi sinh như vậy được xem là dấu hiệu đặc trưng của các mô hình ngôn ngữ lớn mạnh.

- **Mô hình hiệu quả**. Có những mục tiêu hiệu quả khác nhau. Một hệ thống có thể ưu tiên hiệu quả bộ nhớ với các bài toán bị giới hạn

bởi bộ nhớ, hoặc độ trễ thấp khi tốc độ suy luận là then chốt. Cân bằng các yêu cầu này dẫn đến nhiều chiến lược tối ưu khác nhau. Trong Transformer, nhiều tối ưu hóa liên quan đến việc sửa đổi cơ chế chú ý; chẳng hạn, một số biến thể giảm lượng bộ nhớ chiếm dụng khi xử lý chuỗi dài [Tay và cộng sự, 2020b], trong khi một số khác giảm thiểu tính toán để giảm độ trễ. Hơn nữa, Transformer có thể hưởng lợi từ các tối ưu hóa không phụ thuộc kiến trúc vốn điển hình cho mạng nơ-ron, như tính toán có điều kiện, chưng cất tri thức, cắt tỉa có cấu trúc và nén chuỗi. Vấn đề hiệu quả cũng có thể được giải quyết từ góc độ kiến trúc máy tính [Kim và cộng sự, 2023]. Ví dụ, trong các tác vụ chuỗi-sang-chuỗi, quá trình mã hóa thường nặng về tính toán, còn quá trình giải mã tự hồi quy thường nặng về vào/ra (IO). Do đó, ta có thể dùng các phương pháp tối ưu khác nhau cho các thành phần khác nhau của Transformer.

- **Suy luận**. Bài toán suy luận thường được bàn đến trong sinh chuỗi. Trong NLP, ta thường cần tìm giả thuyết "tốt nhất" trong một không gian gồm các chuỗi hàng trăm, thậm chí hàng nghìn token trên một bộ từ vựng. Là một bài toán tìm kiếm kinh điển trong trí tuệ nhân tạo, có thể áp dụng nhiều thuật toán, bao gồm tìm kiếm theo chiều rộng, theo chiều sâu và tìm kiếm A*. Trong nhiều ứng dụng NLP thực tế, hiệu quả của hệ thống tìm kiếm là một yếu tố quan trọng. Vì thế, cần đến các thuật toán tìm kiếm được tối ưu hóa. Phần lớn các thuật toán này đã được khám phá trong dịch máy và ASR, và có thể áp dụng trực tiếp cho các mô hình sinh văn bản nơ-ron như Transformer. Ngoài tìm kiếm truyền thống, các tối ưu hóa gần đây như giải mã suy đoán (*speculative decoding*) được thiết kế riêng cho kiến trúc Transformer [Leviathan và cộng sự, 2023]. Hơn nữa, các mô hình chú ý hiệu quả đã nhắc tới trước đây hiện được triển khai rộng rãi để tăng tốc các mô hình ngôn ngữ lớn và hệ thống dịch máy nơ-ron [Heafield và cộng sự, 2021; Dao và cộng sự, 2023].

- **Ứng dụng**. Các ứng dụng của Transformer bao trùm nhiều bài toán NLP. Trong quá trình phát triển, Transformer thoạt đầu được dùng để xây dựng các mô hình có giám sát thực hiện những tác vụ cụ thể. Về sau, thành công lớn hơn đạt được nhờ dùng chúng làm mạng xương sống cho việc học tự giám sát quy mô lớn các mô hình nền tảng (*foundation model*) [Bommasani và cộng sự, 2021]. Sự chuyển

dịch này đã thay đổi rõ rệt mô thức NLP: một mô hình trước hết được tiền huấn luyện trên lượng văn bản khổng lồ để thu được tri thức ngôn ngữ tổng quát, rồi được điều chỉnh cho các tác vụ phía sau bằng các phương pháp hiệu quả như tinh chỉnh (*fine-tuning*) hoặc dùng prompt. Ngoài NLP, ta đã chứng kiến sự bùng nổ ứng dụng Transformer trong các lĩnh vực như thị giác máy tính, xử lý tiếng nói và tin sinh học. Ý tưởng cốt lõi là biểu diễn các loại dữ liệu đa dạng dưới dạng chuỗi token, mở rộng Transformer thành các mô hình biểu diễn đa năng có khả năng xử lý dữ liệu đa phương thức.

- **Mô hình ngôn ngữ lớn như các mô hình nền tảng**. Transformer là nền tảng của các mô hình ngôn ngữ lớn (*large language model*) gần đây, như dòng GPT, vốn cho thấy những đột phá đáng ngạc nhiên trong NLP [Bubeck và cộng sự, 2023; Yang và cộng sự, 2023a]. Phần lớn nghiên cứu về mô hình ngôn ngữ lớn ít nhiều liên quan đến Transformer. Như đã bàn ở Mục 6.5.1, huấn luyện các mô hình ngôn ngữ này thường đồng nghĩa với huấn luyện các bộ giải mã Transformer quy mô lớn, và những cải tiến kiến trúc cho bộ giải mã trực tiếp mang lại lợi ích cho mô hình ngôn ngữ. Mặt khác, sự phát triển nhanh chóng của mô hình ngôn ngữ lớn đã thúc đẩy thêm đổi mới trong các kỹ thuật liên quan đến Transformer, đặc biệt là phát triển các phương pháp thích ứng hiệu quả và chi phí thấp cho các mô hình khổng lồ.

- **Phân tích lý thuyết**. Trong khi Transformer thống trị về mặt thực nghiệm, nền tảng lý thuyết của chúng trước đây ít được chú trọng hơn so với các cải tiến kỹ thuật — một xu hướng "thực nghiệm trước" phổ biến trong cộng đồng học máy nói chung. Để khắc phục, các nhà nghiên cứu đã bắt đầu phân tích Transformer sâu hơn. Một cách tiếp cận là coi Transformer là mạng nơ-ron sâu và diễn giải chúng bằng các công cụ toán học. Chẳng hạn, các kết nối tắt trong Transformer có thể được xem như phép rời rạc hóa Euler của các ODE. Sự tương đương này gợi ý rằng ta có thể tận dụng hiểu biết từ các phương pháp số giải ODE để định hướng thiết kế mô hình. Một hướng hứa hẹn khác nhằm phát triển hiểu biết lý thuyết về cơ chế tự chú ý, thứ phân biệt Transformer với các mô hình học sâu trước đó. Các nghiên cứu đã diễn giải tự chú ý qua nhiều lăng kính lý thuyết, bao gồm nén dữ liệu [Yu và cộng sự,

2023a], tối ưu hóa [Li và cộng sự, 2022c] và xấp xỉ hàm [Yun và cộng sự, 2019]. Hơn nữa, Transformer có thể được ánh xạ sang các hệ hình thức, như máy Turing [Pérez và cộng sự, 2018], máy đếm [Bhattamishra và cộng sự, 2020], ngôn ngữ chính quy và ngôn ngữ phi ngữ cảnh [Hahn, 2020], mạch Boole [Hao và cộng sự, 2022; Merrill và cộng sự, 2022], ngôn ngữ lập trình [Weiss và cộng sự, 2021], và logic bậc nhất [Chiang và cộng sự, 2023a]. Những liên hệ này cung cấp công cụ lý thuyết để nghiên cứu chặt chẽ năng lực biểu đạt của Transformer. Tuy nhiên, đáng lưu ý là dù ta có thể phân tích Transformer qua những góc nhìn đa dạng này, vẫn chưa có một lý thuyết thống nhất nào giải thích bản chất cơ bản của các mô hình này. Lấp đầy khoảng trống lý thuyết này vẫn là một thách thức trung tâm của lĩnh vực, nhưng lại thiết yếu để phát triển các hệ thống nơ-ron phức tạp có hành vi giải thích được và dự đoán được.

Chú giải của người dịch: Kết nối tắt và phương pháp Euler

Kết nối tắt có dạng $\mathbf{x}_{l+1} = \mathbf{x}_l + F(\mathbf{x}_l)$, còn phương pháp Euler (xem chú thích ở Phương trình (6.166)) có dạng $\mathbf{z}_t = \mathbf{z}_{t-1} + \Delta t \cdot g(\mathbf{z}_{t-1})$. Hai công thức giống hệt nhau nếu coi mỗi tầng là một "bước thời gian" và F đóng vai $\Delta t \cdot g$. Nhờ vậy, một mạng nhiều tầng có thể được hiểu như việc giải gần đúng một phương trình vi phân, và những phương pháp giải số chính xác hơn Euler gợi ý những kiểu kết nối giữa các tầng tốt hơn.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.